```
// src/applicationManager.ts
import Clutter from 'gi://Clutter';
import St from 'gi://St';
import Meta from 'gi://Meta';
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import { LiquidEffect } from './liquidEffect.js';
import Gio from 'gi://Gio';
import GLib from 'gi://GLib';
import { UnpickableClone, UnpickableActor, InverseCornerEffect, getWindowActors, isActorValid } from './utils.js';

import { Logger } from './logger.js';

// Padding to allow the shader to draw effects (like refraction and blur) outside the actor's strict bounds.
const SHADER_PADDING = 10;
// Inward padding for corner rounding
const CORNER_PADDING = 3;

interface WindowState {
  windowActor: Meta.WindowActor;
  // The window's own content/surface actor (the actual client texture). Cached here
  // because windowActor.get_first_child() stops pointing at it once baseActor is
  // inserted below it in _setupWindow — re-deriving it via get_first_child() later
  // (as _updateWindowOpacities/_cleanupState used to) silently targets the wrong actor.
  surfaceActor: Clutter.Actor;
  bgActor: St.Widget;
  clipBox: St.Widget;
  bgClone: InstanceType<typeof UnpickableClone>;
  windowsContainer: Clutter.Actor;
  clones: Map<Meta.WindowActor, Clutter.Actor>;
  effect: LiquidEffect;
  // Unblurred base background, used to reveal the true corners (see InverseCornerEffect below).
  baseActor: St.Widget;
  baseClone: InstanceType<typeof UnpickableClone>;
  baseWindowsContainer: Clutter.Actor;
  baseClones: Map<Meta.WindowActor, Clutter.Actor>;
  // To cut window corners
  roundingEffect: InstanceType<typeof InverseCornerEffect>;
  cornerOverlay: InstanceType<typeof UnpickableActor>;
  cornerOverlayClone: InstanceType<typeof UnpickableClone>;
  signals: { obj: any, id: number }[];
  // Content opacity applied to the window's own surface layer so the glass shows
  // through it; restored when the effect is removed from this window.
  originalOpacity: number;
}

export class ApplicationManager {
  private extensionPath: string;
  private _states: Map<Meta.WindowActor, WindowState>;
  private _settings: Gio.Settings;
  private _logger: Logger;
  private _settingsSignals: number[];
  private _frameSyncId: number;
  private _windowCreatedId: number;
  private _restackedId: number = 0;
  private _rebuildQueued: boolean = false;

  constructor(extensionPath: string, settings: Gio.Settings, logger: Logger) {
    this.extensionPath = extensionPath;
    this._settings = settings;
    this._logger = logger;
    this._states = new Map();
    this._settingsSignals = [];
    this._frameSyncId = 0;
    this._windowCreatedId = 0;
    this._restackedId = 0;
  }

  setup() {
    this._logger.log("[Liquid Glass] ApplicationManager setup starting...");
    this._bindSettings();

    this._windowCreatedId = global.display.connect('window-created', (_d, metaWindow) => {
      this._logger.log(`[Liquid Glass] window-created event: window title = "${metaWindow.get_title()}", class = "$
{metaWindow.get_wm_class()}"`);
      const obj = metaWindow.get_compositor_private();
      if (!obj) {
        this._logger.log("[Liquid Glass] get_compositor_private() returned null");
        return;
      }
      if (!(obj instanceof Meta.WindowActor)) {
        this._logger.log("[Liquid Glass] compositor object is not instance of Meta.WindowActor");
        return;
```

```
      }
      this._logger.log("[Liquid Glass] window compositor actor found. Connecting to first-frame.");
      obj.connect('first-frame', () => {
        this._logger.log("[Liquid Glass] first-frame event fired for window: " + metaWindow.get_title());
        if (this._shouldApplyToWindow(obj)) {
          this._setupWindow(obj);
          this._rebuildAllClones();
        }
      });
    });

    this._restackedId = global.display.connect('restacked', () => {
      this._rebuildAllClones();
    });

    this._logger.log("[Liquid Glass] checking if effect enabled in setup: " + this._isEffectEnabled());
    if (this._isEffectEnabled())
      this._applyEffects();
  }

  cleanup() {
    if (this._windowCreatedId) {
      global.display.disconnect(this._windowCreatedId);
      this._windowCreatedId = 0;
    }

    if (this._restackedId) {
      global.display.disconnect(this._restackedId);
      this._restackedId = 0;
    }

    this._settingsSignals.forEach(id => this._settings.disconnect(id));
    this._settingsSignals = [];

    this._removeAllEffects();
  }

  _bindSettings() {
    const connectSetting = (key: string, callback: () => void) => {
      let id = this._settings.connect(`changed::${key}`, callback);
      this._settingsSignals.push(id);
    };

    connectSetting('enable-application-glass', () => {
      this._logger.log("[Liquid Glass] enable-application-glass setting changed to: " + this._isEffectEnabled());
      if (this._isEffectEnabled())
        this._applyEffects();
      else
        this._removeAllEffects();
    });

    // Apply to every normal/dialog window, bypassing the whitelist entirely.
    connectSetting('application-glass-all-windows', () => this._syncWhitelist());
    // Opacity of the window's own content layer, so the glass underneath is visible through it.
    connectSetting('application-content-opacity', () => this._updateWindowOpacities());
    connectSetting('application-window-whitelist', () => this._syncWhitelist());
    connectSetting('application-tint-color', () => this._updateEffectParams());
    connectSetting('application-tint-strength', () => this._updateEffectParams());
    connectSetting('application-blur-radius', () => this._updateEffectParams());
    connectSetting('application-corner-radius', () => this._updateEffectParams());
    connectSetting('application-brightness', () => this._updateEffectParams());
    connectSetting('application-contrast', () => this._updateEffectParams());
    connectSetting('application-saturation', () => this._updateEffectParams());
  }

  _getContentOpacity(): number {
    return this._settings.get_double('application-content-opacity');
  }

  _updateWindowOpacities() {
    const targetOpacity = Math.round(this._getContentOpacity() * 255);
    this._logger.log("[Liquid Glass] Updating window content opacities to: " + targetOpacity);
    for (let state of this._states.values()) {
      // Use the cached reference, NOT windowActor.get_first_child() — after
      // _setupWindow inserts baseActor below it, get_first_child() returns
      // baseActor instead of the real surface, so it stopped being live-updated.
      if (isActorValid(state.surfaceActor)) {
        state.surfaceActor.opacity = targetOpacity;
      }
    }
  }
```

```
  _isEffectEnabled(): boolean {
    const enabled = this._settings.get_boolean('enable-application-glass');
    return enabled;
  }

  _getWhitelist(): string[] {
    let whitelist = this._settings.get_strv('application-window-whitelist');
    return whitelist;
  }

  _windowMatchesWhitelist(metaWindow: Meta.Window): boolean {
    const whitelist = this._getWhitelist();
    const appName = metaWindow.get_wm_class();
    if (whitelist.length === 0) {
      return false;
    }

    let ret = !!appName && whitelist.includes(appName);
    if (!ret) {
      this._logger.log("[Liquid Glass] window is not in whitelist. name = " + appName);
    } else {
      this._logger.log("[Liquid Glass] window is in whitelist. name = " + appName);
    }
    return ret;
  }

  _shouldApplyToWindow(windowActor: Meta.WindowActor): boolean {
    if (!this._isEffectEnabled()) {
      return false;
    }

    const metaWindow = windowActor.get_meta_window();
    if (!metaWindow) {
      return false;
    }

    // "Apply to all windows" bypasses the whitelist, but is still restricted to
    // normal/dialog windows so we never touch desktop backgrounds, panels, etc.
    const applyAll = this._settings.get_boolean('application-glass-all-windows');
    if (applyAll) {
      const windowType = metaWindow.get_window_type();
      const isNormal = windowType === Meta.WindowType.NORMAL ||
        windowType === Meta.WindowType.DIALOG ||
        windowType === Meta.WindowType.MODAL_DIALOG;
      if (!isNormal) {
        this._logger.log(`[Liquid Glass] window "${metaWindow.get_title()}" has special type ${windowType},
skipping...`);
        return false;
      }
      return true;
    }

    return this._windowMatchesWhitelist(metaWindow);
  }

  _applyEffects() {
    this._logger.log("[Liquid Glass] _applyEffects called");
    this._buildForExistingWindows();
    this._startFrameSync();
  }

  _removeAllEffects() {
    if (this._frameSyncId) {
      if (global.compositor?.get_laters) {
        global.compositor.get_laters().remove(this._frameSyncId);
      }
      this._frameSyncId = 0;
    }

    for (let state of this._states.values())
      this._cleanupState(state);

    this._states.clear();
    this._rebuildQueued = false;
  }

  _syncWhitelist() {
    if (!this._isEffectEnabled()) {
      this._removeAllEffects();
      return;
    }
```

```
    for (let [actor, state] of [...this._states.entries()]) {
      if (!this._shouldApplyToWindow(actor)) {
        this._cleanupState(state);
        this._states.delete(actor);
      }
    }

    for (let actor of getWindowActors()) {
      if (this._shouldApplyToWindow(actor) && !this._states.has(actor))
        this._setupWindow(actor);
    }

    this._rebuildAllClones();
  }

  _updateEffectParams() {
    let tintColorStr = this._settings.get_string('application-tint-color');
    let tintStrength = this._settings.get_double('application-tint-strength');
    let blurRadius = this._settings.get_int('application-blur-radius');
    let cornerRadius = this._settings.get_double('application-corner-radius');
    let brightness = this._settings.get_double('application-brightness');
    let contrast = this._settings.get_double('application-contrast');
    let saturation = this._settings.get_double('application-saturation');

    for (let state of this._states.values()) {
      state.effect.setTintColor(...this._hexToColorArray(tintColorStr));
      state.effect.setTintStrength(tintStrength);
      state.effect.setCornerRadius(cornerRadius);
      state.effect.setBlurRadius(blurRadius);
      state.effect.setBrightness(brightness);
      state.effect.setContrast(contrast);
      state.effect.setSaturation(saturation);
      state.roundingEffect.setRadius(cornerRadius + CORNER_PADDING);
      state.roundingEffect.setInset(this._cornerOverlayInset());
    }
  }

  // [FIX] This used to be SHADER_PADDING + CORNER_PADDING, and
  // InverseCornerEffect used it to shrink its rounded-rect cut inward by the
  // same amount on every side (straight edges included), instead of only
  // pulling the 4 actual corners inward. That revealed a uniform band of
  // raw, unblurred/unshadowed background all the way around the window —
  // see InverseCornerEffect._updateShader() in utils.ts for the full
  // explanation. The overlay now derives the window's true edge from this
  // value alone (it equals SHADER_PADDING, the outward padding this actor
  // has beyond the real window bounds), while CORNER_PADDING is applied
  // only to the radius (below) so it exclusively affects the corner arcs.
  _cornerOverlayInset(): number {
    return SHADER_PADDING;
  }

  _startFrameSync() {
    if (this._frameSyncId === 0)
      this._frameTick();
  }

  _rebuildAllClones() {
    if (this._rebuildQueued) return;
    this._rebuildQueued = true;

    // Debounce to next idle to avoid crashing during rapid restacking/creation
    GLib.idle_add(GLib.PRIORITY_DEFAULT_IDLE, () => {
      if (this._states.size === 0) {
        this._rebuildQueued = false;
        return GLib.SOURCE_REMOVE;
      }
      for (let state of this._states.values()) {
        this._rebuildWindowClones(state);
      }
      this._rebuildQueued = false;
      return GLib.SOURCE_REMOVE;
    });
  }

  _buildForExistingWindows() {
    for (let actor of getWindowActors()) {
      if (this._shouldApplyToWindow(actor))
        this._setupWindow(actor);
    }
  }

  _setupWindow(windowActor: any) {
```

```
    if (!windowActor || !(windowActor instanceof Meta.WindowActor) || this._states.has(windowActor))
      return;

    if (!this._shouldApplyToWindow(windowActor))
      return;

    let surfaceActor = windowActor.get_first_child();
    if (!surfaceActor) {
      return;
    }

    let parent = windowActor.get_parent();
    if (!parent)
      return;

    // Store the surface's original opacity and dial it down so the glass behind
    // it is actually visible; restored in _cleanupState when the effect is removed.
    let originalOpacity = surfaceActor.opacity;
    surfaceActor.opacity = Math.round(this._getContentOpacity() * 255);

    let baseActor = new St.Widget({
      style_class: 'liquid-glass-base-actor',
      reactive: false,
      clip_to_allocation: true,
      visible: true,
    });
    windowActor.insert_child_below(baseActor, surfaceActor);

    let bgActor = new St.Widget({
      style_class: 'liquid-glass-bg-actor',
      reactive: false,
      clip_to_allocation: false,
      visible: true,
    });
    windowActor.insert_child_above(bgActor, baseActor);

    // Both actors sit entirely behind surfaceActor (the window's own content),
    // which is what makes the glass show "through" it once its opacity is
    // dialed down below. Clutter's own occlusion culling doesn't know that
    // surfaceActor is translucent, though -- it treats it as opaque and, on
    // a window's first paint (before anything else has forced a relayout),
    // can conclude baseActor/bgActor are fully covered and skip painting
    // them entirely. That produced a black background behind the window's
    // translucent chrome until something else (resize, move, opening
    // another window, defocusing) forced Clutter to reconsider. Explicitly
    // inhibiting culling on both actors keeps them painting unconditionally.
    baseActor.inhibit_culling();
    bgActor.inhibit_culling();

    let clipBox = new St.Widget({
      clip_to_allocation: true,
      reactive: false,
    });
    bgActor.add_child(clipBox);

    // Size the clones to cover the full monitor so the wallpaper fills correctly.
    let monitor = Main.layoutManager.primaryMonitor;

    let baseClone = new UnpickableClone({
      source: Main.layoutManager._backgroundGroup,
    });
    if (monitor) {
      baseClone.set_size(monitor.width, monitor.height);
    }
    baseActor.add_child(baseClone);

    let baseWindowsContainer = new Clutter.Actor();
    baseActor.add_child(baseWindowsContainer);

    let bgClone = new UnpickableClone({
      source: Main.layoutManager._backgroundGroup,
    });
    if (monitor) {
      bgClone.set_size(monitor.width, monitor.height);
    }
    clipBox.add_child(bgClone);

    let effect = new LiquidEffect({
      extensionPath: this.extensionPath,
      settings: this._settings,
      logger: this._logger,
    } as any);
```

```
    let tintColorStr = this._settings.get_string('application-tint-color');
    let tintStrength = this._settings.get_double('application-tint-strength');
    let cornerRadius = this._settings.get_double('application-corner-radius');
    let blurRadius = this._settings.get_int('application-blur-radius');
    let brightness = this._settings.get_double('application-brightness');
    let contrast = this._settings.get_double('application-contrast');
    let saturation = this._settings.get_double('application-saturation');

    effect.setPadding(SHADER_PADDING);
    effect.setTintColor(...this._hexToColorArray(tintColorStr));
    effect.setTintStrength(tintStrength);
    effect.setCornerRadius(cornerRadius);
    effect.setBlurRadius(blurRadius);
    effect.setBrightness(brightness);
    effect.setContrast(contrast);
    effect.setSaturation(saturation);
    effect.setIsDock(false);
    // Application windows should read as plain "drop shadow + AO" at the
    // edge (like dockManager's shadow treatment), not the dock/menu-style
    // rim + specular + sheen glass glint — that glint sits right at the
    // window's true edge and, combined with the window's own (often
    // non-opaque) content, reads as a distracting bright frame around the
    // window. Blur/tint/refraction are unaffected; only this glint group
    // is turned off, and only for this per-window effect instance — the
    // shared glass-rim-*/glass-sheen-*/glass-specular-* settings still
    // apply normally to the dock, menu, notification, quick-settings and
    // OSD glass.
    effect.setSurfaceLightEnabled(false);
    // Keep the drop-shadow within the small padded border around the window,
    // rather than the huge margin dockManager uses for its full-screen FBO.
    effect.setShadowMaxRadius(SHADER_PADDING);
    bgActor.add_effect(effect);

    let windowsContainer = new Clutter.Actor();
    clipBox.add_child(windowsContainer);

    let cornerOverlay = new UnpickableActor({
      clip_to_allocation: true,
      reactive: false,
    });
    let cornerOverlayClone = new UnpickableClone({ source: baseActor });
    cornerOverlay.add_child(cornerOverlayClone);

    let roundingEffect = new InverseCornerEffect();
    roundingEffect.setRadius(cornerRadius + CORNER_PADDING);
    roundingEffect.setInset(this._cornerOverlayInset());

    cornerOverlay.add_effect(roundingEffect);
    windowActor.add_child(cornerOverlay);

    let state: WindowState = {
      windowActor,
      surfaceActor,
      bgActor,
      clipBox,
      bgClone,
      windowsContainer,
      clones: new Map(),
      effect,
      baseActor,
      baseClone,
      baseWindowsContainer,
      baseClones: new Map(),
      roundingEffect,
      cornerOverlay,
      cornerOverlayClone,
      signals: [],
      originalOpacity,
    };

    this._states.set(windowActor, state);
    this._rebuildWindowClones(state);

    // Immediate sync connections for resize/move using allocation property
    state.signals.push({
      obj: windowActor,
      id: windowActor.connect('notify::allocation', () => this._syncState(state))
    });

    const metaWin = windowActor.get_meta_window();
    if (metaWin) {
```

```
      state.signals.push({
        obj: metaWin,
        id: metaWin.connect('size-changed', () => {
          this._rebuildWindowClones(state);
          this._syncState(state);
        })
      });
      state.signals.push({
        obj: metaWin,
        id: metaWin.connect('position-changed', () => {
          this._syncState(state);
        })
      });
    }

    // Use a later to ensure the initial sync happens after actors are properly added to stage
    global.compositor.get_laters().add(Meta.LaterType.IDLE, () => {
      if (this._states.has(windowActor)) {
        this._syncState(state);
      }
      return false;
    });

    windowActor.connect('destroy', () => {
      this._cleanupState(state);
      this._states.delete(windowActor);
      this._rebuildAllClones();
    });
  }

  _hexToColorArray(hex: string): [number, number, number] {
    if (!hex || typeof hex !== 'string' || !hex.startsWith('#') || hex.length !== 7) return [1.0, 1.0, 1.0];
    let r = parseInt(hex.slice(1, 3), 16) / 255.0;
    let g = parseInt(hex.slice(3, 5), 16) / 255.0;
    let b = parseInt(hex.slice(5, 7), 16) / 255.0;
    return [r, g, b];
  }

  _rebuildWindowClones(state: WindowState) {
    state.clones.forEach(clone => clone.destroy());
    state.clones.clear();
    state.windowsContainer.remove_all_children();

    state.baseClones.forEach(clone => clone.destroy());
    state.baseClones.clear();
    state.baseWindowsContainer.remove_all_children();

    // Get windows in stacking order (bottom to top)
    for (let actor of getWindowActors()) {
      // STOP iterating once we reach our own window.
      // This ensures we ONLY render what is actually BEHIND the app.
      if (actor === state.windowActor)
        break;

      if (!(actor instanceof Meta.WindowActor))
        continue;

      let clone = new UnpickableClone({ source: actor });
      state.windowsContainer.add_child(clone);
      state.clones.set(actor, clone);

      let baseClone = new UnpickableClone({ source: actor });
      state.baseWindowsContainer.add_child(baseClone);
      state.baseClones.set(actor, baseClone);
    }
  }

  _syncState(state: WindowState) {
    let actor = state.windowActor;
    if (!actor || !actor.get_stage() || !actor.mapped) {
      state.bgActor.visible = false;
      state.baseActor.visible = false;
      state.cornerOverlay.visible = false;
      return;
    }

    if (!actor.has_allocation()) {
      return;
    }

    const metaWin = actor.get_meta_window();
    if (!metaWin) return;
```

```
    // PERFORMANCE: Only sync windows on the current active workspace.
    const workspaceManager = global.workspace_manager;
    const activeWorkspace = workspaceManager.get_active_workspace();
    const winWorkspace = metaWin.get_workspace();
    if (winWorkspace && winWorkspace !== activeWorkspace) {
      if (state.bgActor.visible) state.bgActor.visible = false;
      if (state.baseActor.visible) state.baseActor.visible = false;
      if (state.cornerOverlay.visible) state.cornerOverlay.visible = false;
      return;
    }

    const rect = metaWin.get_frame_rect();
    const bufferRect = metaWin.get_buffer_rect();

    if (!rect || !bufferRect || rect.width <= 0 || rect.height <= 0) {
      if (state.bgActor.visible) state.bgActor.visible = false;
      if (state.baseActor.visible) state.baseActor.visible = false;
      if (state.cornerOverlay.visible) state.cornerOverlay.visible = false;
      return;
    }

    if (!state.bgActor.visible) state.bgActor.visible = true;
    if (!state.baseActor.visible) state.baseActor.visible = true;

    // Local offset of the visible frame within the window actor's full buffer.
    const frameLocalX = rect.x - bufferRect.x;
    const frameLocalY = rect.y - bufferRect.y;

    // Base background (unblurred) expanded by expansion margin.
    state.baseActor.set_position(frameLocalX - SHADER_PADDING, frameLocalY - SHADER_PADDING);
    state.baseActor.set_size(rect.width + (SHADER_PADDING * 2), rect.height + (SHADER_PADDING * 2));

    // Glass background (blurred) expanded by padding.
    const bgW = rect.width + (SHADER_PADDING * 2);
    const bgH = rect.height + (SHADER_PADDING * 2);
    const localX = frameLocalX - SHADER_PADDING;
    const localY = frameLocalY - SHADER_PADDING;

    state.bgActor.set_position(localX, localY);
    state.bgActor.set_size(bgW, bgH);

    state.clipBox.set_position(0, 0);
    state.clipBox.set_size(bgW, bgH);

    // Update shader resolution/geometry with the expanded bounds. The glass
    // rect always fills the whole (small, non full-screen) bgActor here, so
    // glass geometry is simply (0, 0, bgW, bgH).
    if (state.effect) {
      state.effect.setResolution(bgW, bgH);
      state.effect.setGlassGeometry(0, 0, bgW, bgH);
    }

    // Use absolute screen coordinates for wallpaper fix
    const absX = rect.x - SHADER_PADDING;
    const absY = rect.y - SHADER_PADDING;

    // Offset for the clipped (blurred) content
    state.bgClone.set_position(-absX, -absY);
    state.windowsContainer.set_position(-absX, -absY);

    // Offset for the base (unblurred) content (baseActor is inset by SHADER_PADDING).
    const baseScreenX = rect.x - SHADER_PADDING;
    const baseScreenY = rect.y - SHADER_PADDING;
    state.baseClone.set_position(-baseScreenX, -baseScreenY);
    state.baseWindowsContainer.set_position(-baseScreenX, -baseScreenY);

    // Sync blurred clones
    for (let [src, clone] of state.clones.entries()) {
      if (!isActorValid(src) || !src.visible || !src.mapped || !src.has_allocation()) {
        if (isActorValid(clone) && clone.visible) clone.hide();
        continue;
      }
      if (isActorValid(clone)) {
        if (!clone.visible) clone.show();

        clone.set_position(src.x, src.y);
        clone.set_size(src.width, src.height);
        clone.set_scale(src.scale_x, src.scale_y);
        clone.opacity = src.opacity;
      }
    }
```

```
    // Sync base clones (unblurred)
    for (let [src, clone] of state.baseClones.entries()) {
      if (!isActorValid(src) || !src.visible || !src.mapped || !src.has_allocation()) {
        if (isActorValid(clone) && clone.visible) clone.hide();
        continue;
      }
      if (isActorValid(clone)) {
        if (!clone.visible) clone.show();

        clone.set_position(src.x, src.y);
        clone.set_size(src.width, src.height);
        clone.set_scale(src.scale_x, src.scale_y);
        clone.opacity = src.opacity;
      }
    }

    if (!state.cornerOverlay.visible) {
      state.cornerOverlay.show();
    }

    const baseW = rect.width + (SHADER_PADDING * 2);
    const baseH = rect.height + (SHADER_PADDING * 2);

    state.cornerOverlay.set_position(
      frameLocalX - SHADER_PADDING,
      frameLocalY - SHADER_PADDING,
    );
    state.cornerOverlay.set_size(baseW, baseH);

    state.cornerOverlayClone.set_position(0, 0);
    state.cornerOverlayClone.set_size(baseW, baseH);
  }

  _frameTick() {
    for (let state of this._states.values()) {
      try {
        this._syncState(state);
      } catch (e) {
        this._logger.error(`[Liquid Glass] Error in _syncState: ${e}`);
      }
    }

    this._frameSyncId = global.compositor.get_laters().add(
      Meta.LaterType.BEFORE_REDRAW,
      () => {
        this._frameTick();
        return false;
      }
    );
  }

  _cleanupState(state: WindowState) {
    if (!state) return;

    // Restore the original opacity of the window's own content layer.
    // Uses the cached surfaceActor reference (see WindowState) rather than
    // windowActor.get_first_child(), which no longer points at the real
    // surface once baseActor has been inserted below it.
    if (state.surfaceActor) {
      try {
        if (isActorValid(state.surfaceActor)) {
          state.surfaceActor.opacity = state.originalOpacity;
        }
      } catch (e) {
        // Window actor may already be destroyed; safe to ignore.
      }
    }

    if (state.signals) {
      state.signals.forEach(sig => {
        try {
          sig.obj.disconnect(sig.id);
        } catch (e) { }
      });
      state.signals = [];
    }

    state.clones.forEach(clone => clone.destroy());
    state.clones.clear();

    state.baseClones.forEach(clone => clone.destroy());
```

```
    state.baseClones.clear();

    if (state.effect) {
      try {
        state.effect.cleanup();
      } catch (e) { }
    }

    if (state.bgActor)
      state.bgActor.destroy();

    if (state.baseActor)
      state.baseActor.destroy();

    if (state.cornerOverlay)
      state.cornerOverlay.destroy();
  }
}
```

```
import GLib from 'gi://GLib';
import Shell from 'gi://Shell';
import GdkPixbuf from 'gi://GdkPixbuf';
import Gio from 'gi://Gio';
import Clutter from 'gi://Clutter';

export const AdaptiveContrastConfig = {
  enabled: true,
  samplePerElement: false, // 要素ごとにサンプリングするか、全体をまとめてサンプリングするか　負荷を考慮してデフォルトはまとめてサンプリング
  sampleIntervalMs: 200, // 5Hz
  luminanceThreshold: 0.42,
  lightTextColor: '#f2f2f2',
  darkTextColor: '#1a1a1a',
};

function _clamp(v: number, min: number, max: number): number {
  return Math.min(max, Math.max(min, v));
}

function _srgbToLinear(c: number): number {
  const n = c / 255.0;
  if (n <= 0.04045)
    return n / 12.92;
  return Math.pow((n + 0.055) / 1.055, 2.4);
}

function _luminanceFromRgb(r: number, g: number, b: number): number {
  const rl = _srgbToLinear(r);
  const gl = _srgbToLinear(g);
  const bl = _srgbToLinear(b);
  return 0.2126 * rl + 0.7152 * gl + 0.0722 * bl;
}

function _trimmedMean(values: number[], trimRatio: number = 0.1): number | null {
  if (values.length === 0)
    return null;

  const sorted = [...values].sort((a, b) => a - b);
  const trim = Math.floor(sorted.length * trimRatio);
  const start = _clamp(trim, 0, sorted.length - 1);
  const end = _clamp(sorted.length - trim, start + 1, sorted.length);

  let sum = 0.0;
  for (let i = start; i < end; i++)
    sum += sorted[i];

  return sum / (end - start);
}

function _getActorRect(actor: Clutter.Actor): { x: number, y: number, width: number, height: number } | null {
  if (!actor)
    return null;

  const [x, y] = actor.get_transformed_position();
  const [w, h] = actor.get_size();

  if ([x, y, w, h].some(Number.isNaN))
    return null;

  const width = Math.max(1, Math.floor(w));
  const height = Math.max(1, Math.floor(h));
  return {
    x: Math.floor(x),
    y: Math.floor(y),
    width,
    height,
  };
}

function _mergeRects(rects: { x: number, y: number, width: number, height: number }[]): { x: number, y: number, width: number, height: number } | null {
  if (rects.length === 0)
    return null;

  let minX = rects[0].x;
  let minY = rects[0].y;
  let maxX = rects[0].x + rects[0].width;
  let maxY = rects[0].y + rects[0].height;

  for (let i = 1; i < rects.length; i++) {
    const r = rects[i];
```

```
    minX = Math.min(minX, r.x);
    minY = Math.min(minY, r.y);
    maxX = Math.max(maxX, r.x + r.width);
    maxY = Math.max(maxY, r.y + r.height);
  }

  return {
    x: minX,
    y: minY,
    width: Math.max(1, maxX - minX),
    height: Math.max(1, maxY - minY),
  };
}

async function _captureAreaToFile(screenshot: Shell.Screenshot, rect: { x: number, y: number, width: number, height:
number }, filePath: string): Promise<boolean> {
  return new Promise((resolve: (success: boolean) => void) => {
    try {
      // 1. 文字列のパスから Gio.File オブジェクトを生成
      const file = Gio.File.new_for_path(filePath) as Gio.File;

      // 2. 上書きモードで書き込み用ストリーム（GOutputStream）を開く
      const stream = file.replace(null, false, Gio.FileCreateFlags.REPLACE_DESTINATION, null);

      // 3. パス文字列の代わりに、作成した stream を引数に渡す
      screenshot.screenshot_area(
        rect.x, rect.y, rect.width, rect.height, stream,
        (obj, res) => {
          try {
            if (!obj) {
              throw new Error("Screenshot object is null");
            }
            // スクリーンショット処理の完了を待機
            const success = obj.screenshot_area_finish(res)[0];

            // ファイルがロックされたままになるのを防ぐためストリームを閉じる
            stream.close(null);
            resolve(success);
          } catch (e) {
            stream.close(null); // エラー時も確実に閉じる
            resolve(false);
          }
        }
      );
    } catch (e) {
      resolve(false);
    }
  });
}

function _buildTempPath(): string {
  const token = `${GLib.get_monotonic_time()}-${Math.floor(Math.random() * 1000000)}`;
  return `${GLib.get_tmp_dir()}/liquid-glass-sample-${token}.png`;
}

export class StageContrastSampler {
  private _screenshot: Shell.Screenshot;

  constructor() {
    this._screenshot = new Shell.Screenshot();
  }

  async sampleLuminance(rect: { x: number, y: number, width: number, height: number }): Promise<number | null> {
    if (!rect || rect.width <= 0 || rect.height <= 0)
      return null;

    const filePath = _buildTempPath();
    const captured = await _captureAreaToFile(this._screenshot, rect, filePath);
    if (!captured)
      return null;

    try {
      const pixbuf = GdkPixbuf.Pixbuf.new_from_file(filePath);
      if (!pixbuf)
        return null;

      const width = pixbuf.get_width();
      const height = pixbuf.get_height();
      const rowstride = pixbuf.get_rowstride();
      const channels = pixbuf.get_n_channels();
      const hasAlpha = pixbuf.get_has_alpha();
      const pixels = pixbuf.get_pixels();
```

```
      const step = Math.max(1, Math.floor(Math.min(width, height) / 48));
      const values: number[] = [];

      for (let y = 0; y < height; y += step) {
        for (let x = 0; x < width; x += step) {
          const idx = y * rowstride + x * channels;
          if (hasAlpha) {
            const a = pixels[idx + 3];
            if (a < 32)
              continue;

            if (a < 255) {
              const scale = 255.0 / a;
              const r = _clamp(Math.round(pixels[idx + 0] * scale), 0, 255);
              const g = _clamp(Math.round(pixels[idx + 1] * scale), 0, 255);
              const b = _clamp(Math.round(pixels[idx + 2] * scale), 0, 255);
              values.push(_luminanceFromRgb(r, g, b));
              continue;
            }
          }

          const r = pixels[idx + 0];
          const g = pixels[idx + 1];
          const b = pixels[idx + 2];
          values.push(_luminanceFromRgb(r, g, b));
        }
      }

      return _trimmedMean(values, 0.10);
    } catch (e) {
      return null;
    } finally {
      try {
        GLib.unlink(filePath);
      } catch (_) {
        // Ignore cleanup errors.
      }
    }
  }

  decideTextColor(luminance: number, config: typeof AdaptiveContrastConfig = AdaptiveContrastConfig): string | null {
    if (luminance === null || luminance === undefined)
      return null;

    return luminance > config.luminanceThreshold
      ? config.darkTextColor
      : config.lightTextColor;
  }

  async chooseColorsForActors(actors: Clutter.Actor[], config: typeof AdaptiveContrastConfig =
AdaptiveContrastConfig): Promise<Map<Clutter.Actor, string>> {
    const rects: { x: number, y: number, width: number, height: number }[] = [];
    const targets: Clutter.Actor[] = [];

    for (const actor of actors) {
      const rect = _getActorRect(actor);
      if (!rect)
        continue;

      targets.push(actor);
      rects.push(rect);
    }

    const result = new Map();
    if (targets.length === 0)
      return result;

    if (!config.samplePerElement) {
      const merged = _mergeRects(rects);
      if (!merged)
        return result;
      const luma = await this.sampleLuminance(merged);
      if (!luma)
        return result;
      const color = this.decideTextColor(luma, config);
      if (!color)
        return result;

      for (const actor of targets)
        result.set(actor, color);
      return result;
```

```
    }

    for (let i = 0; i < targets.length; i++) {
      const luma = await this.sampleLuminance(rects[i]);
      if (!luma)
        return result;
      const color = this.decideTextColor(luma, config);
      if (color)
        result.set(targets[i], color);
    }

    return result;
  }
}
```

```
// src/dockManager.js
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import Clutter from 'gi://Clutter';
import St from 'gi://St';
import GLib from 'gi://GLib';
import Meta from 'gi://Meta';
import { LiquidEffect } from './liquidEffect.js';
import Gio from 'gi://Gio';
import { UnpickableActor, UILayerSampler, WindowCloneManager } from './utils.js';

import { Logger } from './logger.js';

// Padding to allow the shader to draw effects (like refraction and blur) outside the actor's strict bounds.
const SHADER_PADDING = 20;

// Utility: Convert HEX color string (e.g., "#ffffff") to normalized RGB array [1.0, 1.0, 1.0]
function hexToColorArray(hex: string): [number, number, number] {
  if (!hex || typeof hex !== 'string' || !hex.startsWith('#') || hex.length !== 7) {
    return [1.0, 1.0, 1.0];
  }
  let r = parseInt(hex.slice(1, 3), 16) / 255.0;
  let g = parseInt(hex.slice(3, 5), 16) / 255.0;
  let b = parseInt(hex.slice(5, 7), 16) / 255.0;
  return [r, g, b];
}

export class DashManager {
  private extensionPath: string;
  private targetActor: St.Widget;
  private _settings: Gio.Settings;

  // private bgActor: St.Widget | null = null;
  private bgActor: Clutter.Actor | null = null;
  // Delete Shell.BlurEffect and use custom blur (dual kawase) — now handled inside LiquidEffect
  private effect: LiquidEffect | null = null;

  // Nesting order (outermost → innermost):
  //   bgActor (full monitor, no effect)
  //     └─ liquidBox  ← LiquidEffect with built-in dual-Kawase blur
  //          └─ _cloneContainer  ← bgClone + windowClones + uiClones
  private liquidBox: Clutter.Actor | null = null;

  private _lastScreenW: number | undefined;
  private _lastScreenH: number | undefined;

  private _glassExpand: number;


  private _signals: number[];
  private _settingsSignals: number[]; // GSettingsのイベントリスナーを管理
  private _frameSyncId: number;
  private _isEffectActive: boolean; // エフェクトが現在適用されているかのフラグ

  private _originalStyle: string | undefined;
  private _currentMarginStyle: string | undefined;
  private _dockParent: St.Widget | null = null;

  private _cloneContainer: Clutter.Actor | null = null;

  private _lastAbsX: number | undefined;
  private _lastAbsY: number | undefined;
  private _lastTW: number | undefined;
  private _lastTH: number | undefined;
  private _stableDeltaW: number | undefined;
  private _stableDeltaH: number | undefined;
  private _lastBgW: number | undefined;
  private _lastBgH: number | undefined;
  private _lastBgX: number | undefined;
  private _lastBgY: number | undefined;

  private _lastBaseW: number | undefined;
  private _lastBaseH: number | undefined;

  private _outputLogs: boolean = false;

  private _marginValue: number = 0;

  private _uiSampler: UILayerSampler | null = null;
  private _windowCloneManager: WindowCloneManager | null = null;

  private _logger: Logger;
```

```
  // コンストラクタに settings を追加
  constructor(extensionPath: string, targetActor: St.Widget, settings: Gio.Settings, logger: Logger) {
    this.extensionPath = extensionPath;
    this.targetActor = targetActor;
    this._settings = settings; // GSettings object
    this._logger = logger; // Logger object

    this._glassExpand = 0; // ガラスエリアの拡張量（ピクセル）

    this._signals = [];
    this._settingsSignals = []; // GSettingsのイベントリスナーを管理
    this._frameSyncId = 0;
    this._isEffectActive = false; // エフェクトが現在適用されているかのフラグ
  }

  // 拡張機能が有効化された時に呼ばれるエントリーポイント
  setup() {
    if (!this.targetActor || !this._settings) return;

    // 設定の監視を開始
    this._bindSettings();

    // 初回起動時にスイッチがONならエフェクトを適用
    if (this._settings.get_boolean('enable-dock-glass')) {
      this._applyEffect();
    }
  }

  // 設定が変更された時にリアルタイムで反映するためのバインディング
  _bindSettings() {
    const connectSetting = (key, callback) => {
      let id = this._settings.connect(`changed::${key}`, callback.bind(this));
      this._settingsSignals.push(id);
    };

    // ON/OFFスイッチの切り替え
    connectSetting('enable-dock-glass', () => {
      let enabled = this._settings.get_boolean('enable-dock-glass');
      if (enabled && !this._isEffectActive) {
        this._applyEffect();
      } else if (!enabled && this._isEffectActive) {
        this._removeEffect();
      }
    });

    connectSetting('dock-glass-expand', () => {
      if (this.effect && this._isEffectActive) {
        this._glassExpand = this._settings.get_int('dock-glass-expand');
      }
    });

    // マージン変更時
    connectSetting('dock-margin-bottom', () => {
      if (this._isEffectActive) this._applyMargin();
      this._marginValue = this._settings.get_int('dock-margin-bottom') || 0;
    });

    // シェーダーパラメータの動的変更
    connectSetting('dock-tint-color', () => {
      if (this.effect && this._isEffectActive) {
        let colorArray = hexToColorArray(this._settings.get_string('dock-tint-color'));
        this.effect.setTintColor(...colorArray);
      }
    });

    connectSetting('dock-tint-strength', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setTintStrength(this._settings.get_double('dock-tint-strength'));
      }
    });

    connectSetting('dock-blur-radius', () => {
      const radius = this._settings.get_int('dock-blur-radius');
      if (this.effect && this._isEffectActive) this.effect.setBlurRadius(radius);
    });

    connectSetting('dock-corner-radius', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setCornerRadius(this._settings.get_double('dock-corner-radius'));
      }
    });
```

```
    connectSetting('output-logs', () => {
      this._outputLogs = this._settings.get_boolean('output-logs');
    });

    connectSetting('dock-brightness', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setBrightness(this._settings.get_double('dock-brightness'));
      }
    });

    connectSetting('dock-contrast', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setContrast(this._settings.get_double('dock-contrast'));
      }
    });

    connectSetting('dock-saturation', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setSaturation(this._settings.get_double('dock-saturation'));
      }
    });
  }

  // マージンの再計算と適用（動的反映のために独立した関数化）
  _applyMargin() {
    if (!this.targetActor) return;

    let marginBottom = this._settings.get_int('dock-margin-bottom');

    let [w, h] = this.targetActor.get_size();
    let [x, y] = this.targetActor.get_transformed_position();

    let monitorIndex = Main.layoutManager.findIndexForActor(this.targetActor);
    if (monitorIndex < 0) monitorIndex = Main.layoutManager.primaryIndex;
    let monitor = Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;

    // 画面の各エッジとの距離から配置場所を特定する
    let distLeft = x - monitor.x;
    let distRight = (monitor.x + monitor.width) - (x + w);
    let distTop = y - monitor.y;
    let distBottom = (monitor.y + monitor.height) - (y + h);
    let minEdge = Math.min(distLeft, distRight, distTop, distBottom);

    let marginStyle = '';
    if (minEdge === distBottom || minEdge === distTop) {
      if (minEdge === distBottom) {
        marginStyle = `margin-bottom: ${marginBottom}px;`; // 下
      } else {
        marginStyle = `margin-top: ${marginBottom}px;`;    // 上
      }
    } else {
      if (minEdge === distRight) {
        marginStyle = `margin-right: ${marginBottom}px;`;  // 右
      } else {
        marginStyle = `margin-left: ${marginBottom}px;`;   // 左
      }
    }

    if (this._originalStyle === undefined) {
      this._originalStyle = this.targetActor.get_style() || '';
    }
    this._currentMarginStyle = marginStyle;
    this.targetActor.set_style(`${this._originalStyle} ${marginStyle}`);
  }

  _applyEffect() {
    if (this._isEffectActive) return;
    this._isEffectActive = true;

    this.targetActor.add_style_class_name('liquid-glass-transparent');

    this._dockParent = this.targetActor.get_parent() as St.Widget | null;
    if (this._dockParent) {
      this._dockParent.add_style_class_name('liquid-glass-transparent');
    }

    this.bgActor = new UnpickableActor();
    this.bgActor.set_name('liquid-glass-bg-actor');

    this.bgActor.set_size(1.0, 1.0);

    // liquidBox: full-monitor-sized actor that holds the LiquidEffect.
```

```
    // LiquidEffect internally runs dual-Kawase blur passes before the glass composite,
    // so no separate blurBox/Shell.BlurEffect is needed.
    this.liquidBox = new UnpickableActor();
    this.liquidBox.set_name("liquid-box");
    this.liquidBox.set_clip_to_allocation(true);
    this.bgActor.add_child(this.liquidBox);

    // dummyBreaker: dummy actor to break the BMS bug
    let dummyBreaker = new UnpickableActor();
    dummyBreaker.set_name("optimization-breaker");
    dummyBreaker.set_size(1.0, 1.0);
    dummyBreaker.set_opacity(0); // 完全に透明
    this.liquidBox.add_child(dummyBreaker);

    // _cloneContainer lives inside liquidBox.
    // Clones are captured into the LiquidEffect's OffscreenEffect FBO and
    // blurred + distorted by the shader in a single pass.
    this._cloneContainer = new UnpickableActor();
    this._cloneContainer.set_name("clone-container");
    this.liquidBox.add_child(this._cloneContainer);

    // 動的マージンを適用
    this._applyMargin();
    this._marginValue = this._settings.get_int('dock-margin-bottom');
    this._glassExpand = this._settings.get_int("dock-glass-expand");
    this._outputLogs = this._settings.get_boolean('output-logs');

    let dockRoot = this.targetActor;
    while (dockRoot && dockRoot.get_parent() !== Main.layoutManager.uiGroup) {
      let p = dockRoot.get_parent() as St.Widget | null;
      if (!p) break;
      dockRoot = p;
    }

    if (dockRoot && dockRoot.get_parent() === Main.layoutManager.uiGroup) {
      Main.layoutManager.uiGroup.insert_child_below(this.bgActor, dockRoot);
    } else {
      Main.layoutManager.uiGroup.add_child(this.bgActor);
    }

    // 設定から初期値を読み込み
    let blurRadius = this._settings.get_int('dock-blur-radius');
    let tintColorStr = this._settings.get_string('dock-tint-color');
    let tintStrength = this._settings.get_double('dock-tint-strength');
    let cornerRadius = this._settings.get_double('dock-corner-radius');
    let brightness = this._settings.get_double('dock-brightness');
    let contrast = this._settings.get_double('dock-contrast');
    let saturation = this._settings.get_double('dock-saturation');

    this.effect = new LiquidEffect({ extensionPath: this.extensionPath, settings: this._settings, logger:
this._logger } as any);
    this.effect.setPadding(SHADER_PADDING);
    this.effect.setTintColor(...hexToColorArray(tintColorStr));
    this.effect.setTintStrength(tintStrength);
    this.effect.setCornerRadius(cornerRadius);
    this.effect.setBrightness(brightness);
    this.effect.setContrast(contrast);
    this.effect.setSaturation(saturation);
    this.effect.setBlurRadius(blurRadius);

    this.effect.setIsDock(true);
    this.liquidBox.add_effect(this.effect);

    // WindowCloneManager + UILayerSampler deposit their clones inside liquidBox.
    this._windowCloneManager = new WindowCloneManager(this.liquidBox, this._cloneContainer);
    this._uiSampler = new UILayerSampler(this.bgActor, this.liquidBox, [dockRoot, global.windowGroup,
global.window_group], this._cloneContainer);

    this.bgActor.show();

    const laterAdd = (laterType: Meta.LaterType, callback: GLib.SourceFunc) => {
      return global.compositor.get_laters().add(laterType, callback);
    };

    const frameLaterType = Meta.LaterType.BEFORE_REDRAW;

    // Rebuild clones (called on menu open): delegate entirely to WindowCloneManager + UILayerSampler
    let buildClones = () => {
      if (!this.bgActor) return;

      // _uiSampler が存在する場合のみ除外リストへの追加処理を行う
      if (this._uiSampler) {
```

```
        for (let child of Main.layoutManager.uiGroup.get_children()) {
          if (child === this.bgActor) continue;

          // 名前が 'liquid-glass-bg-actor' のもの、または 'liquid-box' を子に持つものを
          // 他のLiquid Glassエフェクトの背景アクターと判定する
          let isLiquidBg = child.name === 'liquid-glass-bg-actor' ||
            (typeof child.get_children === 'function' &&
              child.get_children().some(c => c.name === 'liquid-box'));

          if (isLiquidBg) {
            this._uiSampler.addExclusion(child);
          }
        }
      }

      this._windowCloneManager?.rebuildClones();
      this._uiSampler?.rebindSelf();
      this._uiSampler?.refresh();
    };

    let frameTick = () => {
      this._frameSyncId = 0;
      if (!this.bgActor || !this.targetActor.mapped) return GLib.SOURCE_REMOVE;

      this._syncGeometry();
      this._frameSyncId = laterAdd(frameLaterType, frameTick);
      return GLib.SOURCE_REMOVE;
    };

    let startFrameSync = () => {
      if (this._frameSyncId === 0) {
        buildClones();
        this._frameSyncId = laterAdd(frameLaterType, frameTick);
      }
    };

    let mapSignalId = this.targetActor.connect('notify::mapped', () => {
      if (this.targetActor.mapped) {
        startFrameSync();
      } else {
        if (this._frameSyncId !== 0) {
          this._frameSyncId = 0;
        }
      }
    });
    this._signals.push(mapSignalId);

    if (this.targetActor.mapped) {
      startFrameSync();
    }
  }

  _syncGeometry() {
    if (!this.bgActor || !this.targetActor || !this.targetActor.mapped) return;

    let sourceActor = this.targetActor;
    let children = this.targetActor.get_children() as St.Widget[];
    for (let i = 0; i < children.length; i++) {
      if (children[i].has_style_class_name('dash-background')) {
        children[i].opacity = 0;
        sourceActor = children[i];
      }
    }


    // 1. まず元の背景のサイズと位置を取得
    let [baseW, baseH] = sourceActor.get_size();
    let [absX, absY] = sourceActor.get_transformed_position();
    if (Number.isNaN(absX) || Number.isNaN(absY)) return;

    if (sourceActor !== this.targetActor) {
      let [tX, tY] = this.targetActor.get_transformed_position();
      let [tW, tH] = this.targetActor.get_size();

      // 親コンテナからはみ出した分をカットし、本来のサイズに強制する
      if (absX < tX) { baseW -= (tX - absX); absX = tX; }
      if (absY < tY) { baseH -= (tY - absY); absY = tY; }
      if (absX + baseW > tX + tW) { baseW = (tX + tW) - absX; }
      if (absY + baseH > tY + tH) { baseH = (tY + tH) - absY; }
    }
    // this._logger.log(`[Raw] ${absX}, ${absY}, ${baseW}, ${baseH}`);
```

```
    let monitorIndex = Main.layoutManager.findIndexForActor(this.targetActor);
    if (monitorIndex < 0) {
      monitorIndex = Main.layoutManager.primaryIndex;
    }
    let monitor = Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;
    let minCenterDist = -1;
    let distLeftCenter: number = 0;
    let distRightCenter: number = 0;
    let distTopCenter: number = 0;
    let distBottomCenter: number = 0;

    if (monitor) {
      let dockCenterX = absX + (baseW / 2);
      let dockCenterY = absY + (baseH / 2);

      distLeftCenter = dockCenterX - monitor.x;
      distRightCenter = (monitor.x + monitor.width) - dockCenterX;
      distTopCenter = dockCenterY - monitor.y;
      distBottomCenter = (monitor.y + monitor.height) - dockCenterY;

      minCenterDist = Math.min(distLeftCenter, distRightCenter, distTopCenter, distBottomCenter);
    }
    if (this._lastBaseW !== undefined && this._lastBaseH !== undefined) {
      let isHorizontalDock = (minCenterDist === distTopCenter || minCenterDist === distBottomCenter);

      if (isHorizontalDock) {
        // ▼ 上・下ドックの場合：異常に膨張するのは H（厚み）
        // Hの変化量が「ちょうど marginValue 分」だった場合のみ、そのジャンプを無効化（<= 1 に修正）
        if (Math.abs(Math.abs(baseH - this._lastBaseH) - this._marginValue) <= 1) {
          baseH = this._lastBaseH;
        }
      } else {
        // ▼ 左・右ドックの場合：異常に膨張するのは W（厚み）
        // Wの変化量が「ちょうど marginValue 分」だった場合のみ無効化
        if (Math.abs(Math.abs(baseW - this._lastBaseW) - this._marginValue) <= 1) {
          baseW = this._lastBaseW;
        }
      }
    }
    this._lastBaseW = baseW;
    this._lastBaseH = baseH;
    let refActor = this._findReferenceActor(this.targetActor);
    if (refActor) {
      let [refW, refH] = refActor.get_size();
      let [refX, refY] = refActor.get_transformed_position();
      // this._logger.log(`refActor [Raw]: ${refX}, ${refY}, ${refW}, ${refH}`);

      if (!Number.isNaN(refX) && !Number.isNaN(refY) && refW > 0 && refH > 0) {

        let topGap = refY - absY;
        let bottomGap = (absY + baseH) - (refY + refH);

        // For when the dock is upside down
        if (topGap < 0 || bottomGap < 0) {
          // 原点が下端にあるため、真の左上Y座標は refY - refH になる
          let trueRefY = refY - refH;
          // ギャップを再計算して正常化
          topGap = trueRefY - absY;
          bottomGap = (absY + baseH) - (trueRefY + refH);
        }

        let leftGap = refX - absX;
        let rightGap = (absX + baseW) - (refX + refW);

        // ▼ X軸が反転（左右ミラー）しているかの検知と補正（左/右ドック用）
        if (leftGap < 0 || rightGap < 0) {
          let trueRefX = refX - refW;
          leftGap = trueRefX - absX;
          rightGap = (absX + baseW) - (trueRefX + refW);
        }

        if (baseW >= baseH) {
          // ▼ 横長ドック（上・下ドック）▼
          let diff = Math.abs(bottomGap - topGap);

          // 異常値(高さを超えるようなズレ)は無視する安全装置
          if (diff > 0 && diff < baseH / 2) {
            if (bottomGap > topGap) {
              // 下の隙間の方が広い -> 下を削る
              baseH -= diff;
            } else {
```

```
            // 上の隙間の方が広い -> 開始位置(上)を下げて、高さも削る
            absY += diff;
            baseH -= diff;
          }
        }
      } else {
        // ▼ 縦長ドック（左・右ドック）▼
        let diff = Math.abs(rightGap - leftGap);

        if (diff > 0 && diff < baseW / 2) {

          if (minCenterDist === distLeftCenter) {
            // 左ドック: 中央方向（右側）の余白のみ削る
            // leftGap > rightGap になっても absX を右にズラしてはいけない
            if (rightGap > leftGap) {
              baseW -= diff;
            }
            // leftGap > rightGap の場合は何もしない（誤補正防止）
          } else {
            // 右ドック: 中央方向（左側）の余白を削る
            if (rightGap > leftGap) {
              baseW -= diff;
            } else {
              absX += diff;
              baseW -= diff;
            }
          }
        }
      }
    }
  }
  // this._logger.log(`[Gap] ${absX}, ${absY}, ${baseW}, ${baseH}`);
  // ------------------------------------------------------------------------
  // ------------------------------------------------------------------------
  let marginValue = this._settings.get_int('dock-margin-bottom') || 0;

  if (monitor && marginValue > 0) {

    // アプリ起動時の微小揺れ（誤動作の元）を完全に無視するため、閾値を大きく設定
    let isMoving = false;
    if (this._lastAbsX !== undefined && this._lastAbsY !== undefined) {
      let diffX = Math.abs(absX - this._lastAbsX);
      let diffY = Math.abs(absY - this._lastAbsY);
      if (diffX > 1.0 || diffY > 1.0) {
        isMoving = true;
      }
    }

    // Fix hiding animation bug
    // isMoving = false;
    this._lastAbsX = absX;
    this._lastAbsY = absY;

    let [tW, tH] = this.targetActor.get_size();
    if (this._stableDeltaW === undefined || this._lastTW !== tW) {
      this._stableDeltaW = baseW - tW;
      this._lastTW = tW;
    }
    if (this._stableDeltaH === undefined || this._lastTH !== tH) {
      this._stableDeltaH = baseH - tH;
      this._lastTH = tH;
    }

    let stableBaseW = tW + this._stableDeltaW;
    let stableBaseH = tH + this._stableDeltaH;

    if (!isMoving) {
      if (minCenterDist === distBottomCenter) {
        // 下ドック
        let expectedBottom = monitor.y + monitor.height - marginValue;
        if (absY + baseH > expectedBottom) {
          let overflow = (absY + baseH) - expectedBottom;
          baseH -= overflow;
        }
        if (baseH > stableBaseH) baseH = stableBaseH; // Experimental
      } else if (minCenterDist === distTopCenter) {
        // 上ドック
        let expectedTop = monitor.y + marginValue;
        if (absY < expectedTop) {
          let diff = expectedTop - absY;
          absY = expectedTop;
          baseH -= diff;
```

```
          }
          if (baseH > stableBaseH) baseH = stableBaseH;
        } else if (minCenterDist === distRightCenter) {
          // 右ドック
          let expectedRight = monitor.x + monitor.width - marginValue;
          if (absX + baseW > expectedRight) {
            let overflow = (absX + baseW) - expectedRight;
            baseW -= overflow;
          }
          if (baseW > stableBaseW) baseW = stableBaseW; // Experimental
        } else {
          // 左ドック
          let expectedLeft = monitor.x + marginValue;
          if (absX < expectedLeft) {
            let diff = expectedLeft - absX;
            absX = expectedLeft;
            baseW -= diff;
          }
          if (baseW > stableBaseW) baseW = stableBaseW;
        }
      }
    }
    // this._logger.log(`[Final] ${absX}, ${absY}, ${baseW}, ${baseH}`);
    // ------------------------------------------------------------------

    // 補正されたサイズを適用
    let w = Math.max(1.0, baseW);
    let h = Math.max(1.0, baseH);

    if (baseW <= 9 || baseH <= 9) {
      this.bgActor.hide();
      // stateをリセットして、次の表示時に必ずガードを通過させる
      // Reset state to guard against the next frame tick from applying the guard.
      this._lastBgW = undefined;
      this._lastBgH = undefined;
      this._lastBgX = undefined;
      this._lastBgY = undefined;
      return;
    } else {
      this.bgActor.show();
    }

    this.bgActor.opacity = this.targetActor.opacity;


    /*
    let monitorIndex = Main.layoutManager.findIndexForActor(this.targetActor);
    if (monitorIndex < 0) {
        monitorIndex = Main.layoutManager.primaryIndex;
    }
    let monitor = Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;
    */
    let visibleW = baseW;
    let visibleH = baseH;
    if (monitor) {
      if (absX < monitor.x) visibleW -= (monitor.x - absX);
      if (absY < monitor.y) visibleH -= (monitor.y - absY);
      if (absX + baseW > monitor.x + monitor.width) visibleW -= ((absX + baseW) - (monitor.x + monitor.width));
      if (absY + baseH > monitor.y + monitor.height) visibleH -= ((absY + baseH) - (monitor.y + monitor.height));
    }

    if (visibleW <= 5 || visibleH <= 5) {
      this.bgActor.opacity = 0;
    } else {
      this.bgActor.opacity = this.targetActor.opacity;
    }

    let bgW = Math.max(1.0, w + (SHADER_PADDING * 2) + (this._glassExpand * 2));
    let bgH = Math.max(1.0, h + (SHADER_PADDING * 2) + (this._glassExpand * 2));
    let bgX = absX - SHADER_PADDING - this._glassExpand;
    let bgY = absY - SHADER_PADDING - this._glassExpand;

    // Full-screen FBO geometry.
    // bgActor and liquidBox are both sized to cover the entire monitor.
    // This makes the FBO coordinate system match what BMS expects (full-screen
    // absolute coordinates), eliminating the BMS blur offset and cache-pollution
    // bugs caused by the old dock-sized FBO.
    let screenW = monitor.width;
    let screenH = monitor.height;

    // Dock background position in monitor-local coordinates (monitor origin = 0,0).
    // This is what the shader receives via setGlassGeometry() to reconstruct the
```

```
    // dock-centred local coordinate system inside the full-screen FBO.
    let localBgX = bgX - monitor.x;
    let localBgY = bgY - monitor.y;

    // Detect any change in dock geometry OR monitor size to trigger a rebuild.
    if (this._lastBgW !== bgW || this._lastBgH !== bgH ||
      this._lastBgX !== bgX || this._lastBgY !== bgY ||
      this._lastScreenW !== screenW || this._lastScreenH !== screenH) {

      this.bgActor.remove_transition('size');
      this.bgActor.remove_transition('position');
      this.bgActor.set_position(monitor.x, monitor.y);
      this.bgActor.set_size(screenW, screenH);
      this.bgActor.remove_transition('size');
      this.bgActor.remove_transition('position');

      this.liquidBox?.set_position(0, 0);
      this.liquidBox?.set_size(screenW, screenH);

      this._lastBgW = bgW; this._lastBgH = bgH;
      this._lastBgX = bgX; this._lastBgY = bgY;
      this._lastScreenW = screenW; this._lastScreenH = screenH;
    }


    // Soft clipping via set_clip — limits GPU fragment-shader execution
    // to the dock region + a generous margin for drop-shadow decay, without
    // using clip_to_allocation (which hard-clips child actors and severs shadows).
    //
    // CLIP_PADDING must be at least as large as the maximum shadow_radius
    // setting so the penumbra gradient has room to fade to zero naturally.
    const CLIP_PADDING = 200;
    // Clip only bgActor; liquidBox has no separate clip
    // this.liquidBox?.remove_clip();

    this.bgActor.set_clip(
      localBgX - CLIP_PADDING, localBgY - CLIP_PADDING,
      bgW + CLIP_PADDING * 2, bgH + CLIP_PADDING * 2
    );
    const SHADOW_MAX_RADIUS = CLIP_PADDING - 20;
    this.effect?.setShadowMaxRadius(SHADOW_MAX_RADIUS);

    /*
    if (this._outputLogs) {
      const shadowRadiusSetting = this._settings.get_double('shadow-radius');
      const shadowIntensitySetting = this._settings.get_double('shadow-intensity');
      this._logger.log(`[Shadow Debug] dock(local)=(${localBgX.toFixed(1)}, ${localBgY.toFixed(1)}, ${bgW.toFixed(1)}
x${bgH.toFixed(1)}) ` +
        `clip=(${(localBgX - CLIP_PADDING).toFixed(1)}, ${(localBgY - CLIP_PADDING).toFixed(1)}, ` +
        `${(bgW + CLIP_PADDING * 2).toFixed(1)}x${(bgH + CLIP_PADDING * 2).toFixed(1)}) ` +
        `shadow_max_radius=${SHADOW_MAX_RADIUS} shadow-radius=${shadowRadiusSetting} shadow-intensity=$
{shadowIntensitySetting} ` +
        `screen=${screenW}x${screenH}`);
    }
    */

    // setResolution receives the full monitor dimensions (was bgW/bgH).
    // The shader uses resolution to compute UV-to-pixel mapping over the full FBO.
    this.effect?.setResolution(screenW, screenH);

    // Inform the shader where the dock lives within the full-screen FBO.
    // The shader uses these to compute dock_center and box_size, replacing
    // the old "resolution * 0.5" center assumption that only worked when
    // the FBO was dock-sized.
    this.effect?.setGlassGeometry(localBgX, localBgY, bgW, bgH);

    // Clones in WindowCloneManager are placed at (w.x, w.y) — absolute screen
    // coordinates. The container shift of (-monitor.x, -monitor.y) makes each
    // clone appear at (w.x - monitor.x, w.y - monitor.y) inside the full-screen
    // FBO, which maps back to (w.x, w.y) in screen space once bgActor's
    // monitor-origin position is added by Clutter's scene graph. ✓
    this._windowCloneManager?.setOffset(-monitor.x, -monitor.y);

    // UILayerSampler is synced with the monitor origin and full-screen
    // dimensions instead of the dock-relative bgX/bgY/bgW/bgH.
    this._uiSampler?.refresh();
    this._uiSampler?.sync(monitor.x, monitor.y, screenW, screenH);

    this._windowCloneManager?.sync();
  }

  _syncActorProperties(source, clone) {
```

```
    if (!source || !clone) return;

    // .x, .y, .width を直接読まず、計算済みの「画面上の絶対座標」と「サイズ」を関数で取得
    let [absX, absY] = source.get_transformed_position();
    let [w, h] = source.get_size();

    // NaN（非数）や異常なマイナス値が紛れ込んだ場合は同期をキャンセルして描画を止める
    if (Number.isNaN(absX) || Number.isNaN(absY) || Number.isNaN(w) || Number.isNaN(h) || w <= 0 || h <= 0) {
      clone.visible = false;
      return;
    }

    // 正しい絶対座標とサイズをクローンに適用
    clone.remove_transition('position');
    clone.remove_transition('size');
    clone.set_position(absX, absY);
    clone.set_size(w, h);

    // スケールとピボット
    clone.remove_transition('scale-x');
    clone.remove_transition('scale-y');
    clone.set_scale(source.scale_x, source.scale_y);
    let pX = source.pivot_point ? source.pivot_point.x : 0;
    let pY = source.pivot_point ? source.pivot_point.y : 0;
    clone.set_pivot_point(pX, pY);

    clone.translation_x = 0;
    clone.translation_y = 0;

    // 透明度と表示状態
    clone.opacity = source.opacity;
    clone.visible = source.visible && source.mapped;
  }

  // エフェクトを画面から消し、元に戻す処理
  _removeEffect() {
    if (!this._isEffectActive) return;
    this._isEffectActive = false;

    this._currentMarginStyle = undefined;

    // Safely try to remove styles/signals. If targetActor is already destroyed,
    // this will fail safely without breaking the rest of the cleanup.
    try {
      for (let sigId of this._signals) {
        this.targetActor.disconnect(sigId);
      }
      this.targetActor.remove_style_class_name('liquid-glass-transparent');

      if (this._originalStyle !== undefined) {
        this.targetActor.set_style(this._originalStyle);
        this._originalStyle = undefined;
      }
      let children = this.targetActor.get_children() as St.Widget[];
      for (let i = 0; i < children.length; i++) {
        if (children[i].has_style_class_name('dash-background')) {
          children[i].opacity = 255;
        }
      }
    } catch (e) {
      // Actor was likely destroyed, safe to ignore
    }

    this._signals = [];

    this.targetActor.remove_style_class_name('liquid-glass-transparent');

    try {
      if (this._dockParent) {
        this._dockParent.remove_style_class_name('liquid-glass-transparent');
      }
    } catch (e) { }
    this._dockParent = null;

    if (this._originalStyle !== undefined) {
      this.targetActor.set_style(this._originalStyle);
      this._originalStyle = undefined; // 次回オンになった時に再取得できるようクリア
    }

    let children = this.targetActor.get_children() as St.Widget[];
    for (let i = 0; i < children.length; i++) {
      if (children[i].has_style_class_name('dash-background')) {
```

```
          children[i].opacity = 255;
        }
      }

      if (this._frameSyncId !== 0) {
        if (global.compositor?.get_laters) {
          global.compositor.get_laters().remove(this._frameSyncId);
        } else {
          // Meta.later_remove(this._frameSyncId);
        }
        this._frameSyncId = 0;
      }

      if (this.effect) {
        this.effect.cleanup();
        this.effect = null;
      }

      if (this.bgActor) {
        this.bgActor.destroy();
        this.bgActor = null;
      }
      // liquidBox is a child of bgActor and is already destroyed by bgActor.destroy().
      // Just clear the reference here.
      this.liquidBox = null;

      this._uiSampler?.destroy();
      this._windowCloneManager?.destroy();
    }

    // 拡張機能全体が無効化される時の最終クリーンアップ
    cleanup() {
      // エフェクトを解除
      this._removeEffect();

      // メモリリークを防ぐため、GSettingsのリスナーもすべて解除する
      if (this._settings) {
        for (let id of this._settingsSignals) {
          this._settings.disconnect(id);
        }
        this._settingsSignals = [];
      }
    }

    // ドックの内部から、計算の基準となるアイコンまたはインジケーターを1つ再帰的に探し出す
    private _findReferenceActor(actor: Clutter.Actor): Clutter.Actor | null {
      if (!actor) return null;
      // 1. 安全性のチェック：オブジェクトが存在しない、または get_children がない場合は終了
      if (!actor || typeof actor.get_children !== 'function') {
        return null;
      }

      // 2. 判定条件：文字列化して 'IndicatorDrawingArea' が含まれているか
      if (actor.toString().includes('IndicatorDrawingArea')) {
        return actor;
      }

      // 3. 子要素を再帰的に探索
      const children = actor.get_children();
      for (const child of children) {
        const found = this._findReferenceActor(child);
        if (found) {
          return found; // 見つかったら即座に返す（無駄な探索をしない）
        }
      }

      return null; // 見つからなかった場合
    }
  }
```

```
// src/liquidEffect.ts
//
// ── Design overview ──────────────────────────────────────────────────────
//
//  Old implementation: subclassed Clutter.ShaderEffect and did refraction,
//          rim lighting, and shadowing all in a single glass.frag shader.
//          Blur relied solely on ShaderEffect's cogl_sampler texture
//          sampling, with no dedicated blur pass.
//
//  New implementation: subclasses Clutter.OffscreenEffect and overrides
//          vfunc_paint_target to run a custom multi-pass FBO pipeline.
//
//  Rendering pipeline (per frame):
//
//     ┌──────────────────────────────────────────────────┐
//     │  OffscreenEffect automatically captures the actor's  │
//     │  painted content into an internal FBO                │
//     │  (retrievable via get_texture())                     │
//     └──────────────────────────────────────────────────┘
//                          │ srcTex (full monitor resolution)
//                          ▼
//     ┌──────────────── Downsample ──────────────────────┐
//     │  Pass 0: srcTex    → _blurFbos[0]  (w/2  × h/2)      │
//     │  Pass 1: _tex[0]   → _blurFbos[1]  (w/4  × h/4)      │
//     │  Pass 2: _tex[1]   → _blurFbos[2]  (w/8  × h/8)      │
//     │  Pass 3: _tex[2]   → _blurFbos[3]  (w/16 × h/16)     │
//     │  (shaders/downsample.frag – Dual Kawase, 5-tap)      │
//     └──────────────────────────────────────────────────┘
//                          │
//     ┌──────────────── Upsample ────────────────────────┐
//     │  Pass 3→2: _tex[3] → _blurFbos[2]                    │
//     │  Pass 2→1: _tex[2] → _blurFbos[1]                    │
//     │  Pass 1→0: _tex[1] → _blurFbos[0]  (w/2 × h/2)       │
//     │  (shaders/upsample.frag – Dual Kawase tent, 8-tap)   │
//     └──────────────────────────────────────────────────┘
//                          │ _blurTextures[0] (blurred, w/2 × h/2)
//                          ▼
//     ┌──────────────── Glass composite ─────────────────┐
//     │  shaders/glass.frag is parsed at runtime into a Cogl │
//     │  snippet. cogl_sampler0 = the blurred texture.       │
//     │  Applies refraction / chromatic aberration / rim     │
//     │  lighting / shadow, then draws into screenFb (the    │
//     │  on-screen framebuffer Clutter has prepared).        │
//     └──────────────────────────────────────────────────┘
//
//  The texture pool is rebuilt whenever the resolution changes.
//  Cogl pipelines are compiled once on the first frame and reused after that.
//
// ─────────────────────────────────────────────────────────────────────────

import GObject from 'gi://GObject';
import Clutter from 'gi://Clutter';
import Cogl from 'gi://Cogl';
import Gio from 'gi://Gio';
import GLib from 'gi://GLib';

import { Logger } from './logger.js';

// ── Type helpers ─────────────────────────────────────────────────────────
// In GJS, Cogl.Offscreen inherits from Cogl.Framebuffer, but TypeScript's
// type definitions sometimes require an explicit cast to see that.
type CoglFB = Cogl.Framebuffer;

interface LiquidEffectParams {
  extensionPath?: string;
  settings?: Gio.Settings;
  logger?: Logger;
  [key: string]: any; // spread into super._init(params)
}

// ── Parsed shader source ─────────────────────────────────────────────────
interface ShaderSnippet {
  /** Everything before void main() (uniform declarations / helper functions) */
  decl: string;
  /** The body of void main(), with the surrounding braces stripped */
  body: string;
}

// ── Blur method ──────────────────────────────────────────────────────────
// 0: Separable Gaussian blur (shader source generated dynamically on the TS side)
// 1: Dual Kawase blur (downsample.frag + upsample.frag) — the original implementation
type BlurMethod = 0 | 1;
```

```
// ─── Dynamic Gaussian kernel ─────────────────────────────────────────
// A 1D Gaussian kernel optimized for linear-sampling ("bilinear tap merging").
//   offsets[0] / weights[0] is the center sample (offset is always 0).
//   offsets[i] / weights[i] for i >= 1 is the combined offset/weight for a
//   symmetric left-right (or up-down) pair of taps merged into one fetch.
// Number of texture fetches = 1 (center) + 2 * (offsets.length - 1) (side pairs).
interface GaussianKernel {
  offsets: number[];
  weights: number[];
}

// ─── Main class ──────────────────────────────────────────────────────

export const LiquidEffect = GObject.registerClass({
  GTypeName: 'LiquidGlassEffect',
}, class LiquidEffect extends Clutter.OffscreenEffect {

  // ─── Private fields ────────────────────────────────────────────────

  declare private _extensionPath: string | undefined;
  declare private _settings: Gio.Settings | undefined;
  declare private _settingsIds: number[];
  declare private _logger: Logger | undefined;

  // ── Blur texture pool ──
  // Index 0 = w/2 × h/2 (first half-res level)
  // Index N = w/(2^(N+1)) × h/(2^(N+1))
  declare private _blurTextures: Cogl.Texture2D[];
  declare private _blurFbos: Cogl.Offscreen[];

  // ── Intermediate buffers for the Gaussian blur ──
  // Holds the output of the horizontal pass (same resolution as _blurTextures
  // at the corresponding index).
  declare private _gaussianTempTextures: Cogl.Texture2D[];
  declare private _gaussianTempFbos: Cogl.Offscreen[];

  // ── Compiled pipelines, reused across frames ──
  // Dual Kawase
  declare private _downsamplePipeline: Cogl.Pipeline | null;
  declare private _upsamplePipeline: Cogl.Pipeline | null;
  // Separable Gaussian
  declare private _gaussianHPipeline: Cogl.Pipeline | null; // horizontal pass
  declare private _gaussianVPipeline: Cogl.Pipeline | null; // vertical pass
  declare private _compositePipeline: Cogl.Pipeline | null;

  // ── Active blur method (0: Gaussian, 1: Dual Kawase) ──
  declare private _blurMethod: BlurMethod;

  // ── State for dynamic Gaussian shader generation ──
  // The kernel currently compiled into the H/V pipelines (its tap count
  // determines the shader's structure).
  declare private _gaussianKernel: GaussianKernel | null;
  // A kernel waiting to be compiled; picked up safely inside vfunc_paint_target.
  declare private _pendingGaussianKernel: GaussianKernel | null;
  // While true, the Gaussian H/V pipelines will be recompiled on the next paint.
  declare private _gaussianPipelineDirty: boolean;
  // The sigma (in half-res texels) that the currently compiled kernel targets.
  // Small changes in radius that don't change the tap count are absorbed via
  // the kernel_scale ratio below instead of triggering a recompile.
  declare private _gaussianBaseSigma: number;
  // kernel_scale uniform sent to the shader (= current sigma / _gaussianBaseSigma).
  declare private _gaussianScale: number;
  // Number of fetch pairs in the currently compiled (or pending) kernel.
  declare private _gaussianFetchPairs: number;

  // ── Uniform location cache for the composite pipeline ──
  declare private _compUniforms: Map<string, number>;

  // ── Uniforms set before the pipeline existed, applied once it's created ──
  declare private _pendingUniforms: Map<string, number>;

  // ── Crop texture pool ──
  // Some Cogl/Clutter versions return a get_texture() whose size is larger
  // than the actor's logical size (internal FBO padding). When that happens,
  // this texture holds a cropped copy containing only the valid region, and
  // all later passes (blur, composite) use it as their input instead.
  declare private _cropTexture: Cogl.Texture2D | null;
  declare private _cropFbo: Cogl.Offscreen | null;
  declare private _cropPoolW: number;
  declare private _cropPoolH: number;
```

```
    declare private _poolWidth: number;
    declare private _poolHeight: number;

    // Number of blur passes. Each direction runs PASS_COUNT passes.
    // With 4: 1/2 → 1/4 → 1/8 → 1/16 → (turnaround) → 1/8 → 1/4 → 1/2
    declare private PASS_COUNT: number;

    // Blur radius, forwarded to the down/upsample shaders' blur_radius uniform.
    // Any real number >= 0.5; larger values produce a stronger blur.
    // Default for downsample is 0.5 (the original Kawase value), default for
    // upsample is 1.0 (the original tent-filter value).
    declare private _blurRadiusDown: number;
    declare private _blurRadiusUp: number;

    // The last radius requested by the caller (before method-specific mapping).
    declare private _targetRadius: number;

    // ── Shader Sources ──
    declare private _downsampleSource: string | null;
    declare private _upsampleSource: string | null;
    declare private _glassSource: string | null;
    declare private _shadersLoaded: boolean;

    // ── [DIAG] Black-background investigation ──
    // Tracks whether/how often vfunc_paint_target actually gets invoked by
    // Clutter for this instance, and whether it ever renders a frame that
    // doesn't fall back to super.vfunc_paint_target() (i.e. an actual glass
    // composite). If this instance's window is showing the black-background
    // bug and _diagPaintCount never advances (or never reaches "composited"),
    // that's direct evidence Clutter is skipping/culling this actor's paint
    // entirely rather than the content being wrong.
    declare private _diagPaintCount: number;
    declare private _diagCompositedPaintCount: number;
    declare private _diagLastPaintLogAt: number;
    declare private _diagFirstPaintLogged: boolean;

    // ─── _init ──────────────────────────────────────────────────────────

    _init(params: LiquidEffectParams) {
      const extensionPath = params.extensionPath;
      const settings = params.settings;
      const logger = params.logger;
      delete params.extensionPath;
      delete params.settings;
      delete params.logger;

      super._init(params);

      this._blurTextures = [];
      this._blurFbos = [];
      this._gaussianTempTextures = [];
      this._gaussianTempFbos = [];
      this._downsamplePipeline = null;
      this._upsamplePipeline = null;
      this._gaussianHPipeline = null;
      this._gaussianVPipeline = null;
      this._compositePipeline = null;
      this._compUniforms = new Map();
      this._pendingUniforms = new Map();
      this._poolWidth = 0;
      this._poolHeight = 0;
      this._cropTexture = null;
      this._cropFbo = null;
      this._cropPoolW = 0;
      this._cropPoolH = 0;

      this.PASS_COUNT = 4;
      this._blurRadiusDown = 0.5;
      this._blurRadiusUp = 1.0;
      this._blurMethod = 1; // default: Dual Kawase
      this._targetRadius = 15.0;

      this._gaussianKernel = null;
      this._pendingGaussianKernel = null;
      this._gaussianPipelineDirty = false;
      this._gaussianBaseSigma = 0;
      this._gaussianScale = 1.0;
      this._gaussianFetchPairs = 0;


      this._downsampleSource = null;
      this._upsampleSource = null;
```

```
    this._glassSource = null;
    this._shadersLoaded = false;
    this._diagPaintCount = 0;
    this._diagCompositedPaintCount = 0;
    this._diagLastPaintLogAt = 0;
    this._diagFirstPaintLogged = false;

    this._extensionPath = extensionPath;
    this._settings = settings;
    this._logger = logger;

    // ── Default values for the composite shader's uniforms ──
    // The pipeline doesn't exist yet at this point, so these are buffered
    // into _pendingUniforms and applied once the pipeline is created.

    this._setFloat('resolution_x', 0.0);
    this._setFloat('resolution_y', 0.0);
    this._setFloat('pointer_x', -100.0);
    this._setFloat('pointer_y', -100.0);
    this._setFloat('intensity', 0.0);
    this._setFloat('corner_radius', 60.0);
    this._setFloat('brightness', 1.0);
    this._setFloat('contrast', 1.0);
    this._setFloat('saturation', 1.0);
    this._setFloat('padding', 20.0);
    // Distinct from the small optical 'padding' uniform (20px, only
    // meant to give the refraction/blur shader room past the actor's strict
    // bounds). shadow_max_radius instead reflects how much room the drop
    // shadow actually has to render outward before it would run into the
    // bgActor's own clip in dockManager.ts (CLIP_PADDING). Previously the
    // shader reused 'padding' for this, capping shadow_radius at ~18px no
    // matter how high the 0-100 prefs.js slider was set. Overwritten by
    // setShadowMaxRadius() once dockManager starts syncing geometry; this
    // default only matters before the first sync.
    this._setFloat('shadow_max_radius', 180.0);
    this._setFloat('isDock', 0.0);
    // Rim/specular/sheen "glass surface glint" terms, gated together by
    // setSurfaceLightEnabled(). Defaults to enabled (1.0) so dock/menu/
    // notification/quick-settings/osd — which never call the setter — keep
    // their existing look unchanged. applicationManager.ts turns this off
    // for application windows, which should only show the outer drop
    // shadow and the inner AO darkening (both already independent of this
    // uniform — see the addedLight gating in glass.frag), not the
    // dock-style rim/specular/sheen highlight.
    this._setFloat('surface_light_enabled', 1.0);

    // Full-screen FBO mode: lets the shader know where the dock sits.
    this._setFloat('dock_x', 0.0);
    this._setFloat('dock_y', 0.0);
    this._setFloat('dock_w', 0.0);
    this._setFloat('dock_h', 0.0);

    this._settingsIds = [];
    if (this._settings) {
      this._bindSettings();
    } else {
      // Fallback defaults used when no GSettings schema is available.
      this._setFloat('max_z', 25.0);
      this._setFloat('displacement_scale', 78.5);
      this._setFloat('edge_smoothing', 2.0);
      this._setFloat('profile_shape_n', 7.0);
      this._setFloat('ior', 2.40);
      this._setFloat('chroma_strength', 0.006);
      this._setFloat('specular_intensity', 0.0);
      this._setFloat('shininess', 42.0);
      this._setFloat('rim_width', 5.0);
      this._setFloat('rim_intensity', 0.6);
      this._setFloat('rim_directional_power', 2.7);
      this._setFloat('rim_power', 6.0);
      this._setFloat('rim_light_color_intensity', 1.4);
      this._setFloat('sheen_intensity', 0.32);
      this._setFloat('light_angle_deg', 0.0);
      this._setFloat('shadow_radius', 8.0);
      this._setFloat('shadow_intensity', 0.55);
      // Inner edge AO darkening (independent of rim_width/shadow_radius).
      // ~7.5px matches the old rim_width*1.5-derived falloff at the default
      // rim_width of 5.0, so the look is unchanged until the user retunes it.
      this._setFloat('ao_intensity', 0.25);
      this._setFloat('ao_radius', 7.5);
      this._setFloat('tint_strength', 0.0);
      this._setFloat('tint_r', 1.0);
      this._setFloat('tint_g', 1.0);
```

```
      this._setFloat('tint_b', 1.0);
    }
    this._loadAllShadersAsync();
  }

  /**
  * Load all shader files asynchronously.
  */
  private async _loadAllShadersAsync(): Promise<void> {
    // [DIAG] Black-background investigation: each LiquidEffect instance loads
    // its own copy of the 3 shader files independently (no cross-instance
    // cache), so a brand-new window's glass literally cannot render until
    // this completes. Log start/duration to see how long this actually takes
    // relative to the window's own open animation, and to correlate with the
    // applicationManager diag logs (search for "[Liquid Glass][diag]").
    const diagStart = GLib.get_monotonic_time();
    this._logger?.log(`[Liquid Glass][diag] LiquidEffect: starting async shader load at t=${diagStart}us ` +
      `(extensionPath=${this._extensionPath})`);
    try {
      this._downsampleSource = await this._readFileAsync(`${this._extensionPath}/shaders/downsample.frag`);
      this._upsampleSource = await this._readFileAsync(`${this._extensionPath}/shaders/upsample.frag`);
      this._glassSource = await this._readFileAsync(`${this._extensionPath}/shaders/glass.frag`);

      this._shadersLoaded = true;

      const elapsedMs = (GLib.get_monotonic_time() - diagStart) / 1000;
      this._logger?.log(`[Liquid Glass][diag] LiquidEffect: async shader load finished in ${elapsedMs.toFixed(1)}ms, `
+
        `calling queue_repaint() now. If the on-screen black-background bug is still visible ` +
        `after this point, the shader load itself is not the (sole) cause -- the issue is in ` +
        `getting this repaint request actually flushed to the display.`);

      // 読み込み完了後に再描画をリクエストし、パイプラインを初期化させる
      this.queue_repaint();
    } catch (e) {
      this._logger?.error(`[Liquid Glass] Failed to load shaders asynchronously: ${e}`);
    }
  }

  /**
  * Gio.File を使ってファイルを非同期で読み込み、文字列として返すPromise関数
  */
  private _readFileAsync(path: string): Promise<string> {
    return new Promise((resolve, reject) => {
      const file = Gio.File.new_for_path(path);
      file.load_contents_async(null, (_, res) => {
        try {
          const [ok, bytes] = file.load_contents_finish(res);
          if (!ok) {
            reject(new Error(`load_contents_finish returned false for ${path}`));
          } else {
            resolve(new TextDecoder('utf-8').decode(bytes));
          }
        } catch (e) {
          reject(e);
        }
      });
    });
  }

  // ── Pipeline initialization (deferred until the first frame, once a Cogl context exists) ──

  /**
   * Compiles and caches the downsample / upsample / composite Cogl.Pipeline
   * objects. Call only once.
   */
  private _initPipelines(ctx: Cogl.Context): void {
    // ── Downsample pipeline ──────────────────────────────────────────
    this._downsamplePipeline = Cogl.Pipeline.new(ctx);
    this._configureSamplerLayer(this._downsamplePipeline, 0);

    if (this._downsampleSource) {
      const downSnippet = this._splitShader(this._downsampleSource);
      const s = Cogl.Snippet.new(Cogl.SnippetHook.FRAGMENT, downSnippet.decl, null);
      s.set_replace(downSnippet.body);
      this._downsamplePipeline.add_snippet(s);
    }

    // ── Upsample pipeline ────────────────────────────────────────────
    this._upsamplePipeline = Cogl.Pipeline.new(ctx);
    this._configureSamplerLayer(this._upsamplePipeline, 0);
```

```
    if (this._upsampleSource) {
      const upSnippet = this._splitShader(this._upsampleSource);
      const s = Cogl.Snippet.new(Cogl.SnippetHook.FRAGMENT, upSnippet.decl, null);
      s.set_replace(upSnippet.body);
      this._upsamplePipeline.add_snippet(s);
    }
    // ── Gaussian H/V pipelines ──────────────────────────────────────────
    // Not precompiled here: the separable Gaussian blur builds its shader
    // source dynamically from the kernel computed in setBlurRadius(), and
    // _compileGaussianPipelines() compiles it lazily inside
    // vfunc_paint_target (see _computeGaussianKernel / _buildGaussianSnippet).

    // ── Composite pipeline (glass.frag) ─────────────────────────────────
    this._compositePipeline = Cogl.Pipeline.new(ctx);
    this._configureSamplerLayer(this._compositePipeline, 0);

    // Standard premultiplied-alpha blending, equivalent to ShaderEffect's default:
    // "src.rgb + dst.rgb * (1 - src.a)"
    this._compositePipeline.set_blend(
      'RGBA = ADD(SRC_COLOR, DST_COLOR * (1 - SRC_COLOR[A]))',
    );

    this._loadCompositeShader();

    // Apply any uniforms that were buffered before the pipeline existed.
    this._applyPendingUniforms();
  }

  /**
   * Shared helper: sets bilinear filtering and clamp-to-edge wrapping on
   * layer 0 of a pipeline.
   */
  private _configureSamplerLayer(pipeline: Cogl.Pipeline, layer: number): void {
    pipeline.set_layer_wrap_mode(layer, Cogl.PipelineWrapMode.CLAMP_TO_EDGE);
    pipeline.set_layer_filters(
      layer,
      Cogl.PipelineFilter.LINEAR,     // minification
      Cogl.PipelineFilter.LINEAR      // magnification
    );
  }

  /**
   * Splits a GLSL source string into { decl, body } at the "void main()" boundary.
   */
  private _splitShader(src: string): ShaderSnippet {
    const match = src.match(/void\s+main\s*\(\s*\)\s*\{/);
    if (!match || match.index === undefined) {
      this._logger?.warn('[Liquid Glass] void main() not found; treating entire source as decl.');
      return { decl: src, body: '' };
    }

    const decl = src.substring(0, match.index);
    const rest = src.substring(match.index + match[0].length);

    // Find the matching closing brace.
    let depth = 1;
    let bodyEnd = 0;
    for (let i = 0; i < rest.length; i++) {
      if (rest[i] === '{') depth++;
      else if (rest[i] === '}') {
        depth--;
        if (depth === 0) { bodyEnd = i; break; }
      }
    }

    return { decl, body: rest.substring(0, bodyEnd) };
  }

  /**
   * Loads glass.frag, rewrites its "cogl_sampler" (the uniform name used by
   * the old ShaderEffect) to "cogl_sampler0" (the name Cogl auto-declares for
   * a FRAGMENT-hook layer 0), and adds it to the composite pipeline as a
   * snippet.
   *
   * The original "uniform sampler2D cogl_sampler;" declaration is stripped
   * since cogl_sampler0 is already declared automatically by Cogl.
   */
  private _loadCompositeShader(): void {
    if (!this._compositePipeline || !this._glassSource) return;

    let { decl, body } = this._splitShader(this._glassSource);
```

```
    // Rewrite the ShaderEffect-style sampler name to the FRAGMENT-hook name.
    decl = decl.replace(/uniform\s+sampler2D\s+cogl_sampler\d*\s*;[^\n]*/g, '');
    body = body.replace(/\bcogl_sampler\b/g, 'cogl_sampler0');

    const snippet = Cogl.Snippet.new(Cogl.SnippetHook.FRAGMENT, decl, null);
    snippet.set_replace(body);
    this._compositePipeline.add_snippet(snippet);
  }

  // ─── Dynamic Gaussian kernel computation / shader generation ──────────────

  /**
   * Computes a linear-sampling-optimized 1D Gaussian kernel from a standard
   * deviation (sigma, in half-res texels) and a target number of fetch pairs.
   *
   * Method:
   *   1. Compute discrete Gaussian weights for i = 0..(fetchPairs*2) and normalize.
   *   2. i = 0 (the center) stays a single, standalone sample.
   *   3. Merge each (i, i+1) pair into a single fetch (bilinear-tap merging):
   *        combined weight  = w(i) + w(i+1)
   *        combined offset  = (i * w(i) + (i+1) * w(i+1)) / combined weight
   *
   * For a fixed fetchPairs, the resulting offsets/weights (and therefore the
   * shader's structure) are deterministic. As long as fetchPairs doesn't
   * change, sigma changes only need to update the kernel_scale uniform — see
   * setBlurRadius() — without any shader recompilation.
   */
  private _computeGaussianKernel(sigma: number, fetchPairs: number): GaussianKernel {
    const sideTaps = Math.max(2, fetchPairs * 2);

    // Compute and normalize discrete Gaussian weights for i = 0..sideTaps.
    const raw: number[] = [];
    let sum = 0;
    for (let i = 0; i <= sideTaps; i++) {
      const w = Math.exp(-(i * i) / (2 * sigma * sigma));
      raw.push(w);
      sum += (i === 0) ? w : w * 2;
    }
    for (let i = 0; i <= sideTaps; i++) {
      raw[i] /= sum;
    }

    const offsets: number[] = [0];
    const weights: number[] = [raw[0]];

    for (let p = 0; p < fetchPairs; p++) {
      const i = p * 2 + 1;
      const j = i + 1;
      const w0 = raw[i] ?? 0;
      const w1 = (j <= sideTaps) ? raw[j] : 0;
      const wSum = w0 + w1;
      const offset = wSum > 0 ? (i * w0 + j * w1) / wSum : i;
      offsets.push(offset);
      weights.push(wSum);
    }

    return { offsets, weights };
  }

  /**
   * Builds a GLSL fragment shader snippet string from a GaussianKernel
   * (fully unrolled — no for loop is used at runtime).
   *
   * Offsets are baked in as GLSL constants; the kernel_scale uniform is
   * multiplied in at runtime so sigma can be fine-tuned without recompiling.
   * Weights define the kernel's shape (fetch count) and are only baked in
   * again when a recompile actually happens.
   */
  private _buildGaussianSnippet(kernel: GaussianKernel, direction: 'h' | 'v'): ShaderSnippet {
    const decl =
      `uniform vec2 inv_size;    /* 1/width, 1/height of the SOURCE texture */\n` +
      `uniform float kernel_scale; /* dynamic scale based on the sigma ratio, avoids recompiling */\n`;

    const lines: string[] = [];
    lines.push(`vec2 uv = cogl_tex_coord_in[0].st;`);
    lines.push(`vec4 col = texture2D(cogl_sampler0, uv) * ${kernel.weights[0].toFixed(8)};`);

    for (let i = 1; i < kernel.offsets.length; i++) {
      const off = kernel.offsets[i].toFixed(8);
      const w = kernel.weights[i].toFixed(8);
      const plusVec = direction === 'h'
        ? `vec2(${off} * kernel_scale * inv_size.x, 0.0)`
```

```
        : `vec2(0.0, ${off} * kernel_scale * inv_size.y)`;
      lines.push(`col += texture2D(cogl_sampler0, uv + ${plusVec}) * ${w};`);
      lines.push(`col += texture2D(cogl_sampler0, uv - ${plusVec}) * ${w};`);
    }

    lines.push(`cogl_color_out = col;`);

    return { decl, body: '\n    ' + lines.join('\n    ') + '\n' };
  }

  /**
   * Compiles the H/V pipelines from a dynamically generated GaussianKernel.
   * The caller is responsible for having already dropped any previous
   * pipeline reference (we never call run_dispose(), see _destroyTexturePool).
   */
  private _compileGaussianPipelines(ctx: Cogl.Context, kernel: GaussianKernel): void {
    this._gaussianHPipeline = Cogl.Pipeline.new(ctx);
    this._configureSamplerLayer(this._gaussianHPipeline, 0);
    const hSnippet = this._buildGaussianSnippet(kernel, 'h');
    const hSnip = Cogl.Snippet.new(Cogl.SnippetHook.FRAGMENT, hSnippet.decl, null);
    hSnip.set_replace(hSnippet.body);
    this._gaussianHPipeline.add_snippet(hSnip);

    this._gaussianVPipeline = Cogl.Pipeline.new(ctx);
    this._configureSamplerLayer(this._gaussianVPipeline, 0);
    const vSnippet = this._buildGaussianSnippet(kernel, 'v');
    const vSnip = Cogl.Snippet.new(Cogl.SnippetHook.FRAGMENT, vSnippet.decl, null);
    vSnip.set_replace(vSnippet.body);
    this._gaussianVPipeline.add_snippet(vSnip);

    this._gaussianKernel = kernel;
    this._gaussianPipelineDirty = false;
    this._pendingGaussianKernel = null;
  }

  // ─── Texture pool management ──────────────────────────────────────

  /**
   * Allocates the blur texture + FBO pairs for resolution (w, h).
   *
   * Index-to-resolution mapping:
   *   [0]: w>>1 × h>>1  (= w/2)
   *   [1]: w>>2 × h>>2  (= w/4)
   *   ...
   *   [PASS_COUNT-1]: w >> PASS_COUNT
   */
  private _buildTexturePool(ctx: Cogl.Context, w: number, h: number): void {
    this._destroyTexturePool();

    let pw = Math.max(w >> 1, 1);
    let ph = Math.max(h >> 1, 1);

    for (let i = 0; i < this.PASS_COUNT; i++) {
      try {
        // Main buffer, shared by Dual Kawase and Gaussian.
        const tex = Cogl.Texture2D.new_with_size(ctx, pw, ph);
        const fbo = Cogl.Offscreen.new_with_texture(tex);
        this._blurTextures.push(tex);
        this._blurFbos.push(fbo);

        // Intermediate buffer for the Gaussian horizontal pass (same resolution).
        const tmpTex = Cogl.Texture2D.new_with_size(ctx, pw, ph);
        const tmpFbo = Cogl.Offscreen.new_with_texture(tmpTex);
        this._gaussianTempTextures.push(tmpTex);
        this._gaussianTempFbos.push(tmpFbo);
      } catch (e) {
        this._logger?.error(`[Liquid Glass] Failed to build texture pool at pass ${i} (${pw}x${ph}): ${e}`);
        this._destroyTexturePool();
        return;
      }

      pw = Math.max(pw >> 1, 1);
      ph = Math.max(ph >> 1, 1);
    }

    this._poolWidth = w;
    this._poolHeight = h;
  }

  /**
   * Runs the Dual Kawase blur.
   *
```

```
     *   Downsample phase: srcTex → [0] → [1] → ... → [PASS_COUNT-1]
     *   Upsample phase:   [PASS_COUNT-1] → ... → [0]
     *
     * The result ends up in _blurTextures[0].
     */
    private _runDualKawaseBlur(srcTex: Cogl.Texture): void {
      let currentSrc: Cogl.Texture = srcTex;

      // ── Downsample phase ──────────────────────────────────────────
      for (let i = 0; i < this.PASS_COUNT; i++) {
        const destFbo = this._blurFbos[i] as unknown as CoglFB;
        const destTex = this._blurTextures[i];
        const destW = destTex.get_width();
        const destH = destTex.get_height();

        const invW = 1.0 / currentSrc.get_width();
        const invH = 1.0 / currentSrc.get_height();

        this._downsamplePipeline!.set_layer_texture(0, currentSrc);
        this._setPipelineVec2(this._downsamplePipeline!, 'inv_size', invW, invH);
        this._setPipelineFloat(this._downsamplePipeline!, 'blur_radius', this._blurRadiusDown);

        destFbo.clear4f(Cogl.BufferBit.COLOR, 0.0, 0.0, 0.0, 0.0);
        destFbo.orthographic(0, 0, destW, destH, -1, 1);
        destFbo.draw_textured_rectangle(
          this._downsamplePipeline!, 0, 0, destW, destH, 0, 0, 1, 1
        );
        // Flush right after each pass to break the FBO dependency chain.
        destFbo.flush();

        currentSrc = destTex;
      }

      // ── Upsample phase ────────────────────────────────────────────
      for (let i = this.PASS_COUNT - 1; i > 0; i--) {
        const srcTexture = this._blurTextures[i];
        const destFbo = this._blurFbos[i - 1] as unknown as CoglFB;
        const destTex = this._blurTextures[i - 1];
        const destW = destTex.get_width();
        const destH = destTex.get_height();

        const invW = 1.0 / srcTexture.get_width();
        const invH = 1.0 / srcTexture.get_height();

        this._upsamplePipeline!.set_layer_texture(0, srcTexture);
        this._setPipelineVec2(this._upsamplePipeline!, 'inv_size', invW, invH);
        this._setPipelineFloat(this._upsamplePipeline!, 'blur_radius', this._blurRadiusUp);

        destFbo.clear4f(Cogl.BufferBit.COLOR, 0.0, 0.0, 0.0, 0.0);
        destFbo.orthographic(0, 0, destW, destH, -1, 1);
        destFbo.draw_textured_rectangle(
          this._upsamplePipeline!, 0, 0, destW, destH, 0, 0, 1, 1
        );
        destFbo.flush();
      }
    }

    /**
     * Runs the separable Gaussian blur.
     *
     * PASS_COUNT is always fixed to 1 for this method, and the texture pool
     * only uses a single w/2 × h/2 level (no pool rebuild / pass-count change
     * happens when the radius changes).
     *
     *   srcTex → [gaussianTemp[0]] (horizontal pass) → [blurTextures[0]] (vertical pass)
     *
     * The H/V pipelines are the ones dynamically built from the kernel
     * computed in setBlurRadius() (fully unrolled). Result ends up in
     * _blurTextures[0].
     */
    private _runGaussianBlur(srcTex: Cogl.Texture): void {
      const tempFbo = this._gaussianTempFbos[0] as unknown as CoglFB;
      const tempTex = this._gaussianTempTextures[0];
      const destFbo = this._blurFbos[0] as unknown as CoglFB;
      const destTex = this._blurTextures[0];
      const destW = destTex.get_width();
      const destH = destTex.get_height();
      const srcW = srcTex.get_width();
      const srcH = srcTex.get_height();

      // ── 0. Pre-pass: srcTex (full res) → destTex (half res) ──────────
      // A plain bilinear downsample so the H/V passes can operate entirely in
```

```
    // half-resolution space. (Reuses the Dual Kawase downsample pipeline with
    // radius 0.)
    this._downsamplePipeline!.set_layer_texture(0, srcTex);
    this._setPipelineVec2(this._downsamplePipeline!, 'inv_size', 1.0 / srcW, 1.0 / srcH);
    this._setPipelineFloat(this._downsamplePipeline!, 'blur_radius', 0.0);

    destFbo.clear4f(Cogl.BufferBit.COLOR, 0.0, 0.0, 0.0, 0.0);
    destFbo.orthographic(0, 0, destW, destH, -1, 1);
    destFbo.draw_textured_rectangle(
      this._downsamplePipeline!, 0, 0, destW, destH, 0, 0, 1, 1
    );
    destFbo.flush();

    // ── 1. Horizontal pass: destTex (half res) → tempTex (half res) ──────────
    // Input is already half-resolution, so inv_size uses destW/destH directly.
    this._gaussianHPipeline!.set_layer_texture(0, destTex);
    this._setPipelineVec2(this._gaussianHPipeline!, 'inv_size', 1.0 / destW, 1.0 / destH);
    this._setPipelineFloat(this._gaussianHPipeline!, 'kernel_scale', this._gaussianScale);

    tempFbo.clear4f(Cogl.BufferBit.COLOR, 0.0, 0.0, 0.0, 0.0);
    tempFbo.orthographic(0, 0, destW, destH, -1, 1);
    tempFbo.draw_textured_rectangle(
      this._gaussianHPipeline!, 0, 0, destW, destH, 0, 0, 1, 1
    );
    tempFbo.flush();

    // ── 2. Vertical pass: tempTex (half res) → destTex (half res) ────────────
    this._gaussianVPipeline!.set_layer_texture(0, tempTex);
    this._setPipelineVec2(this._gaussianVPipeline!, 'inv_size', 1.0 / destW, 1.0 / destH);
    this._setPipelineFloat(this._gaussianVPipeline!, 'kernel_scale', this._gaussianScale);

    destFbo.clear4f(Cogl.BufferBit.COLOR, 0.0, 0.0, 0.0, 0.0);
    destFbo.orthographic(0, 0, destW, destH, -1, 1);
    destFbo.draw_textured_rectangle(
      this._gaussianVPipeline!, 0, 0, destW, destH, 0, 0, 1, 1
    );
    destFbo.flush();
  }

  /**
   * Drops the texture pool and resets the related fields.
   *
   * We never call run_dispose() on these GJS-managed Cogl objects: GJS's own
   * garbage collector would later try to unref them again, causing a double
   * free ("free(): invalid size" → SIGABRT). Simply clearing the references
   * lets the GC reclaim the VRAM safely.
   */
  private _destroyTexturePool(): void {
    this._blurFbos = [];
    this._blurTextures = [];
    this._gaussianTempFbos = [];
    this._gaussianTempTextures = [];
    this._poolWidth = 0;
    this._poolHeight = 0;
  }

  // ─── Crop pass (works around OffscreenEffect FBO padding) ──────────────────
  //
  // Background: on some Cogl/Clutter versions, the texture returned by
  // get_texture() can be a few pixels larger than the actor's logical size
  // (e.g. alloc=1920x1080 but tex=1923x1083). This appears to be fixed
  // internal padding added by OffscreenEffect's FBO allocation, unrelated to
  // any user setting.
  //
  // If vfunc_paint_target treated that padded size as the "true" resolution,
  // the extra pixels would leak into both the final composite draw rect and
  // the blur texture pool's resolution chain, producing undefined-content
  // artifacts and small misalignments between sharp and blurred layers.
  //
  // Fix: never trust get_texture()'s size — actor.get_size() is always the
  // source of truth. When they differ, crop out just the valid region into
  // a dedicated texture (_cropTexture) once per frame, and use that as the
  // input for every later pass. No extra shader is needed: downsample.frag
  // run with blur_radius = 0 collapses its 5-tap Kawase kernel onto the
  // center sample, so it doubles as a plain UV-remapping passthrough.

  /**
   * (Re)allocates the crop FBO/texture at size (w, h), reusing the existing
   * one if the size hasn't changed.
   */
  private _ensureCropTarget(ctx: Cogl.Context, w: number, h: number): boolean {
    if (this._cropTexture && this._cropFbo &&
```

```
      this._cropPoolW === w && this._cropPoolH === h) {
      return true;
    }

    // Just clear the old references and let the GC handle them (same
    // reasoning as _destroyTexturePool).
    this._cropTexture = null;
    this._cropFbo = null;
    this._cropPoolW = 0;
    this._cropPoolH = 0;

    try {
      const tex = Cogl.Texture2D.new_with_size(ctx, w, h);
      const fbo = Cogl.Offscreen.new_with_texture(tex);
      this._cropTexture = tex;
      this._cropFbo = fbo;
      this._cropPoolW = w;
      this._cropPoolH = h;
      return true;
    } catch (e) {
      this._logger?.error(`[Liquid Glass] Failed to create crop texture (${w}x${h}): ${e}`);
      return false;
    }
  }

  /**
   * Crops srcTex (the OffscreenEffect's raw texture, which may include
   * padding) down to exactly the actor's logical size (allocW x allocH).
   * If no cropping is needed (no padding), returns srcTex unchanged.
   *
   * Padding is assumed to be distributed evenly around the content (i.e. the
   * valid region is centered within srcTex). This anchor was determined
   * empirically to match observed behavior across the tested Cogl/Clutter
   * versions.
   */
  private _cropSourceTexture(
    ctx: Cogl.Context,
    srcTex: Cogl.Texture2D,
    srcW: number, srcH: number,
    allocW: number, allocH: number
  ): Cogl.Texture2D {
    if (allocW === srcW && allocH === srcH) {
      return srcTex;
    }
    if (!this._downsamplePipeline) {
      // Pipeline isn't ready yet, so cropping isn't possible; fall back to the raw texture.
      return srcTex;
    }
    if (!this._ensureCropTarget(ctx, allocW, allocH)) {
      return srcTex;
    }

    // Assume padding is split evenly on all sides (center anchor).
    const padW = srcW - allocW;
    const padH = srcH - allocH;
    const offX = padW / 2;
    const offY = padH / 2;

    const uMin = offX / srcW;
    const vMin = offY / srcH;
    const uMax = Math.min(1.0, (offX + allocW) / srcW);
    const vMax = Math.min(1.0, (offY + allocH) / srcH);

    const fbo = this._cropFbo as unknown as CoglFB;
    const pipeline = this._downsamplePipeline!;

    pipeline.set_layer_texture(0, srcTex);
    this._setPipelineVec2(pipeline, 'inv_size', 1.0 / srcW, 1.0 / srcH);
    // blur_radius = 0 collapses every tap in the 5-tap kernel onto the center
    // sample, turning this into a plain UV resample (i.e. a crop).
    this._setPipelineFloat(pipeline, 'blur_radius', 0.0);

    fbo.clear4f(Cogl.BufferBit.COLOR, 0.0, 0.0, 0.0, 0.0);
    fbo.orthographic(0, 0, allocW, allocH, -1, 1);
    // Draw across the full (0,0)-(allocW,allocH) output rect while sampling
    // only the (uMin,vMin)-(uMax,vMax) input UV range — stretching the
    // padding-free region to fill the output, i.e. cropping.
    fbo.draw_textured_rectangle(
      pipeline, 0, 0, allocW, allocH, uMin, vMin, uMax, vMax
    );
    fbo.flush();
```

```
    return this._cropTexture!;
  }

  private _destroyCropTarget(): void {
    this._cropTexture = null;
    this._cropFbo = null;
    this._cropPoolW = 0;
    this._cropPoolH = 0;
  }

  /**
   * Overrides the Clutter.OffscreenEffect hook.
   *
   * Called after OffscreenEffect has rendered the actor's content into its
   * internal FBO, at the point where that FBO texture is normally composited
   * onto the screen.
   *
   * The default super.vfunc_paint_target() just draws the FBO straight to
   * the screen; here we instead run the blur pipeline followed by the glass
   * composite pass.
   *
   * @param _paintNode   Clutter's paint node (new signature since GNOME 45+)
   * @param paintContext Current paint context, holding a reference to the on-screen framebuffer
   */
  vfunc_paint_target(_paintNode: Clutter.PaintNode, paintContext: Clutter.PaintContext): void {
    // ── [DIAG] Black-background investigation ──────────────────────────
    // If Clutter culls/skips this actor entirely (e.g. because it decides
    // it's fully occluded by the window content painted above it), this
    // function never runs at all -- which would show up here as a call count
    // that never advances past whatever it was when the window opened, even
    // though _frameTick keeps calling set_size()/queue_redraw() at 60fps.
    this._diagPaintCount++;
    {
      const now = GLib.get_monotonic_time();
      const actorTitle = (() => {
        try {
          const a = this.get_actor() as any;
          return a?.get_meta_window?.()?.get_title?.() ?? a?.get_name?.() ?? '?';
        } catch (e) { return '?'; }
      })();
      if (!this._diagFirstPaintLogged) {
        this._diagFirstPaintLogged = true;
        this._diagLastPaintLogAt = now;
        this._logger?.log(`[Liquid Glass][diag] LiquidEffect.vfunc_paint_target: FIRST call for "${actorTitle}" ` +
          `(paintCount=${this._diagPaintCount}, shadersLoaded=${this._shadersLoaded})`);
      } else if (now - this._diagLastPaintLogAt > 2000 * 1000) {
        this._logger?.log(`[Liquid Glass][diag] LiquidEffect.vfunc_paint_target: heartbeat for "${actorTitle}", ` +
          `paintCount=${this._diagPaintCount}, compositedCount=${this._diagCompositedPaintCount}`);
        this._diagLastPaintLogAt = now;
      }
    }

    // ── Wait for async shaders ─────────────────────────────────────────
    if (!this._shadersLoaded) {
      super.vfunc_paint_target(_paintNode, paintContext);
      return;
    }
    // ── Deferred pipeline initialization ───────────────────────────────
    if (!this._compositePipeline) {
      try {
        const ctx = this._getCoglContext();
        if (!ctx) throw new Error('Could not obtain a Cogl context');
        this._initPipelines(ctx);
      } catch (e) {
        this._logger?.error(`[Liquid Glass] Pipeline initialization failed: ${e}`);
        // Fall back to OffscreenEffect's default drawing.
        super.vfunc_paint_target(_paintNode, paintContext);
        return;
      }
    }

    // ── Guard check ────────────────────────────────────────────────────
    // The Gaussian H/V pipelines don't exist until a radius has been set
    // (they're built dynamically), so they're intentionally excluded from
    // this required-pipeline check.
    if (!this._compositePipeline || !this._downsamplePipeline || !this._upsamplePipeline) {
      super.vfunc_paint_target(_paintNode, paintContext);
      return;
    }

    // ── Deferred compilation of the Gaussian shaders ───────────────────
    // Whenever setBlurRadius() changes the tap count, compile the new H/V
```

```
    // pipelines here, where a Cogl context is guaranteed to be available.
    // Old pipeline references are left for GJS's GC rather than disposed
    // manually.
    if (this._gaussianPipelineDirty && this._pendingGaussianKernel) {
      try {
        const ctx = this._getCoglContext();
        if (!ctx) throw new Error('Could not obtain a Cogl context');
        this._gaussianHPipeline = null;
        this._gaussianVPipeline = null;
        this._compileGaussianPipelines(ctx, this._pendingGaussianKernel);
      } catch (e) {
        this._logger?.error(`[Liquid Glass] Failed to build Gaussian pipelines: ${e}`);
      }
    }

    // Grab the FBO texture OffscreenEffect captured from the actor.
    const srcTex = this.get_texture() as Cogl.Texture2D | null;
    if (!srcTex) {
      super.vfunc_paint_target(_paintNode, paintContext);
      return;
    }

    const srcW = srcTex.get_width();
    const srcH = srcTex.get_height();

    // ── Trust the actor's logical size over get_texture()'s reported size ──
    // get_texture() can be a few pixels larger than the actor's logical size
    // due to internal FBO padding (see the crop-pass comment above), so
    // actor.get_size() is used as the source of truth from here on.
    const actor = this.get_actor();
    let allocW = srcW;
    let allocH = srcH;
    if (actor) {
      const [aw, ah] = actor.get_size();
      if (Number.isFinite(aw) && aw > 0) allocW = Math.round(aw);
      if (Number.isFinite(ah) && ah > 0) allocH = Math.round(ah);
    }

    // ── Only run the crop pass when padding is actually present ─────────────
    // (When sizes match, _cropSourceTexture just returns srcTex unchanged,
    //  so the normal-case overhead is a single size comparison.)
    let effectiveTex: Cogl.Texture2D = srcTex;
    let effectiveW = srcW;
    let effectiveH = srcH;

    if (allocW !== srcW || allocH !== srcH) {
      try {
        const ctx = this._getCoglContext();
        if (!ctx) throw new Error('Could not obtain a Cogl context');
        effectiveTex = this._cropSourceTexture(ctx, srcTex, srcW, srcH, allocW, allocH);
        // Only update the effective dimensions if cropping actually succeeded
        // (on failure, _cropSourceTexture returns srcTex unchanged, so we keep
        // the padded dimensions).
        if (effectiveTex !== srcTex) {
          effectiveW = allocW;
          effectiveH = allocH;
        }
      } catch (e) {
        this._logger?.error(`[Liquid Glass] Crop pass failed; continuing with the padded texture: ${e}`);
      }
    }

    // ── Rebuild the texture pool when the resolution changes ────────────────
    // Based on the cropped ("true") resolution — using the padded size here
    // would cause rounding error from bit-shifting (w >> 1) an odd value to
    // accumulate across passes, misaligning the sharp and blurred layers.
    if (effectiveW !== this._poolWidth || effectiveH !== this._poolHeight) {
      try {
        const ctx = this._getCoglContext();
        if (!ctx) throw new Error('Could not obtain a Cogl context');
        this._buildTexturePool(ctx, effectiveW, effectiveH);
      } catch (e) {
        this._logger?.error(`[Liquid Glass] Failed to rebuild the texture pool: ${e}`);
        super.vfunc_paint_target(_paintNode, paintContext);
        return;
      }
    }

    if (this.PASS_COUNT > 0 && !this._blurFbos.length) {
      super.vfunc_paint_target(_paintNode, paintContext);
      return;
    }
```

```
    // ──────────────────────────────────────────────────────────────────
    // Blur pass: which blur method runs depends on _blurMethod
    //   0: Separable Gaussian blur
    //   1: Dual Kawase blur (original implementation)
    // Always takes effectiveTex (cropped, padding-free) as input.
    // ──────────────────────────────────────────────────────────────────
    if (this.PASS_COUNT > 0) {
      if (this._blurMethod === 0) {
        if (this._gaussianHPipeline && this._gaussianVPipeline) {
          this._runGaussianBlur(effectiveTex);
        }
      } else {
        this._runDualKawaseBlur(effectiveTex);
      }
    }

    // ──────────────────────────────────────────────────────────────────
    // Final pass: glass composite.
    //   Binds _blurTextures[0] (blurred, w/2 × h/2) as cogl_sampler0 and runs
    //   glass.frag (refraction / rim lighting / shadow) to draw onto the screen.
    //
    //   Clutter has already set up the actor's model-view transform on
    //   screenFb, so drawing in actor-local coordinates
    //   (0, 0)-(effectiveW, effectiveH) is all that's needed. Using
    //   (0,0,srcW,srcH) here used to leak the padding onto the screen.
    // ──────────────────────────────────────────────────────────────────
    const compFb = paintContext.get_framebuffer();
    const compPipeline = this._compositePipeline!;

    // Layer 0: the sharp, unblurred capture (used as the basis for refraction).
    // Using the cropped texture means UV (0,0)-(1,1) lines up exactly with
    // the actor's logical size.
    compPipeline.set_layer_texture(0, effectiveTex);
    this._configureSamplerLayer(compPipeline, 0);

    // Layer 1: the heavily blurred texture (used for the background blur).
    // Falls back to effectiveTex when no blur pass ran.
    if (this.PASS_COUNT > 0 && this._blurTextures.length > 0) {
      compPipeline.set_layer_texture(1, this._blurTextures[0]);
    } else {
      compPipeline.set_layer_texture(1, effectiveTex);
    }
    this._configureSamplerLayer(compPipeline, 1);

    // Manually sync pending uniforms into the composite pipeline.
    // Without this, values like dock_x would stay at 0 and the whole screen
    // would be misdetected as being inside the dock mask.
    this._applyPendingUniforms();

    compFb.push_matrix();

    const screenFb = paintContext.get_framebuffer() as unknown as CoglFB;
    screenFb.draw_textured_rectangle(
      this._compositePipeline!, 0, 0, effectiveW, effectiveH, 0, 0, 1, 1
    );

    compFb.pop_matrix();

    // [DIAG] This is the actual "real glass" draw path (not a super.vfunc_paint_target()
    // fallback). If the black-background bug is visible on screen while this counter
    // keeps advancing at 60fps, the composite pass itself is running fine and drawing
    // into the on-screen framebuffer every frame -- the bug is not about painting
    // being skipped/culled or about stale content, it's about what's being composited.
    this._diagCompositedPaintCount++;
  }

  // ─── Uniform helpers ──────────────────────────────────────────────

  /**
   * Sets a vec2 uniform on a pipeline. Cogl caches the uniform location
   * internally, so calling this every frame is safe.
   */
  private _setPipelineVec2(
    pipeline: Cogl.Pipeline, name: string, x: number, y: number
  ): void {
    const loc = pipeline.get_uniform_location(name);
    // set_uniform_float(loc, n_components, count, values[])
    pipeline.set_uniform_float(loc, 2, 1, [x, y]);
  }

  /**
```

```
     * Sets a scalar float uniform on a pipeline.
     */
    private _setPipelineFloat(
      pipeline: Cogl.Pipeline, name: string, value: number
    ): void {
      const loc = pipeline.get_uniform_location(name);
      pipeline.set_uniform_float(loc, 1, 1, [value]);
    }

    /**
     * Sets a float uniform on the composite pipeline. If the pipeline hasn't
     * been created yet, the value is buffered in _pendingUniforms and applied
     * later in _applyPendingUniforms().
     */
    private _setFloat(name: string, value: number): void {
      this._pendingUniforms.set(name, value);
      if (this._compositePipeline) {
        this._applyUniform(name, value);
      }
    }

    private _applyUniform(name: string, value: number): void {
      if (!this._compositePipeline) return;

      // Cache the uniform location to avoid a get_uniform_location() call every frame.
      let loc = this._compUniforms.get(name);
      if (loc === undefined) {
        loc = this._compositePipeline.get_uniform_location(name);
        this._compUniforms.set(name, loc);
      }
      // set_uniform_float(loc, 1 component, 1 element, [value])
      this._compositePipeline.set_uniform_float(loc, 1, 1, [value]);
    }

    private _applyPendingUniforms(): void {
      for (const [name, value] of this._pendingUniforms) {
        this._applyUniform(name, value);
      }
    }

    // ─── Cogl context lookup ─────────────────────────────────────────

    private _getCoglContext(): Cogl.Context | null {
      try {
        // Clutter.get_default_backend() is available from GJS.
        // On GNOME 50, get_cogl_context() returns a Cogl.Context.
        const backend = Clutter.get_default_backend();
        return backend.get_cogl_context() as Cogl.Context;
      } catch (e) {
        this._logger?.error(`[Liquid Glass] Failed to obtain the Cogl context: ${e}`);
        return null;
      }
    }

    // ─── GSettings bindings ──────────────────────────────────────────

    _bindSettings(): void {
      const mappings: { key: string; uniform: string }[] = [
        { key: 'glass-max-z', uniform: 'max_z' },
        { key: 'glass-displacement-scale', uniform: 'displacement_scale' },
        { key: 'glass-edge-smoothing', uniform: 'edge_smoothing' },
        { key: 'glass-profile-shape-n', uniform: 'profile_shape_n' },
        { key: 'glass-ior', uniform: 'ior' },
        { key: 'glass-chroma-strength', uniform: 'chroma_strength' },
        { key: 'glass-specular-intensity', uniform: 'specular_intensity' },
        { key: 'glass-shininess', uniform: 'shininess' },
        { key: 'glass-rim-width', uniform: 'rim_width' },
        { key: 'glass-rim-intensity', uniform: 'rim_intensity' },
        { key: 'glass-rim-directional-power', uniform: 'rim_directional_power' },
        { key: 'glass-rim-power', uniform: 'rim_power' },
        { key: 'glass-rim-light-color-intensity', uniform: 'rim_light_color_intensity' },
        { key: 'glass-sheen-intensity', uniform: 'sheen_intensity' },
        { key: 'glass-light-angle-deg', uniform: 'light_angle_deg' },
        { key: 'shadow-radius', uniform: 'shadow_radius' },
        { key: 'shadow-intensity', uniform: 'shadow_intensity' },
        // Inner edge AO darkening — independent of rim_width and of the
        // outer drop shadow's radius/intensity pair above.
        { key: 'glass-ao-intensity', uniform: 'ao_intensity' },
        { key: 'glass-ao-radius', uniform: 'ao_radius' },
      ];

      const settings = this._settings;
```

```
    if (!settings) return;

    mappings.forEach(map => {
      // Apply the initial value.
      this._setFloat(map.uniform, settings.get_double(map.key));
      // Watch for changes.
      const id = settings.connect(`changed::${map.key}`, () => {
        this._setFloat(map.uniform, settings.get_double(map.key));
      });
      this._settingsIds.push(id);
    });

    // ── blur-method (int): 0 = Gaussian, 1 = Dual Kawase ──────────────────
    // Assumes the GSettings schema defines this key as an int.
    const applyBlurMethod = () => {
      const raw = settings.get_int('blur-method');
      this.setBlurMethod(raw === 0 ? 0 : 1);
    };
    applyBlurMethod();
    const blurMethodId = settings.connect('changed::blur-method', applyBlurMethod);
    this._settingsIds.push(blurMethodId);

  }

  // ─── Public API (compatible with the previous ShaderEffect-based interface) ──

  cleanup(): void {
    // Disconnect GSettings signal handlers.
    if (this._settings && this._settingsIds) {
      this._settingsIds.forEach(id => this._settings?.disconnect(id));
      this._settingsIds = [];
    }

    // Free the texture pool (reference clear only — run_dispose() would double-free).
    this._destroyTexturePool();
    this._destroyCropTarget();

    // Clear pipeline references (GJS's GC reclaims the VRAM).
    // Never call run_dispose() here — it would double-unref a GJS-managed object.
    this._downsamplePipeline = null;
    this._upsamplePipeline = null;
    this._gaussianHPipeline = null;
    this._gaussianVPipeline = null;
    this._compositePipeline = null;
    this._compUniforms.clear();
    this._pendingUniforms.clear();

    // Reset the dynamic Gaussian shader generation state too.
    this._gaussianKernel = null;
    this._pendingGaussianKernel = null;
    this._gaussianPipelineDirty = false;
    this._gaussianBaseSigma = 0;
    this._gaussianScale = 1.0;
    this._gaussianFetchPairs = 0;
  }

  setIsDock(isDock: boolean): void {
    this._setFloat('isDock', isDock ? 1.0 : 0.0);
  }

  /**
   * Enables/disables the rim light + specular + sheen "glass surface
   * glint" terms as a group (see addedLight in glass.frag). The outer
   * drop shadow and inner AO edge-darkening are unaffected either way —
   * they're computed independently of this uniform. Used by
   * applicationManager.ts to give application windows a plainer
   * "shadow + AO only" edge instead of the dock/menu-style glass glint,
   * without touching the shared rim/specular/sheen settings that dock,
   * menu, notification, quick-settings and OSD still use.
   */
  setSurfaceLightEnabled(enabled: boolean): void {
    this._setFloat('surface_light_enabled', enabled ? 1.0 : 0.0);
    this.queue_repaint();
  }

  setPadding(pad: number): void {
    this._setFloat('padding', pad);
  }

  /**
   * Tells the shader how much room (in px) the drop shadow actually
   * has to render outward, independent of the small optical `padding`
```

```
     * uniform. Should be kept in sync with dockManager's CLIP_PADDING (minus
     * a small safety margin) so shadow_radius can use its full prefs.js
     * range (0-100) without being invisibly clamped or hitting a hard edge
     * at the bgActor's own clip boundary.
     */
    setShadowMaxRadius(radius: number): void {
      this._setFloat('shadow_max_radius', radius);
    }

    /**
     * [DEBUG] Forces glass.frag (and the downsample/upsample shaders) to be
     * re-read from disk and recompiled into fresh Cogl.Pipelines on the next
     * paint.
     *
     * Why this exists: _initPipelines() only ever runs once per LiquidEffect
     * instance, guarded by `if (!this._compositePipeline)` in
     * vfunc_paint_target(). The instance itself only gets recreated when
     * dockManager tears down and rebuilds the effect (extension disable/
     * re-enable, or the dock actor being destroyed). So editing glass.frag on
     * disk while the shell keeps running has NO effect on what's on screen
     * until one of those happens — the exact same (possibly still-buggy)
     * compiled shader keeps executing every frame regardless of what the
     * source file now says. This silently made prior shader fixes look like
     * they hadn't worked. Call this after saving shader edits to pick them up
     * immediately instead.
     */
    reloadShaders(): void {
      this._compositePipeline = null;
      this._downsamplePipeline = null;
      this._upsamplePipeline = null;
      this._gaussianHPipeline = null;
      this._gaussianVPipeline = null;
      this._gaussianKernel = null;
      this._gaussianFetchPairs = 0;
      this._compUniforms.clear();
      // _pendingUniforms is intentionally left intact: it holds every uniform
      // value currently in effect, and _initPipelines() re-applies all of them
      // to the freshly-compiled pipeline via _applyPendingUniforms().
      // Re-derive the Gaussian kernel (if that's the active blur method) so
      // _gaussianPipelineDirty / _pendingGaussianKernel get set correctly
      // instead of leaving the Gaussian pass permanently skipped.
      this.setBlurRadius(this._targetRadius);
      this.queue_repaint();
    }

    setTintColor(r: number, g: number, b: number): void {
      this._setFloat('tint_r', r);
      this._setFloat('tint_g', g);
      this._setFloat('tint_b', b);
      this.queue_repaint();
    }

    setTintStrength(strength: number): void {
      this._setFloat('tint_strength', strength);
      this.queue_repaint();
    }

    setCornerRadius(radius: number): void {
      this._setFloat('corner_radius', radius);
      this.queue_repaint();
    }

    setAnimationScale(scale: number): void {
      const settings = this._settings;
      if (!settings) return;
      this._setFloat('displacement_scale',
        settings.get_double('glass-displacement-scale') * scale);
      this._setFloat('max_z',
        settings.get_double('glass-max-z') * scale);
      this._setFloat('chroma_strength',
        settings.get_double('glass-chroma-strength') * scale);
      this.queue_repaint();
    }

    setPointerPosition(x: number, y: number, intensity: number): void {
      this._setFloat('pointer_x', x);
      this._setFloat('pointer_y', y);
      this._setFloat('intensity', intensity);
    }

    /**
     * Syncs the actor's logical size to the shader's resolution uniform.
```

```
     *
     * The texture pool itself is rebuilt automatically inside
     * vfunc_paint_target based on get_texture()'s size, so no extra work is
     * needed here.
     */
    setResolution(width: number, height: number): void {
      this._setFloat('resolution_x', width);
      this._setFloat('resolution_y', height);
      this.queue_repaint();
    }

    /**
     * Full-screen FBO mode: passes the dock's monitor-relative geometry to the
     * shader (see the dock_x/y/w/h comments in glass.frag for details).
     */
    setGlassGeometry(x: number, y: number, w: number, h: number): void {
      this._setFloat('dock_x', x);
      this._setFloat('dock_y', y);
      this._setFloat('dock_w', w);
      this._setFloat('dock_h', h);
      this.queue_repaint();
    }

    setBrightness(brightness: number): void {
      this._setFloat('brightness', brightness);
      this.queue_repaint();
    }

    setContrast(contrast: number): void {
      this._setFloat('contrast', contrast);
      this.queue_repaint();
    }

    setSaturation(saturation: number): void {
      this._setFloat('saturation', saturation);
      this.queue_repaint();
    }

    /**
     * Dynamically switches the blur method.
     *
     * @param method 0: separable Gaussian blur, 1: Dual Kawase blur
     *
     * The Dual Kawase pipelines are already compiled in _initPipelines on the
     * first frame. The Gaussian pipelines are built dynamically: setBlurRadius()
     * computes the kernel for the current radius, and it's lazily compiled on
     * the next vfunc_paint_target only if needed.
     * The texture pool is shared between both methods (see _buildTexturePool),
     * so no manual rebuild is required when switching — queue_repaint() alone
     * is enough for the new method to take effect on the next frame.
     */
    setBlurMethod(method: BlurMethod): void {
      if (this._blurMethod === method) return;
      this._blurMethod = method;
      this.setBlurRadius(this._targetRadius);
      this.queue_repaint();
    }

    /**
     * Dynamically sets the blur radius. The calculation branches depending on
     * the active method (Gaussian / Dual Kawase).
     */
    setBlurRadius(radius: number): void {
      this._targetRadius = radius;

      if (this._blurMethod === 0) {
        this._setGaussianBlurRadius(radius);
        return;
      }

      this._setDualKawaseBlurRadius(radius);
    }

    /**
     * Radius setter for the separable Gaussian blur (dynamic shader generation).
     *
     * Basic approach:
     *   - PASS_COUNT is always fixed to 1. The texture pool only uses a single
     *     w/2 × h/2 level, so changing the radius never triggers a pool
     *     rebuild (avoids visible stutter).
     *   - The number of fetch pairs (tap count) is derived from the radius
     *     (= sigma, in original-resolution pixels). As long as the fetch count
```

```
     *     doesn't change, the existing compiled shader is reused as-is and only
     *     the kernel_scale uniform is updated (skips an unnecessary recompile).
     *
     * Derivation:
     *   1. Compute the effective standard deviation sigma in half-resolution
     *      space: sigma = radius / RES_SCALE (RES_SCALE = 2.0; at half
     *      resolution, 1 texel = 2 original pixels).
     *   2. Clamp to a maximum radius of 30px (15 texels in half-res space).
     *   3. Determine how many one-sided taps are needed for the Gaussian
     *      weights to decay close enough to zero (the "3 sigma" rule), then
     *      convert that into a fetch-pair count (2 taps merged per fetch).
     *   4. If the fetch-pair count matches the previous one, skip regenerating
     *      the shader string and recompiling the pipeline — just update
     *      kernel_scale = sigma / base sigma.
     *      If it changed, stage a new kernel in _pendingGaussianKernel to be
     *      compiled safely on the next vfunc_paint_target.
     */
    private _setGaussianBlurRadius(radius: number): void {
      const RES_SCALE = 2.0;        // half resolution: 1 texel = 2 original pixels
      const MAX_SIGMA_TEXEL = 15.0; // physical cap of 30px (= 15 texels in half-res space)

      // ── Minimum sigma guarantee ──────────────────────────────────────
      // Downsampling to half resolution (bilinear 2x) is effectively a 2px-wide
      // box filter, which aliases high-frequency content such as text. To
      // counteract that aliasing, the H/V kernel's effective width needs to
      // exceed 1.0 half-res texel (= 2 original pixels).
      // So sigma is floored at MIN_SIGMA_TEXEL = 1.0, guaranteeing at least a
      // minimal amount of smoothing even for a very small requested radius.
      // For small radii, kernel_scale ends up < 1.0, pulling the taps toward
      // the center — functioning simply as a "weaker blur" (the anti-aliasing
      // effect is preserved).
      const MIN_SIGMA_TEXEL = 1.0;

      if (radius <= 0) {
        if (this.PASS_COUNT !== 0) {
          this.PASS_COUNT = 0;
          this._destroyTexturePool();
        }
        this._gaussianScale = 0.0;
        this.queue_repaint();
        return;
      }

      const sigmaTexel = Math.min(radius / RES_SCALE, MAX_SIGMA_TEXEL);

      // Use a sigma floored at MIN_SIGMA_TEXEL to decide the kernel shape
      // (fetch-pair count), so a wide-enough kernel gets compiled even for
      // small radii.
      const kernelSigma = Math.max(sigmaTexel, MIN_SIGMA_TEXEL);

      // Number of one-sided taps needed to satisfy the 4-sigma rule (changed from 3
      // to prevent abrupt truncation ringing/grid artifacts at integer multiples),
      // converted to fetch pairs (2 taps per fetch). At least 2 pairs (5-tap equivalent)
      // are guaranteed so bilinear-downsample aliasing is reliably absorbed.
      const sideTaps = Math.max(2, Math.ceil(kernelSigma * 4));
      const fetchPairs = Math.max(2, Math.ceil(sideTaps / 2));

      const needsRecompile =
        this._gaussianFetchPairs !== fetchPairs ||
        (!this._gaussianKernel && !this._pendingGaussianKernel);

      if (needsRecompile) {
        const kernel = this._computeGaussianKernel(kernelSigma, fetchPairs);
        this._pendingGaussianKernel = kernel;
        this._gaussianPipelineDirty = true;
        this._gaussianFetchPairs = fetchPairs;
        this._gaussianBaseSigma = kernelSigma;
        // kernel_scale = actual sigma / sigma at compile time.
        // When sigmaTexel < kernelSigma, scale < 1.0, giving a weaker blur.
        this._gaussianScale = sigmaTexel / kernelSigma;
      } else {
        // Fetch count (shader structure) is unchanged — only update
        // kernel_scale and skip the recompile.
        this._gaussianScale = this._gaussianBaseSigma > 0
          ? sigmaTexel / this._gaussianBaseSigma
          : 1.0;
      }

      // Gaussian always uses a single level (w/2 × h/2).
      // A pool rebuild is only needed when PASS_COUNT transitions 0 → 1
      // (recovering from a disabled-blur state).
      if (this.PASS_COUNT !== 1) {
```

```
      this.PASS_COUNT = 1;
      // Only force a rebuild if the pool wasn't built yet, or previously had
      // a different number of levels (e.g. coming from Dual Kawase). The
      // actual rebuild happens next frame once vfunc_paint_target notices
      // the resolution mismatch.
      this._destroyTexturePool();
    }

    this.queue_repaint();
  }

  /**
   * Radius setter for the Dual Kawase blur (original implementation, logic unchanged).
   */
  private _setDualKawaseBlurRadius(radius: number): void {
    let newPassCount = 0;
    let offsetDown = 0.0;
    let offsetUp = 0.0;

    if (radius > 0) {
      // 1. Derive the optimal integer pass count P from the physical radius R
      //    (empirical blur-falloff model).
      let p = Math.floor(Math.log2(radius + 1));

      // Clamp the pass count to the shader/FBO limit of [1, 4].
      newPassCount = Math.max(1, Math.min(4, p));

      // 2. Compute a linear normalized progress t within the pass interval.
      let baseR = (newPassCount === 1) ? 0 : Math.pow(2, newPassCount) - 1;
      let nextR = Math.pow(2, newPassCount + 1) - 1;

      let t = (radius - baseR) / (nextR - baseR);
      t = Math.max(0.0, Math.min(1.0, t));

      // 3. A piecewise cubic Hermite spline, chosen for C1 continuity.
      let s = 0.25 * Math.pow(t, 3) - 0.75 * Math.pow(t, 2) + 1.5 * t;

      // 4. Map to an offset range that guarantees anti-aliasing.
      let minOffset = (newPassCount === 1) ? 0.0 : 0.5;
      let maxOffset = 1.0;

      let r = minOffset + s * (maxOffset - minOffset);

      offsetDown = r;
      offsetUp = r * 1.5;
    }

    // Check whether anything actually changed.
    if (this.PASS_COUNT !== newPassCount ||
      this._blurRadiusDown !== offsetDown ||
      this._blurRadiusUp !== offsetUp) {

      const passCountChanged = this.PASS_COUNT !== newPassCount;

      this.PASS_COUNT = newPassCount;
      this._blurRadiusDown = offsetDown;
      this._blurRadiusUp = offsetUp;

      // A pass-count change requires rebuilding the FBO pool.
      if (passCountChanged) {
        this._destroyTexturePool();
      }

      this.queue_repaint();
    }
  }
});

export type LiquidEffect = InstanceType<typeof LiquidEffect>;
```

```
import Gio from "gi://Gio"

// Logger class. Used by other classes and initialized in extension.js.
export class Logger {
  private _settings: Gio.Settings;
  private _outputLogs: boolean;
  private _settingsIds: number[] = [];

  constructor(settings: Gio.Settings) {
    this._settings = settings;
    this._outputLogs = this._settings.get_boolean('output-logs');
    this._bindSettings();
  }

  _bindSettings() {

    const connectSetting = (key: string, callback: Function) => {
      let id = this._settings.connect(`changed::${key}`, callback.bind(this));
      this._settingsIds.push(id);
    };
    connectSetting('output-logs', () => {
      this._outputLogs = this._settings.get_boolean('output-logs');
    });
  }

  log(...args: any[]) {
    if (!this._outputLogs) return;
    console.log(...args);
  }

  error(...args: any[]) {
    if (!this._outputLogs) return;
    console.error(...args);
  }

  warn(...args: any[]) {
    if (!this._outputLogs) return;
    console.warn(...args);
  }

  debug(...args: any[]) {
    if (!this._outputLogs) return;
    console.debug(...args);
  }

  cleanup() {
    if (this._settings) {
      for (const id of this._settingsIds) {
        this._settings.disconnect(id);
      }
      this._settingsIds = [];
    }
  }
}
```

```
// src/notificationManager.ts
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import Clutter from 'gi://Clutter';
import St from 'gi://St';
import GLib from 'gi://GLib';
import Meta from 'gi://Meta';
import { LiquidEffect } from './liquidEffect.js';
import { StageContrastSampler, AdaptiveContrastConfig } from './contrastSampler.js';
import Gio from 'gi://Gio';
import {
  UnpickableActor,
  UILayerSampler,
  WindowCloneManager,
} from './utils.js';

import { Logger } from './logger.js';


// ========== Configuration Parameters (Defaults, overridden by settings) ==========
const SHADER_PADDING = 20;
const HIDE_SAFETY_MARGIN = 7;

interface CustomBannerActor extends St.Widget {
  _colorTweenId?: number;
  _currentTargetColor?: string;
}

export class NotificationManager {
  private extensionPath: string;
  private _settings: Gio.Settings;
  private _logger: Logger;
  private tray: St.Widget;

  private currentBanner: St.Widget | null = null;

  // Full-screen FBO actor hierarchy (matches dockManager pattern)
  //   bgActor (full monitor, no effect)
  //     └─ liquidBox  ← LiquidEffect with built-in dual-Kawase blur
  //          ├─ _cloneContainer ← WindowCloneManager + UILayerSampler deposits here
  //          └─ dummyBreaker (prevents BMS black-screen optimization bug)
  private bgActor: Clutter.Actor | null = null;
  private liquidBox: Clutter.Actor | null = null;
  private _cloneContainer: Clutter.Actor | null = null;
  private effect: LiquidEffect | null = null;

  private _windowCloneManager: WindowCloneManager | null = null;
  private _uiSampler: UILayerSampler | null = null;

  private _signals: number[];
  private _settingsSignals: number[];
  private _frameSyncId: number;
  private _isEffectActive: boolean;

  private _stableBaseW: number | undefined;
  private _lastBgW: number | undefined;
  private _lastBgH: number | undefined;
  private _lastBgX: number | undefined;
  private _lastBgY: number | undefined;

  private _lastScreenW: number | undefined;
  private _lastScreenH: number | undefined;

  private _contrastSampler: StageContrastSampler;
  private _adaptiveConfig: typeof AdaptiveContrastConfig;
  private _adaptiveTimerId: number;
  private _adaptiveInFlight: boolean;
  private _styledActors: Map<Clutter.Actor, string>;

  private _glassExpand: number;
  private _baseTint: number;
  private _currentTint: number;
  private _notificationYOffset: number;

  private _isFirstAdaptiveRun: boolean = true;

  constructor(extensionPath: string, settings: Gio.Settings, logger: Logger) {
    this.extensionPath = extensionPath;
    this._settings = settings;
    this._logger = logger;
    this.tray = Main.messageTray;

    this._signals = [];
```

```
    this._settingsSignals = [];
    this._frameSyncId = 0;
    this._isEffectActive = false;

    this._contrastSampler = new StageContrastSampler();
    this._adaptiveConfig = {
      ...AdaptiveContrastConfig,
      enabled: true,
      samplePerElement: false,
      sampleIntervalMs: 400,
    };
    this._adaptiveTimerId = 0;
    this._adaptiveInFlight = false;
    this._styledActors = new Map();

    this._glassExpand = 12;
    this._baseTint = 0.08;
    this._currentTint = 0.08;
    this._notificationYOffset = 10;
  }

  setup() {
    if (!this._settings) return;
    this._bindSettings();

    if (this._settings.get_boolean('enable-notification-glass')) {
      this._applyEffect();
    }
  }

  // Utility: Convert HEX color string to normalized RGB array
  _hexToColorArray(hex: string): [number, number, number] {
    if (!hex || typeof hex !== 'string' || !hex.startsWith('#') || hex.length !== 7)
      return [1.0, 1.0, 1.0];
    let r = parseInt(hex.slice(1, 3), 16) / 255.0;
    let g = parseInt(hex.slice(3, 5), 16) / 255.0;
    let b = parseInt(hex.slice(5, 7), 16) / 255.0;
    return [r, g, b];
  }

  _bindSettings() {
    const connectSetting = (key: string, callback: Function) => {
      let id = this._settings.connect(`changed::${key}`, callback.bind(this));
      this._settingsSignals.push(id);
    };

    connectSetting('enable-notification-glass', () => {
      let enabled = this._settings.get_boolean('enable-notification-glass');
      if (enabled && !this._isEffectActive) this._applyEffect();
      else if (!enabled && this._isEffectActive) this._removeEffect();
    });

    connectSetting('notification-tint-color', () => {
      if (this.effect && this._isEffectActive) {
        let colorArray = this._hexToColorArray(this._settings.get_string('notification-tint-color'));
        this.effect.setTintColor(...colorArray);
      }
    });

    connectSetting('notification-tint-strength', () => {
      if (this.effect && this._isEffectActive) {
        this._baseTint = this._settings.get_double('notification-tint-strength');
        this._currentTint = this._baseTint;
        this.effect.setTintStrength(this._baseTint);
      }
    });

    connectSetting('notification-blur-radius', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setBlurRadius(this._settings.get_int('notification-blur-radius'));
      }
    });

    connectSetting('notification-corner-radius', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setCornerRadius(this._settings.get_double('notification-corner-radius'));
      }
    });

    connectSetting('notification-glass-expand', () => {
      if (this._isEffectActive) {
        this._glassExpand = this._settings.get_int('notification-glass-expand');
```

```
      }
    });

    // Brightness / Saturation / Contrast — dynamic application from settings
    connectSetting('notification-brightness', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setBrightness(this._settings.get_double('notification-brightness'));
      }
    });

    connectSetting('notification-saturation', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setSaturation(this._settings.get_double('notification-saturation'));
      }
    });

    connectSetting('notification-contrast', () => {
      if (this.effect && this._isEffectActive) {
        this.effect.setContrast(this._settings.get_double('notification-contrast'));
      }
    });

    connectSetting('notification-enable-adaptive-text-color', () => {
      this._adaptiveConfig.enabled = this._settings.get_boolean('notification-enable-adaptive-text-color');
    });

    connectSetting('notification-sample-interval-ms', () => {
      this._adaptiveConfig.sampleIntervalMs = this._settings.get_int('notification-sample-interval-ms');
    });

    connectSetting('notification-y-offset', () => {
      this._notificationYOffset = this._settings.get_int('notification-y-offset');
    });
  }

  _applyEffect() {
    if (this._isEffectActive) return;
    this._isEffectActive = true;

    // @ts-expect-error: _bannerBin is an internal property
    let bannerBin = this.tray._bannerBin;
    if (!bannerBin) {
      this._logger.error('[Liquid Glass] _bannerBin is not found. GNOME internal structure might have changed.');
      return;
    }

    // Apply settings initially
    this._adaptiveConfig.enabled = this._settings.get_boolean('notification-enable-adaptive-text-color');
    this._adaptiveConfig.sampleIntervalMs = this._settings.get_int('notification-sample-interval-ms');
    this._glassExpand = this._settings.get_int('notification-glass-expand');
    this._baseTint = this._settings.get_double('notification-tint-strength');
    this._currentTint = this._baseTint;
    this._notificationYOffset = this._settings.get_int('notification-y-offset');

    // Listen for new notifications
    this._signals.push(bannerBin.connect('child-added', (container, actor: St.Widget) => {
      if (actor === this.bgActor || actor.get_name?.() === 'liquid-glass-bg-actor') return;

      GLib.idle_add(GLib.PRIORITY_DEFAULT, () => {
        // @ts-expect-error: _banner is an internal property
        let banner = this.tray._banner || actor;
        if (banner && banner !== this.currentBanner) {
          this._cleanupCurrentBanner();
          this.currentBanner = banner;
          this._setupBannerEffect(banner);
        }
        return GLib.SOURCE_REMOVE;
      });
    }));

    this._signals.push(bannerBin.connect('child-removed', (container, actor: St.Widget) => {
      if (actor === this.bgActor || actor.get_name?.() === 'liquid-glass-bg-actor') return;
      this._cleanupCurrentBanner();
    }));

    // @ts-expect-error
    if (this.tray._banner) {
      // @ts-expect-error
      this.currentBanner = this.tray._banner;
      // @ts-expect-error
      this._setupBannerEffect(this.tray._banner);
    }
```

```
  }

  _setupBannerEffect(targetActor: St.Widget) {
    targetActor.add_style_class_name('liquid-glass-transparent');

    // @ts-expect-error
    if (this.tray._bannerBin) {
      // @ts-expect-error
      this.tray._bannerBin.translation_y = this._settings.get_int('notification-y-offset');
    }

    // ── 1. bgActor: full monitor, no effect ──────────────────────────────
    this.bgActor = new UnpickableActor();
    this.bgActor.set_name('liquid-glass-bg-actor');
    this.bgActor.set_size(1.0, 1.0);
    this.bgActor.set_pivot_point(0.0, 0.0);

    // ── 2. liquidBox: outer layer — LiquidEffect with built-in dual-Kawase blur ─
    this.liquidBox = new UnpickableActor();
    this.liquidBox.set_name('liquid-box');
    this.liquidBox.set_clip_to_allocation(true);
    this.bgActor.add_child(this.liquidBox);

    // dummyBreaker: prevents BMS black-screen optimization bug
    let dummyBreaker = new UnpickableActor();
    dummyBreaker.set_name('optimization-breaker');
    dummyBreaker.set_size(1.0, 1.0);
    dummyBreaker.set_opacity(0);
    this.liquidBox.add_child(dummyBreaker);

    // ── 3. _cloneContainer: sub-container inside liquidBox ──────────────────
    this._cloneContainer = new UnpickableActor();
    this._cloneContainer.set_name('clone-container');
    this.liquidBox.add_child(this._cloneContainer);

    // ── Find the bannerBin's ancestor that is a direct child of uiGroup ──────
    // @ts-expect-error
    let bannerBin = this.tray._bannerBin;
    let bannerRoot: Clutter.Actor = bannerBin ?? targetActor;
    while (bannerRoot.get_parent() && bannerRoot.get_parent() !== Main.layoutManager.uiGroup) {
      const p = bannerRoot.get_parent();
      if (!p) break;
      bannerRoot = p;
    }

    // Insert bgActor below the notification root in uiGroup to prevent recursive
    // clone loops (same pattern as dockManager / uiManager)
    if (bannerRoot.get_parent() === Main.layoutManager.uiGroup) {
      Main.layoutManager.uiGroup.insert_child_below(this.bgActor, bannerRoot);
    } else {
      Main.layoutManager.uiGroup.add_child(this.bgActor);
    }

    // ── 4. Read effect parameters from settings ──────────────────────────────
    let blurRadius = this._settings.get_int('notification-blur-radius');
    let tintColorStr = this._settings.get_string('notification-tint-color');
    let cornerRadius = this._settings.get_double('notification-corner-radius');
    let tintStrength = this._settings.get_double('notification-tint-strength');
    let brightness = this._settings.get_double('notification-brightness');
    let saturation = this._settings.get_double('notification-saturation');
    let contrast = this._settings.get_double('notification-contrast');
    this._baseTint = tintStrength;

    // LiquidEffect on liquidBox (includes built-in dual-Kawase blur)
    this.effect = new LiquidEffect({ extensionPath: this.extensionPath, settings: this._settings } as any);
    this.effect.setPadding(SHADER_PADDING);
    this.effect.setTintColor(...this._hexToColorArray(tintColorStr));
    this.effect.setTintStrength(this._baseTint);
    this.effect.setCornerRadius(cornerRadius);
    this.effect.setIsDock(false);
    this.effect.setBrightness(brightness);
    this.effect.setSaturation(saturation);
    this.effect.setContrast(contrast);
    this.effect.setBlurRadius(blurRadius);
    this.liquidBox.add_effect(this.effect);

    // ── 5. WindowCloneManager + UILayerSampler ──────────────────────────────
    this._windowCloneManager = new WindowCloneManager(this.liquidBox, this._cloneContainer);
    this._uiSampler = new UILayerSampler(
      this.bgActor,
      this.liquidBox,
      [bannerRoot, global.windowGroup, global.window_group],
```

```
      this._cloneContainer
    );

    this.bgActor.show();

    // Initial clone build (also applies liquid-glass mutual exclusions)
    this._buildClones();

    // ── 7. Frame-render loop ──────────────────────────────────────────
    const frameLaterType = Meta.LaterType.BEFORE_REDRAW;
    const frameTick = () => {
      this._frameSyncId = 0;
      if (!this.bgActor || !this.currentBanner) return GLib.SOURCE_REMOVE;

      this._syncGeometry();

      // Hover tint animation (notification-specific behaviour preserved)
      let isHovered = this.currentBanner.hover;
      let targetTint = isHovered ? (this._baseTint + 0.1) : this._baseTint;
      if (Math.abs(this._currentTint - targetTint) > 0.001) {
        this._currentTint += (targetTint - this._currentTint) * 0.1;
        this.effect?.setTintStrength(this._currentTint);
      }

      this._frameSyncId = this._laterAdd(frameLaterType, frameTick);
      return GLib.SOURCE_REMOVE;
    };

    this._frameSyncId = this._laterAdd(frameLaterType, frameTick);
    this._isFirstAdaptiveRun = true;
    this._startAdaptiveColorSampling();
  }

  // ── Geometry synchronisation ──────────────────────────────────────
  // Called every frame. Uses full-screen FBO architecture so that BMS coordinate
  // assumptions are satisfied (all actors cover the entire monitor).
  _syncGeometry() {
    if (!this.bgActor || !this.currentBanner) return;

    let [w, h] = this.currentBanner.get_size();
    let [absX, absY] = this.currentBanner.get_transformed_position();
    if (Number.isNaN(absX) || Number.isNaN(absY)) return;

    this.bgActor.opacity = this.currentBanner.opacity;

    let themeNode = this.currentBanner.get_theme_node();
    let mL = themeNode ? themeNode.get_margin(St.Side.LEFT) : 0;
    let mR = themeNode ? themeNode.get_margin(St.Side.RIGHT) : 0;
    let mT = themeNode ? themeNode.get_margin(St.Side.TOP) : 0;
    let mB = themeNode ? themeNode.get_margin(St.Side.BOTTOM) : 0;

    // Hide when the notification banner has slid completely off-screen
    if (absY + h <= mB + HIDE_SAFETY_MARGIN) {
      this.bgActor.hide();
      return;
    } else if (!this.bgActor.visible) {
      this.bgActor.show();
    }

    // Margin-bloat compensation (same logic as before, now used only for shader geometry)
    let marginW = mL + mR;
    let marginH = mT + mB;

    if (this._stableBaseW === undefined) {
      this._stableBaseW = w;
    }
    if (Math.abs(this._stableBaseW - (w + marginW)) <= 1) {
      this._stableBaseW = w;
    }

    let isBloated = Math.abs(w - (this._stableBaseW + marginW)) <= 1;
    let visualW = isBloated ? w - marginW : w;
    let visualH = isBloated ? h - marginH : h;
    if (!isBloated) this._stableBaseW = w;

    let visualX = absX;
    let visualY = absY;

    let bgW = visualW + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgH = visualH + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgX_abs = visualX - this._glassExpand - SHADER_PADDING;
    let bgY_abs = visualY - this._glassExpand - SHADER_PADDING;
```

```
    // ── Monitor geometry ──────────────────────────────────────────
    let monitorIndex = Main.layoutManager.findIndexForActor(this.tray);
    if (monitorIndex < 0) monitorIndex = Main.layoutManager.primaryIndex;
    let monitor = Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;

    let monitorX = monitor?.x ?? 0;
    let monitorY = monitor?.y ?? 0;
    let screenW = Math.max(1, monitor?.width ?? 1);
    let screenH = Math.max(1, monitor?.height ?? 1);

    // Monitor-local coordinates (shader uses these)
    let localBgX = bgX_abs - monitorX;
    let localBgY = bgY_abs - monitorY;

    // ── Update actors only when geometry actually changed ─────────
    if (this._lastBgW !== bgW || this._lastBgH !== bgH ||
      this._lastBgX !== bgX_abs || this._lastBgY !== bgY_abs ||
      this._lastScreenW !== screenW || this._lastScreenH !== screenH) {

      // bgActor: full monitor size, positioned at monitor origin
      this.bgActor.remove_transition('size');
      this.bgActor.remove_transition('position');
      this.bgActor.set_position(monitorX, monitorY);
      this.bgActor.set_size(screenW, screenH);
      this.bgActor.remove_transition('size');
      this.bgActor.remove_transition('position');

      // liquidBox fills the entire bgActor
      this.liquidBox?.set_position(0, 0);
      this.liquidBox?.set_size(screenW, screenH);

      // Soft clip — limits GPU work to the notification area + generous margin
      const CLIP_PADDING = 200;
      this.liquidBox?.remove_clip();
      this.bgActor.set_clip(
        localBgX - CLIP_PADDING, localBgY - CLIP_PADDING,
        bgW + CLIP_PADDING * 2, bgH + CLIP_PADDING * 2
      );

      const SHADOW_MAX_RADIUS = CLIP_PADDING - 20;
      this.effect?.setShadowMaxRadius(SHADOW_MAX_RADIUS);

      // Inform the shader of the full-screen resolution and where the
      // notification lives within the FBO (mirrors dockManager.setGlassGeometry)
      this.effect?.setResolution(screenW, screenH);
      this.effect?.setGlassGeometry(localBgX, localBgY, bgW, bgH);

      this._lastBgW = bgW; this._lastBgH = bgH;
      this._lastBgX = bgX_abs; this._lastBgY = bgY_abs;
      this._lastScreenW = screenW; this._lastScreenH = screenH;
    }

    // ── Sync clones every frame (dockManager pattern) ─────────────
    this._windowCloneManager?.setOffset(-monitorX, -monitorY);
    this._uiSampler?.refresh();
    this._uiSampler?.sync(monitorX, monitorY, screenW, screenH);
    this._windowCloneManager?.sync();
  }

  // Called once when the banner effect is first set up (and after monitor changes).
  // Applies mutual exclusions between multiple Liquid Glass bgActors, then
  // delegates clone construction to WindowCloneManager + UILayerSampler.
  _buildClones() {
    if (!this.bgActor) return;

    if (this._uiSampler) {
      for (let child of Main.layoutManager.uiGroup.get_children()) {
        if (child === this.bgActor) continue;
        let isLiquidBg = child.get_name?.() === 'liquid-glass-bg-actor' ||
          (typeof child.get_children === 'function' &&
            child.get_children().some((c: Clutter.Actor) => c.get_name?.() === 'liquid-box'));
        if (isLiquidBg) this._uiSampler.addExclusion(child);
      }
    }

    this._windowCloneManager?.rebuildClones();
    this._uiSampler?.rebindSelf();
    this._uiSampler?.refresh();
  }

  // ── Per-banner cleanup ──────────────────────────────────────────
```

```
  _cleanupCurrentBanner() {
    this._stopAdaptiveColorSampling();
    this._clearAdaptiveStyles();

    // @ts-expect-error
    if (this.tray._bannerBin) {
      // @ts-expect-error
      this.tray._bannerBin.translation_y = 0;
    }

    if (this.currentBanner) {
      this.currentBanner.remove_style_class_name('liquid-glass-transparent');
      this.currentBanner.translation_y = 0;
      this.currentBanner = null;
    }

    if (this._frameSyncId !== 0) {
      if (global.compositor?.get_laters) global.compositor.get_laters().remove(this._frameSyncId);
      this._frameSyncId = 0;
    }

    // DESTROY EFFECT FIRST (must happen before bgActor.destroy())
    if (this.effect) {
      this.effect.cleanup();
      this.effect = null;
    }

    // DESTROY ACTOR HIERARCHY — bgActor.destroy() cascades through
    // liquidBox → _cloneContainer and all their children.
    if (this.bgActor) {
      this.bgActor.destroy();
      this.bgActor = null;
    }
    this.liquidBox = null;
    this._cloneContainer = null;

    // Clean up managers (their destroy() guards against already-destroyed actors)
    this._uiSampler?.destroy();
    this._uiSampler = null;
    this._windowCloneManager?.destroy();
    this._windowCloneManager = null;

    // Reset cached geometry state
    this._lastBgW = undefined;
    this._lastBgH = undefined;
    this._lastBgX = undefined;
    this._lastBgY = undefined;
    this._lastScreenW = undefined;
    this._lastScreenH = undefined;
    this._stableBaseW = undefined;

    this._isFirstAdaptiveRun = true;
  }

  // ── Effect remove / cleanup ──────────────────────────────────────
  _removeEffect() {
    if (!this._isEffectActive) return;
    this._isEffectActive = false;

    // @ts-expect-error
    let bannerBin = this.tray._bannerBin;
    for (let sigId of this._signals) {
      try { bannerBin.disconnect(sigId); } catch (e) { }
    }
    this._signals = [];

    this._cleanupCurrentBanner();
  }

  cleanup() {
    for (let sigId of this._settingsSignals) {
      this._settings.disconnect(sigId);
    }
    this._settingsSignals = [];
    this._removeEffect();
  }

  // ── Adaptive text colour helpers (unchanged logic) ──────────────

  _collectAdaptiveTextTargets(actor: Clutter.Actor | null = this.currentBanner, targets: Clutter.Actor[] = []):
Clutter.Actor[] {
    if (!actor) return targets;
```

```
    return this._findAllTextActors(actor);
  }

  _setActorColor(actor: CustomBannerActor, color: string, skipAnimations = false) {
    if (!actor || typeof actor.set_style !== 'function') return;
    if (actor._currentTargetColor === color) return;
    actor._currentTargetColor = color;
    this._animateActorColor(actor, color, 380, skipAnimations);
  }

  _clearAdaptiveStyles() {
    for (const [actor, style] of this._styledActors.entries() as MapIterator<[CustomBannerActor, string]>) {
      if (actor && typeof actor.set_style === 'function') {
        if (actor._colorTweenId !== undefined) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        actor._currentTargetColor = undefined;
        actor.remove_style_class_name('adaptive-text-transition');
        actor.remove_style_class_name('adaptive-color-light');
        actor.remove_style_class_name('adaptive-color-dark');
        actor.set_style(style);
      }
    }
    this._styledActors.clear();
  }

  _applyAdaptiveColorMap(colorMap: Map<Clutter.Actor, string>, skipAnimations = false) {
    if (!colorMap || colorMap.size === 0) return;
    for (const [actor, color] of colorMap.entries()) {
      this._setActorColor(actor as unknown as CustomBannerActor, color, skipAnimations);
    }
  }

  _startAdaptiveColorSampling() {
    if (!this._adaptiveConfig.enabled) return;
    this._updateAdaptiveTextColors();

    if (this._adaptiveTimerId !== 0) return;

    this._adaptiveTimerId = GLib.timeout_add(
      GLib.PRIORITY_DEFAULT,
      this._adaptiveConfig.sampleIntervalMs,
      () => {
        if (!this.currentBanner || !this.bgActor) {
          this._adaptiveTimerId = 0;
          return GLib.SOURCE_REMOVE;
        }
        this._updateAdaptiveTextColors();
        return GLib.SOURCE_CONTINUE;
      }
    );
  }

  _stopAdaptiveColorSampling() {
    if (this._adaptiveTimerId !== 0) {
      GLib.source_remove(this._adaptiveTimerId);
      this._adaptiveTimerId = 0;
    }
  }

  _findAllTextActors(actor: Clutter.Actor, foundActors: Clutter.Actor[] = []) {
    if (!actor) return foundActors;

    if (actor instanceof St.Label || actor instanceof Clutter.Text || actor instanceof St.Button) {
      if (actor.visible) foundActors.push(actor);
    }

    let children = actor.get_children();
    for (let i = 0; i < children.length; i++) {
      this._findAllTextActors(children[i], foundActors);
    }
    return foundActors;
  }

  _updateAdaptiveTextColors() {
    if (!this._adaptiveConfig.enabled || this._adaptiveInFlight) return;

    let [absX, absY] = this.currentBanner?.get_transformed_position() ?? [0, 0];
    if (absY < 0) return;

    const targets = this._collectAdaptiveTextTargets();
```

```
    if (targets.length === 0) return;

    this._adaptiveInFlight = true;

    this._contrastSampler
      .chooseColorsForActors(targets, this._adaptiveConfig)
      .then(colorMap => {
        this._applyAdaptiveColorMap(colorMap, this._isFirstAdaptiveRun);
        this._isFirstAdaptiveRun = false;
      })
      .catch(e => {
        this._logger.error(`[Liquid Glass] Notification adaptive color update failed: ${e}`);
      })
      .finally(() => {
        this._adaptiveInFlight = false;
      });
  }

  _hexToRgb(hex: string) {
    let bigint = parseInt(hex.replace('#', ''), 16);
    return { r: (bigint >> 16) & 255, g: (bigint >> 8) & 255, b: bigint & 255 };
  }

  _rgbToHex(r: number, g: number, b: number) {
    return '#' + (1 << 24 | r << 16 | g << 8 | b).toString(16).slice(1);
  }

  _animateActorColor(actor: CustomBannerActor, targetHexColor: string, durationMs = 380, skipAnimations = false) {
    if (!actor || Object.keys(actor).length === 0) return;

    if (actor._colorTweenId) {
      GLib.source_remove(actor._colorTweenId);
      actor._colorTweenId = undefined;
    }

    let themeNode = actor.get_theme_node();
    let startColor = themeNode.get_foreground_color();
    let targetRgb = this._hexToRgb(targetHexColor);
    let startTime = GLib.get_monotonic_time();

    if (skipAnimations) {
      actor.set_style(`color: ${targetHexColor}; -st-icon-foreground-color: ${targetHexColor};`);
      return;
    }

    actor._colorTweenId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 16, () => {
      if (!actor || Object.keys(actor).length === 0) return GLib.SOURCE_REMOVE;

      let currentTime = GLib.get_monotonic_time();
      let elapsedMs = (currentTime - startTime) / 1000;
      let progress = Math.min(elapsedMs / durationMs, 1.0);
      let ease = progress < 0.5
        ? 2 * progress * progress
        : 1 - Math.pow(-2 * progress + 2, 2) / 2;

      let r = Math.round(startColor.red + (targetRgb.r - startColor.red) * ease);
      let g = Math.round(startColor.green + (targetRgb.g - startColor.green) * ease);
      let b = Math.round(startColor.blue + (targetRgb.b - startColor.blue) * ease);
      actor.set_style(`color: ${this._rgbToHex(r, g, b)}; -st-icon-foreground-color: ${this._rgbToHex(r, g, b)};`);

      if (progress >= 1.0) {
        actor._colorTweenId = undefined;
        return GLib.SOURCE_REMOVE;
      }
      return GLib.SOURCE_CONTINUE;
    });
  }

  _hasStyleClass(actor: St.Widget, className: string) {
    return typeof actor?.has_style_class_name === 'function' &&
      actor.has_style_class_name(className);
  }

  _laterAdd(laterType: Meta.LaterType, callback: GLib.SourceFunc) {
    return global.compositor?.get_laters?.().add(laterType, callback);
  }
}
```

```
// src/osdManager.ts
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import Clutter from 'gi://Clutter';
import St from 'gi://St';
import GLib from 'gi://GLib';
import Meta from 'gi://Meta';
import { LiquidEffect } from './liquidEffect.js';
import { StageContrastSampler, AdaptiveContrastConfig } from './contrastSampler.js';
import Gio from 'gi://Gio';
import {
  UnpickableActor,
  UILayerSampler,
  WindowCloneManager,
} from './utils.js';

import { Logger } from './logger.js';

interface CustomBannerActor extends St.Widget {
  _colorTweenId?: number;
  _currentTargetColor?: string;
}

// ── Per-monitor OSD state ──────────────────────────────────────
interface OsdState {
  osdWindow: any;   // GNOME internal OsdWindow (no strict type)
  targetBox: St.Widget;

  bgActor: Clutter.Actor | null;

  // Full-screen FBO actor hierarchy
  liquidBox: Clutter.Actor | null;  // LiquidEffect (with built-in blur) lives here
  _cloneContainer: Clutter.Actor | null;

  effect: LiquidEffect | null;

  // Clone managers (replace manual clone tracking)
  _windowCloneManager: WindowCloneManager | null;
  _uiSampler: UILayerSampler | null;

  // Shader geometry cache
  _lastBgW?: number;
  _lastBgH?: number;
  _lastBgX?: number;
  _lastBgY?: number;

  // Monitor size cache for change detection
  _lastScreenW?: number;
  _lastScreenH?: number;

  // OSD-specific state
  _stableBaseH?: number;
  _currentTint: number;
  _wasVisible: boolean;
  _isFirstAdaptiveRun: boolean;
  _destroyId: number;
}

// ========== Configuration Parameters (Defaults, overridden by settings) ==========
const SHADER_PADDING = 20;

export class OsdManager {
  private extensionPath: string;
  private _settings: Gio.Settings;
  private _logger: Logger;

  private _settingsSignals: number[];
  private _frameSyncId: number;
  private _isEffectActive: boolean;

  private _osdYOffset: number;
  private _osdStates: OsdState[];
  private _baseTint: number;
  private _glassExpand: number;
  private _monitorsChangedId: number;
  private _contrastSampler: StageContrastSampler;
  private _adaptiveConfig: typeof AdaptiveContrastConfig;
  private _adaptiveTimerId: number;
  private _adaptiveInFlight: boolean;
  private _styledActors: Map<Clutter.Actor, string>;
  private _isFirstAdaptiveRun: boolean;

  constructor(extensionPath: string, settings: Gio.Settings, logger: Logger) {
```

```
    this.extensionPath = extensionPath;
    this._settings = settings;
    this._logger = logger;

    this._osdStates = [];
    this._settingsSignals = [];
    this._frameSyncId = 0;
    this._monitorsChangedId = 0;
    this._isEffectActive = false;

    this._contrastSampler = new StageContrastSampler();
    this._adaptiveConfig = {
      ...AdaptiveContrastConfig,
      enabled: true,
      samplePerElement: false,
      sampleIntervalMs: 400,
    };
    this._adaptiveTimerId = 0;
    this._adaptiveInFlight = false;
    this._styledActors = new Map();

    this._glassExpand = 12;
    this._baseTint = 0.08;
    this._osdYOffset = 0;
    this._isFirstAdaptiveRun = true;
  }

  setup() {
    if (!this._settings) return;
    this._bindSettings();

    if (this._settings.get_boolean('enable-osd-glass')) {
      this._applyEffect();
    }
  }

  _hexToColorArray(hex: string): [number, number, number] {
    if (!hex || typeof hex !== 'string' || !hex.startsWith('#') || hex.length !== 7)
      return [1.0, 1.0, 1.0];
    let r = parseInt(hex.slice(1, 3), 16) / 255.0;
    let g = parseInt(hex.slice(3, 5), 16) / 255.0;
    let b = parseInt(hex.slice(5, 7), 16) / 255.0;
    return [r, g, b];
  }

  _bindSettings() {
    const connectSetting = (key: string, callback: Function) => {
      let id = this._settings.connect(`changed::${key}`, callback.bind(this));
      this._settingsSignals.push(id);
    };

    connectSetting('enable-osd-glass', () => {
      let enabled = this._settings.get_boolean('enable-osd-glass');
      if (enabled && !this._isEffectActive) this._applyEffect();
      else if (!enabled && this._isEffectActive) this._removeEffect();
    });

    connectSetting('osd-tint-color', () => {
      if (this._isEffectActive) {
        let colorArray = this._hexToColorArray(this._settings.get_string('osd-tint-color'));
        for (let state of this._osdStates) {
          if (state.effect) state.effect.setTintColor(...colorArray);
        }
      }
    });

    connectSetting('osd-tint-strength', () => {
      if (this._isEffectActive) {
        this._baseTint = this._settings.get_double('osd-tint-strength');
        for (let state of this._osdStates) {
          state._currentTint = this._baseTint;
          if (state.effect) state.effect.setTintStrength(this._baseTint);
        }
      }
    });

    connectSetting('osd-blur-radius', () => {
      if (this._isEffectActive) {
        let radius = this._settings.get_int('osd-blur-radius');
        for (let state of this._osdStates) {
          if (state.effect) state.effect.setBlurRadius(radius);
        }
```

```
      }
    });

    connectSetting('osd-corner-radius', () => {
      if (this._isEffectActive) {
        let radius = this._settings.get_double('osd-corner-radius');
        for (let state of this._osdStates) {
          if (state.effect) state.effect.setCornerRadius(radius);
        }
      }
    });

    connectSetting('osd-glass-expand', () => {
      if (this._isEffectActive) {
        this._glassExpand = this._settings.get_int('osd-glass-expand');
      }
    });

    // ----------  Brightness / Saturation / Contrast  ----------
    connectSetting('osd-brightness', () => {
      if (this._isEffectActive) {
        let v = this._settings.get_double('osd-brightness');
        for (let state of this._osdStates) {
          if (state.effect) state.effect.setBrightness(v);
        }
      }
    });

    connectSetting('osd-saturation', () => {
      if (this._isEffectActive) {
        let v = this._settings.get_double('osd-saturation');
        for (let state of this._osdStates) {
          if (state.effect) state.effect.setSaturation(v);
        }
      }
    });

    connectSetting('osd-contrast', () => {
      if (this._isEffectActive) {
        let v = this._settings.get_double('osd-contrast');
        for (let state of this._osdStates) {
          if (state.effect) state.effect.setContrast(v);
        }
      }
    });

    connectSetting('osd-enable-adaptive-text-color', () => {
      this._adaptiveConfig.enabled = this._settings.get_boolean('osd-enable-adaptive-text-color');
      if (this._adaptiveConfig.enabled) this._startAdaptiveColorSampling();
      else {
        this._stopAdaptiveColorSampling();
        this._clearAdaptiveStyles();
      }
    });

    connectSetting('osd-sample-interval-ms', () => {
      this._adaptiveConfig.sampleIntervalMs = this._settings.get_int('osd-sample-interval-ms');
    });

    connectSetting('osd-y-offset', () => {
      this._osdYOffset = this._settings.get_int('osd-y-offset');
      for (let state of this._osdStates) {
        if (state.targetBox) {
          state.targetBox.translation_y = -this._osdYOffset;
        }
      }
    });
  }

  _applyEffect() {
    if (this._isEffectActive) return;
    this._isEffectActive = true;

    this._adaptiveConfig.enabled = this._settings.get_boolean('osd-enable-adaptive-text-color');
    this._adaptiveConfig.sampleIntervalMs = this._settings.get_int('osd-sample-interval-ms');
    this._glassExpand = this._settings.get_int('osd-glass-expand');
    this._baseTint = this._settings.get_double('osd-tint-strength');
    this._osdYOffset = this._settings.get_int('osd-y-offset');

    let osdWindows = Main.osdWindowManager._osdWindows;
    if (!osdWindows) return;
```

```
    // Set up each monitor's OSD independently
    for (let osdWindow of osdWindows) {
      this._setupOsdEffect(osdWindow);
    }

    // After all states exist, apply mutual exclusions so each UILayerSampler
    // does not clone the other monitors' liquid-glass bgActors
    for (let state of this._osdStates) {
      if (!state._uiSampler) continue;
      // Add the bgActors of all OTHER states as exclusions
      for (let other of this._osdStates) {
        if (other !== state && other.bgActor) {
          state._uiSampler.addExclusion(other.bgActor);
        }
      }
      // Also exclude any other liquid-glass bgActors already in uiGroup
      for (let child of Main.layoutManager.uiGroup.get_children()) {
        if (child === state.bgActor) continue;
        let isLiquidBg = child.get_name?.() === 'liquid-glass-bg-actor' ||
          (typeof child.get_children === 'function' &&
            child.get_children().some((c: Clutter.Actor) => c.get_name?.() === 'liquid-box'));
        if (isLiquidBg) state._uiSampler.addExclusion(child);
      }
    }

    // Global frame-render loop (covers all OSD states)
    const frameLaterType = Meta.LaterType.BEFORE_REDRAW;
    const frameTick = () => {
      this._frameSyncId = 0;
      if (!this._isEffectActive) return GLib.SOURCE_REMOVE;

      for (let state of this._osdStates) {
        this._syncGeometry(state);
      }

      this._frameSyncId = this._laterAdd(frameLaterType, frameTick);
      return GLib.SOURCE_REMOVE;
    };

    this._frameSyncId = this._laterAdd(frameLaterType, frameTick);
    this._startAdaptiveColorSampling();

    // Rebuild everything if monitor configuration changes
    this._monitorsChangedId = Main.layoutManager.connect('monitors-changed', () => {
      this._removeEffect();
      if (this._settings.get_boolean('enable-osd-glass')) {
        this._applyEffect();
      }
    });
  }

  _setupOsdEffect(osdWindow) {
    // ── Find the OSD's targetBox (the St.Widget with style methods) ──────────
    let targetBox: St.Widget | null = null;
    if (osdWindow._icon && osdWindow._icon.get_parent) {
      targetBox = osdWindow._icon.get_parent();
    } else if (typeof osdWindow.get_first_child === 'function') {
      targetBox = osdWindow.get_first_child();
    }

    if (targetBox && typeof targetBox.add_style_class_name !== 'function') {
      let children = osdWindow.get_children();
      for (let child of children) {
        if (typeof child.add_style_class_name === 'function') {
          targetBox = child;
          break;
        }
      }
    }

    if (!targetBox || typeof targetBox.add_style_class_name !== 'function') {
      this._logger.warn('[Liquid Glass] OSD UI container not found.');
      return;
    }

    targetBox.add_style_class_name('liquid-glass-transparent');
    targetBox.translation_y = -this._osdYOffset;

    // ── Determine which monitor this OSD belongs to ──────────────────────────
    let monitorIndex = Main.layoutManager.findIndexForActor(osdWindow);
    if (monitorIndex < 0) monitorIndex = Main.layoutManager.primaryIndex;
    let monitor = Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;
```

```
// ── 1. bgActor: full monitor size, no effect ──────────────────────────
let bgActor = new UnpickableActor();
bgActor.set_name('liquid-glass-bg-actor');
bgActor.set_size(1.0, 1.0);
bgActor.set_pivot_point(0.0, 0.0);

// ── 2. liquidBox: outer layer — LiquidEffect with built-in dual-Kawase blur —
let liquidBox = new UnpickableActor();
liquidBox.set_name('liquid-box');
liquidBox.set_clip_to_allocation(true);
bgActor.add_child(liquidBox);

// dummyBreaker: prevents BMS black-screen optimization bug
let dummyBreaker = new UnpickableActor();
dummyBreaker.set_name('optimization-breaker');
dummyBreaker.set_size(1.0, 1.0);
dummyBreaker.set_opacity(0);
liquidBox.add_child(dummyBreaker);

// ── 3. _cloneContainer: sub-container inside liquidBox ────────────────
let cloneContainer = new UnpickableActor();
cloneContainer.set_name('clone-container');
liquidBox.add_child(cloneContainer);

// ── Find the OSD's ancestor that is a direct child of uiGroup ─────────
let osdRoot: Clutter.Actor = osdWindow;
while (osdRoot.get_parent() && osdRoot.get_parent() !== Main.layoutManager.uiGroup) {
  const p = osdRoot.get_parent();
  if (!p) break;
  osdRoot = p;
}

// Insert bgActor below the OSD root in uiGroup
if (osdRoot.get_parent() === Main.layoutManager.uiGroup) {
  Main.layoutManager.uiGroup.insert_child_below(bgActor, osdRoot);
} else {
  Main.layoutManager.uiGroup.add_child(bgActor);
}

// ── 4. Read effect parameters ─────────────────────────────────────────
let blurRadius = this._settings.get_int('osd-blur-radius');
let tintColorStr = this._settings.get_string('osd-tint-color');
let cornerRadius = this._settings.get_double('osd-corner-radius');
let brightness = this._settings.get_double('osd-brightness');
let saturation = this._settings.get_double('osd-saturation');
let contrast = this._settings.get_double('osd-contrast');

// LiquidEffect on liquidBox (includes built-in dual-Kawase blur)
let effect = new LiquidEffect({ extensionPath: this.extensionPath, settings: this._settings } as any);
effect.setPadding(SHADER_PADDING);
effect.setTintColor(...this._hexToColorArray(tintColorStr));
effect.setTintStrength(this._baseTint);
effect.setCornerRadius(cornerRadius);
effect.setIsDock(false);
effect.setBrightness(brightness);
effect.setSaturation(saturation);
effect.setContrast(contrast);
effect.setBlurRadius(blurRadius);
liquidBox.add_effect(effect);


bgActor.hide();

// ── 5. WindowCloneManager + UILayerSampler ────────────────────────────
let windowCloneManager = new WindowCloneManager(liquidBox, cloneContainer);
let uiSampler = new UILayerSampler(
  bgActor,
  liquidBox,
  [osdRoot, global.windowGroup, global.window_group],
  cloneContainer
);

// ── 7. Build initial clones ───────────────────────────────────────────
windowCloneManager.rebuildClones();
uiSampler.rebindSelf();
uiSampler.refresh();

// ── 6. Compose the state object ───────────────────────────────────────
let state: OsdState = {
  osdWindow,
  targetBox,
```

```
      bgActor,
      liquidBox,
      _cloneContainer: cloneContainer,
      effect,
      _windowCloneManager: windowCloneManager,
      _uiSampler: uiSampler,
      _lastBgW: undefined,
      _lastBgH: undefined,
      _lastBgX: undefined,
      _lastBgY: undefined,
      _lastScreenW: undefined,
      _lastScreenH: undefined,
      _stableBaseH: undefined,
      _currentTint: this._baseTint,
      _wasVisible: false,
      _isFirstAdaptiveRun: true,
      _destroyId: 0,
    };
    this._osdStates.push(state);

    // When the OSD window itself is destroyed, clean up our matching state
    state._destroyId = osdWindow.connect('destroy', () => {
      this._osdStates = this._osdStates.filter(s => s !== state);
      // effect must be cleaned up before bgActor.destroy()
      if (state.effect) {
        try { state.effect.cleanup(); } catch (e) { }
        state.effect = null;
      }
      if (state.bgActor) {
        try { state.bgActor.destroy(); } catch (e) { }
        state.bgActor = null;
      }
      state._uiSampler?.destroy();
      state._windowCloneManager?.destroy();
    });
  }

  // ── Per-state geometry synchronisation ──────────────────────────────
  // Full-screen FBO approach: bgActor covers the full monitor, shader is told
  // where the OSD lives within that FBO via setGlassGeometry().
  _syncGeometry(state: OsdState) {
    if (!state.bgActor || !state.targetBox) return;

    let [w, h] = state.targetBox.get_size();
    let [absX, absY] = state.targetBox.get_transformed_position();
    if (Number.isNaN(absX) || Number.isNaN(absY)) return;

    // Respect GNOME's OSD fade animation.
    //
    // Check visible, mapped, opacity states of the OSD window and targetBox.
    let osdWindowVisible = state.osdWindow.visible && state.osdWindow.mapped;
    let targetBoxVisible = state.targetBox.visible && state.targetBox.mapped;

    let currentOpacity = (osdWindowVisible && targetBoxVisible)
      ? Math.min(state.osdWindow.opacity, state.targetBox.opacity)
      : 0;
    let isVisible = currentOpacity > 0;

    if (isVisible && !state._wasVisible) {
      state._wasVisible = true;
      this._isFirstAdaptiveRun = true;
      this._updateAdaptiveTextColors();
    } else if (!isVisible && state._wasVisible) {
      state._wasVisible = false;
    }
    state.bgActor.opacity = currentOpacity;

    if (!isVisible) {
      state.bgActor.hide();
      return;
    } else if (!state.bgActor.visible) {
      state.bgActor.show();
    }

    // OSD-specific margin-bottom bloat compensation (icon switch glitch)
    let themeNode = state.targetBox.get_theme_node();
    let mB = themeNode ? themeNode.get_margin(St.Side.BOTTOM) : 0;

    if (state._stableBaseH === undefined) {
      let initH = h;
      try {
        let [_, naturalH] = state.targetBox.get_preferred_height(-1);
```

```
        if (naturalH > 0) {
          initH = naturalH;
        }
      } catch (e) {
        this._logger.warn(`[Liquid Glass] Failed to get preferred height for OSD initialization: ${e}`);
      }
      state._stableBaseH = initH;
    }
    let isHeightBloated = Math.abs(h - (state._stableBaseH + mB)) <= 1;

    let visualW = w;
    let visualH = isHeightBloated ? h - mB : h;
    if (!isHeightBloated) state._stableBaseH = h;

    let visualX = absX;
    let visualY = absY;

    let bgW = visualW + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgH = visualH + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgX_abs = visualX - this._glassExpand - SHADER_PADDING;
    let bgY_abs = visualY - this._glassExpand - SHADER_PADDING;

    // ── Monitor geometry ──────────────────────────────────────────
    let monitorIndex = Main.layoutManager.findIndexForActor(state.osdWindow);
    if (monitorIndex < 0) monitorIndex = Main.layoutManager.primaryIndex;
    let monitor = Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;

    let monitorX = monitor?.x ?? 0;
    let monitorY = monitor?.y ?? 0;
    let screenW = Math.max(1, monitor?.width ?? 1);
    let screenH = Math.max(1, monitor?.height ?? 1);

    // Monitor-local shader coordinates
    let localBgX = bgX_abs - monitorX;
    let localBgY = bgY_abs - monitorY;

    // ── Update actors only when geometry changed ──────────────────
    if (state._lastBgW !== bgW || state._lastBgH !== bgH ||
      state._lastBgX !== bgX_abs || state._lastBgY !== bgY_abs ||
      state._lastScreenW !== screenW || state._lastScreenH !== screenH) {

      // bgActor: full monitor size, positioned at monitor origin
      state.bgActor.remove_transition('size');
      state.bgActor.remove_transition('position');
      state.bgActor.set_position(monitorX, monitorY);
      state.bgActor.set_size(screenW, screenH);
      state.bgActor.remove_transition('size');
      state.bgActor.remove_transition('position');

      // liquidBox fills the entire bgActor
      state.liquidBox?.set_position(0, 0);
      state.liquidBox?.set_size(screenW, screenH);

      // Soft clip — limits GPU work to the OSD area + generous margin
      const CLIP_PADDING = 200;
      state.liquidBox?.remove_clip();
      state.bgActor.set_clip(
        localBgX - CLIP_PADDING, localBgY - CLIP_PADDING,
        bgW + CLIP_PADDING * 2, bgH + CLIP_PADDING * 2
      );

      const SHADOW_MAX_RADIUS = CLIP_PADDING - 20;
      state.effect?.setShadowMaxRadius(SHADOW_MAX_RADIUS);

      // Inform shader of full-screen resolution and OSD position within FBO
      state.effect?.setResolution(screenW, screenH);
      state.effect?.setGlassGeometry(localBgX, localBgY, bgW, bgH);

      state._lastBgW = bgW; state._lastBgH = bgH;
      state._lastBgX = bgX_abs; state._lastBgY = bgY_abs;
      state._lastScreenW = screenW; state._lastScreenH = screenH;
    }

    // ── Sync clones every frame (dockManager pattern) ─────────────
    state._windowCloneManager?.setOffset(-monitorX, -monitorY);
    state._uiSampler?.refresh();
    state._uiSampler?.sync(monitorX, monitorY, screenW, screenH);
    state._windowCloneManager?.sync();
  }

  // ── Effect remove / cleanup ─────────────────────────────────────
  _removeEffect() {
```

```
    if (!this._isEffectActive) return;
    this._isEffectActive = false;

    this._stopAdaptiveColorSampling();
    this._clearAdaptiveStyles();

    if (this._monitorsChangedId !== 0) {
      Main.layoutManager.disconnect(this._monitorsChangedId);
      this._monitorsChangedId = 0;
    }

    if (this._frameSyncId !== 0) {
      if (global.compositor?.get_laters) global.compositor.get_laters().remove(this._frameSyncId);
      this._frameSyncId = 0;
    }

    for (let state of this._osdStates) {
      this._cleanupOsdState(state);
    }
    this._osdStates = [];
  }

  _cleanupOsdState(state: OsdState) {

    // Disconnect destroy watcher
    if (state.osdWindow && state._destroyId) {
      try { state.osdWindow.disconnect(state._destroyId); } catch (e) { }
      state._destroyId = 0;
    }

    // Restore target box
    if (state.targetBox) {
      try {
        state.targetBox.remove_style_class_name('liquid-glass-transparent');
        state.targetBox.translation_y = 0;
      } catch (e) { }
    }

    // DESTROY EFFECT FIRST (before bgActor.destroy())
    if (state.effect) {
      try { state.effect.cleanup(); } catch (e) { }
      state.effect = null;
    }

    // DESTROY ACTOR HIERARCHY — cascades through liquidBox → _cloneContainer
    if (state.bgActor) {
      try { state.bgActor.hide(); } catch (e) { }
      try { state.bgActor.destroy(); } catch (e) { }
      state.bgActor = null;
    }
    state.liquidBox = null;
    state._cloneContainer = null;

    // Clean up managers
    try { state._uiSampler?.destroy(); } catch (e) { }
    state._uiSampler = null;
    try { state._windowCloneManager?.destroy(); } catch (e) { }
    state._windowCloneManager = null;
  }

  cleanup() {
    for (let sigId of this._settingsSignals) {
      this._settings.disconnect(sigId);
    }
    this._settingsSignals = [];
    this._removeEffect();
  }

  // ── Adaptive text colour helpers ──────────────────────────────────

  _collectAdaptiveTextTargets() {
    let targets: Clutter.Actor[] = [];
    for (let state of this._osdStates) {
      if (state.osdWindow && state.osdWindow.opacity > 0 && state.osdWindow.visible) {
        this._findAllTextActors(state.targetBox, targets);
      }
    }
    return targets;
  }

  _findAllTextActors(actor: St.Widget, foundActors: Clutter.Actor[] = []) {
    if (!actor) return foundActors;
```

```
    let isProgressBar = actor.has_style_class_name && actor.has_style_class_name('level');
    if (actor instanceof St.Label || actor instanceof Clutter.Text ||
      actor instanceof St.Button || actor instanceof St.Icon || isProgressBar) {
      if (actor.visible) foundActors.push(actor);
    }

    let children = actor.get_children() as St.Widget[];
    for (let i = 0; i < children.length; i++) {
      this._findAllTextActors(children[i], foundActors);
    }
    return foundActors;
  }

  _setActorColor(actor: CustomBannerActor, color: string, skipAnimations = false) {
    if (!actor || typeof actor.set_style !== 'function') return;
    if (actor._currentTargetColor === color) return;
    actor._currentTargetColor = color;
    this._animateActorColor(actor, color, 380, skipAnimations);
  }

  _clearAdaptiveStyles() {
    for (const [actor, style] of this._styledActors.entries() as MapIterator<[CustomBannerActor, string]>) {
      if (actor && typeof actor.set_style === 'function') {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        actor._currentTargetColor = undefined;
        actor.remove_style_class_name('adaptive-text-transition');
        actor.remove_style_class_name('adaptive-color-light');
        actor.remove_style_class_name('adaptive-color-dark');
        actor.set_style(style);
      }
    }
    this._styledActors.clear();
  }

  _applyAdaptiveColorMap(colorMap: Map<Clutter.Actor, string>, skipAnimations = false) {
    if (!colorMap || colorMap.size === 0) return;
    for (const [actor, color] of colorMap.entries()) {
      this._setActorColor(actor as unknown as CustomBannerActor, color, skipAnimations);
    }
  }

  _startAdaptiveColorSampling() {
    if (!this._adaptiveConfig.enabled) return;
    this._updateAdaptiveTextColors();
    if (this._adaptiveTimerId !== 0) return;

    this._adaptiveTimerId = GLib.timeout_add(
      GLib.PRIORITY_DEFAULT,
      this._adaptiveConfig.sampleIntervalMs,
      () => {
        let isActive = this._osdStates.some(s => s.osdWindow && s.osdWindow.visible);
        if (isActive) this._updateAdaptiveTextColors();
        return GLib.SOURCE_CONTINUE;
      }
    );
  }

  _stopAdaptiveColorSampling() {
    if (this._adaptiveTimerId !== 0) {
      GLib.source_remove(this._adaptiveTimerId);
      this._adaptiveTimerId = 0;
    }
  }

  _updateAdaptiveTextColors() {
    if (!this._adaptiveConfig.enabled || this._adaptiveInFlight) return;

    const targets = this._collectAdaptiveTextTargets();
    if (targets.length === 0) return;

    this._adaptiveInFlight = true;
    let isFirst = this._isFirstAdaptiveRun;
    this._isFirstAdaptiveRun = false;

    this._contrastSampler
      .chooseColorsForActors(targets, this._adaptiveConfig)
      .then(colorMap => {
        this._applyAdaptiveColorMap(colorMap, isFirst);
```

```
      })
      .catch(e => {
        this._logger.error(`[Liquid Glass] OSD adaptive color update failed: ${e}`);
      })
      .finally(() => {
        this._adaptiveInFlight = false;
      });
  }

  _hexToRgb(hex: string) {
    let bigint = parseInt(hex.replace('#', ''), 16);
    return { r: (bigint >> 16) & 255, g: (bigint >> 8) & 255, b: bigint & 255 };
  }

  _rgbToHex(r: number, g: number, b: number) {
    return '#' + (1 << 24 | r << 16 | g << 8 | b).toString(16).slice(1);
  }

  _animateActorColor(actor: CustomBannerActor, targetHexColor: string, durationMs = 380, skipAnimations = false) {
    if (!actor || Object.keys(actor).length === 0) return;

    if (actor._colorTweenId) {
      GLib.source_remove(actor._colorTweenId);
      actor._colorTweenId = undefined;
    }

    let themeNode = actor.get_theme_node();
    let startColor = themeNode.get_foreground_color();
    let startBgColor = themeNode.get_background_color();
    let targetRgb = this._hexToRgb(targetHexColor);

    // OSD-specific: BarLevel progress bar colour interpolation
    let isProgressBar = actor.has_style_class_name && actor.has_style_class_name('level');
    let trackTargetRgb = targetRgb;

    if (isProgressBar) {
      let lightHex = this._adaptiveConfig?.lightTextColor || '#ffffff';
      let darkHex = this._adaptiveConfig?.darkTextColor || '#000000';
      let isTargetLight = targetHexColor.toLowerCase() === lightHex.toLowerCase();
      let otherRgb = this._hexToRgb(isTargetLight ? darkHex : lightHex);
      let lerpRatio = 0.7;
      trackTargetRgb = {
        r: Math.round(targetRgb.r + (otherRgb.r - targetRgb.r) * lerpRatio),
        g: Math.round(targetRgb.g + (otherRgb.g - targetRgb.g) * lerpRatio),
        b: Math.round(targetRgb.b + (otherRgb.b - targetRgb.b) * lerpRatio),
      };
    }

    if (skipAnimations) {
      let finalHex = this._rgbToHex(targetRgb.r, targetRgb.g, targetRgb.b);
      if (isProgressBar) {
        let finalBgHex = this._rgbToHex(trackTargetRgb.r, trackTargetRgb.g, trackTargetRgb.b);
        actor.set_style(`-barlevel-active-background-color: ${finalHex}; -barlevel-background-color: ${finalBgHex};`);
      } else {
        actor.set_style(`color: ${finalHex}; -st-icon-foreground-color: ${finalHex};`);
      }
      return;
    }

    let startTime = GLib.get_monotonic_time();
    actor._colorTweenId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 16, () => {
      if (!actor || Object.keys(actor).length === 0) return GLib.SOURCE_REMOVE;

      let currentTime = GLib.get_monotonic_time();
      let elapsedMs = (currentTime - startTime) / 1000;
      let progress = Math.min(elapsedMs / durationMs, 1.0);
      let ease = progress < 0.5
        ? 2 * progress * progress
        : 1 - Math.pow(-2 * progress + 2, 2) / 2;

      let r = Math.round(startColor.red + (targetRgb.r - startColor.red) * ease);
      let g = Math.round(startColor.green + (targetRgb.g - startColor.green) * ease);
      let b = Math.round(startColor.blue + (targetRgb.b - startColor.blue) * ease);
      let currentHex = this._rgbToHex(r, g, b);

      if (isProgressBar) {
        let bgR = Math.round(startBgColor.red + (trackTargetRgb.r - startBgColor.red) * ease);
        let bgG = Math.round(startBgColor.green + (trackTargetRgb.g - startBgColor.green) * ease);
        let bgB = Math.round(startBgColor.blue + (trackTargetRgb.b - startBgColor.blue) * ease);
        actor.set_style(`-barlevel-active-background-color: ${currentHex}; -barlevel-background-color: $
{this._rgbToHex(bgR, bgG, bgB)};`);
      } else {
```

```
          actor.set_style(`color: ${currentHex}; -st-icon-foreground-color: ${currentHex};`);
        }

        if (progress >= 1.0) {
          actor._colorTweenId = undefined;
          return GLib.SOURCE_REMOVE;
        }
        return GLib.SOURCE_CONTINUE;
      });
    }

    _laterAdd(laterType: Meta.LaterType, callback: GLib.SourceFunc) {
      return global.compositor?.get_laters?.().add(laterType, callback);
    }
}
```

```
// src/quickSettingsManager.ts
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import Clutter from 'gi://Clutter';
import St from 'gi://St';
import GLib from 'gi://GLib';
import Meta from 'gi://Meta';
import { LiquidEffect } from './liquidEffect.js';
import { StageContrastSampler, AdaptiveContrastConfig } from './contrastSampler.js';
import Gio from 'gi://Gio';
import {
  UnpickableActor,
  UILayerSampler,
  WindowCloneManager,
} from './utils.js';

import { Logger } from './logger.js';

// ========== Configuration Parameters ==========

// Transparent padding outside the glass area.
// This prevents the shader distortion or rounded corners from being clipped by the actor bounds.
const SHADER_PADDING = 20;

// Adaptive text color flags
const SAMPLE_PER_ELEMENT = false;

interface CustomBannerActor extends St.Widget {
  _colorTweenId?: number;
  _currentTargetColor?: string;
  _currentInsensitiveState?: boolean;
  _isUpdatingAlpha?: boolean;
}
// ==============================================

export class QuickSettingsManager {
  private extensionPath: string;
  private _settings: Gio.Settings;
  private _logger: Logger;
  private targetActor: St.Widget;
  private menu: any;
  private animActor: St.Widget;

  private bgActor: Clutter.Actor | null;
  private liquidBox: Clutter.Actor | null = null;
  private _cloneContainer: Clutter.Actor | null = null;
  private effect: LiquidEffect | null;

  private _windowCloneManager: WindowCloneManager | null = null;
  private _uiSampler: UILayerSampler | null = null;

  // Cached monitor dimensions for change detection
  private _lastScreenW: number | undefined;
  private _lastScreenH: number | undefined;

  private _isEffectActive: boolean;
  private buttonAlpha: number;
  private _buttonTimerId: number;
  private _styledButtons: Map<Clutter.Actor, string>;
  private _buttonSignalIds: Map<Clutter.Actor, number[]>;
  private _signals: { target: any, id: number }[];
  private _animSignalId: number = 0;
  private _frameSyncId: number;
  private _glassExpand: number;
  private _menuXoffset: number;
  private _menuYoffset: number;

  // Spring physics parameters
  private _springScale: Spring;
  private _springPos: Spring;
  private _springStiffness: number;
  private _springDamping: number;
  private _springMass: number;

  private _enableAnimation: boolean;

  private _tickId: number;
  private _contrastSampler: StageContrastSampler;
  private _adaptiveTimerId: number;
  private _adaptiveInFlight: boolean;
  private _styledActors: Map<Clutter.Actor, string>;
  private _hasAutoRefreshed: boolean;
  private _settingsSignals: number[];
```

```
  private _adaptiveConfig!: typeof AdaptiveContrastConfig;

  // Used in _syncGeometry
  private _stableBaseW: number | undefined;
  private _stableBaseH: number | undefined;
  private _lastValidAnimAbsX: number | undefined;
  private _lastValidAnimAbsY: number | undefined;
  private _lastBgW: number | undefined;
  private _lastBgH: number | undefined;
  private _lastBgX: number | undefined;
  private _lastBgY: number | undefined;

  private _cornerRadius: number = 0;
  private _animationInterval: number = 16;

  // Flag to forcefully move submenus using translation_x, translation_y
  private _enableSubmenuFix: boolean = false;

  private _cachedSubmenus: Clutter.Actor[] | null = null; // Cache of submenus

  constructor(extensionPath: string, settings: Gio.Settings, logger: Logger) {
    this.extensionPath = extensionPath;
    this._settings = settings;
    this._logger = logger;

    // Target the main container of the Quick Settings menu
    this.targetActor = Main.panel.statusArea.quickSettings.menu.actor;
    this.menu = Main.panel.statusArea.quickSettings.menu;
    // Target for animations and visual offsets (The inner content)
    this.animActor = Main.panel.statusArea.quickSettings.menu.box;

    this.bgActor = null;
    this.effect = null;

    this._signals = [];
    this._frameSyncId = 0;
    this._isEffectActive = false;
    this._hasAutoRefreshed = false;

    this._glassExpand = 0;
    this._menuXoffset = 0;
    this._menuYoffset = 0;

    // Custom spring physics parameters for the open/close animation
    // Spring(stiffness, damping, mass)
    this._springScale = new Spring(120, 8, 1.0);
    this._springPos = new Spring(300, 12, 1.0);
    this._springStiffness = 120;
    this._springDamping = 8;
    this._springMass = 1.0;
    this._enableAnimation = true;
    this._tickId = 0;

    this._contrastSampler = new StageContrastSampler();
    this._adaptiveTimerId = 0;
    this._adaptiveInFlight = false;
    this._styledActors = new Map();

    this._settingsSignals = [];

    this.buttonAlpha = 0.8;
    this._buttonTimerId = 0;
    this._styledButtons = new Map();
    this._buttonSignalIds = new Map();

    this._enableSubmenuFix = true;
  }

  setup() {
    if (!this._settings) return;
    this._bindSettings();

    // Setup spring parameters
    this._enableAnimation = this._settings.get_boolean('enable-quick-settings-animation');
    this._springStiffness = this._settings.get_double('quick-settings-spring-stiffness');
    this._springDamping = this._settings.get_double('quick-settings-spring-damping');
    this._springMass = this._settings.get_double('quick-settings-spring-mass');
    this._springScale.updateParams(this._springStiffness, this._springDamping, this._springMass);
    this._springPos.updateParams(this._springStiffness, this._springDamping, this._springMass);

    if (this._settings.get_boolean('enable-quick-settings-glass')) {
      this._applyEffect();
```

```
    }
  }

  // Utility: Convert HEX color string to normalized RGB array
  _hexToColorArray(hex: string): [number, number, number] {
    if (!hex || typeof hex !== 'string' || !hex.startsWith('#') || hex.length !== 7)
      return [1.0, 1.0, 1.0];
    let r = parseInt(hex.slice(1, 3), 16) / 255.0;
    let g = parseInt(hex.slice(3, 5), 16) / 255.0;
    let b = parseInt(hex.slice(5, 7), 16) / 255.0;
    return [r, g, b];
  }

  _getMenuMonitorGeometry() {
    let monitorIndex = Main.layoutManager.findIndexForActor(this.targetActor);
    if (monitorIndex < 0) monitorIndex = Main.layoutManager.primaryIndex;
    return Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;
  }

  _applyMenuOffsets() {
    if (!this.targetActor) return;
    this.targetActor.translation_y = this._menuYoffset;
    this.targetActor.translation_x = this._menuXoffset;
  }

  // Dynamically apply settings changes
  _bindSettings() {
    const connectSetting = (key: string, callback: Function) => {
      let id = this._settings.connect(`changed::${key}`, callback.bind(this));
      this._settingsSignals.push(id);
    };

    // ON/OFF toggle
    connectSetting('enable-quick-settings-glass', () => {
      let enabled = this._settings.get_boolean('enable-quick-settings-glass');
      if (enabled && !this._isEffectActive) this._applyEffect();
      else if (!enabled && this._isEffectActive) this._removeEffect();
    });

    connectSetting('enable-quick-settings-animation', () => {
      this._enableAnimation = this._settings.get_boolean('enable-quick-settings-animation');
    });

    connectSetting('quick-settings-spring-stiffness', () => {
      this._springStiffness = this._settings.get_double('quick-settings-spring-stiffness');
      if (this._springScale) this._springScale.updateParams(this._springStiffness, this._springDamping,
this._springMass);
    });

    connectSetting('quick-settings-spring-damping', () => {
      this._springDamping = this._settings.get_double('quick-settings-spring-damping');
      if (this._springScale) this._springScale.updateParams(this._springStiffness, this._springDamping,
this._springMass);
    });

    connectSetting('quick-settings-spring-mass', () => {
      this._springMass = this._settings.get_double('quick-settings-spring-mass');
      if (this._springScale) this._springScale.updateParams(this._springStiffness, this._springDamping,
this._springMass);
    });

    connectSetting('quick-settings-animation-interval-ms', () => {
      this._animationInterval = this._settings.get_int('quick-settings-animation-interval-ms');
    });

    connectSetting('quick-settings-tint-color', () => {
      if (this.effect) {
        let colorArray = this._hexToColorArray(this._settings.get_string('quick-settings-tint-color'));
        this.effect.setTintColor(...colorArray);
      }
    });

    connectSetting('quick-settings-tint-strength', () => {
      if (this.effect) {
        this.effect.setTintStrength(this._settings.get_double('quick-settings-tint-strength'));
      }
    });

    connectSetting('quick-settings-blur-radius', () => {
      if (this.effect) {
        this.effect.setBlurRadius(this._settings.get_int('quick-settings-blur-radius'));
      }
```

```
    });

    connectSetting('quick-settings-corner-radius', () => {
      if (this.effect) {
        this._cornerRadius = this._settings.get_double('quick-settings-corner-radius');
        this.effect.setCornerRadius(this._cornerRadius);
      }
    });

    connectSetting('quick-settings-glass-expand', () => {
      if (this.effect) {
        this._glassExpand = this._settings.get_int('quick-settings-glass-expand');
      }
    });

    connectSetting('quick-settings-y-offset', () => {
      if (this.targetActor) {
        this._menuYoffset = this._settings.get_int('quick-settings-y-offset');
        this._applyMenuOffsets();
      }
    });

    connectSetting('quick-settings-x-offset', () => {
      if (this.targetActor) {
        this._menuXoffset = this._settings.get_int('quick-settings-x-offset');
        this._applyMenuOffsets();
      }
    });

    connectSetting('quick-settings-enable-adaptive-text-color', () => {
      this._adaptiveConfig.enabled = this._settings.get_boolean('quick-settings-enable-adaptive-text-color');
    });

    connectSetting('quick-settings-sample-interval-ms', () => {
      this._adaptiveConfig.sampleIntervalMs = this._settings.get_int('quick-settings-sample-interval-ms');
    });

    // Brightness / Saturation / Contrast — dynamic application from settings
    connectSetting('quick-settings-brightness', () => {
      if (this.effect) {
        this.effect.setBrightness(this._settings.get_double('quick-settings-brightness'));
      }
    });

    connectSetting('quick-settings-saturation', () => {
      if (this.effect) {
        this.effect.setSaturation(this._settings.get_double('quick-settings-saturation'));
      }
    });

    connectSetting('quick-settings-contrast', () => {
      if (this.effect) {
        this.effect.setContrast(this._settings.get_double('quick-settings-contrast'));
      }
    });
  }

  _applyClassStyles() {
    if (!this.targetActor) return;
    if (!this._hasStyleClass(this.targetActor, 'liquid-glass-transparent'))
      this.targetActor.add_style_class_name('liquid-glass-transparent');
    if (!this._hasStyleClass(this.animActor, 'liquid-glass-transparent'))
      this.animActor.add_style_class_name('liquid-glass-transparent');
    if (!this._hasStyleClass(this.animActor, 'liquid-glass-qs-root'))
      this.animActor.add_style_class_name('liquid-glass-qs-root');
  }

  _applyEffect() {
    if (this._isEffectActive) return;
    this._isEffectActive = true;

    if (!this.targetActor) return;

    // Shift the menu down to prevent it from clipping into the top bar
    this._menuYoffset = this._settings.get_int('quick-settings-y-offset');
    this._menuXoffset = this._settings.get_int('quick-settings-x-offset');
    this._glassExpand = this._settings.get_int('quick-settings-glass-expand');
    this._animationInterval = this._settings.get_int('quick-settings-animation-interval-ms');

    this._adaptiveConfig = {
      ...AdaptiveContrastConfig,
      enabled: this._settings.get_boolean('quick-settings-enable-adaptive-text-color'),
```

```
    samplePerElement: SAMPLE_PER_ELEMENT,
    sampleIntervalMs: this._settings.get_int('quick-settings-sample-interval-ms'),
  };

  // ── 1. bgActor: full monitor, no effect ──────────────────────────────
  // Create the main background actor that covers the full monitor
  this.bgActor = new UnpickableActor();
  this.bgActor.set_name('liquid-glass-bg-actor');
  // Set an initial size of 1x1. Passing a 0x0 size to the Cogl engine
  // while applying a shader will immediately crash the GNOME Shell.
  this.bgActor.set_size(1.0, 1.0);
  this.bgActor.set_pivot_point(0.0, 0.0);

  // ── 2. liquidBox: outer layer — LiquidEffect with built-in dual-Kawase blur ─
  this.liquidBox = new UnpickableActor();
  this.liquidBox.set_name('liquid-box');
  this.liquidBox.set_clip_to_allocation(true);
  this.bgActor.add_child(this.liquidBox);

  // dummyBreaker: prevents BMS black-screen optimization bug
  let dummyBreaker = new UnpickableActor();
  dummyBreaker.set_name('optimization-breaker');
  dummyBreaker.set_size(1.0, 1.0);
  dummyBreaker.set_opacity(0);
  this.liquidBox.add_child(dummyBreaker);

  // ── 3. _cloneContainer: sub-container inside liquidBox ──────────────
  this._cloneContainer = new UnpickableActor();
  this._cloneContainer.set_name('clone-container');
  this.liquidBox.add_child(this._cloneContainer);

  // Scale pivot points
  // The menu scales from the top-center (0.5, 0.0)
  this.animActor.set_pivot_point(0.5, 0.0);
  // bgActor scales from the top-left (0.0, 0.0) because we manually sync its exact coordinates
  this.bgActor.set_pivot_point(0.0, 0.0);

  // ── Find the menuActor's ancestor that is a direct child of uiGroup ─────
  let menuRoot: Clutter.Actor = this.menu.actor;
  while (menuRoot.get_parent() && menuRoot.get_parent() !== Main.layoutManager.uiGroup) {
    const p = menuRoot.get_parent();
    if (!p) break;
    menuRoot = p;
  }

  // Insert bgActor below menuRoot in uiGroup to prevent recursive clone loops
  // Insert the custom background *underneath* the actual menu UI
  if (menuRoot.get_parent() === Main.layoutManager.uiGroup) {
    Main.layoutManager.uiGroup.insert_child_below(this.bgActor, menuRoot);
  } else {
    // Fallback: If it has no parent yet, add it directly to the UI group
    Main.layoutManager.uiGroup.add_child(this.bgActor);
  }

  // ── 5. Read effect parameters from settings ──────────────────────────
  let blurRadius = this._settings.get_int('quick-settings-blur-radius');
  let tintColorStr = this._settings.get_string('quick-settings-tint-color');
  let tintStrength = this._settings.get_double('quick-settings-tint-strength');
  this._cornerRadius = this._settings.get_double('quick-settings-corner-radius');
  let brightness = this._settings.get_double('quick-settings-brightness');
  let saturation = this._settings.get_double('quick-settings-saturation');
  let contrast = this._settings.get_double('quick-settings-contrast');

  // LiquidEffect on liquidBox (includes built-in dual-Kawase blur)
  // Apply our custom GLSL liquid shader to the outer background actor
  this.effect = new LiquidEffect({ extensionPath: this.extensionPath, settings: this._settings } as any);
  // Tell the shader about the padding so it calculates refraction coordinates correctly
  this.effect.setPadding(SHADER_PADDING);
  this.effect.setTintColor(...this._hexToColorArray(tintColorStr));
  this.effect.setTintStrength(tintStrength);
  this.effect.setCornerRadius(this._cornerRadius);
  this.effect.setIsDock(false);
  this.effect.setBrightness(brightness);
  this.effect.setSaturation(saturation);
  this.effect.setContrast(contrast);
  this.effect.setBlurRadius(blurRadius);
  this.liquidBox.add_effect(this.effect);

  // ── 5. WindowCloneManager + UILayerSampler ──────────────────────────
  this._windowCloneManager = new WindowCloneManager(this.liquidBox, this._cloneContainer);
  this._uiSampler = new UILayerSampler(
    this.bgActor,
```

```
      this.liquidBox,
      [menuRoot, global.windowGroup, global.window_group],
      this._cloneContainer
    );

    this.bgActor.hide();

    // ── Helper functions for GNOME's render pipeline ──────────────────────────
    const laterAdd = (laterType: Meta.LaterType, callback: GLib.SourceFunc) => {
      return global.compositor?.get_laters?.().add(laterType, callback);
    };
    const laterRemove = (id: number) => {
      if (!id) return;
      if (global.compositor?.get_laters) global.compositor.get_laters().remove(id);
    };
    // Hook into the frame right before it is painted to the screen
    const frameLaterType = Meta.LaterType.BEFORE_REDRAW;

    // Clone build: applies mutual liquid-glass exclusions, then delegates to managers
    let buildClones = () => {
      if (!this.bgActor) return;

      if (this._uiSampler) {
        for (let child of Main.layoutManager.uiGroup.get_children()) {
          if (child === this.bgActor) continue;
          let isLiquidBg = child.get_name?.() === 'liquid-glass-bg-actor' ||
            (typeof child.get_children === 'function' &&
              child.get_children().some((c: Clutter.Actor) => c.get_name?.() === 'liquid-box'));
          if (isLiquidBg) this._uiSampler!.addExclusion(child);
        }
      }

      this._windowCloneManager?.rebuildClones();
      this._uiSampler?.rebindSelf();
      this._uiSampler?.refresh();
    };

    // Frame render loop (runs every frame while the menu is mapped)
    let frameTick = () => {
      this._frameSyncId = 0;
      if (!this.bgActor || !this.targetActor.mapped) return GLib.SOURCE_REMOVE;

      this._syncGeometry();
      this._frameSyncId = laterAdd(frameLaterType, frameTick);
      return GLib.SOURCE_REMOVE;
    };

    // Starts the render loop and builds fresh clones when the menu is opened
    let startFrameSync = () => {
      if (this._frameSyncId === 0) {
        buildClones();
        this._frameSyncId = laterAdd(frameLaterType, frameTick);
      }
    };

    let stopFrameSync = () => {
      if (this._frameSyncId !== 0) {
        laterRemove(this._frameSyncId);
        this._frameSyncId = 0;
      }
    };

    if (this._hasAutoRefreshed === undefined) this._hasAutoRefreshed = false;
    this._signals = [];

    // Handle the first open as a plain GNOME quick settings open; apply custom behavior only afterwards.
    this._animSignalId = this.menu.connect('open-state-changed', (menu, isOpen: boolean) => {
      if (isOpen) {
        this._cachedSubmenus = null; // Reset submenu cache
        if (!this._hasAutoRefreshed) this._hasAutoRefreshed = true;

        this._applyClassStyles();
        this._applyMenuOffsets();

        this._stableBaseW = undefined;
        this._stableBaseH = undefined;
        startFrameSync();
        // Skip animations on the first open for instant feedback
        this._startAdaptiveColorSampling(true);
        this._startButtonAlphaSampling();
        this._startAnimation(1);
        return;
```

```
      }

      this._applyClassStyles();
      this._applyMenuOffsets();
      this._stopAdaptiveColorSampling();
      this._stopButtonAlphaSampling();
      this._startAnimation(0);
    });

    // Monitor the signal when the menu's mapped state changes
    // Stop the render loop when the menu unmaps (fully hidden)
    this._signals.push({
      target: this.menu.actor,
      id: this.menu.actor.connect('notify::mapped', () => {
        // When the menu is completely hidden from the screen
        if (!this.menu.actor.mapped) {
          // Stop the render/sync loop here for the first time
          stopFrameSync();

          // Ensure cleanup is done reliably
          if (this.bgActor) {
            this.bgActor.hide();
            this.bgActor.opacity = 0;
          }
          if (this.animActor) {
            this.animActor.opacity = 0;
          }
        }
      })
    });

    this._updateResolution();
    if (this.targetActor.mapped) {
      startFrameSync();
    }
  }

  // ── Geometry synchronisation ──────────────────────────────────────
  // Calculates and synchronizes the position/size of the glass background every frame
  // Full-screen FBO approach: bgActor covers the entire monitor, shader is told
  // where the menu lives within that FBO via setGlassGeometry().
  _syncGeometry() {
    if (!this.bgActor || !this.targetActor || !this.targetActor.mapped) {
      if (this.bgActor && this.bgActor.visible) this.bgActor.hide();
      return;
    }
    if (!this.bgActor.visible) this.bgActor.show();

    if (!this._enableAnimation) {
      if (this.targetActor !== null)
        this.bgActor.opacity = this.targetActor.get_first_child()?.opacity ?? 255;
    }

    let [inW, inH] = this.animActor.get_size();
    let [outW, outH] = this.targetActor.get_size();

    inW = Number.isNaN(inW) || inW <= 0 ? (this._stableBaseW || 1) : inW;
    inH = Number.isNaN(inH) || inH <= 0 ? (this._stableBaseH || 1) : inH;

    let [scaleX, scaleY] = this.animActor.get_scale();

    if (!this._enableAnimation) {
      // For default GNOME animation: the transparent wrapper directly under BoxPointer is the actual animated entity
      let gnomeAnimContainer = this.targetActor.get_first_child();
      if (gnomeAnimContainer) {
        scaleX *= gnomeAnimContainer.scale_x;
        scaleY *= gnomeAnimContainer.scale_y;
      }
    } else {
      scaleX *= this.targetActor.get_scale()[0];
      scaleY *= this.targetActor.get_scale()[1];
    }

    let themeNode = this.animActor.get_theme_node();
    let mL = themeNode ? themeNode.get_margin(St.Side.LEFT) : 0;
    let mR = themeNode ? themeNode.get_margin(St.Side.RIGHT) : 0;
    let mT = themeNode ? themeNode.get_margin(St.Side.TOP) : 0;
    let mB = themeNode ? themeNode.get_margin(St.Side.BOTTOM) : 0;
    let marginW = mL + mR;
    let marginH = mT + mB;

    let targetW = Math.round(inW);
```

```
    let targetH = Math.round(inH);

    // GNOME Shell Hover Bug Compensation
    // Detects when the menu tries to unexpectedly shrink by a few pixels
    if (Math.abs(inW - outW) <= 2 && marginW > 0) {
      targetW = Math.round(inW - marginW);
      targetH = Math.round(inH - marginH);
    }

    this._stableBaseW = targetW;
    this._stableBaseH = targetH;

    // Multiply by the current animation scale.
    // Math.max guarantees the size never drops below 1px (prevents Cogl crashes).
    let w = Math.max(1, this._stableBaseW * scaleX);
    let h = Math.max(1, this._stableBaseH * scaleY);

    // Get correct coordinates directly from animActor, which is the actual UI content area
    let [animAbsX, animAbsY] = this.animActor.get_transformed_position();

    // ------------------------------------------------------
    // Advanced Fallback Logic for NaN Coordinates
    // GNOME sometimes fails to report actor positions during the very first frame
    // of an animation. This logic predicts where the menu should be.
    // ------------------------------------------------------
    if (Number.isNaN(animAbsX) || Number.isNaN(animAbsY)) {
      if (this._lastValidAnimAbsX !== undefined && this._lastValidAnimAbsY !== undefined) {
        // Use the last known good coordinates if available
        animAbsX = this._lastValidAnimAbsX;
        animAbsY = this._lastValidAnimAbsY;
      } else {
        // Ultimate fallback: Just place it in the top-center of the primary monitor
        let monitor = Main.layoutManager.primaryMonitor;
        if (monitor) {
          animAbsX = (monitor.width / 2) - (w / 2);
          animAbsY = (Main.panel.height || 27) + (this._menuYoffset ?? 0);
        } else {
          animAbsX = 0;
          animAbsY = 0;
        }
      }
    } else {
      // Save successful coordinates for future fallbacks
      this._lastValidAnimAbsX = animAbsX;
      this._lastValidAnimAbsY = animAbsY;
    }

    // The background needs to be larger than the UI to account for the glass expansion
    // and the extra padding required by the shader for edge refraction.
    let bgW = w + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgH = h + (this._glassExpand * 2) + (SHADER_PADDING * 2);

    // Cover the background by purely subtracting the padding from the exact UI coordinates
    let bgX = animAbsX - this._glassExpand - SHADER_PADDING;
    let bgY = animAbsY - this._glassExpand - SHADER_PADDING;

    if (!Number.isNaN(bgX) && !Number.isNaN(bgY) && w >= 1.0 && h >= 1.0) {
      // ── Monitor geometry ──────────────────────────────────────────
      let monitor = this._getMenuMonitorGeometry();
      let monitorX = monitor?.x ?? 0;
      let monitorY = monitor?.y ?? 0;
      let screenW = Math.max(1, monitor?.width ?? 1);
      let screenH = Math.max(1, monitor?.height ?? 1);

      // Monitor-local coordinates (shader uses these)
      let localBgX = bgX - monitorX;
      let localBgY = bgY - monitorY;

      // ── Update actors only when geometry changed ──────────────────
      // Only update positions/sizes if they actually changed to save CPU cycles
      if (this._lastBgW !== bgW || this._lastBgH !== bgH ||
        this._lastBgX !== bgX || this._lastBgY !== bgY ||
        this._lastScreenW !== screenW || this._lastScreenH !== screenH) {

        // bgActor: full monitor size, positioned at monitor origin
        this.bgActor.remove_transition('size');
        this.bgActor.remove_transition('position');
        this.bgActor.set_position(monitorX, monitorY);
        this.bgActor.set_size(screenW, screenH);
        this.bgActor.remove_transition('size');
        this.bgActor.remove_transition('position');
```

```
        // liquidBox fills the entire bgActor
        this.liquidBox?.set_position(0, 0);
        this.liquidBox?.set_size(screenW, screenH);

        // Soft clip — limits GPU work to the menu area + generous margin
        const CLIP_PADDING = 200;
        this.liquidBox?.remove_clip();
        this.bgActor.set_clip(
          localBgX - CLIP_PADDING, localBgY - CLIP_PADDING,
          bgW + CLIP_PADDING * 2, bgH + CLIP_PADDING * 2
        );

        const SHADOW_MAX_RADIUS = CLIP_PADDING - 20;
        this.effect?.setShadowMaxRadius(SHADOW_MAX_RADIUS);

        // Inform shader of full-screen resolution and where the menu lives in the FBO
        this.effect?.setResolution(screenW, screenH);
        this.effect?.setGlassGeometry(localBgX, localBgY, bgW, bgH);

        this._lastBgW = bgW; this._lastBgH = bgH;
        this._lastBgX = bgX; this._lastBgY = bgY;
        this._lastScreenW = screenW; this._lastScreenH = screenH;
      }

      // ── Sync clones every frame (dockManager pattern) ──────────────────
      this._windowCloneManager?.setOffset(-monitorX, -monitorY);
      this._uiSampler?.refresh();
      this._uiSampler?.sync(monitorX, monitorY, screenW, screenH);
      this._windowCloneManager?.sync();
    }

    // Scale-aware corner radius
    // Use the smaller of the X/Y scales to prevent corners from squishing incorrectly
    if (this.effect && typeof this.effect.setCornerRadius === 'function') {
      let currentScale = Math.min(scaleX, scaleY);
      this.effect.setCornerRadius(this._cornerRadius * currentScale);
      if (typeof this.effect.setAnimationScale === 'function') {
        this.effect.setAnimationScale(currentScale);
      }
    }

    this._adjustSubmenuPositions();
  }

  // Updates the shader resolution based on the current background actor size
  _updateResolution() {
    if (!this.bgActor || !this.effect) return;
    let [width, height] = this.bgActor.get_size();
    if (Number.isFinite(width) && Number.isFinite(height) && width > 0 && height > 0) {
      this.effect.setResolution(width, height);
    }
  }

  // Utility function to safely check if an actor has a specific style class
  _hasStyleClass(actor: Clutter.Actor, className: string) {
    return actor instanceof St.Widget && actor.has_style_class_name(className);
  }

  _collectAdaptiveTextTargets(actor = this.menu?.actor, targets = []) {
    if (!actor) return targets;
    return this._findAllTextActors(this.menu?.actor);
  }

  _findAllTextActors(actor: Clutter.Actor, foundActors: Clutter.Actor[] = []) {
    if (!actor) return foundActors;

    // Collect applicable text or button elements that are currently visible
    if (actor instanceof St.Label || actor instanceof Clutter.Text ||
      actor instanceof St.Button || actor instanceof St.Icon) {
      if (actor.visible) foundActors.push(actor);
    }

    // Recursively scan child elements
    let children = typeof actor.get_children === 'function' ? actor.get_children() : [];
    for (let i = 0; i < children.length; i++) {
      this._findAllTextActors(children[i], foundActors);
    }
    return foundActors;
  }

  // Initiates the color change for a specific actor
  _setActorColor(actor: CustomBannerActor, color: string, skipAnimations = false) {
```

```
    if (!actor || typeof actor.set_style !== 'function') return;

    if (!this._styledActors.has(actor)) {
      let origStyle = typeof actor.get_style === 'function' ? actor.get_style() : null;
      this._styledActors.set(actor, origStyle || '');

      actor.connect('destroy', () => {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        this._styledActors.delete(actor);
      });
    }

    let isInsensitive = false;
    if (actor instanceof St.Button) {
      isInsensitive = (actor.reactive === false) ||
        (typeof actor.has_style_pseudo_class === 'function' && actor.has_style_pseudo_class('insensitive'));
    }

    if (actor._currentTargetColor === color && actor._currentInsensitiveState === isInsensitive) return;
    actor._currentTargetColor = color;
    actor._currentInsensitiveState = isInsensitive;

    this._animateActorColor(actor, color, isInsensitive, 380, skipAnimations);
  }

  _clearAdaptiveStyles() {
    for (const [actor, originalStyle] of this._styledActors.entries() as MapIterator<[CustomBannerActor, string]>) {
      if (actor && typeof actor.set_style === 'function') {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        actor._currentTargetColor = undefined;
        actor._currentInsensitiveState = undefined;
        actor.remove_style_class_name('adaptive-text-transition');
        actor.remove_style_class_name('adaptive-color-light');
        actor.remove_style_class_name('adaptive-color-dark');
        actor.set_style(originalStyle || null);
      }
    }
    this._styledActors.clear();

    const currentTargets = this._collectAdaptiveTextTargets() as CustomBannerActor[];
    for (let actor of currentTargets) {
      if (actor && typeof actor.set_style === 'function') {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        actor._currentTargetColor = undefined;
        actor._currentInsensitiveState = undefined;
        actor.set_style(null);
      }
    }
  }

  // Iterates through the color map and applies the new target colors to the respective actors
  _applyAdaptiveColorMap(colorMap: Map<Clutter.Actor, string>, skipAnimations = false) {
    if (!colorMap || colorMap.size === 0) return;
    for (const [actor, color] of colorMap.entries()) {
      this._setActorColor(actor as unknown as CustomBannerActor, color, skipAnimations);
    }
  }

  // Starts the timer for periodically sampling contrast and updating adaptive text colors
  _startAdaptiveColorSampling(skipAnimations = false) {
    if (!this._adaptiveConfig.enabled) return;
    this._updateAdaptiveTextColors(skipAnimations);
    if (this._adaptiveTimerId !== 0) return;

    this._adaptiveTimerId = GLib.timeout_add(
      GLib.PRIORITY_DEFAULT,
      this._adaptiveConfig.sampleIntervalMs,
      () => {
        if (!this.menu?.isOpen) {
          this._adaptiveTimerId = 0;
          return GLib.SOURCE_REMOVE;
        }
        this._updateAdaptiveTextColors(false);
```

```
        return GLib.SOURCE_CONTINUE;
      }
    );
  }

  // Stops the adaptive color sampling timer
  _stopAdaptiveColorSampling() {
    if (this._adaptiveTimerId !== 0) {
      GLib.source_remove(this._adaptiveTimerId);
      this._adaptiveTimerId = 0;
    }
  }

  // Collects target actors, samples their contrast, and triggers color updates
  _updateAdaptiveTextColors(skipAnimations = false) {
    if (!this._adaptiveConfig.enabled || this._adaptiveInFlight) return;

    const targets = this._collectAdaptiveTextTargets();
    if (targets.length === 0) return;

    this._adaptiveInFlight = true;

    this._contrastSampler
      .chooseColorsForActors(targets, this._adaptiveConfig)
      .then(colorMap => {
        this._applyAdaptiveColorMap(colorMap, skipAnimations);
      })
      .catch(e => {
        this._logger.error(`[Liquid Glass] Quick Settings adaptive color update failed: ${e}`);
      })
      .finally(() => {
        this._adaptiveInFlight = false;
      });
  }

  _hexToRgb(hex: string) {
    let bigint = parseInt(hex.replace('#', ''), 16);
    return { r: (bigint >> 16) & 255, g: (bigint >> 8) & 255, b: bigint & 255 };
  }

  _rgbToHex(r: number, g: number, b: number) {
    return '#' + (1 << 24 | r << 16 | g << 8 | b).toString(16).slice(1);
  }

  _animateActorColor(actor: CustomBannerActor, targetHexColor: string, isInsensitive: boolean, durationMs = 380,
skipAnimations = false) {
    if (!actor || Object.keys(actor).length === 0) return;

    if (actor._colorTweenId) {
      GLib.source_remove(actor._colorTweenId);
      actor._colorTweenId = undefined;
    }

    let themeNode = actor.get_theme_node();
    let startColor = themeNode.get_foreground_color();
    let targetRgb = this._hexToRgb(targetHexColor);
    let targetAlpha = isInsensitive ? 0.5 : 1.0;
    let startAlpha = startColor.alpha / 255.0;

    if (skipAnimations) {
      let alphaStr = targetAlpha.toFixed(3);
      let targetRgba = `rgba(${targetRgb.r}, ${targetRgb.g}, ${targetRgb.b}, ${alphaStr})`;
      actor.set_style(`color: ${targetRgba}; -st-icon-foreground-color: ${targetRgba};`);
      return;
    }

    let startTime = GLib.get_monotonic_time();

    actor._colorTweenId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 32, () => {
      if (!actor || Object.keys(actor).length === 0) return GLib.SOURCE_REMOVE;

      let currentTime = GLib.get_monotonic_time();
      let elapsedMs = (currentTime - startTime) / 1000;
      let progress = Math.min(elapsedMs / durationMs, 1.0);
      // Standard ease-in-out easing function
      let ease = progress < 0.5
        ? 2 * progress * progress
        : 1 - Math.pow(-2 * progress + 2, 2) / 2;

      // Linearly interpolate (lerp) each RGB channel individually
      let r = Math.round(startColor.red + (targetRgb.r - startColor.red) * ease);
      let g = Math.round(startColor.green + (targetRgb.g - startColor.green) * ease);
```

```
      let b = Math.round(startColor.blue + (targetRgb.b - startColor.blue) * ease);

      // Interpolate the alpha value and generate the rgba() format
      // Safely clamp between 0.0 and 1.0
      let a = Math.max(0.0, Math.min(1.0, startAlpha + (targetAlpha - startAlpha) * ease));
      // Up to 3 decimal places for CSS
      let currentRgba = `rgba(${r}, ${g}, ${b}, ${a.toFixed(3)})`;

      // Override text color and icon foreground color directly using inline CSS
      actor.set_style(`color: ${currentRgba}; -st-icon-foreground-color: ${currentRgba};`);

      if (progress >= 1.0) {
        actor._colorTweenId = undefined;
        return GLib.SOURCE_REMOVE;
      }
      return GLib.SOURCE_CONTINUE;
    });
  }

  // ── Button alpha sampling (QuickSettings-specific) ──────────────────────

  _findAllButtons(actor: Clutter.Actor, foundButtons: Clutter.Actor[] = []) {
    if (!actor) return foundButtons;

    let isQuickSlider = false;
    let isToggleContainer = false;
    let isButton = actor instanceof St.Button;

    if (actor instanceof St.Widget) {
      isQuickSlider = actor.has_style_class_name('quick-slider');
      isToggleContainer = actor.has_style_class_name('quick-toggle');
    }

    if (actor.visible && !isQuickSlider) {
      if (isButton || isToggleContainer) foundButtons.push(actor);
    }

    let children = typeof actor.get_children === 'function' ? actor.get_children() : [];
    for (let i = 0; i < children.length; i++) {
      this._findAllButtons(children[i], foundButtons);
    }
    return foundButtons;
  }

  _updateSingleButtonAlpha(button: CustomBannerActor, targetAlpha: number) {
    if (!button || button._isUpdatingAlpha) return;
    button._isUpdatingAlpha = true;

    let origStyle = this._styledButtons.get(button) || '';
    button.set_style(origStyle || null);
    button.ensure_style();

    let themeNode = button.get_theme_node();
    if (themeNode) {
      let bgColor = themeNode.get_background_color();

      if (bgColor) {
        let isToggleContainer = button instanceof St.Widget && button.has_style_class_name('quick-toggle');

        // FIX 1: If this is a parent toggle container, hide its background if any child is active/colored.
        // This prevents the dark pod background from muddying the semi-transparent orange child button.
        if (isToggleContainer) {
          let hasColoredChild = false;
          let children = typeof button.get_children === 'function' ? button.get_children() : [];
          for (let i = 0; i < children.length; i++) {
            let child = children[i];
            if (child instanceof St.Widget) {
              let childTheme = child.get_theme_node();
              if (childTheme) {
                let childBg = childTheme.get_background_color();
                if (childBg && childBg.alpha > 0) { hasColoredChild = true; break; }
              }
            }
          }
          if (hasColoredChild) {
            let newStyle = origStyle
              ? `${origStyle} background-color: transparent !important;`
              : `background-color: transparent !important;`;
            button.set_style(newStyle);
            button._isUpdatingAlpha = false;
            return;
          }
```

```
        }

        // FIX 2: If the button is completely transparent by default (like power/lock buttons), keep it transparent.
        if (bgColor.alpha === 0) {
          // Keep transparent buttons transparent
        } else {
          // Apply target alpha for normally visible buttons
          let rgbaStr = `rgba(${bgColor.red}, ${bgColor.green}, ${bgColor.blue}, ${targetAlpha})`;
          let newStyle = origStyle ? `${origStyle} background-color: ${rgbaStr};` : `background-color: ${rgbaStr};`;
          button.set_style(newStyle);

          // Ensure the parent toggle container is also updated dynamically.
          // If a child button changes state, we must force the parent to re-evaluate its transparency.
          let parent = typeof button.get_parent === 'function' ? button.get_parent() : null;
          if (parent && parent instanceof St.Widget && parent.has_style_class_name('quick-toggle')) {
            this._updateSingleButtonAlpha(parent as CustomBannerActor, targetAlpha);
          }
        }
      }
    }

    button._isUpdatingAlpha = false;
  }

  _updateButtonAlpha() {
    if (!this.menu?.isOpen) return;

    const buttons = this._findAllButtons(this.menu?.actor);
    if (buttons.length === 0) return;

    let targetAlpha = this.buttonAlpha !== undefined ? this.buttonAlpha : 0.5;

    for (let button of buttons) {
      if (!this._styledButtons.has(button)) {
        if (button instanceof St.Widget) {
          let origStyle = typeof button.get_style === 'function' ? button.get_style() : null;
          this._styledButtons.set(button, origStyle || '');
        }

        const updateHandler = () => {
          if (!this.menu?.isOpen) return;
          GLib.idle_add(GLib.PRIORITY_DEFAULT, () => {
            this._updateSingleButtonAlpha(button as CustomBannerActor, targetAlpha);
            return GLib.SOURCE_REMOVE;
          });
        };

        let signalIds: number[] = [];
        signalIds.push(button.connect('notify::hover', updateHandler));
        signalIds.push(button.connect('notify::active', updateHandler));
        signalIds.push(button.connect('notify::checked', updateHandler));
        signalIds.push(button.connect('notify::reactive', updateHandler));
        signalIds.push(button.connect('notify::mapped', updateHandler));
        signalIds.push(button.connect('key-focus-in', updateHandler));
        signalIds.push(button.connect('key-focus-out', updateHandler));
        this._buttonSignalIds.set(button, signalIds);
      }

      // Apply style safely
      this._updateSingleButtonAlpha(button as unknown as CustomBannerActor, targetAlpha);
    }
  }

  // Start sampling timer
  _startButtonAlphaSampling() {
    this._updateButtonAlpha();
    if (this._buttonTimerId !== 0) return;

    this._buttonTimerId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 400, () => {
      if (!this.menu?.isOpen) {
        this._buttonTimerId = 0;
        return GLib.SOURCE_REMOVE;
      }
      this._updateButtonAlpha();
      return GLib.SOURCE_CONTINUE;
    });
  }

  _stopButtonAlphaSampling() {
    if (this._buttonTimerId !== 0) {
      GLib.source_remove(this._buttonTimerId);
      this._buttonTimerId = 0;
```

```
    }
  }

  // Revert processing when extension is disabled, etc.
  _clearButtonStyles() {
    this._stopButtonAlphaSampling();
    if (this._buttonSignalIds) {
      for (const [button, signalIds] of this._buttonSignalIds.entries()) {
        if (button) {
          for (const id of signalIds) {
            try { button.disconnect(id); } catch (e) { }
          }
        }
      }
      this._buttonSignalIds.clear();
    }
    for (const [button, originalStyle] of this._styledButtons.entries()) {
      if (button && button instanceof St.Widget && typeof button.set_style === 'function') {
        button.set_style(originalStyle || null);
      }
    }
    this._styledButtons.clear();
  }

  // ── Spring animation (QuickSettings-specific) ──────────────────────

  _startAnimation(targetValue: number) {
    if (this._tickId !== 0) {
      GLib.source_remove(this._tickId);
      this._tickId = 0;
    }

    if (!this._enableAnimation) {
      if (this.bgActor) {
        this.bgActor.remove_all_transitions();
        this.bgActor.opacity = 255;
        this.bgActor.set_scale(1.0, 1.0);
        if (this.animActor) {
          this.animActor.set_scale(1.0, 1.0);
          this.animActor.opacity = 255;
        }
      }
      return;
    }

    if (this.animActor) this.animActor.remove_all_transitions();
    if (this.bgActor) this.bgActor.remove_all_transitions();

    this._springScale.target = targetValue;
    this._springPos.target = targetValue;

    if (this._tickId === 0) {
      let lastTime = GLib.get_monotonic_time();

      this._tickId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, this._animationInterval, () => {
        if (!this.bgActor || !this.targetActor) {
          this._tickId = 0;
          return GLib.SOURCE_REMOVE;
        }

        let currentTime = GLib.get_monotonic_time();
        let elapsedMs = (currentTime - lastTime) / 1000;
        lastTime = currentTime;

        let isClosing = (this._springScale.target === 0);
        let dt = elapsedMs / 1000;
        if (dt > 0.033) dt = 0.033;

        let stopped = false;
        let s: number, p: number;

        if (isClosing) {
          // Use a simple exponential decay for closing (faster, no bounce)
          let speed = 15.0;
          this._springScale.value += (0 - this._springScale.value) * (1.0 - Math.exp(-speed * dt));
          this._springPos.value += (0 - this._springPos.value) * (1.0 - Math.exp(-speed * dt));
          s = this._springScale.value;
          p = this._springPos.value;
          // Stop animation completely when it's virtually invisible
          if (s < 0.005) { s = 0; p = 0; stopped = true; }
        } else {
          // Use Hooke's law spring physics for opening (creates a nice bounce effect)
```

```
          stopped = this._springScale.update(elapsedMs) && this._springPos.update(elapsedMs);
          s = this._springScale.value;
          p = this._springPos.value;
          // Magnet effect: Snap to exactly 1.0 when the bounce is almost settled.
          // This prevents indefinite micro-stuttering at the end of the animation.
          if (Math.abs(1.0 - s) < 0.002 && Math.abs(this._springScale.velocity) < 0.03) {
            s = 1.0; p = 1.0; stopped = true;
          }
        }

        let currentScale: number;
        let opacity: number;

        if (isClosing) {
          // Clamp to 0.001 because scale = 0 crashes Cogl
          currentScale = Math.max(0.001, s);
          // Fade out opacity faster than the scale shrinks (fades between scale 1.0 and 0.3)
          opacity = Math.min(255, Math.max(0, (s - 0.3) / 0.7 * 255));
        } else {
          currentScale = 0.2 + (s * 0.8);
          opacity = Math.min(255, Math.max(0, (s / 0.3) * 255));
        }

        this.animActor.set_scale(currentScale, currentScale);
        this.bgActor.opacity = opacity;
        this.animActor.opacity = opacity;

        this._syncGeometry();

        if (stopped) {
          this._tickId = 0;

          if (isClosing && this.menu.actor) {
            this.menu.actor.hide(); // Tell GNOME the menu is officially closed
            this.bgActor.opacity = 0;
            this.animActor.opacity = 0;
          }

          if (!isClosing) {
            this.animActor.set_scale(1.0, 1.0);
            this.animActor.opacity = 255;
            this.bgActor.opacity = 255;
            this._syncGeometry();
          }
          return GLib.SOURCE_REMOVE;
        }
        return GLib.SOURCE_CONTINUE;
      });
    }
  }

  // ── Submenu position fix (QuickSettings-specific) ──────────────────────
  // Fix: Force submenu position to the center of the parent menu

  _adjustSubmenuPositions() {
    if (!this._enableSubmenuFix || !this.menu?.isOpen || !this.animActor) return;

    // Scan when there's no cached submenus yet
    if (!this._cachedSubmenus) {
      this._cachedSubmenus = [];
      let deepScan = (actor: Clutter.Actor) => {
        if (!actor) return;
        if (actor instanceof St.Widget) {
          let css = actor.get_style_class_name ? actor.get_style_class_name() : '';
          if (css && css.split(' ').includes('quick-toggle-menu')) this._cachedSubmenus!.push(actor);
        }
        let children = typeof actor.get_children === 'function' ? actor.get_children() : [];
        for (let child of children) deepScan(child);
      };
      deepScan(this.menu.actor);
    }

    let foundMenus: Clutter.Actor[] = this._cachedSubmenus;

    if (foundMenus.length === 0) return;

    // Get the absolute coordinates and size as the parent's base (animActor = the visual bounding box of the menu)
    let [parentAbsX, parentAbsY] = this.animActor.get_transformed_position();
    let [parentW, parentH] = this.animActor.get_size();

    if (Number.isNaN(parentAbsX) || Number.isNaN(parentAbsY) ||
      Number.isNaN(parentW) || Number.isNaN(parentH) ||
```

```
      parentW <= 0 || parentH <= 0) return;

    for (let submenu of foundMenus) {
      if (!submenu.mapped || !submenu.visible) continue;

      let [subAbsX, subAbsY] = submenu.get_transformed_position();
      let [subW, subH] = submenu.get_size();

      if (Number.isNaN(subAbsX) || Number.isNaN(subAbsY) ||
        Number.isNaN(subW) || Number.isNaN(subH) ||
        subW <= 0 || subH <= 0) continue;

      // X: centre-align submenu within parent
      let currentTranslationX = submenu.translation_x || 0;
      let baseRelativeX = subAbsX - parentAbsX - currentTranslationX;
      let targetRelativeX = (parentW - subW) / 2;
      let newTranslationX = targetRelativeX - baseRelativeX;
      if (Math.abs(currentTranslationX - newTranslationX) > 0.5) {
        submenu.translation_x = newTranslationX;
      }

      // Y: centre-align submenu in the available gap between neighbours
      let currentTranslationY = submenu.translation_y || 0;
      let baseAbsY = subAbsY - currentTranslationY;
      let subCenterY = baseAbsY + (subH / 2);
      let aboveMaxY = parentAbsY;
      let belowMinY = parentAbsY + parentH;

      let findBoundaries = (n: Clutter.Actor) => {
        if (!n || !n.visible || !n.mapped || n === submenu) return;
        if (typeof n.contains === 'function' && n.contains(submenu)) {
          let children = typeof n.get_children === 'function' ? n.get_children() : [];
          for (let child of children) findBoundaries(child);
          return;
        }
        let [, nodeY] = n.get_transformed_position();
        let [nodeW, nodeH] = n.get_size();
        if (Number.isNaN(nodeY) || Number.isNaN(nodeW) || Number.isNaN(nodeH) ||
          nodeH <= 5 || nodeW <= 5) return;

        // Separate and evaluate elements above and below based on the submenu's "center point"
        if (nodeY + (nodeH / 2) < subCenterY) {
          // Elements above: Their bottom edge doesn't cross the submenu center, and are the lowest among them
          if (nodeY + nodeH <= subCenterY && nodeY + nodeH > aboveMaxY) aboveMaxY = nodeY + nodeH;
        } else {
          // Elements below: Their top edge doesn't cross the submenu center, and are the highest among them
          if (nodeY >= subCenterY && nodeY < belowMinY) belowMinY = nodeY;
        }
        let children = typeof n.get_children === 'function' ? n.get_children() : [];
        for (let child of children) findBoundaries(child);
      };

      // Execute boundary scan starting from the direct children of the box (parent container)
      let parentChildren = typeof this.animActor.get_children === 'function' ? this.animActor.get_children() : [];
      for (let child of parentChildren) findBoundaries(child);

      // Calculate the target value to place the submenu in the center of the identified vertical gap
      let targetTranslationY = (aboveMaxY + (belowMinY - aboveMaxY) / 2) - (subH / 2) - baseAbsY;

      // Chattering prevention (update only if there's a difference of 0.5px or more from the current movement)
      if (Math.abs(currentTranslationY - targetTranslationY) > 0.5) {
        submenu.translation_y = targetTranslationY;
      }
    }
  }

  _clearSubmenuFix() {
    // Scan when there's no cached submenus yet
    let foundMenus: Clutter.Actor[] = this._cachedSubmenus || [];

    if (foundMenus.length === 0) {
      let deepScan = (actor: Clutter.Actor) => {
        if (!actor) return;
        if (actor instanceof St.Widget) {
          let css = actor.get_style_class_name ? actor.get_style_class_name() : '';
          if (css && css.split(' ').includes('quick-toggle-menu')) foundMenus.push(actor);
        }
        let children = typeof actor.get_children === 'function' ? actor.get_children() : [];
        for (let child of children) deepScan(child);
      };
      if (this.menu?.actor) deepScan(this.menu.actor);
    }
```

```
    for (let submenu of foundMenus) {
      try { submenu.translation_x = 0; } catch (e) { }
    }

    this._cachedSubmenus = null; // Clear cache
  }

  // ── Effect remove / cleanup ──────────────────────────────────────
  _removeEffect() {
    if (!this._isEffectActive) return;
    this._isEffectActive = false;

    this._stopAdaptiveColorSampling();
    this._clearAdaptiveStyles();
    this._clearButtonStyles();
    this._clearSubmenuFix();

    // Disconnect all event listeners
    for (let sig of this._signals) {
      try { if (sig && sig.id) sig.target.disconnect(sig.id); } catch (e) { }
    }
    this._signals = [];

    if (this._animSignalId) {
      try { this.menu.disconnect(this._animSignalId); } catch (e) { }
      this._animSignalId = 0;
    }

    // Stop the render frame loop
    if (this._frameSyncId !== 0) {
      if (global.compositor?.get_laters)
        global.compositor.get_laters().remove(this._frameSyncId);
      this._frameSyncId = 0;
    }

    // Remove transparent CSS overrides
    this.targetActor.remove_style_class_name('liquid-glass-transparent');
    if (this.animActor) {
      this.animActor.remove_style_class_name('liquid-glass-transparent');
      this.animActor.remove_style_class_name('liquid-glass-qs-root');
      this.animActor.translation_x = 0;
      this.animActor.translation_y = 0;
      this.animActor.set_scale(1.0, 1.0);
      this.animActor.opacity = 255;
    }

    this.targetActor.translation_y = 0;
    this.targetActor.translation_x = 0;
    this.targetActor.set_scale(1.0, 1.0);
    this.targetActor.opacity = 255;

    if (this.menu.actor) {
      this.menu.actor.opacity = 255;
      this.menu.actor.translation_x = 0;
      this.menu.actor.translation_y = 0;
      if (this.menu.isOpen) this.menu.close(false);
    }

    // DESTROY EFFECT FIRST (before bgActor.destroy())
    if (this.effect) {
      this.effect.cleanup();
      this.effect = null;
    }

    // DESTROY ACTOR HIERARCHY — bgActor.destroy() cascades through
    // liquidBox → _cloneContainer and all their children.
    if (this.bgActor) {
      this.bgActor.destroy();
      this.bgActor = null;
    }
    this.liquidBox = null;
    this._cloneContainer = null;

    // Clean up managers (their destroy() guards against already-destroyed actors)
    this._uiSampler?.destroy();
    this._uiSampler = null;
    this._windowCloneManager?.destroy();
    this._windowCloneManager = null;

    this._stableBaseW = undefined;
    this._stableBaseH = undefined;
```

```
    this._lastScreenW = undefined;
    this._lastScreenH = undefined;
  }

  cleanup() {
    for (let sigId of this._settingsSignals) {
      try { this._settings.disconnect(sigId); } catch (e) { }
    }
    this._settingsSignals = [];

    if (!this.targetActor) return;
    this._removeEffect();
  }
}

// A straightforward mathematical implementation of Hooke's Law for spring physics
class Spring {
  private stiffness: number;
  private damping: number;
  private mass: number;
  public value: number;
  public velocity: number;
  public target: number;
  constructor(stiffness: number, damping: number, mass: number) {
    this.stiffness = stiffness; // How rigid the spring is (higher = faster, more snappy)
    this.damping = damping;     // Friction (higher = less bounce, settles quicker)
    this.mass = mass;           // Weight of the object

    this.value = 0;             // Current position/scale
    this.velocity = 0;          // Current speed
    this.target = 0;            // Destination value
  }

  updateParams(stiffness: number, damping: number, mass: number) {
    this.stiffness = stiffness; // How rigid the spring is (higher = faster, more snappy)
    this.damping = damping;     // Friction (higher = less bounce, settles quicker)
    this.mass = mass;           // Weight of the object
  }

  update(elapsedMs: number) {
    // Cap max delta time to prevent the spring from violently exploding during heavy CPU load
    let dt = elapsedMs / 1000;
    if (dt > 0.033) dt = 0.033;

    // F = -k * x
    let springForce = -this.stiffness * (this.value - this.target);

    // F = -c * v
    let dampingForce = -this.damping * this.velocity;

    // a = F / m
    let acceleration = (springForce + dampingForce) / this.mass;

    // Update velocity and position using Euler integration
    this.velocity += acceleration * dt;
    this.value += this.velocity * dt;

    // Return true if the spring has virtually stopped moving and reached its destination
    return Math.abs(this.velocity) < 0.01 && Math.abs(this.value - this.target) < 0.001;
  }
}
```

```
// src/uiManager.ts
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import Clutter from 'gi://Clutter';
import St from 'gi://St';
import GLib from 'gi://GLib';
import Meta from 'gi://Meta';
import Gio from 'gi://Gio';
import { LiquidEffect } from './liquidEffect.js';
import { StageContrastSampler, AdaptiveContrastConfig } from './contrastSampler.js';
import { UnpickableActor, UILayerSampler, UnpickableWidget, WindowCloneManager } from './utils.js';

import { Logger } from './logger.js';

// ========== Configuration Parameters ==========

// Transparent padding outside the glass area.
// This prevents the shader distortion or rounded corners from being clipped by the actor bounds.
const SHADER_PADDING = 20;

// Adaptive text color flags
const SAMPLE_PER_ELEMENT = false;

interface CustomBannerActor extends St.Widget {
  _colorTweenId?: number;
  _currentTargetColor?: string;
  _currentInsensitiveState?: boolean;
  _isUpdatingAlpha?: boolean;
}
// ==============================================

export class UIManager {
  private extensionPath: string;
  private _settings: Gio.Settings;
  private _logger: Logger;
  private targetActor: St.Widget;
  private menu: any;
  private animActor: St.Widget;
  private bgActor: Clutter.Actor | null;
  private effect: LiquidEffect | null;

  private _cloneContainer: Clutter.Actor | null = null;
  private _windowCloneManager: WindowCloneManager | null = null;

  private _signals: { target: any, id: number }[];
  private _animSignalId: number = 0;
  private _frameSyncId: number;
  private _glassExpand: number;
  private _menuXoffset: number;
  private _menuYoffset: number;
  private _tickId: number;
  private _contrastSampler: StageContrastSampler;
  private _adaptiveTimerId: number;
  private _adaptiveInFlight: boolean;
  private _styledActors: Map<Clutter.Actor, string>;
  private _settingsSignals: number[];
  private _isEffectActive: boolean;
  private _adaptiveConfig!: typeof AdaptiveContrastConfig;
  private liquidBox: Clutter.Actor | null = null;
  private _stableBaseW: number | undefined;
  private _stableBaseH: number | undefined;
  private _lastValidAnimAbsX: number | undefined;
  private _lastValidAnimAbsY: number | undefined;
  private _lastBgW: number | undefined;
  private _lastBgH: number | undefined;
  private _lastBgX: number | undefined;
  private _lastBgY: number | undefined;

  // Spring physics parameters
  private _springScale: Spring;
  private _springPos: Spring;
  private _springStiffness: number;
  private _springDamping: number;
  private _springMass: number;

  // SwiftUI Animation parameters
  private _swiftAnimation: boolean = false;
  private _swiftResponse: number = 0.3;
  private _swiftDampingFraction: number = 0.65;

  private _swiftSpringScale: SwiftSpring;
  private _swiftSpringPos: SwiftSpring;
```

```
  private _enableAnimation: boolean;

  private _interfaceSettings: Gio.Settings | null = null;
  private _accentColorSignalId: number = 0;

  private _dynamicCssFile: Gio.File | null = null;
  private _cornerRadius: number = 0;

  private _animationInterval: number = 16;
  private _uiSampler: UILayerSampler | null = null;

  private _lastScreenW: number | undefined;
  private _lastScreenH: number | undefined;

  constructor(extensionPath: string, settings: Gio.Settings, logger: Logger) {
    this.extensionPath = extensionPath;
    this._settings = settings;
    this._logger = logger;

    // Target the main container of the Date/Calendar menu
    this.targetActor = Main.panel.statusArea.dateMenu.menu.actor as St.Widget;
    this.menu = Main.panel.statusArea.dateMenu.menu;

    // Target for animations and visual offsets (The inner content)
    // @ts-expect-error
    this.animActor = Main.panel.statusArea.dateMenu.menu.box as St.Widget;

    this.bgActor = null;
    this.effect = null;

    this._signals = [];
    this._frameSyncId = 0;

    this._glassExpand = 0;
    this._menuXoffset = 0;
    this._menuYoffset = 0;

    // Custom spring physics parameters for the open/close animation
    this._springScale = new Spring(120, 8, 1.0);
    this._springPos = new Spring(300, 12, 1.0);
    this._springStiffness = 120;
    this._springDamping = 8;
    this._springMass = 1.0;

    this._swiftSpringScale = new SwiftSpring(this._swiftResponse, this._swiftDampingFraction);
    this._swiftSpringPos = new SwiftSpring(this._swiftResponse, this._swiftDampingFraction);

    this._enableAnimation = false;
    this._tickId = 0;

    this._contrastSampler = new StageContrastSampler();
    this._adaptiveTimerId = 0;
    this._adaptiveInFlight = false;
    this._styledActors = new Map();

    this._settingsSignals = [];
    this._isEffectActive = false;

    // Listen for the menu opening/closing to trigger our custom physics animation
    this._animSignalId = this.menu.connect('open-state-changed', (menu: any, isOpen: boolean) => {
      if (isOpen) {
        this._startAnimation(1); // Target scale: 1.0 (fully open)
      } else {
        this._startAnimation(0); // Target scale: 0.0 (closed)
      }
    });
  }

  setup() {
    if (!this._settings) return;
    this._bindSettings();

    this._enableAnimation = this._settings.get_boolean('enable-menu-animation');
    this._springStiffness = this._settings.get_double('menu-spring-stiffness');
    this._springDamping = this._settings.get_double('menu-spring-damping');
    this._springMass = this._settings.get_double('menu-spring-mass');
    this._springScale.updateParams(this._springStiffness, this._springDamping, this._springMass);
    this._springPos.updateParams(this._springStiffness, this._springDamping, this._springMass);

    this._interfaceSettings = new Gio.Settings({ schema_id: 'org.gnome.desktop.interface' });
    this._accentColorSignalId = this._interfaceSettings.connect('changed::accent-color', () => {
      // console.log(`[Liquid Glass] System accent color changed.`);
```

```
      GLib.timeout_add(GLib.PRIORITY_DEFAULT, 200, () => {
        this._applySystemAccentColor();
        return GLib.SOURCE_REMOVE;
      });
    });

    // 初回実行
    this._applySystemAccentColor();

    if (this._settings.get_boolean('enable-menu-glass')) {
      this._applyEffect();
    }
  }

  private _applySystemAccentColor() {
    if (!this.targetActor) return;

    // 1. 親要素と子要素を作成して、GNOMEテーマが要求する正しい階層を再現
    const parent = new UnpickableWidget({ style_class: 'calendar' });
    const child = new UnpickableWidget({ style_class: 'calendar-day calendar-today' });
    parent.add_child(child);

    // 2. UIグループに追加してスタイルを強制計算させる
    Main.layoutManager.uiGroup.add_child(parent);
    child.ensure_style();

    // 3. 計算済みの色を取得
    const themeNode = child.get_theme_node();
    const bgColor = themeNode.get_background_color();

    // 4. 用が済んだらすぐお掃除
    Main.layoutManager.uiGroup.remove_child(parent);
    parent.destroy();

    // 5. HEXに変換
    const colorStr = this._rgbToHex(bgColor.red, bgColor.green, bgColor.blue);
    // console.log(`[Liquid Glass] Set system accent color to ${colorStr}`);

    const cssContent = `
      .liquid-glass-menu-root .calendar-today,
      .liquid-glass-menu-root .calendar-today:hover,
      .liquid-glass-menu-root .calendar-today:active,
      .liquid-glass-menu-root .calendar-today:checked,
      .liquid-glass-menu-root .calendar-today:focus {
        background-color: ${colorStr} !important;
        color: white !important;
      }
    `;

    try {
      const cacheDir = GLib.get_user_cache_dir();
      const filePath = GLib.build_filenamev([cacheDir, 'liquid-glass-accent.css']);

      GLib.file_set_contents(filePath, cssContent);

      const themeContext = St.ThemeContext.get_for_stage(global.stage);
      const theme = themeContext.get_theme();

      if (this._dynamicCssFile) {
        theme.unload_stylesheet(this._dynamicCssFile);
      }

      this._dynamicCssFile = Gio.File.new_for_path(filePath);
      theme.load_stylesheet(this._dynamicCssFile);

      this._logger.log(`[Liquid Glass] [UIManager] System accent color applied: ${colorStr}`);
    } catch (e) {
      this._logger.error(`[Liquid Glass] [UIManager] Failed to apply system accent color: ${e}`);
    }
  }

  // Utility: Convert HEX color string to normalized RGB array
  _hexToColorArray(hex: string): [number, number, number] {
    if (!hex || typeof hex !== 'string' || !hex.startsWith('#') || hex.length !== 7) return [1.0, 1.0, 1.0];
    let r = parseInt(hex.slice(1, 3), 16) / 255.0;
    let g = parseInt(hex.slice(3, 5), 16) / 255.0;
    let b = parseInt(hex.slice(5, 7), 16) / 255.0;
    return [r, g, b];
  }

  _getMenuMonitorGeometry() {
    let monitorIndex = Main.layoutManager.findIndexForActor(this.targetActor);
```

```
    if (monitorIndex < 0) {
      monitorIndex = Main.layoutManager.primaryIndex;
    }

    return Main.layoutManager.monitors[monitorIndex] || Main.layoutManager.primaryMonitor;
  }

  // 設定の動的反映
  _bindSettings() {
    const connectSetting = (key: string, callback: Function) => {
      let id = this._settings.connect(`changed::${key}`, callback.bind(this));
      this._settingsSignals.push(id);
    };

    // ON/OFF切り替え
    connectSetting('enable-menu-glass', () => {
      let enabled = this._settings.get_boolean('enable-menu-glass');
      if (enabled && !this._isEffectActive) this._applyEffect();
      else if (!enabled && this._isEffectActive) this._removeEffect();
    });

    connectSetting('enable-menu-animation', () => {
      this._enableAnimation = this._settings.get_boolean('enable-menu-animation');
    });

    connectSetting('menu-spring-stiffness', () => {
      this._springStiffness = this._settings.get_double('menu-spring-stiffness');
      if (this._springScale) this._springScale.updateParams(this._springStiffness, this._springDamping,
this._springMass);
    });

    connectSetting('menu-spring-damping', () => {
      this._springDamping = this._settings.get_double('menu-spring-damping');
      if (this._springScale) this._springScale.updateParams(this._springStiffness, this._springDamping,
this._springMass);
    });

    connectSetting('menu-spring-mass', () => {
      this._springMass = this._settings.get_double('menu-spring-mass');
      if (this._springScale) this._springScale.updateParams(this._springStiffness, this._springDamping,
this._springMass);
    });

    connectSetting('menu-animation-interval-ms', () => {
      this._animationInterval = this._settings.get_int('menu-animation-interval-ms');
    });

    connectSetting('menu-tint-color', () => {
      if (this.effect) {
        let colorArray = this._hexToColorArray(this._settings.get_string('menu-tint-color'));
        this.effect.setTintColor(...colorArray);
      }
    });

    connectSetting('menu-tint-strength', () => {
      if (this.effect) {
        this.effect.setTintStrength(this._settings.get_double('menu-tint-strength'));
      }
    });

    connectSetting('menu-blur-radius', () => {
      if (this.effect) {
        this.effect.setBlurRadius(this._settings.get_int('menu-blur-radius'));
      }
    });

    connectSetting('menu-brightness', () => {
      if (this.effect) {
        this.effect.setBrightness(this._settings.get_double('menu-brightness'));
      }
    });

    connectSetting('menu-contrast', () => {
      if (this.effect) {
        this.effect.setContrast(this._settings.get_double('menu-contrast'));
      }
    });

    connectSetting('menu-saturation', () => {
      if (this.effect) {
        this.effect.setSaturation(this._settings.get_double('menu-saturation'));
      }
```

```
    });

    connectSetting('menu-corner-radius', () => {
      if (this.effect) {
        this._cornerRadius = this._settings.get_double('menu-corner-radius');
        this.effect.setCornerRadius(this._cornerRadius);
      }
    });

    connectSetting('menu-glass-expand', () => {
      if (this.effect) {
        this._glassExpand = this._settings.get_int('menu-glass-expand');
      }
    });

    connectSetting('menu-x-offset', () => {
      if (this.animActor) {
        this._menuXoffset = this._settings.get_int('menu-x-offset');
        this.animActor.translation_x = this._menuXoffset;
      }
    });

    connectSetting('menu-y-offset', () => {
      if (this.animActor) {
        this._menuYoffset = this._settings.get_int('menu-y-offset');
        this.animActor.translation_y = this._menuYoffset;
      }
    });

    connectSetting('menu-enable-adaptive-text-color', () => {
      this._adaptiveConfig.enabled = this._settings.get_boolean('menu-enable-adaptive-text-color');
    });

    connectSetting('menu-sample-interval-ms', () => {
      this._adaptiveConfig.sampleIntervalMs = this._settings.get_int('menu-sample-interval-ms');
    });
  }

  _applyEffect() {
    if (this._isEffectActive) return;
    this._isEffectActive = true;

    if (!this.targetActor) return;

    // Remove default GNOME styling and make the background transparent
    this.targetActor.add_style_class_name('liquid-glass-transparent');
    this.animActor.add_style_class_name('liquid-glass-transparent');
    this.animActor.add_style_class_name('liquid-glass-menu-root');

    // Shift the menu to apply user offsets
    this._menuXoffset = this._settings.get_int('menu-x-offset');
    this._menuYoffset = this._settings.get_int('menu-y-offset');
    this.animActor.translation_x = this._menuXoffset;
    this.animActor.translation_y = this._menuYoffset;

    this._glassExpand = this._settings.get_int('menu-glass-expand');
    this._animationInterval = this._settings.get_int('menu-animation-interval-ms');

    this._adaptiveConfig = {
      ...AdaptiveContrastConfig,
      enabled: this._settings.get_boolean('menu-enable-adaptive-text-color'),
      samplePerElement: SAMPLE_PER_ELEMENT,
      sampleIntervalMs: this._settings.get_int('menu-sample-interval-ms'),
    };

    // 1. bgActor: full monitor, no effect — starts 1×1, _syncGeometry expands it immediately
    this.bgActor = new UnpickableActor();
    this.bgActor.set_name('liquid-glass-bg-actor');
    this.bgActor.set_size(1.0, 1.0);

    // 2. liquidBox: outer layer — LiquidEffect with built-in dual-Kawase blur
    this.liquidBox = new UnpickableActor();
    this.liquidBox.set_name("liquid-box");
    this.liquidBox.set_clip_to_allocation(true);
    this.bgActor.add_child(this.liquidBox);

    // dummyBreaker: transparent actor to prevent BMS black-screen optimization bug
    let dummyBreaker = new UnpickableActor();
    dummyBreaker.set_name("optimization-breaker");
    dummyBreaker.set_size(1.0, 1.0);
    dummyBreaker.set_opacity(0);
    this.liquidBox.add_child(dummyBreaker);
```

```
// 3. _cloneContainer: explicit sub-container inside liquidBox.
//    UILayerSampler deposits its _uiClonesContainer here.
//    WindowCloneManager places bgClone + windowClonesContainer directly in liquidBox.
this._cloneContainer = new UnpickableActor();
this._cloneContainer.set_name("clone-container");
this.liquidBox.add_child(this._cloneContainer);

// Set pivot points for scaling.
// The menu scales from the top-center (0.5, 0.0)
this.animActor.set_pivot_point(0.5, 0.0);
// bgActor scales from the top-left because we manually sync its exact coordinates
this.bgActor.set_pivot_point(0.0, 0.0);

// Find the uiGroup-direct ancestor of the menu actor so we can insert bgActor below it
let menuRoot: Clutter.Actor = this.menu.actor;
while (menuRoot.get_parent() && menuRoot.get_parent() !== Main.layoutManager.uiGroup) {
  const p = menuRoot.get_parent();
  if (!p) break;
  menuRoot = p;
}

// Insert bgActor below menuRoot in uiGroup to prevent recursive clone loops
if (menuRoot.get_parent() === Main.layoutManager.uiGroup) {
  // Main.layoutManager.uiGroup.insert_child_below(this.bgActor, menuRoot);
  Main.layoutManager.uiGroup.insert_child_above(this.bgActor, Main.layoutManager.panelBox);
} else {
  Main.layoutManager.uiGroup.add_child(this.bgActor);
}

// 4. WindowCloneManager: handles wallpaper clone + window actor clones
this._windowCloneManager = new WindowCloneManager(this.liquidBox, this._cloneContainer);

// 5. UILayerSampler: handles uiGroup child clones (panels, notifications, overview, etc.)
//    Exclude menuRoot and window groups to prevent recursive cloning and BMS loops.
this._uiSampler = new UILayerSampler(
  this.bgActor,
  this.liquidBox,
  [menuRoot, global.windowGroup, global.window_group],
  this._cloneContainer
);

let blurRadius = this._settings.get_int('menu-blur-radius');
let tintColorStr = this._settings.get_string('menu-tint-color');
let tintStrength = this._settings.get_double('menu-tint-strength');
let brightness = this._settings.get_double('menu-brightness');
let contrast = this._settings.get_double('menu-contrast');
let saturation = this._settings.get_double('menu-saturation');
this._cornerRadius = this._settings.get_double('menu-corner-radius');

// Apply our custom GLSL liquid shader to liquidBox (includes built-in dual-Kawase blur)
this.effect = new LiquidEffect({ extensionPath: this.extensionPath, settings: this._settings } as any);
this.effect.setPadding(SHADER_PADDING);
this.effect.setTintColor(...this._hexToColorArray(tintColorStr));
this.effect.setTintStrength(tintStrength);
this.effect.setCornerRadius(this._cornerRadius);
this.effect.setIsDock(false);
this.effect.setBrightness(brightness);
this.effect.setContrast(contrast);
this.effect.setSaturation(saturation);
this.effect.setBlurRadius(blurRadius);
this.liquidBox.add_effect(this.effect);

this.bgActor.hide();

// Helper functions to hook into GNOME's render pipeline
const laterAdd = (laterType: Meta.LaterType, callback: GLib.SourceFunc) => {
  return global.compositor?.get_laters?.().add(laterType, callback);
};

const laterRemove = (id: number) => {
  if (!id) return;
  if (global.compositor?.get_laters)
    global.compositor.get_laters().remove(id);
};

const frameLaterType = Meta.LaterType.BEFORE_REDRAW;

// Rebuild clones (called on menu open): delegate entirely to WindowCloneManager + UILayerSampler
let buildClones = () => {
  if (!this.bgActor) return;
```

```
        // _uiSampler が存在する場合のみ除外リストへの追加処理を行う
        if (this._uiSampler) {
          for (let child of Main.layoutManager.uiGroup.get_children()) {
            if (child === this.bgActor) continue;

            // 名前が 'liquid-glass-bg-actor' のもの、または 'liquid-box' を子に持つものを
            // 他のLiquid Glassエフェクトの背景アクターと判定する
            let isLiquidBg = child.name === 'liquid-glass-bg-actor' ||
              (typeof child.get_children === 'function' &&
                child.get_children().some(c => c.name === 'liquid-box'));

            if (isLiquidBg) {
              this._uiSampler.addExclusion(child);
            }
          }
        }

        this._windowCloneManager?.rebuildClones();
        this._uiSampler?.rebindSelf();
        this._uiSampler?.refresh();
      };

      // Render loop: called every frame while the menu is visible
      let frameTick = () => {
        this._frameSyncId = 0;
        if (!this.bgActor || !this.targetActor.mapped)
          return GLib.SOURCE_REMOVE;

        this._syncGeometry();
        this._frameSyncId = laterAdd(frameLaterType, frameTick);
        return GLib.SOURCE_REMOVE;
      };

      // Starts the render loop and builds fresh clones when the menu is opened
      let startFrameSync = () => {
        if (this._frameSyncId === 0) {
          buildClones();
          this._frameSyncId = laterAdd(frameLaterType, frameTick);
        }
      };

      let stopFrameSync = () => {
        if (this._frameSyncId !== 0) {
          laterRemove(this._frameSyncId);
          this._frameSyncId = 0;
        }
      };

      // Clear the cached size whenever the menu opens so it can recalculate
      // based on any new notifications or calendar events
      this._signals.push({
        target: this.menu,
        id: this.menu.connect('open-state-changed', (menu: any, isOpen: boolean) => {
          if (isOpen) {
            this._stableBaseW = undefined;
            this._stableBaseH = undefined;
            startFrameSync();
            this._startAdaptiveColorSampling(true);
          } else {
            this._stopAdaptiveColorSampling();
          }
        })
      });

      // Stop the render loop when the menu is fully hidden (mapped = false)
      this._signals.push({
        target: this.menu.actor,
        id: this.menu.actor.connect('notify::mapped', () => {
          if (!this.menu.actor.mapped) {
            stopFrameSync();

            if (this.bgActor) {
              this.bgActor.hide();
              this.bgActor.opacity = 0;
            }
            if (this.animActor) {
              this.animActor.opacity = 0;
            }
          }
        })
      });
```

```
    this._updateResolution();
    if (this.targetActor.mapped) {
      startFrameSync();
    }
  }

  // Calculates and synchronizes the position/size of the glass background every frame
  _syncGeometry() {
    if (!this.bgActor || !this.targetActor || !this.targetActor.mapped) {
      if (this.bgActor && this.bgActor.visible) {
        this.bgActor.hide();
      }
      return;
    }
    if (!this.bgActor.visible) {
      this.bgActor.show();
    }
    if (!this._enableAnimation) {
      this.bgActor.opacity = this.targetActor.opacity;
    }
    let [inW, inH] = this.animActor.get_size();
    let [outW, outH] = this.targetActor.get_size();
    let [scaleX, scaleY] = this.animActor.get_scale();

    inW = Number.isNaN(inW) || inW <= 0 ? (this._stableBaseW || 1) : inW;
    inH = Number.isNaN(inH) || inH <= 0 ? (this._stableBaseH || 1) : inH;
    scaleX = Number.isNaN(scaleX) ? 1.0 : scaleX;
    scaleY = Number.isNaN(scaleY) ? 1.0 : scaleY;

    scaleX *= this.targetActor.get_scale()[0];
    scaleY *= this.targetActor.get_scale()[1];

    let themeNode = this.animActor.get_theme_node();
    let mL = themeNode ? themeNode.get_margin(St.Side.LEFT) : 0;
    let mR = themeNode ? themeNode.get_margin(St.Side.RIGHT) : 0;
    let mT = themeNode ? themeNode.get_margin(St.Side.TOP) : 0;
    let mB = themeNode ? themeNode.get_margin(St.Side.BOTTOM) : 0;

    let marginW = mL + mR;
    let marginH = mT + mB;

    let targetW = Math.round(inW);
    let targetH = Math.round(inH);

    // GNOME Shell Hover Bug Compensation:
    if (Math.abs(inW - outW) <= 2 && marginW > 0) {
      targetW = Math.round(inW - marginW);
      targetH = Math.round(inH - marginH);
    }

    this._stableBaseW = targetW;
    this._stableBaseH = targetH;

    // Multiply by the current animation scale.
    let w = Math.max(1, this._stableBaseW * scaleX);
    let h = Math.max(1, this._stableBaseH * scaleY);

    // Get the absolute position of the inner content actor
    let [animAbsX, animAbsY] = this.animActor.get_transformed_position();

    // Advanced Fallback Logic for NaN Coordinates
    if (Number.isNaN(animAbsX) || Number.isNaN(animAbsY)) {
      if (this._lastValidAnimAbsX !== undefined && this._lastValidAnimAbsY !== undefined) {
        animAbsX = this._lastValidAnimAbsX;
        animAbsY = this._lastValidAnimAbsY;
      } else {
        let monitor = Main.layoutManager.primaryMonitor;
        if (monitor) {
          animAbsX = (monitor.width / 2) - (w / 2) + this._menuXoffset;
          animAbsY = (Main.panel.height || 27) + this._menuYoffset;
        } else {
          animAbsX = 0;
          animAbsY = 0;
        }
      }
    } else {
      this._lastValidAnimAbsX = animAbsX;
      this._lastValidAnimAbsY = animAbsY;
    }

    // The background needs to be larger than the UI to account for the glass expansion
    // and the extra padding required by the shader for edge refraction.
```

```
    let bgW = w + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgH = h + (this._glassExpand * 2) + (SHADER_PADDING * 2);
    let bgX = animAbsX - this._glassExpand - SHADER_PADDING;
    let bgY = animAbsY - this._glassExpand - SHADER_PADDING;

    // Monitor geometry — always valid (defaults to 0 if monitor is null)
    let monitor = this._getMenuMonitorGeometry();
    let monitorX = monitor?.x ?? 0;
    let monitorY = monitor?.y ?? 0;
    let screenW = Math.max(1, monitor?.width ?? 1);
    let screenH = Math.max(1, monitor?.height ?? 1);

    if (!Number.isNaN(bgX) && !Number.isNaN(bgY) && w >= 1.0 && h >= 1.0) {

      // Menu position in monitor-local coordinates (shader uses these)
      let localBgX = bgX - monitorX;
      let localBgY = bgY - monitorY;

      // Only update positions/sizes if they actually changed to save CPU cycles
      if (this._lastBgW !== bgW || this._lastBgH !== bgH ||
        this._lastBgX !== bgX || this._lastBgY !== bgY ||
        this._lastScreenW !== screenW || this._lastScreenH !== screenH) {

        // 1. bgActor: full monitor size, positioned at monitor origin
        this.bgActor.remove_transition('size');
        this.bgActor.remove_transition('position');
        this.bgActor.set_position(monitorX, monitorY);
        this.bgActor.set_size(screenW, screenH);
        this.bgActor.remove_transition('size');
        this.bgActor.remove_transition('position');

        // 2. liquidBox: full monitor size (relative to bgActor = 0,0)
        this.liquidBox?.set_position(0, 0);
        this.liquidBox?.set_size(screenW, screenH);

        // 3. GPU-efficient soft clip — limits rendering to the menu region +
        //    generous margin for drop-shadow decay without hard-clipping children.
        const CLIP_PADDING = 200;
        // this.liquidBox?.remove_clip();

        this.bgActor.set_clip(
          localBgX - CLIP_PADDING, localBgY - CLIP_PADDING,
          bgW + CLIP_PADDING * 2, bgH + CLIP_PADDING * 2
        );

        const SHADOW_MAX_RADIUS = CLIP_PADDING - 20;
        this.effect?.setShadowMaxRadius(SHADOW_MAX_RADIUS);

        // 4. Update shader with full-screen resolution
        this.effect?.setResolution(screenW, screenH);

        // 5. Tell the shader where the menu lives within the full-screen FBO
        //    (matches the dockManager setGlassGeometry pattern)
        this.effect?.setGlassGeometry(localBgX, localBgY, bgW, bgH);

        this._lastBgW = bgW; this._lastBgH = bgH;
        this._lastBgX = bgX; this._lastBgY = bgY;
        this._lastScreenW = screenW; this._lastScreenH = screenH;
      }
    }

    if (this.effect) {
      let currentScale = Math.min(scaleX, scaleY);
      this.effect.setCornerRadius(this._cornerRadius * currentScale);

      if (typeof this.effect.setAnimationScale === 'function') {
        this.effect.setAnimationScale(currentScale);
      }
    }

    // Clone sync every frame (dockManager pattern).
    // WindowCloneManager handles background + window actor clones.
    // UILayerSampler handles all uiGroup children — including the overview actors
    // automatically, so no separate overview/isOverview branch is needed.
    this._windowCloneManager?.setOffset(-monitorX, -monitorY);
    this._uiSampler?.refresh();
    this._uiSampler?.sync(monitorX, monitorY, screenW, screenH);
    this._windowCloneManager?.sync();
  }

  // Updates the shader resolution based on the current background actor size
  _updateResolution() {
```

```
    if (!this.bgActor || !this.effect) return;
    let [width, height] = this.bgActor.get_size();
    if (Number.isFinite(width) && Number.isFinite(height) && width > 0 && height > 0) {
      this.effect.setResolution(width, height);
    }
  }

  // Utility function to safely check if an actor has a specific style class
  _hasStyleClass(actor: Clutter.Actor, className: string) {
    return actor instanceof St.Widget &&
      actor.has_style_class_name(className);
  }

  _collectAdaptiveTextTargets(actor: Clutter.Actor = this.menu?.actor, targets: Clutter.Actor[] = []) {
    if (!actor) return targets;
    return this._findAllTextActors(this.menu?.actor);
  }

  _findAllTextActors(actor: Clutter.Actor, foundActors: Clutter.Actor[] = []) {
    if (!actor) return foundActors;

    if (actor instanceof St.Label || actor instanceof Clutter.Text || actor instanceof St.Button || actor instanceof
St.Icon) {
      if (actor.visible) {
        foundActors.push(actor);
      }
    }

    let children = typeof actor.get_children === 'function' ? actor.get_children() : [];
    for (let i = 0; i < children.length; i++) {
      this._findAllTextActors(children[i], foundActors);
    }

    return foundActors;
  }

  // Initiates the color change for a specific actor
  _setActorColor(actor: CustomBannerActor, color: string, skipAnimations = false) {
    if (!actor || typeof actor.set_style !== 'function') return;

    if (!this._styledActors.has(actor)) {
      let origStyle = typeof actor.get_style === 'function' ? actor.get_style() : null;
      this._styledActors.set(actor, origStyle || '');

      actor.connect('destroy', () => {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        this._styledActors.delete(actor);
      });
    }

    let isInsensitive = false;
    if (actor instanceof St.Button) {
      isInsensitive = (actor.reactive === false) || (typeof actor.has_style_pseudo_class === 'function' &&
actor.has_style_pseudo_class('insensitive'));
    }

    if (actor._currentTargetColor === color && actor._currentInsensitiveState === isInsensitive) return;
    actor._currentTargetColor = color;
    actor._currentInsensitiveState = isInsensitive;

    this._animateActorColor(actor, color, isInsensitive, 380, skipAnimations);
  }

  // Removes all dynamically applied adaptive text color styles and stops related animations
  _clearAdaptiveStyles() {
    for (const [actor, originalStyle] of this._styledActors.entries() as MapIterator<[CustomBannerActor, string]>) {
      if (actor && typeof actor.set_style === 'function') {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        actor._currentTargetColor = undefined;
        actor._currentInsensitiveState = undefined;
        try {
          actor.remove_style_class_name('adaptive-text-transition');
          actor.remove_style_class_name('adaptive-color-light');
          actor.remove_style_class_name('adaptive-color-dark');
          actor.set_style(originalStyle || null);
        } catch (e) { }
```

```
      }
    }
    this._styledActors.clear();

    const currentTargets = this._collectAdaptiveTextTargets() as CustomBannerActor[];
    for (let actor of currentTargets) {
      if (actor && typeof actor.set_style === 'function') {
        if (actor._colorTweenId) {
          GLib.source_remove(actor._colorTweenId);
          actor._colorTweenId = undefined;
        }
        actor._currentTargetColor = undefined;
        actor._currentInsensitiveState = undefined;
        try {
          actor.set_style(null);
        } catch (e) { }
      }
    }
  }

  // Iterates through the color map and applies the new target colors to the respective actors
  _applyAdaptiveColorMap(colorMap: Map<Clutter.Actor, string>, skipAnimations = false) {
    if (!colorMap || colorMap.size === 0)
      return;

    for (const [actor, color] of colorMap.entries()) {
      this._setActorColor(actor as unknown as CustomBannerActor, color, skipAnimations);
    }
  }

  // Starts the timer for periodically sampling contrast and updating adaptive text colors
  _startAdaptiveColorSampling(skipAnimations = false) {
    if (!this._adaptiveConfig.enabled)
      return;

    this._updateAdaptiveTextColors(skipAnimations);

    if (this._adaptiveTimerId !== 0)
      return;

    this._adaptiveTimerId = GLib.timeout_add(
      GLib.PRIORITY_DEFAULT,
      this._adaptiveConfig.sampleIntervalMs,
      () => {
        if (!this.menu?.isOpen) {
          this._adaptiveTimerId = 0;
          return GLib.SOURCE_REMOVE;
        }

        this._updateAdaptiveTextColors(false);
        return GLib.SOURCE_CONTINUE;
      }
    );
  }

  // Stops the adaptive color sampling timer
  _stopAdaptiveColorSampling() {
    if (this._adaptiveTimerId !== 0) {
      GLib.source_remove(this._adaptiveTimerId);
      this._adaptiveTimerId = 0;
    }
  }

  // Collects target actors, samples their contrast, and triggers color updates
  _updateAdaptiveTextColors(skipAnimations = false) {
    if (!this._adaptiveConfig.enabled || this._adaptiveInFlight)
      return;

    const targets = this._collectAdaptiveTextTargets();
    if (targets.length === 0)
      return;

    this._adaptiveInFlight = true;

    this._contrastSampler
      .chooseColorsForActors(targets, this._adaptiveConfig)
      .then(colorMap => {
        this._applyAdaptiveColorMap(colorMap, skipAnimations);
      })
      .catch(e => {
        this._logger.error(`[Liquid Glass] Menu adaptive color update failed: ${e}`);
      })
```

```
      .finally(() => {
        this._adaptiveInFlight = false;
      });
  }

  // Converts a hexadecimal color code string to an RGB object.
  _hexToRgb(hex: string) {
    let bigint = parseInt(hex.replace('#', ''), 16);
    return {
      r: (bigint >> 16) & 255,
      g: (bigint >> 8) & 255,
      b: bigint & 255
    };
  }

  // Converts RGB numerical values to a hexadecimal color string.
  _rgbToHex(r: number, g: number, b: number) {
    return "#" + (1 << 24 | r << 16 | g << 8 | b).toString(16).slice(1);
  }

  _animateActorColor(actor: CustomBannerActor, targetHexColor: string, isInsensitive: boolean, durationMs = 380,
skipAnimations = false) {
    if (!actor || Object.keys(actor).length === 0) return;

    if (actor._colorTweenId) {
      GLib.source_remove(actor._colorTweenId);
      actor._colorTweenId = undefined;
    }

    let themeNode = actor.get_theme_node();
    let startColor = themeNode.get_foreground_color();

    let targetRgb = this._hexToRgb(targetHexColor);

    let targetAlpha = isInsensitive ? 0.5 : 1.0;
    let startAlpha = startColor.alpha / 255.0;

    if (skipAnimations) {
      let alphaStr = targetAlpha.toFixed(3);
      let targetRgba = `rgba(${targetRgb.r}, ${targetRgb.g}, ${targetRgb.b}, ${alphaStr})`;
      try { actor.set_style(`color: ${targetRgba}; -st-icon-foreground-color: ${targetRgba};`); } catch (e) { }
      return;
    }

    let startTime = GLib.get_monotonic_time();

    actor._colorTweenId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 32, () => {
      if (!actor || Object.keys(actor).length === 0) return GLib.SOURCE_REMOVE;

      let currentTime = GLib.get_monotonic_time();
      let elapsedMs = (currentTime - startTime) / 1000;
      let progress = Math.min(elapsedMs / durationMs, 1.0);

      let easeProgress = progress < 0.5
        ? 2 * progress * progress
        : 1 - Math.pow(-2 * progress + 2, 2) / 2;

      let r = Math.round(startColor.red + (targetRgb.r - startColor.red) * easeProgress);
      let g = Math.round(startColor.green + (targetRgb.g - startColor.green) * easeProgress);
      let b = Math.round(startColor.blue + (targetRgb.b - startColor.blue) * easeProgress);

      let a = startAlpha + (targetAlpha - startAlpha) * easeProgress;
      a = Math.max(0.0, Math.min(1.0, a));

      let alphaStr = a.toFixed(3);
      let currentRgba = `rgba(${r}, ${g}, ${b}, ${alphaStr})`;

      try { actor.set_style(`color: ${currentRgba}; -st-icon-foreground-color: ${currentRgba};`); } catch (e) { }

      if (progress >= 1.0) {
        actor._colorTweenId = undefined;
        return GLib.SOURCE_REMOVE;
      }
      return GLib.SOURCE_CONTINUE;
    });
  }

  // Handles the custom bounce/spring physics when the menu opens or closes
  _startAnimation(targetValue: number) {
    let isClosing = (targetValue === 0);
    if (this._tickId !== 0) {
      GLib.source_remove(this._tickId);
```

```
      this._tickId = 0;
    }
    // If animation is disabled, just reset to default state
    if (!this._enableAnimation) {
      if (this.bgActor) {
        this.bgActor.remove_all_transitions();
        this.bgActor.opacity = 255;
        this.bgActor.set_scale(1.0, 1.0);

        if (this.animActor) {
          this.animActor.set_scale(1.0, 1.0);
          this.animActor.opacity = 255;
        }
      }
      return;
    }

    if (this.animActor) this.animActor.remove_all_transitions();
    if (this.bgActor) this.bgActor.remove_all_transitions();

    if (this._swiftAnimation) {
      this._swiftSpringScale.updateParams(this._swiftResponse, this._swiftDampingFraction);
      this._swiftSpringPos.updateParams(this._swiftResponse, this._swiftDampingFraction);
      this._swiftSpringScale.target = targetValue;
      this._swiftSpringPos.target = targetValue;
      if (Number.isNaN(this._swiftSpringScale.value)) this._swiftSpringScale.value = 0;
      if (Number.isNaN(this._swiftSpringPos.value)) this._swiftSpringPos.value = 0;
    } else {
      this._springScale.target = targetValue;
      this._springPos.target = targetValue;
    }

    if (this._tickId === 0) {
      let lastTime = GLib.get_monotonic_time();

      this._tickId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, this._animationInterval, () => {
        if (!this.bgActor || !this.targetActor) {
          this._tickId = 0;
          return GLib.SOURCE_REMOVE;
        }

        let currentTime = GLib.get_monotonic_time();
        let elapsedMs = (currentTime - lastTime) / 1000;
        lastTime = currentTime;

        let isClosing = this._swiftAnimation ? (this._swiftSpringScale.target === 0) : (this._springScale.target ===
0);

        let dt = elapsedMs / 1000;
        if (dt > 0.033) dt = 0.033;

        let stopped = false;
        let s: number, p: number;

        if (isClosing) {
          let speed = 15.0;
          if (this._swiftAnimation) {
            this._swiftSpringScale.value += (0 - this._swiftSpringScale.value) * (1.0 - Math.exp(-speed * dt));
            this._swiftSpringPos.value += (0 - this._swiftSpringPos.value) * (1.0 - Math.exp(-speed * dt));
            s = this._swiftSpringScale.value;
            p = this._swiftSpringPos.value;
          } else {
            this._springScale.value += (0 - this._springScale.value) * (1.0 - Math.exp(-speed * dt));
            this._springPos.value += (0 - this._springPos.value) * (1.0 - Math.exp(-speed * dt));
            s = this._springScale.value;
            p = this._springPos.value;
          }

          if (s < 0.005) {
            s = 0; p = 0;
            stopped = true;
          }
        } else {
          if (this._swiftAnimation) {
            stopped = this._swiftSpringScale.update(elapsedMs) && this._swiftSpringPos.update(elapsedMs);
            s = this._swiftSpringScale.value;
            p = this._swiftSpringPos.value;
          } else {
            stopped = this._springScale.update(elapsedMs) && this._springPos.update(elapsedMs);
            s = this._springScale.value;
            p = this._springPos.value;
          }
```

```
          if (Math.abs(1.0 - s) < 0.002 && Math.abs(this._swiftAnimation ? this._swiftSpringScale.velocity :
this._springScale.velocity) < 0.03) {
            s = 1.0;
            p = 1.0;
            stopped = true;
          }
        }

        let currentScale: number;
        let opacity: number;

        if (isClosing) {
          currentScale = Math.max(0.001, s);
          opacity = Math.min(255, Math.max(0, (s - 0.3) / 0.7 * 255));
        } else {
          currentScale = 0.2 + (s * 0.8);
          opacity = Math.min(255, Math.max(0, (s / 0.3) * 255));
        }

        this.animActor.set_scale(currentScale, currentScale);

        this.bgActor.opacity = opacity;
        this.animActor.opacity = opacity;

        this._syncGeometry();

        if (stopped) {
          this._tickId = 0;

          if (isClosing && this.menu.actor) {
            this.menu.actor.hide();
            this.bgActor.opacity = 0;
            this.animActor.opacity = 0;
          }

          if (!isClosing) {
            this.animActor.set_scale(1.0, 1.0);
            this.animActor.opacity = 255;
            this.bgActor.opacity = 255;
            this._syncGeometry();
          }
          return GLib.SOURCE_REMOVE;
        }
        return GLib.SOURCE_CONTINUE;
      });
    }
  }

  _removeEffect() {
    if (!this._isEffectActive) return;
    this._isEffectActive = false;

    this._stopAdaptiveColorSampling();
    this._clearAdaptiveStyles();

    // Disconnect all event listeners
    for (let sig of this._signals) {
      try {
        if (sig && sig.id) sig.target.disconnect(sig.id);
      } catch (e) { }
    }
    this._signals = [];

    if (this._tickId && this._tickId !== 0) {
      GLib.Source.remove(this._tickId);
      this._tickId = 0;
    }

    // Stop the render frame loop
    if (this._frameSyncId !== 0) {
      if (global.compositor?.get_laters)
        global.compositor.get_laters().remove(this._frameSyncId);
      this._frameSyncId = 0;
    }

    if (this._interfaceSettings && this._accentColorSignalId) {
      this._interfaceSettings.disconnect(this._accentColorSignalId);
      this._accentColorSignalId = 0;
      this._interfaceSettings = null;
    }
```

```
    // Remove transparent CSS overrides
    this.targetActor.remove_style_class_name('liquid-glass-transparent');
    if (this.animActor) {
      this.animActor.remove_style_class_name('liquid-glass-transparent');
      this.animActor.remove_style_class_name('liquid-glass-menu-root');

      this.animActor.translation_y = 0;
      this.animActor.set_scale(1.0, 1.0);
      this.animActor.opacity = 255;
    }
    if (this._dynamicCssFile) {
      const themeContext = St.ThemeContext.get_for_stage(global.stage);
      const theme = themeContext.get_theme();
      theme.unload_stylesheet(this._dynamicCssFile);
      this._dynamicCssFile = null;
    }

    this.targetActor.translation_y = 0;
    this.targetActor.set_scale(1.0, 1.0);
    this.targetActor.opacity = 255;

    if (this.menu.actor) {
      this.menu.actor.opacity = 255;

      if (this.menu.isOpen) {
        this.menu.close(false);
      }
    }

    // DESTROY EFFECT FIRST
    if (this.effect) {
      this.effect.cleanup();
      this.effect = null;
    }

    // DESTROY ACTOR SECOND
    // bgActor.destroy() cascades through liquidBox → _cloneContainer
    // and its children, so we only need to null the references afterwards.
    if (this.bgActor) {
      this.bgActor.destroy();
      this.bgActor = null;
    }
    this.liquidBox = null;
    this._cloneContainer = null;

    // Clean up managers (try-catch in their destroy() handles already-destroyed actors)
    this._uiSampler?.destroy();
    this._uiSampler = null;
    this._windowCloneManager?.destroy();
    this._windowCloneManager = null;

    this._stableBaseW = undefined;
    this._stableBaseH = undefined;
  }

  cleanup() {
    for (let sigId of this._settingsSignals) {
      try { this._settings.disconnect(sigId); } catch (e) { }
    }
    this._settingsSignals = [];

    if (!this.targetActor) return;
    this._removeEffect();
  }
}

// A straightforward mathematical implementation of Hooke's Law for spring physics
class Spring {
  private stiffness: number;
  private damping: number;
  private mass: number;
  public value: number;
  public velocity: number;
  public target: number;

  constructor(stiffness: number, damping: number, mass: number) {
    this.stiffness = stiffness;
    this.damping = damping;
    this.mass = mass;

    this.value = 0;
    this.velocity = 0;
```

```
    this.target = 0;
  }

  updateParams(stiffness: number, damping: number, mass: number) {
    this.stiffness = stiffness;
    this.damping = damping;
    this.mass = mass;
  }

  update(elapsedMs: number) {
    let dt = elapsedMs / 1000;
    if (dt > 0.033) dt = 0.033;

    let springForce = -this.stiffness * (this.value - this.target);
    let dampingForce = -this.damping * this.velocity;
    let acceleration = (springForce + dampingForce) / this.mass;

    this.velocity += acceleration * dt;
    this.value += this.velocity * dt;

    return Math.abs(this.velocity) < 0.01 && Math.abs(this.value - this.target) < 0.001;
  }
}

class SwiftSpring {
  response: number;
  dampingFraction: number;
  mass: number;

  value: number;
  velocity: number;
  target: number;

  constructor(response: number, dampingFraction: number, mass: number = 1.0) {
    this.response = typeof response === 'number' && !isNaN(response) && response > 0.01 ? response : 0.4;
    this.dampingFraction = typeof dampingFraction === 'number' && !isNaN(dampingFraction) && dampingFraction >= 0 ?
dampingFraction : 0.7;
    this.mass = typeof mass === 'number' && !isNaN(mass) && mass > 0.01 ? mass : 1.0;

    this.value = 0;
    this.velocity = 0;
    this.target = 0;
  }

  updateParams(response: number, dampingFraction: number, mass: number = 1.0) {
    if (typeof response === 'number' && !isNaN(response) && response > 0.01) this.response = response;
    if (typeof dampingFraction === 'number' && !isNaN(dampingFraction) && dampingFraction >= 0) this.dampingFraction =
dampingFraction;
    if (typeof mass === 'number' && !isNaN(mass) && mass > 0.01) this.mass = mass;
  }

  update(elapsedMs: number): boolean {
    let dt = elapsedMs / 1000;

    if (isNaN(dt) || dt <= 0) return false;
    if (dt > 0.1) dt = 0.1;

    if (isNaN(this.value) || !isFinite(this.value) || isNaN(this.velocity) || !isFinite(this.velocity)) {
      this.value = this.target;
      this.velocity = 0;
      return true;
    }

    const x0 = this.value - this.target;
    const v0 = this.velocity;

    if (Math.abs(x0) < 0.001 && Math.abs(v0) < 0.001) {
      this.value = this.target;
      this.velocity = 0;
      return true;
    }

    const omega0 = (2 * Math.PI) / this.response;
    const zeta = this.dampingFraction;

    let x_t = 0;
    let v_t = 0;

    // Analytical solution — no numerical explosion regardless of spring stiffness
    if (zeta < 0.999) {
      // 1. Underdamped — standard bouncy motion
      const omegaD = omega0 * Math.sqrt(1.0 - zeta * zeta);
```

```
      const alpha = zeta * omega0;
      const exp = Math.exp(-alpha * dt);
      const cos = Math.cos(omegaD * dt);
      const sin = Math.sin(omegaD * dt);

      x_t = exp * (x0 * cos + ((v0 + alpha * x0) / omegaD) * sin);
      v_t = exp * (v0 * cos - ((alpha * v0 + omega0 * omega0 * x0) / omegaD) * sin);
    } else if (zeta > 1.001) {
      // 2. Overdamped — slow, viscous motion
      const beta = omega0 * Math.sqrt(zeta * zeta - 1.0);
      const gamma1 = -zeta * omega0 + beta;
      const gamma2 = -zeta * omega0 - beta;
      const exp1 = Math.exp(gamma1 * dt);
      const exp2 = Math.exp(gamma2 * dt);

      const c1 = (v0 - gamma2 * x0) / (gamma1 - gamma2);
      const c2 = x0 - c1;

      x_t = c1 * exp1 + c2 * exp2;
      v_t = c1 * gamma1 * exp1 + c2 * gamma2 * exp2;
    } else {
      // 3. Critically damped — fastest settle without overshoot
      const exp = Math.exp(-omega0 * dt);
      x_t = exp * (x0 + (v0 + omega0 * x0) * dt);
      v_t = exp * (v0 - omega0 * (v0 + omega0 * x0) * dt);
    }

    this.value = x_t + this.target;
    this.velocity = v_t;

    if (isNaN(this.value) || !isFinite(this.value)) {
      this.value = this.target;
      this.velocity = 0;
      return true;
    }
    this.value = Math.max(-0.5, Math.min(2.5, this.value));

    if (Math.abs(this.value - this.target) < 0.001 && Math.abs(this.velocity) < 0.001) {
      this.value = this.target;
      this.velocity = 0;
      return true;
    }

    return false;
  }
}
```

```
// utils.ts
//
// Shared helpers for the Liquid Glass extension: actors that stay invisible
// to Looking Glass's picker, the UI-layer sampler that clones the desktop
// behind the glass, and the special-case handling needed to render a blurred
// panel (from the Blur My Shell extension) inside the glass without breaking
// the real panel's own blur.
import GObject from 'gi://GObject';
import Clutter from 'gi://Clutter';
import Cogl from 'gi://Cogl';
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import St from 'gi://St';
import Mtk from 'gi://Mtk';


/**
 * Captures a small rectangle of the screen (the panel area) into a
 * `Clutter.Content`, for use as the "blurred panel" backdrop inside the
 * glass, while structurally guaranteeing the glass never captures itself.
 *
 * Background: Blur My Shell (BMS) blurs the real top panel using a native
 * (non-JS) Clutter effect. That effect has no public API to read its result,
 * and — critically — it assumes it is the *only* consumer of its target
 * actor's paint output. Cloning the BMS target directly (`Clutter.Clone`)
 * makes BMS think a second consumer has taken over, and the *real* panel
 * loses its blur. So we never clone or paint the BMS actor at all; instead
 * we take an independent snapshot of "what the screen looks like there".
 *
 * Why `paint_to_content()` specifically: it performs a one-off, synchronous
 * render of a stage rectangle into an offscreen buffer, completely separate
 * from the actual on-screen frame. That gives us two things a live
 * `Clutter.Clone` cannot:
 *   1. Self-exclusion: our own glass root (`bgActor`) sits directly above
 *      the panel in z-order and can visually overlap it (e.g. when a
 *      panel-anchored popup is open). If we captured "the whole composited
 *      screen" while our own glass was visible, we would capture our own
 *      glass along with the panel — and since we redraw using that captured
 *      image every frame, this becomes a runaway feedback loop: each new
 *      capture already contains yesterday's capture, nested one level
 *      deeper, forever. (Diagnostic tip that confirmed this: the nesting
 *      alternated right-side-up / upside-down with each additional level,
 *      matching a V-flip correction from an older capture method being
 *      compounded once per loop iteration.)
 *      We avoid this entirely by hiding our own root actor for the single
 *      synchronous `paint_to_content()` call, then restoring it immediately
 *      — so the glass structurally cannot appear in its own snapshot.
 *   2. No visible flicker: hide → capture → show all happens synchronously,
 *      before control returns to Clutter's normal repaint cycle, so the
 *      actual displayed frame is never affected.
 *
 * We also always pass `Clutter.PaintFlag.NO_CURSORS`. GNOME Shell's own
 * screenshot code (shell-screenshot.c) does the same for this exact API —
 * without it, the mouse pointer sprite gets composited into the snapshot,
 * which shows up as cursor smearing inside the glass.
 */
export class SelfExcludingSnapshotCapture {
  private _content: any = null;
  private _rectGetter: () => [number, number, number, number];

  private _hideActors: Set<Clutter.Actor> = new Set();
  private _stage: Clutter.Stage;
  private _refCount: number = 0;
  private _afterPaintId: number = 0;
  private _destroyed: boolean = false;

  // Re-capture on every 'after-paint' rather than a fixed timer: this way
  // updates only happen (and only cost anything) while the screen is
  // actually changing, and are as fresh as the display's own refresh rate.
  // Raise FRAME_SKIP if this ever proves too expensive on slower hardware
  // (2 = every other frame, etc.) — 1 keeps it perfectly in sync.
  private static readonly FRAME_SKIP = 1;
  private _frameCounter: number = 0;

  constructor(stage: Clutter.Stage, hideActor: Clutter.Actor, rectGetter: () => [number, number, number, number]) {
    this._stage = stage;
    if (hideActor) this._hideActors.add(hideActor);
    this._rectGetter = rectGetter;
    this._captureOnce();
    try {
      this._afterPaintId = (this._stage as any).connect('after-paint', () => {
        if (this._destroyed) return;
        this._frameCounter++;
```

```
        if (this._frameCounter % SelfExcludingSnapshotCapture.FRAME_SKIP !== 0) return;
        this._captureOnce();
      });
    } catch (e) {
    }
  }

  retain(): void { this._refCount++; }
  release(): boolean {
    this._refCount--;
    if (this._refCount <= 0) { this.destroy(); return true; }
    return false;
  }

  /** Registers another Liquid Glass instance's root as needing to be hidden during capture. */
  addHideActor(actor: Clutter.Actor | null | undefined): void {
    if (actor) this._hideActors.add(actor);
  }

  /** Unregisters a previously-added hide actor (called when that instance releases the capture). */
  removeHideActor(actor: Clutter.Actor | null | undefined): void {
    if (actor) this._hideActors.delete(actor);
  }

  private _captureOnce(): void {
    const [x, y, w, h] = this._rectGetter();
    if (w <= 0 || h <= 0) return;

    // Hide every registered instance's root, not just a single
    // one, so a shared capture never leaks any glass instance into itself.
    const hidden: Clutter.Actor[] = [];
    try {
      for (const actor of this._hideActors) {
        try {
          if (actor && actor.visible) {
            actor.hide();
            hidden.push(actor);
          }
        } catch (_) { /* actor may have been destroyed; skip it */ }
      }

      const rect = new Mtk.Rectangle({ x: Math.round(x), y: Math.round(y), width: Math.round(w), height:
Math.round(h) });
      const scale = 1; // TODO: honor per-monitor resource scale if this is ever used on HiDPI setups.

      // Signature is (rect, scale, color_state, paint_flags); color_state
      // of null uses the default color space. NO_CURSORS excludes the
      // mouse pointer sprite from the snapshot (see class doc comment).
      const paintFlags = (Clutter as any).PaintFlag?.NO_CURSORS ?? 0;
      const content = (this._stage as any).paint_to_content?.(rect, scale, null, paintFlags);
      if (content) this._content = content;
    } catch (e) {
    } finally {
      for (const actor of hidden) {
        try { actor.show(); } catch (_) { /* actor may have been destroyed; skip it */ }
      }
    }
  }

  getContent(): any | null {
    return this._content;
  }

  destroy(): void {
    this._destroyed = true;
    if (this._afterPaintId) {
      try { (this._stage as any).disconnect(this._afterPaintId); } catch (_) { /* noop */ }
      this._afterPaintId = 0;
    }
  }
}

// Shared pool: multiple UILayerSampler instances (e.g. a permanent dock glass
// and a popup-menu glass) may want to capture the same BMS target. Keying by
// source actor lets them share a single capture instead of duplicating work.
const _selfExcludingSnapshotRegistry: Map<Clutter.Actor, SelfExcludingSnapshotCapture> = new Map();

function acquireSelfExcludingSnapshot(
  sourceActor: Clutter.Actor,
  stage: Clutter.Stage,
  hideActor: Clutter.Actor,
  rectGetter: () => [number, number, number, number]
```

```
): SelfExcludingSnapshotCapture {
  let cap = _selfExcludingSnapshotRegistry.get(sourceActor);
  if (!cap) {
    cap = new SelfExcludingSnapshotCapture(stage, hideActor, rectGetter);
    _selfExcludingSnapshotRegistry.set(sourceActor, cap);
  } else {
    cap.addHideActor(hideActor);
  }
  cap.retain();
  return cap;
}

function releaseSelfExcludingSnapshot(sourceActor: Clutter.Actor, hideActor?: Clutter.Actor): void {
  const cap = _selfExcludingSnapshotRegistry.get(sourceActor);
  if (!cap) return;
  // Unregister our hide actor first so a capture that outlives us (still
  // retained by another instance) doesn't keep trying to hide an actor we
  // no longer care about.
  cap.removeHideActor(hideActor);
  if (cap.release()) {
    _selfExcludingSnapshotRegistry.delete(sourceActor);
  }
}

/**
 * A Clutter.Clone whose pick pass is a no-op, so Looking Glass's actor
 * picker sees through it to whatever is behind.
 */
export const UnpickableClone = GObject.registerClass(
  class UnpickableClone extends Clutter.Clone {
    vfunc_pick(_pickContext: any): void {
      // No-op: never respond to picking.
    }
  }
);

/**
 * A plain container actor with the same "invisible to picking" behavior as
 * UnpickableClone. Uses Clutter.Actor rather than St.Widget to avoid St's
 * CSS/theming padding interfering with pixel-precise layout.
 */
export const UnpickableActor = GObject.registerClass(
  class UnpickableActor extends Clutter.Actor {
    vfunc_pick(_pickContext: any): void {
      // No-op: never respond to picking.
    }
  }
);

/**
 * St.Widget variant of the same "invisible to picking" behavior, for cases
 * that need St's styling/layout features.
 */
export const UnpickableWidget = GObject.registerClass(
  class UnpickableWidget extends St.Widget {
    vfunc_pick(_pickContext: any): void {
      // No-op: never respond to picking.
    }
  }
);

/**
 * Paints a captured texture stretched to fill its own allocation, without
 * ever triggering the source actor's own paint. Used for the "read an
 * existing OffscreenEffect's texture" fallback path (see
 * UILayerSampler._createExistingEffectBlitActor): unlike Clutter.Clone,
 * this never re-evaluates the source's effect chain, so it can't cause the
 * "two consumers" ownership conflict described on SelfExcludingSnapshotCapture.
 */
export const TextureBlitActor = GObject.registerClass({
  GTypeName: 'LiquidGlassTextureBlitActor',
}, class TextureBlitActor extends Clutter.Actor {

  declare private _getTexture: (() => Cogl.Texture2D | null) | null;
  declare private _sourceActor: Clutter.Actor | null;
  declare private _pipeline: Cogl.Pipeline | null;

  _init(params: any = {}) {
    super._init(params);
    this._getTexture = null;
    this._sourceActor = null;
    this._pipeline = null;
```

```
  }

  vfunc_pick(_pickContext: any): void { }

  setTextureGetter(fn: () => Cogl.Texture2D | null): void {
    this._getTexture = fn;
  }

  setSourceActor(actor: Clutter.Actor): void {
    this._sourceActor = actor;
  }

  private _getCoglContext(): Cogl.Context | null {
    try {
      const backend = Clutter.get_default_backend();
      return backend.get_cogl_context() as Cogl.Context;
    } catch (e) {
      return null;
    }
  }

  vfunc_paint(paintContext: Clutter.PaintContext): void {
    if (!this._getTexture) return;
    const tex = this._getTexture();
    if (!tex) return;

    try {
      if (!this._pipeline) {
        const ctx = this._getCoglContext();
        if (!ctx) return;
        this._pipeline = Cogl.Pipeline.new(ctx);
        this._pipeline.set_layer_wrap_mode(0, Cogl.PipelineWrapMode.CLAMP_TO_EDGE);
        this._pipeline.set_layer_filters(
          0, Cogl.PipelineFilter.LINEAR, Cogl.PipelineFilter.LINEAR
        );
      }

      const texW = tex.get_width();
      const texH = tex.get_height();

      // Some OffscreenEffect implementations pad their captured texture a
      // few pixels beyond the actor's logical size (Cogl FBO alignment).
      // If so, sample only the centered sub-rectangle matching the source's
      // actual allocated size.
      let uMin = 0, vMin = 0, uMax = 1, vMax = 1;
      const src = this._sourceActor;
      if (src) {
        const [rawW, rawH] = src.get_size();
        const allocW = Number.isFinite(rawW) && rawW > 0 ? Math.round(rawW) : texW;
        const allocH = Number.isFinite(rawH) && rawH > 0 ? Math.round(rawH) : texH;

        if ((allocW !== texW || allocH !== texH) && texW > 0 && texH > 0) {
          const padW = texW - allocW;
          const padH = texH - allocH;
          uMin = (padW / 2) / texW;
          vMin = (padH / 2) / texH;
          uMax = Math.min(1.0, uMin + allocW / texW);
          vMax = Math.min(1.0, vMin + allocH / texH);
        }
      }

      this._pipeline.set_layer_texture(0, tex);

      const [w, h] = this.get_size();
      if (!(w > 0) || !(h > 0)) return;

      const fb = paintContext.get_framebuffer() as unknown as Cogl.Framebuffer;
      fb.draw_textured_rectangle(this._pipeline, 0, 0, w, h, uMin, vMin, uMax, vMax);
    } catch (e) {
    }
  }
});
export type TextureBlitActor = InstanceType<typeof TextureBlitActor>;

/**
 * Clones every actor under `Main.layoutManager.uiGroup` into a private
 * container so the glass can render a distorted/blurred view of "everything
 * behind it" (panel, windows, other extensions' UI). One instance per glass
 * (permanent dock glass, popup-menu glass, etc).
 */
export class UILayerSampler {
  private readonly _selfActor: Clutter.Actor;
```

```
  private readonly _container: Clutter.Actor;
  private readonly _extraExclusions: Set<Clutter.Actor>;

  private _selfRoot: Clutter.Actor | null = null;
  private _clones: Map<Clutter.Actor, Clutter.Actor> = new Map();
  private _uiClonesContainer: Clutter.Actor | null = null;

  // Read-only cache: for each uiGroup child, either the (actor, effect) pair
  // of an existing Clutter.OffscreenEffect found in its subtree, or null if
  // none was found. Never written to by us (no actor tree mutation), so
  // sharing this cache across multiple UILayerSampler instances is safe.
  private _existingEffectCache: Map<Clutter.Actor, { actor: Clutter.Actor; effect: Clutter.OffscreenEffect } | null> =
new Map();

  // While true, a uiGroup child containing the Blur My Shell target is
  // rendered via _createExistingEffectBlitActor() if a usable
  // Clutter.OffscreenEffect is found in its subtree. BMS's actual blur is a
  // native (non-JS) effect that never matches this, so BMS itself always
  // falls through — this toggle mainly matters for *other* extensions that
  // implement their effects as a JS Clutter.OffscreenEffect subclass.
  private _useCaptureFixForBms: boolean = true;

  // clone actor -> { source actor (BMS target's uiGroup child), hideActor (our own selfRoot) }
  // Both are needed on destroy to release exactly what we registered on the
  // (possibly shared) SelfExcludingSnapshotCapture.
  private _delayedCaptureOwners: Map<Clutter.Actor, { source: Clutter.Actor; hideActor: Clutter.Actor }> = new Map();

  constructor(
    selfActor: Clutter.Actor,
    container: Clutter.Actor,
    extraExclusions: Clutter.Actor[] = [],
    cloneContainer: Clutter.Actor | null = null
  ) {
    this._selfActor = selfActor;
    this._container = container;
    this._extraExclusions = new Set(extraExclusions);
    this._selfRoot = this._findUiGroupAncestor(selfActor);

    this._uiClonesContainer = new UnpickableActor();
    this._uiClonesContainer.set_name("ui-clones-container");

    // Connect to the destroy signal and assign null
    this._uiClonesContainer.connect('destroy', () => {
      this._uiClonesContainer = null;
    });

    if (cloneContainer) {
      cloneContainer.add_child(this._uiClonesContainer);
    } else {
      this._container.add_child(this._uiClonesContainer);
    }
  }

  private _findUiGroupAncestor(actor: Clutter.Actor): Clutter.Actor | null {
    const uiGroup = Main.layoutManager.uiGroup;
    let current: Clutter.Actor | null = actor;
    while (current) {
      if (current.get_parent() === uiGroup) return current;
      current = current.get_parent();
    }
    return null;
  }

  /** Adds an actor to the set of uiGroup children that should never be cloned. */
  addExclusion(actor: Clutter.Actor) {
    if (!actor) return;
    this._extraExclusions.add(actor);
  }

  /**
   * Resolves the Blur My Shell panel-blur target actor via
   * Main.extensionManager, if BMS is installed and enabled and its internal
   * structure matches what we expect. Everything here is best-effort and
   * guarded: if BMS is absent or has changed shape, this simply returns
   * null and callers fall back to normal cloning.
   */
  private _resolveBmsTargetActor(): Clutter.Actor | null {
    try {
      const ext = (Main as any).extensionManager?.lookup?.('blur-my-shell@aunetx');
      const actor = ext?.stateObj?._panel_blur?.actors_list?.[0]?.bg_manager?.backgroundActor;
      return (actor as Clutter.Actor) ?? null;
    } catch (_) {
```

```
      return null;
    }
  }

  /**
   * Returns the BMS target actor if `child` (a direct uiGroup child) either
   * *is* the BMS target or contains it as a descendant — i.e. whether
   * cloning `child` would also clone BMS's blurred panel.
   */
  private _findBmsDescendant(child: Clutter.Actor): Clutter.Actor | null {
    const target = this._resolveBmsTargetActor();
    if (!target) return null;
    if (child === target) return target;
    try {
      if (typeof (child as any).contains === 'function' && (child as any).contains(target)) {
        return target;
      }
    } catch (_) { /* noop */ }
    return null;
  }

  /**
   * @deprecated no-op, kept only so older callers that still toggle these
   * debug switches don't break. The multi-paint diagnostic probe and the
   * "force-hide the BMS clone" A/B switch they used to control have both
   * been removed now that the real fix (SelfExcludingSnapshotCapture) is in
   * place.
   */
  setDebugDisableBmsClone(_disabled: boolean): void { /* no-op */ }
  /** @deprecated no-op, see setDebugDisableBmsClone. */
  setDebugBmsProbeEnabled(_enabled: boolean): void { /* no-op */ }

  /**
   * Primary path for rendering the BMS-blurred panel inside the glass. See
   * SelfExcludingSnapshotCapture for the full rationale. Returns null (and
   * lets the caller fall back) if this Clutter version lacks
   * `Stage.paint_to_content()`.
   */
  private _createSelfExcludingSnapshotActor(child: Clutter.Actor): Clutter.Actor | null {
    try {
      const stage = child.get_stage() as Clutter.Stage | null;
      if (!stage) return null;
      if (typeof (stage as any).paint_to_content !== 'function') return null;
      if (!this._selfRoot) return null;
      const selfRoot = this._selfRoot;

      const rectGetter = (): [number, number, number, number] => {
        const [x, y] = child.get_transformed_position();
        const [w, h] = child.get_size();
        if (Number.isNaN(x) || Number.isNaN(y) || w <= 0 || h <= 0) {
          return [0, 0, 0, 0];
        }
        return [x, y, w, h];
      };

      const capture = acquireSelfExcludingSnapshot(child, stage, selfRoot, rectGetter);

      const actor = new UnpickableActor();
      actor.set_name(`${child.name}-selfExcludingSnapshot`);

      // Push new content onto the actor whenever the capture updates,
      // rather than polling on a timer, so this stays in lockstep with
      // SelfExcludingSnapshotCapture's own 'after-paint'-driven refresh.
      const applyContent = () => {
        if ((actor as any)._isDisposed) return;
        const content = capture.getContent();
        if (content && actor.content !== content) {
          actor.content = content;
        }
      };
      let afterPaintId = 0;
      try {
        afterPaintId = (stage as any).connect('after-paint', applyContent);
      } catch (e) {
      }
      applyContent();

      this._delayedCaptureOwners.set(actor, { source: child, hideActor: selfRoot });
      actor.connect('destroy', () => {
        (actor as any)._isDisposed = true;
        if (afterPaintId) { try { (stage as any).disconnect(afterPaintId); } catch (_) { /* noop */ } }
        const owner = this._delayedCaptureOwners.get(actor);
```

```
        if (owner) {
          releaseSelfExcludingSnapshot(owner.source, owner.hideActor);
          this._delayedCaptureOwners.delete(actor);
        }
      });

      return actor;
    } catch (e) {
      return null;
    }
  }

  /** Toggle for the OffscreenEffect-reading fallback (see _useCaptureFixForBms). */
  setUseCaptureFixForBms(enabled: boolean): void {
    this._useCaptureFixForBms = enabled;
  }


  /**
   * Searches `root`'s subtree (read-only, no mutation) for an existing
   * Clutter.OffscreenEffect — e.g. a blur implemented as a JS effect by some
   * other extension. Our own debug effects (GTypeName starting with
   * "LiquidGlass") are skipped so we never pick up our own instrumentation.
   */
  private _findExistingOffscreenEffect(
    root: Clutter.Actor
  ): { actor: Clutter.Actor; effect: Clutter.OffscreenEffect } | null {
    const stack: Clutter.Actor[] = [root];
    const visited = new Set<Clutter.Actor>();

    while (stack.length > 0) {
      const actor = stack.pop()!;
      if (visited.has(actor)) continue;
      visited.add(actor);

      try {
        const effects: Clutter.Effect[] = (actor as any).get_effects?.() ?? [];
        for (const effect of effects) {
          if (!(effect instanceof Clutter.OffscreenEffect)) continue;
          const gtypeName = (effect.constructor as any)?.$gtype?.name ?? '';
          if (gtypeName.startsWith('LiquidGlass')) continue;
          return { actor, effect: effect as Clutter.OffscreenEffect };
        }

        const children: Clutter.Actor[] = (actor as any).get_children?.() ?? [];
        for (const c of children) stack.push(c);
      } catch (_) { /* noop */ }
    }
    return null;
  }

  /**
   * Fallback for BMS-target children when SelfExcludingSnapshotCapture is
   * unavailable: reads an existing OffscreenEffect's captured texture
   * directly, without adding anything to the actor tree. This never
   * matches BMS's own native blur effect (see class doc comment on
   * _useCaptureFixForBms) but can help other, JS-effect-based extensions.
   * Returns null if nothing suitable was found.
   */
  private _createExistingEffectBlitActor(child: Clutter.Actor): Clutter.Actor | null {
    let found = this._existingEffectCache.get(child);
    if (found === undefined) {
      found = this._findExistingOffscreenEffect(child);
      this._existingEffectCache.set(child, found);
    }
    if (!found) return null;

    const { actor: effectOwner, effect } = found;
    const blit = new TextureBlitActor();
    blit.setSourceActor(effectOwner);
    blit.setTextureGetter(() => effect.get_texture() as Cogl.Texture2D | null);
    return blit;
  }

  rebindSelf() {
    this._selfRoot = this._findUiGroupAncestor(this._selfActor);
  }

  /**
   * Returns true if `root`'s subtree contains the root actor of *another*
   * Liquid Glass instance (bgActor, named 'liquid-glass-bg-actor', or its
   * child liquidBox, named 'liquid-box'). Searched recursively with no
```

```
     * depth limit — a shallow, direct-child-only check is not enough: if a
     * glass instance's root ends up nested more than one level below a
     * uiGroup child (e.g. when multiple popups are open at once, or another
     * container wraps it), a shallow check silently misses it and that whole
     * instance — including whatever it has already rendered — gets cloned
     * into this glass, producing a visible "glass inside glass" nesting
     * artifact.
     */
    private _containsOtherLiquidGlassRoot(root: Clutter.Actor): boolean {
      const stack: Clutter.Actor[] = [root];
      const visited = new Set<Clutter.Actor>();
      while (stack.length > 0) {
        const actor = stack.pop()!;
        if (visited.has(actor)) continue;
        visited.add(actor);
        try {
          const name = (actor as any).name;
          if (name === 'liquid-glass-bg-actor' || name === 'liquid-box') return true;
          const children: Clutter.Actor[] = (actor as any).get_children?.() ?? [];
          for (const c of children) stack.push(c);
        } catch (_) { /* noop */ }
      }
      return false;
    }

    /**
     * Repositions a freshly-added clone within `_uiClonesContainer` to match
     * `child`'s real z-order among `uiGroup`'s children, rather than leaving
     * it wherever `add_child()` put it (always the front).
     *
     * Without this, any uiGroup child that appears *after* the glass was
     * already showing other clones — e.g. the full-screen blurred backdrop
     * GNOME's Activities/Overview creates — ends up rendered in front of
     * clones added earlier, regardless of its real stacking order on screen.
     * (The real screen is unaffected since this only concerns our own clone
     * container's internal ordering.)
     */
    private _insertCloneInZOrder(child: Clutter.Actor, clone: Clutter.Actor): void {
      if (!this._uiClonesContainer) return;
      try {
        const uiGroup = Main.layoutManager.uiGroup;
        const siblings = uiGroup.get_children();
        const idx = siblings.indexOf(child);
        if (idx < 0) return; // Not found: leave it at the front.

        let insertAboveClone: Clutter.Actor | null = null;
        for (let i = idx - 1; i >= 0; i--) {
          const prevClone = this._clones.get(siblings[i]);
          if (prevClone && !(prevClone as any)._isDisposed) {
            insertAboveClone = prevClone;
            break;
          }
        }
        if (insertAboveClone) {
          this._uiClonesContainer.set_child_above_sibling(clone, insertAboveClone);
        } else {
          // No cloned sibling sits below `child` in uiGroup's real order, so
          // this one is currently the backmost among cloned siblings.
          this._uiClonesContainer.set_child_below_sibling(clone, null);
        }
      } catch (e) {
      }
    }

    /**
     * Scans uiGroup's current children, creating/destroying clones as needed.
     * Call whenever the set of top-level UI actors may have changed (e.g. a
     * menu opening or closing).
     */
    refresh() {
      if (!this._selfRoot) this._selfRoot = this._findUiGroupAncestor(this._selfActor);

      const uiGroup = Main.layoutManager.uiGroup;
      const children = uiGroup.get_children();
      const seen = new Set<Clutter.Actor>();

      for (const child of children) {
        if ((child as any)._isDisposed) continue;
        if (child === this._selfActor || child === this._selfRoot) continue;
        if (child === Main.layoutManager._backgroundGroup) continue;
        if (this._extraExclusions.has(child)) continue;
        if (!child.visible || !child.mapped) continue;
```

```
      // if (this._containsOtherLiquidGlassRoot(child)) continue;
      // Deep scan for nested Liquid Glass roots only once per newly discovered actor.
      // Doing this every frame causes massive performance drops in the Overview.
      if (!this._clones.has(child) && this._containsOtherLiquidGlassRoot(child)) {
        this.addExclusion(child);
        continue;
      }
      seen.add(child);
      if (!this._clones.has(child)) {
        child.connect('destroy', () => {
          (child as any)._isDisposed = true;
          const clone = this._clones.get(child);
          if (clone) {
            this._clones.delete(child);
            try { clone.destroy(); } catch (_) { }
          }
        });

        const bmsTarget = this._findBmsDescendant(child);

        let sourceClone: Clutter.Actor | null = null;
        if (bmsTarget) {
          // 1st: the real fix — a snapshot that structurally cannot
          // include ourselves (see SelfExcludingSnapshotCapture).
          sourceClone = this._createSelfExcludingSnapshotActor(child);
          // 2nd: read an existing OffscreenEffect's texture (useful for
          // other extensions; BMS's native effect never matches this).
          if (!sourceClone && this._useCaptureFixForBms) {
            sourceClone = this._createExistingEffectBlitActor(child);
          }
        }
        // Fallback: an ordinary unpickable clone.
        if (!sourceClone) {
          sourceClone = new UnpickableClone({ source: child });
        }
        sourceClone.set_name(`${child.name}-sourceClone`);

        sourceClone.connect('destroy', () => {
          this._clones.delete(child);
        });

        this._uiClonesContainer?.add_child(sourceClone);
        this._clones.set(child, sourceClone);
        this._insertCloneInZOrder(child, sourceClone);
      }
    }

    for (const [actor, sourceClone] of this._clones) {
      if (!seen.has(actor)) {
        try { sourceClone.destroy(); } catch (_) { }
      }
    }
  }

  private static _stageToLocal(
    actor: Clutter.Actor,
    stageX: number,
    stageY: number
  ): [number, number] {
    try {
      const res = (actor as any).transform_stage_point(stageX, stageY);
      if (Array.isArray(res) && res[0] === true) {
        return [res[1] as number, res[2] as number];
      }
    } catch (_) { }

    try {
      const [cx, cy] = actor.get_transformed_position();
      return [
        stageX - (Number.isNaN(cx) ? 0 : cx),
        stageY - (Number.isNaN(cy) ? 0 : cy),
      ];
    } catch (_) {
      return [stageX, stageY];
    }
  }

  /**
   * Copies `source`'s current position/size/opacity/visibility onto its
   * clone, and culls the clone if it falls outside the given container
   * bounds.
   */
```

```
    syncProperties(
      source: Clutter.Actor,
      sourceClone: Clutter.Actor,
      containerW: number,
      containerH: number,
      cX: number,
      cY: number
    ) {
      if (!source || !sourceClone) return;
      try {
        const [absX, absY] = source.get_transformed_position();
        const [w, h] = source.get_size();

        if (Number.isNaN(absX) || Number.isNaN(absY) || w <= 0 || h <= 0) {
          sourceClone.visible = false;
          return;
        }

        const scaleX = source.scale_x;
        const scaleY = source.scale_y;

        // get_transformed_position() already folds in source's own
        // scale/pivot (it maps the local origin through the full accumulated
        // transform). So we must NOT also apply scale/pivot again on the
        // clone — doing so double-counts the pivot offset
        // (pivot * size * (1 - scale)), which is invisible when scale is 1
        // and pivot is (0,0) but shows up as a few pixels of drift on
        // anything that scales on hover/press (e.g. the calendar's "today"
        // highlight, panel buttons). Instead, bake the visual scale directly
        // into the clone's size and leave its own scale at 1.
        const scaledW = w * scaleX;
        const scaledH = h * scaleY;

        sourceClone.set_position(absX, absY);
        sourceClone.translation_x = 0;
        sourceClone.translation_y = 0;

        sourceClone.set_size(scaledW, scaledH);
        sourceClone.set_scale(1.0, 1.0);
        sourceClone.set_pivot_point(0, 0);

        sourceClone.opacity = source.opacity;

        const localX = absX - cX;
        const localY = absY - cY;

        const isVisible = source.visible && source.mapped;

        if (isVisible && containerW > 0 && containerH > 0) {
          const isIntersecting =
            localX < containerW &&
            (localX + scaledW) > 0 &&
            localY < containerH &&
            (localY + scaledH) > 0;

          sourceClone.visible = isIntersecting;
        } else {
          sourceClone.visible = isVisible;
        }
      } catch (_) { }
    }

    // Repositions the UI-clone container and culls off-screen clones.
    //
    // In the full-screen-FBO architecture, callers pass
    //   sync(monitor.x, monitor.y, screenW, screenH)
    // rather than the dock's own local (bgX, bgY, bgW, bgH).
    //
    // Effect: _uiClonesContainer is placed at (-monitor.x, -monitor.y) so a
    // clone at absolute screen position (absX, absY) ends up at:
    //   monitor.x + (-monitor.x + absX) = absX  ✓
    // The wider container dimensions (screenW, screenH) relax the cull
    // frustum to the full monitor; actual rendering is still limited to the
    // dock area by the clip applied to liquidBox/blurBox elsewhere.
    sync(cX?: number, cY?: number, cW?: number, cH?: number) {
      let contW = cW ?? 0;
      let contH = cH ?? 0;
      let contAbsX = cX ?? 0;
      let contAbsY = cY ?? 0;

      if (cX === undefined || cY === undefined) {
        try {
```

```
        const [cw, ch] = this._container.get_size();
        if (!Number.isNaN(cw)) contW = cw;
        if (!Number.isNaN(ch)) contH = ch;

        const [tx, ty] = this._container.get_transformed_position();
        contAbsX = Number.isNaN(tx) ? 0 : tx;
        contAbsY = Number.isNaN(ty) ? 0 : ty;
      } catch (_) { }
    }
    // Always bring the UI clones container to the front, regardless of its parent.
    // This prevents WindowCloneManager's rebuilds from placing windows above the UI.
    const parent = this._uiClonesContainer?.get_parent();
    if (parent && this._uiClonesContainer) {
      const siblings = parent.get_children();
      if (siblings[siblings.length - 1] !== this._uiClonesContainer) {
        parent.set_child_above_sibling(this._uiClonesContainer, null);
      }
    }
    // Sign is flipped relative to WindowCloneManager.setOffset(x, y).
    this._uiClonesContainer?.set_position(-contAbsX, -contAbsY);

    for (const [actor, sourceClone] of this._clones) {
      this.syncProperties(actor, sourceClone, contW, contH, contAbsX, contAbsY);
    }
  }

  destroy() {
    if (this._uiClonesContainer) {
      try { this._uiClonesContainer.destroy(); } catch (_) { }
    }
    this._clones.clear();
    this._selfRoot = null;
    this._existingEffectCache.clear();
  }
}


export class WindowCloneManager {
  private windowClonesContainer: Clutter.Actor | null = null;
  private _windowClones: Map<Clutter.Actor, Clutter.Clone>;
  private bgClone: Clutter.Clone | null = null;

  private container: Clutter.Actor | null = null;
  private cloneContainer: Clutter.Actor | null = null;

  constructor(container: Clutter.Actor, cloneContainer: Clutter.Actor | null = null) {
    this.container = container;
    this._windowClones = new Map();

    this.bgClone = new UnpickableClone({ source: Main.layoutManager._backgroundGroup });
    this.bgClone.connect('destroy', () => { this.bgClone = null; });

    this.windowClonesContainer = new UnpickableActor();
    this.windowClonesContainer.connect('destroy', () => { this.windowClonesContainer = null; });

    this.cloneContainer = cloneContainer;

    // windowClonesContainer can only have one parent, so it's added either
    // to cloneContainer or to container directly — never both. As long as
    // cloneContainer is added to container after bgClone, the intended
    // z-order (bgClone behind, window clones in front) holds regardless.
    if (this.cloneContainer) {
      this.cloneContainer.add_child(this.windowClonesContainer);
    } else {
      this.container.add_child(this.windowClonesContainer);
    }

    // bgClone (the wallpaper) always sits at the very back of container.
    this.container.insert_child_at_index(this.bgClone, 0);
  }

  rebuildClones() {
    if (!this.container) return;

    if (this.bgClone) { this.bgClone.destroy(); }
    if (this.windowClonesContainer) { this.windowClonesContainer.destroy(); }

    this.bgClone = new UnpickableClone({ source: Main.layoutManager._backgroundGroup });
    this.bgClone.connect('destroy', () => { this.bgClone = null; });

    this.windowClonesContainer = new UnpickableActor();
    this.windowClonesContainer.connect('destroy', () => { this.windowClonesContainer = null; });
```

```
  if (this.cloneContainer) {
    this.cloneContainer.add_child(this.windowClonesContainer);
  } else {
    this.container.add_child(this.windowClonesContainer);
  }
  this.container.insert_child_at_index(this.bgClone, 0);

  this._windowClones.clear();
  this.sync();
}

// Shifts the entire clone subtree within the full-screen FBO.
//
// In the full-screen-FBO architecture, the caller (dockManager) passes
// (-monitor.x, -monitor.y) rather than the dock's own (-bgX, -bgY).
//
// Rationale: clones sit at their absolute screen coordinates (w.x, w.y).
// blurBox/liquidBox start at (0,0) inside bgActor, which itself sits at
// (monitor.x, monitor.y). Offsetting this container by
// (-monitor.x, -monitor.y) makes each clone's net screen position:
//   monitor.x + 0 + (-monitor.x + w.x) = w.x  ✓
setOffset(x: number, y: number) {
  this.windowClonesContainer?.set_position(x, y);
  this.bgClone?.set_position(x, y);
}

sync() {
  let windows = getWindowActors();
  let activeWindows = new Set();
  let zIndex = 0;

  for (let w of windows) {
    let metaWindow = w.get_meta_window();
    if (!metaWindow || metaWindow.minimized || !w.visible) continue;

    // Read position/size directly rather than via the more expensive
    // get_transformed_position().
    let width = w.width;
    let height = w.height;

    if (width <= 0 || height <= 0) continue;

    activeWindows.add(w);

    let clone;
    if (!this._windowClones.has(w)) {
      clone = new UnpickableClone({ source: w });
      this.windowClonesContainer?.add_child(clone);
      this._windowClones.set(w, clone);
    } else {
      clone = this._windowClones.get(w);
    }

    clone.remove_transition('position');
    clone.remove_transition('size');
    clone.set_position(w.x, w.y);
    clone.set_size(width, height);

    clone.remove_transition('scale-x');
    clone.remove_transition('scale-y');
    clone.set_scale(w.scale_x, w.scale_y);

    // Copy translation directly too, so animation interpolation is
    // reflected immediately rather than lagging a frame behind.
    clone.translation_x = w.translation_x;
    clone.translation_y = w.translation_y;

    let pX = w.pivot_point ? w.pivot_point.x : 0;
    let pY = w.pivot_point ? w.pivot_point.y : 0;
    clone.set_pivot_point(pX, pY);

    this.windowClonesContainer?.set_child_at_index(clone, zIndex);
    zIndex++;
  }

  // Remove clones for windows that closed, or all of them when the
  // Overview starts.
  for (let [w, clone] of this._windowClones.entries()) {
    if (!activeWindows.has(w)) {
      clone.destroy();
      this._windowClones.delete(w);
```

```
      }
    }
  }

  destroy() {
    if (this.windowClonesContainer) {
      this.windowClonesContainer.destroy();
    }
    this._windowClones.clear();
    if (this.bgClone) {
      this.bgClone.destroy();
    }
    this.container = null;
  }
}

/**
 * A ShaderEffect that punches a rounded-rectangle hole out of whatever it's
 * attached to, leaving only the (inset) corner regions visible.
 *
 * Used by ApplicationManager: real application windows have square surfaces,
 * but the liquid-glass background behind them is rendered with rounded
 * corners (via LiquidEffect's corner_radius uniform). Without this effect the
 * window's own opaque content would square off the corners again, breaking
 * the illusion. Applied to a small overlay actor stacked above the window's
 * content and fed a clone of the true (unblurred) background, it reveals
 * exactly the true corner pixels while leaving the rest of the window alone.
 */
export const InverseCornerEffect = GObject.registerClass(
  {
    GTypeName: 'LiquidGlassInverseCornerEffect',
  },
  class InverseCornerEffect extends Clutter.ShaderEffect {
    private _radius: number = 0;
    private _inset: number = 0;

    setRadius(radius: number) {
      this._radius = radius;
      this._updateShader();
    }

    setInset(inset: number) {
      this._inset = inset;
      this._updateShader();
    }

    _updateShader() {
      const shader = `
        uniform sampler2D cogl_sampler;
        uniform float radius;
        uniform float inset;
        uniform float width;
        uniform float height;

        float sdRoundRect(vec2 p, vec2 b, float r) {
          vec2 d = abs(p) - b + vec2(r);
          return min(max(d.x, d.y), 0.0) + length(max(d, 0.0)) - r;
        }

        void main() {
          vec2 st = cogl_tex_coord_in[0].st;
          vec2 resolution = vec2(width, height);
          vec2 p = (st * resolution) - (resolution * 0.5);

          // [FIX] Box half-size at the window's TRUE edge, not shrunk by
          // "inset" on every side. This overlay's actor is padded by
          // SHADER_PADDING beyond the real window bounds on all sides, and
          // "inset" here is exactly that padding — so windowHalf lands
          // precisely on the real window edge.
          //
          // Previously this used (resolution - inset*2)*0.5 with
          // inset = SHADER_PADDING + CORNER_PADDING, which shrank the box by
          // the SAME amount on every side, not just near the corners. Per
          // sdRoundRect's construction, its straight-edge (non-corner)
          // zero-crossing sits exactly at the box half-size regardless of
          // "radius" — only points within "radius" of an actual corner get
          // pulled inward. So shrinking the box itself (rather than only
          // widening "radius") revealed a uniform band along the ENTIRE
          // perimeter — straight edges included — instead of just the 4
          // corners. That band unconditionally painted this overlay's raw,
          // unblurred/untinted/un-shadowed source at full alpha, erasing the
          // drop shadow right next to the window on every edge (reported as
```

```
          // an unnatural halo/"frame" around the window), and — since its
          // width is a fixed pixel count independent of actor scale — became
          // sharply more visible whenever GNOME Shell's open/close animation
          // scaled the window down (the same fixed-pixel band read as a much
          // larger fraction of the shrunk window).
          //
          // "radius" (cornerRadius + CORNER_PADDING, set by the caller) is
          // intentionally a couple pixels larger than the glass shape's own
          // corner_radius so the cut safely over-reveals past the glass's
          // own antialiased corner — but that over-cut should only pull the
          // 4 corners inward, not shift the straight edges too.
          vec2 windowHalf = max(resolution * 0.5 - vec2(inset), vec2(1.0));

          float d = sdRoundRect(p, windowHalf, radius);

          // Sharper transition for the corner cut to avoid dark fringes
          float alpha = smoothstep(-0.5, 0.5, d);

          // Fade out at the very edges of the overlay actor to ensure it blends seamlessly
          // with the background and hides any potential window shadow cutoff.
          vec2 edgeDist = min(st, 1.0 - st) * resolution;
          float edgeFade = smoothstep(0.0, 10.0, min(edgeDist.x, edgeDist.y));
          alpha *= edgeFade;

          cogl_color_out = texture2D(cogl_sampler, st) * alpha * cogl_color_in;
        }
      `;
      this.set_shader_source(shader);
      this._updateUniforms();
    }

    _setUniform(name: string, value: number) {
      let gval = new GObject.Value();
      gval.init(GObject.TYPE_FLOAT);
      gval.set_float(value);
      this.set_uniform_value(name, gval);
    }

    _updateUniforms() {
      let actor = (this as any).get_actor();
      if (!actor) return;

      let w = actor.width;
      let h = actor.height;

      if (Number.isNaN(w) || Number.isNaN(h) || w <= 0 || h <= 0) return;

      this._setUniform('radius', this._radius);
      this._setUniform('inset', this._inset);
      this._setUniform('width', w);
      this._setUniform('height', h);
    }

    vfunc_paint_target(node: any, paint_context: any): void {
      this._updateUniforms();
      super.vfunc_paint_target(node, paint_context);
    }
  }
);

/**
 * Safe helper to retrieve window actors, compatible with GNOME Shell
 * pre-48 (global.get_window_actors) and 48+/GNOME 50 (Mutter moved it to
 * global.compositor.get_window_actors). Every call site in this extension
 * that needs the current list of window actors should go through this
 * instead of calling either API directly.
 */
export function getWindowActors(): any[] {
  if (global.compositor && typeof (global.compositor as any).get_window_actors === 'function') {
    return (global.compositor as any).get_window_actors();
  }
  if (typeof (global as any).get_window_actors === 'function') {
    return (global as any).get_window_actors();
  }
  return [];
}

/**
 * Safe helper to check whether a Clutter actor (GObject) is still valid and
 * has not been disposed. Touching a property on a disposed GObject throws;
 * this is used to guard per-frame sync loops (e.g. ApplicationManager, which
 * juggles many short-lived per-window clones) against that.
```

```
 */
export function isActorValid(actor: any): boolean {
  if (!actor) return false;
  try {
    let _v = actor.visible;
    return true;
  } catch (e) {
    return false;
  }
}
```

```
import { Extension } from 'resource:///org/gnome/shell/extensions/extension.js';
import * as Main from 'resource:///org/gnome/shell/ui/main.js';
import { UIManager } from './dist/uiManager.js';
import { DashManager } from './dist/dockManager.js';
import { NotificationManager } from './dist/notificationManager.js';
import { QuickSettingsManager } from './dist/quickSettingsManager.js';
import { OsdManager } from './dist/osdManager.js';
import { ApplicationManager } from './dist/applicationManager.js';
import { Logger } from './dist/logger.js';
import GLib from 'gi://GLib';

export default class LiquidGlassExtension extends Extension {
  enable() {
    this._settings = this.getSettings("org.gnome.shell.extensions.liquid-glass@thinkingcoding1231.gmail.com");

    // Initialize the logger
    this._logger = new Logger(this._settings);

    this._logger.log(`[Liquid Glass] Enabled. UUID: ${this.uuid}`);

    // Initialize the UI manager for the top panel (e.g., Date Menu)
    // Pass the extension path so it can properly load the GLSL shader files
    this._uiManager = new UIManager(this.dir.get_path(), this._settings, this._logger);
    this._uiManager.setup();

    // Initialize the notification manager to apply effects to notifications
    this._notificationManager = new NotificationManager(this.dir.get_path(), this._settings, this._logger);
    this._notificationManager.setup();

    // Initialize the OSD manager to apply effects to on-screen displays (like volume changes)
    this._osdManager = new OsdManager(this.dir.get_path(), this._settings, this._logger);
    this._osdManager.setup();

    // Initialize the Application manager to apply effects to whitelisted (or all) application windows
    this._applicationManager = new ApplicationManager(this.dir.get_path(), this._settings, this._logger);
    this._applicationManager.setup();

    this._quickSettingsTimeoutId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 1500, () => {
      this._quickSettingsManager = new QuickSettingsManager(this.dir.get_path(), this._settings, this._logger);
      this._quickSettingsManager.setup();
      this._quickSettingsTimeoutId = 0;
      return GLib.SOURCE_REMOVE;
    });

    // Variable to store the timeout ID so we can cancel it if the extension is disabled quickly
    this._timeoutId = 0;

    this._reconnectTimeoutId = 0; // Timeout ID for reconnecting to Dash to Dock signals if it's not found immediately
    this._dashDestroyId = 0;      // ID for the Dash to Dock destroy signal connection, so we can clean it up properly

    // Dash to Dock might not be fully loaded when this extension is enabled at startup.
    // We set a 2-second (2000ms) delay before searching for its UI container.
    this._timeoutId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 2000, () => {
      this._findDashToDock();

      // Reset the ID after execution
      this._timeoutId = 0;

      // Return SOURCE_REMOVE to ensure this timer only runs exactly once
      return GLib.SOURCE_REMOVE;
    });
  }

  _findDashToDock() {
    // A helper function to recursively search the GNOME UI tree for a specific actor name
    const findActorByName = (actor, name) => {
      if (actor.get_name && actor.get_name() === name) {
        return actor;
      }

      // Traverse through all children elements
      let children = actor.get_children();
      for (let i = 0; i < children.length; i++) {
        let found = findActorByName(children[i], name);
        if (found) return found;
      }
      return null;
    };

    // Search the entire GNOME UI group for the main Dash to Dock container
    let dashContainer = findActorByName(Main.layoutManager.uiGroup, 'dashtodockDashContainer');
```

```
    if (dashContainer) {
      this._logger.log("[Liquid Glass] Found Dash to Dock container!", dashContainer);

      // Initialize the dock manager and apply the liquid glass effect
      this._dashManager = new DashManager(this.dir.get_path(), dashContainer, this._settings, this._logger);
      this._dashManager.setup();

      this._dashDestroyId = dashContainer.connect('destroy', () => {
        this._logger.log("[Liquid Glass] Dash to Dock container destroyed (settings changed?). Restarting search...");
        this._dashDestroyId = 0; // Reset the destroy signal ID since the container is gone

        // Cleanup the existing Dash manager to avoid memory leaks or orphaned actors
        if (this._dashManager) {
          this._dashManager.cleanup();
          this._dashManager = null;
        }

        // Clear any existing reconnect timeout to prevent multiple timers from stacking up
        if (this._reconnectTimeoutId !== 0) {
          GLib.Source.remove(this._reconnectTimeoutId);
        }

        // Set a short delay before trying to find Dash to Dock again, as it might be reloaded shortly after being
destroyed
        this._reconnectTimeoutId = GLib.timeout_add(GLib.PRIORITY_DEFAULT, 2000, () => {
          // Try to find Dash to Dock again after the delay. If it's found, the timer will be removed. If not, it will
continue to check every 2 seconds until it is found.
          let isFound = this._findDashToDock();

          if (isFound) {
            // If found, reset the timeout ID and remove the timer
            this._reconnectTimeoutId = 0;
            return GLib.SOURCE_REMOVE;
          }

          // If not found, continue the loop and check again in 2 seconds
          return GLib.SOURCE_CONTINUE;
        });

        return true; // Return true to indicate that the signal was handled
      });
      return true;

    } else {
      // Note: If it's still not found, the user might not have Dash to Dock installed,
      // or it requires a more complex monitoring system to detect late loads.
      this._logger.log("[Liquid Glass] Dash to Dock was not found.");
      return false; // Return false to indicate that Dash to Dock was not found
    }
  }

  disable() {
    this._logger.log(`[Liquid Glass] Disabling...`);

    if (this._quickSettingsTimeoutId && this._quickSettingsTimeoutId !== 0) {
      GLib.Source.remove(this._quickSettingsTimeoutId);
      this._quickSettingsTimeoutId = 0;
    }

    // Clear any pending timeouts to prevent them from executing after the extension is disabled
    if (this._timeoutId !== 0) {
      GLib.Source.remove(this._timeoutId);
      this._timeoutId = 0;
    }

    if (this._reconnectTimeoutId !== 0) {
      GLib.Source.remove(this._reconnectTimeoutId);
      this._reconnectTimeoutId = 0;
    }

    // Crucial: Always restore the UI to its original state when the extension is disabled
    // Failing to clean up can result in invisible menus or memory leaks
    if (this._uiManager) {
      this._uiManager.cleanup();
      this._uiManager = null;
    }

    if (this._quickSettingsManager) {
      this._quickSettingsManager.cleanup();
      this._quickSettingsManager = null;
    }
```

```
    if (this._dashManager) {
      // Disconnect the destroy signal if it was connected
      if (this._dashDestroyId !== 0 && this._dashManager.targetActor) {
        try {
          this._dashManager.targetActor.disconnect(this._dashDestroyId);
        } catch (e) { }
        this._dashDestroyId = 0;
      }
      this._dashManager.cleanup();
      this._dashManager = null;
    }

    if (this._notificationManager) {
      this._notificationManager.cleanup();
      this._notificationManager = null;
    }

    if (this._osdManager) {
      this._osdManager.cleanup();
      this._osdManager = null;
    }

    if (this._applicationManager) {
      this._applicationManager.cleanup();
      this._applicationManager = null;
    }

    this._settings = null;

    if (this._logger) {
      this._logger.cleanup();
      this._logger = null;
    }
  }
}
```

```
import { ExtensionPreferences } from 'resource:///org/gnome/Shell/Extensions/js/extensions/prefs.js';
import Adw from 'gi://Adw';
import Gtk from 'gi://Gtk';
import Gio from 'gi://Gio';
import Gdk from 'gi://Gdk';

export default class LiquidGlassPreferences extends ExtensionPreferences {
  fillPreferencesWindow(window) {
    const settings = this.getSettings("org.gnome.shell.extensions.liquid-glass@thinkingcoding1231.gmail.com");

    const blurRadiusRows = []; // blur-radius のテキストボックスの row を保持する

    // 拡張機能のディレクトリから resources.gresource の絶対パスを取得する
    const resourceFile = this.dir.get_child('resources.gresource');
    const resource = Gio.Resource.load(resourceFile.get_path());
    resource._register();
    const iconTheme = Gtk.IconTheme.get_for_display(Gdk.Display.get_default());
    iconTheme.add_resource_path('/com/example/my-app/icons');

    // --- Dock タブ ---
    const dockPage = new Adw.PreferencesPage({
      title: 'Dock',
      icon_name: 'dock-bottom-symbolic',
    });
    window.add(dockPage);

    const dockGroup = new Adw.PreferencesGroup({
      title: 'Dock Settings',
      description: 'Configure the liquid glass effect for the Dash to Dock',
    });
    dockPage.add(dockGroup);

    // 有効化スイッチ
    this._addSwitchRow(dockGroup, settings, 'enable-dock-glass', 'Enable Glass Effect', 'Apply the effect to the
dock');
    // 各種パラメータ
    this._addSliderRow(dockGroup, settings, 'dock-glass-expand', 'Glass Expand', 'Extra area for the effect', 0, 50,
1);
    this._addSliderRow(dockGroup, settings, 'dock-margin-bottom', 'Margin Bottom', 'Bottom spacing', -5, 30, 1);
    this._addColorRow(dockGroup, settings, 'dock-tint-color', 'Tint Color', 'Color of the glass tint');
    this._addSliderRow(dockGroup, settings, 'dock-tint-strength', 'Tint Strength', 'Intensity of the color tint', 0.0,
1.0, 0.01);
    const dockBlurRow = this._addSliderRow(dockGroup, settings, 'dock-blur-radius', 'Blur Radius', '', 0, 30, 1);
    blurRadiusRows.push(dockBlurRow);
    this._addSliderRow(dockGroup, settings, 'dock-corner-radius', 'Corner Radius', 'Roundness of the corners', 0, 200,
1);

    // Advancedグループ (開閉可能)
    const dockAdvanced = new Adw.ExpanderRow({
      title: 'Advanced',
      subtitle: 'Color adjustments (Brightness, Contrast, Saturation)'
    });
    dockGroup.add(dockAdvanced);

    this._addSliderRow(dockAdvanced, settings, 'dock-brightness', 'Brightness', 'Adjusts brightness', 0.5, 1.5, 0.01);
    this._addSliderRow(dockAdvanced, settings, 'dock-contrast', 'Contrast', 'Adjusts contrast', 0.5, 1.5, 0.01);
    this._addSliderRow(dockAdvanced, settings, 'dock-saturation', 'Saturation', 'Adjusts saturation', 0.0, 2.0, 0.01);


    // --- Menu タブ ---
    const menuPage = new Adw.PreferencesPage({
      title: 'Menu',
      icon_name: 'view-list-symbolic',
    });
    window.add(menuPage);

    const menuGroup = new Adw.PreferencesGroup({ title: 'Menu Settings' });
    menuPage.add(menuGroup);

    this._addSwitchRow(menuGroup, settings, 'enable-menu-glass', 'Enable Glass Effect', 'Apply to menus and popups');
    this._addSwitchRow(menuGroup, settings, "enable-menu-animation", "Enable Menu Animation", "Animate menu
transitions using spring physics");

    this._addSliderRow(menuGroup, settings, 'menu-glass-expand', 'Glass Expand', 'Extra area for the effect', 0, 50,
1);
    this._addSliderRow(menuGroup, settings, 'menu-x-offset', 'X Offset', 'Horizontal offset adjustment', -200, 200,
1);
    this._addSliderRow(menuGroup, settings, 'menu-y-offset', 'Y Offset', 'Vertical offset adjustment', -50, 100, 1);

    this._addSwitchRow(menuGroup, settings, 'menu-enable-adaptive-text-color', 'Adaptive Text Color', 'Adjust text
contrast automatically');
```

```
    const menuSampleIntervalRow = this._addSliderRow(menuGroup, settings, 'menu-sample-interval-ms', 'Sample Interval
(ms)', 'Contrast update frequency', 100, 2000, 50);
    // Adaptive Text Color連動の非表示化
    settings.bind('menu-enable-adaptive-text-color', menuSampleIntervalRow, 'visible', Gio.SettingsBindFlags.GET);

    this._addColorRow(menuGroup, settings, 'menu-tint-color', 'Tint Color', 'Color of the glass tint');
    this._addSliderRow(menuGroup, settings, 'menu-tint-strength', 'Tint Strength', 'Intensity of the color tint', 0.0,
1.0, 0.01);
    const menuBlurRow = this._addSliderRow(menuGroup, settings, 'menu-blur-radius', 'Blur Radius', '', 0, 30, 1);
    blurRadiusRows.push(menuBlurRow);
    this._addSliderRow(menuGroup, settings, 'menu-corner-radius', 'Corner Radius', 'Roundness of the corners', 0, 200,
1);

    // Advancedグループ（開閉可能） - Spring関連とカラー調整をここに集約
    const menuAdvanced = new Adw.ExpanderRow({
      title: 'Advanced',
      subtitle: 'Spring physics, color adjustments (Brightness, Contrast, Saturation)'
    });
    menuGroup.add(menuAdvanced);

    // 【修正】Spring物理挙動パラメータをAdvanced内へ移動
    const menuStiffnessRow = this._addSliderRow(menuAdvanced, settings, 'menu-spring-stiffness', 'Spring Stiffness',
'Spring stiffness', 0.0, 1000.0, 0.1);
    const menuDampingRow = this._addSliderRow(menuAdvanced, settings, 'menu-spring-damping', 'Spring Damping', 'Spring
damping', 0.0, 1000.0, 0.1);
    const menuMassRow = this._addSliderRow(menuAdvanced, settings, 'menu-spring-mass', 'Spring Mass', 'Spring mass',
0.0, 1.0, 0.1);
    const menuIntervalRow = this._addSliderRow(menuAdvanced, settings, 'menu-animation-interval-ms', 'Animation
Interval (ms)', 'Animation interval', 0, 1000, 1);

    // アニメーションOFF時に項目を非表示にするバインド（Advanced内にあっても正常に動作します）
    settings.bind('enable-menu-animation', menuStiffnessRow, 'visible', Gio.SettingsBindFlags.GET);
    settings.bind('enable-menu-animation', menuDampingRow, 'visible', Gio.SettingsBindFlags.GET);
    settings.bind('enable-menu-animation', menuMassRow, 'visible', Gio.SettingsBindFlags.GET);
    settings.bind('enable-menu-animation', menuIntervalRow, 'visible', Gio.SettingsBindFlags.GET);

    // カラー調整（Advanced内）
    this._addSliderRow(menuAdvanced, settings, 'menu-brightness', 'Brightness', 'Adjusts brightness', 0.5, 1.5, 0.01);
    this._addSliderRow(menuAdvanced, settings, 'menu-contrast', 'Contrast', 'Adjusts contrast', 0.5, 1.5, 0.01);
    this._addSliderRow(menuAdvanced, settings, 'menu-saturation', 'Saturation', 'Adjusts saturation', 0.0, 2.0, 0.01);


    // --- Notifications タブ ---
    const notifPage = new Adw.PreferencesPage({
      title: 'Notifications',
      icon_name: 'preferences-system-notifications-symbolic',
    });
    window.add(notifPage);

    const notifGroup = new Adw.PreferencesGroup({ title: 'Notification Settings' });
    notifPage.add(notifGroup);

    this._addSwitchRow(notifGroup, settings, 'enable-notification-glass', 'Enable Glass Effect', 'Apply to
notification banners');

    this._addSwitchRow(notifGroup, settings, 'notification-enable-adaptive-text-color', 'Adaptive Text Color', 'Adjust
text contrast automatically');
    const notifSampleIntervalRow = this._addSliderRow(notifGroup, settings, 'notification-sample-interval-ms', 'Sample
Interval (ms)', 'Contrast update frequency', 100, 2000, 50);
    // Adaptive Text Color連動の非表示化
    settings.bind('notification-enable-adaptive-text-color', notifSampleIntervalRow, 'visible',
Gio.SettingsBindFlags.GET);

    this._addSliderRow(notifGroup, settings, 'notification-glass-expand', 'Glass Expand', 'Extra area for the effect',
0, 50, 1);
    this._addSliderRow(notifGroup, settings, 'notification-y-offset', 'Y Offset', 'Vertical offset adjustment', 0,
100, 1);
    this._addColorRow(notifGroup, settings, 'notification-tint-color', 'Tint Color', 'Color of the glass tint');
    this._addSliderRow(notifGroup, settings, 'notification-tint-strength', 'Tint Strength', 'Intensity of the color
tint', 0.0, 1.0, 0.01);
    const notifBlurRow = this._addSliderRow(notifGroup, settings, 'notification-blur-radius', 'Blur Radius', '', 0,
30, 1);
    blurRadiusRows.push(notifBlurRow);
    this._addSliderRow(notifGroup, settings, 'notification-corner-radius', 'Corner Radius', 'Roundness of the
corners', 0, 200, 1);

    // Advancedグループ（開閉可能）
    const notifAdvanced = new Adw.ExpanderRow({
      title: 'Advanced',
      subtitle: 'Color adjustments (Brightness, Contrast, Saturation)'
    });
    notifGroup.add(notifAdvanced);
```

```
    this._addSliderRow(notifAdvanced, settings, 'notification-brightness', 'Brightness', 'Adjusts brightness', 0.5,
1.5, 0.01);
    this._addSliderRow(notifAdvanced, settings, 'notification-contrast', 'Contrast', 'Adjusts contrast', 0.5, 1.5,
0.01);
    this._addSliderRow(notifAdvanced, settings, 'notification-saturation', 'Saturation', 'Adjusts saturation', 0.0,
2.0, 0.01);


    // --- Quick Settings タブ ---
    const qsPage = new Adw.PreferencesPage({
      title: 'Quick Settings',
      icon_name: 'shapes-large-symbolic',
    });
    window.add(qsPage);

    const qsGroup = new Adw.PreferencesGroup({ title: 'Quick Settings Settings (Experimental)' });
    qsPage.add(qsGroup);

    this._addSwitchRow(qsGroup, settings, 'enable-quick-settings-glass', 'Enable Glass Effect', 'Apply to quick
settings panel');
    this._addSwitchRow(qsGroup, settings, "enable-quick-settings-animation", "Enable Quick Settings Animation", "Apply
to quick settings panel");

    this._addSwitchRow(qsGroup, settings, 'quick-settings-enable-adaptive-text-color', 'Adaptive Text Color', 'Adjust
text contrast automatically');
    const qsSampleIntervalRow = this._addSliderRow(qsGroup, settings, 'quick-settings-sample-interval-ms', 'Sample
Interval (ms)', 'Contrast update frequency', 100, 2000, 50);
    // Adaptive Text Color連動の非表示化
    settings.bind('quick-settings-enable-adaptive-text-color', qsSampleIntervalRow, 'visible',
Gio.SettingsBindFlags.GET);

    this._addSliderRow(qsGroup, settings, 'quick-settings-glass-expand', 'Glass Expand', 'Extra area for the effect',
0, 50, 1);
    this._addSliderRow(qsGroup, settings, 'quick-settings-x-offset', 'X Offset', 'Horizontal offset adjustment', -100,
100, 1);
    this._addSliderRow(qsGroup, settings, 'quick-settings-y-offset', 'Y Offset', 'Vertical offset adjustment', -100,
100, 1);
    this._addColorRow(qsGroup, settings, 'quick-settings-tint-color', 'Tint Color', 'Color of the glass tint');
    this._addSliderRow(qsGroup, settings, 'quick-settings-tint-strength', 'Tint Strength', 'Intensity of the color
tint', 0.0, 1.0, 0.01);
    const qsBlurRow = this._addSliderRow(qsGroup, settings, 'quick-settings-blur-radius', 'Blur Radius', '', 0, 30,
1);
    blurRadiusRows.push(qsBlurRow);
    this._addSliderRow(qsGroup, settings, 'quick-settings-corner-radius', 'Corner Radius', 'Roundness of the corners',
0, 200, 1);

    // Advancedグループ（開閉可能） - Spring関連とカラー調整をここに集約
    const qsAdvanced = new Adw.ExpanderRow({
      title: 'Advanced',
      subtitle: 'Spring physics, color adjustments (Brightness, Contrast, Saturation)'
    });
    qsGroup.add(qsAdvanced);

    const quickSettingsStiffnessRow = this._addSliderRow(qsAdvanced, settings, 'quick-settings-spring-stiffness',
'Spring Stiffness', 'Spring stiffness', 0.0, 1000.0, 0.1);
    const quickSettingsDampingRow = this._addSliderRow(qsAdvanced, settings, 'quick-settings-spring-damping', 'Spring
Damping', 'Spring damping', 0.0, 1000.0, 0.1);
    const quickSettingsMassRow = this._addSliderRow(qsAdvanced, settings, 'quick-settings-spring-mass', 'Spring Mass',
'Spring mass', 0.0, 1.0, 0.1);
    const quickSettingsIntervalRow = this._addSliderRow(qsAdvanced, settings, 'quick-settings-animation-interval-ms',
'Animation Interval (ms)', 'Animation interval', 0, 1000, 1);

    // アニメーションOFF時に項目を非表示にするバインド
    settings.bind('enable-quick-settings-animation', quickSettingsStiffnessRow, 'visible', Gio.SettingsBindFlags.GET);
    settings.bind('enable-quick-settings-animation', quickSettingsDampingRow, 'visible', Gio.SettingsBindFlags.GET);
    settings.bind('enable-quick-settings-animation', quickSettingsMassRow, 'visible', Gio.SettingsBindFlags.GET);
    settings.bind('enable-quick-settings-animation', quickSettingsIntervalRow, 'visible', Gio.SettingsBindFlags.GET);

    // カラー調整（Advanced内）
    this._addSliderRow(qsAdvanced, settings, 'quick-settings-brightness', 'Brightness', 'Adjusts brightness', 0.5,
1.5, 0.01);
    this._addSliderRow(qsAdvanced, settings, 'quick-settings-contrast', 'Contrast', 'Adjusts contrast', 0.5, 1.5,
0.01);
    this._addSliderRow(qsAdvanced, settings, 'quick-settings-saturation', 'Saturation', 'Adjusts saturation', 0.0,
2.0, 0.01);


    // --- OSD タブ ---
    const osdPage = new Adw.PreferencesPage({
      title: 'OSD',
      icon_name: 'audio-volume-medium-symbolic',
```

```
    });
    window.add(osdPage);

    const osdGroup = new Adw.PreferencesGroup({ title: 'OSD Settings (Experimental)' });
    osdPage.add(osdGroup);

    this._addSwitchRow(osdGroup, settings, 'enable-osd-glass', 'Enable Glass Effect', 'Apply to on-screen displays
(like volume changes)');

    this._addSwitchRow(osdGroup, settings, 'osd-enable-adaptive-text-color', 'Adaptive Text Color', 'Adjust text
contrast automatically');
    const osdSampleIntervalRow = this._addSliderRow(osdGroup, settings, 'osd-sample-interval-ms', 'Sample Interval
(ms)', 'Contrast update frequency', 100, 2000, 50);
    // Adaptive Text Color連動の非表示化
    settings.bind('osd-enable-adaptive-text-color', osdSampleIntervalRow, 'visible', Gio.SettingsBindFlags.GET);

    this._addSliderRow(osdGroup, settings, 'osd-glass-expand', 'Glass Expand', 'Extra area for the effect', 0, 50, 1);
    this._addSliderRow(osdGroup, settings, 'osd-y-offset', 'Y Offset', 'Vertical offset adjustment', -100, 100, 1);
    this._addColorRow(osdGroup, settings, 'osd-tint-color', 'Tint Color', 'Color of the glass tint');
    this._addSliderRow(osdGroup, settings, 'osd-tint-strength', 'Tint Strength', 'Intensity of the color tint', 0.0,
1.0, 0.01);
    const osdBlurRow = this._addSliderRow(osdGroup, settings, 'osd-blur-radius', 'Blur Radius', '', 0, 30, 1);
    blurRadiusRows.push(osdBlurRow);
    this._addSliderRow(osdGroup, settings, 'osd-corner-radius', 'Corner Radius', 'Roundness of the corners', 0, 200,
1);

    // Advancedグループ（開閉可能）
    const osdAdvanced = new Adw.ExpanderRow({
      title: 'Advanced',
      subtitle: 'Color adjustments (Brightness, Contrast, Saturation)'
    });
    osdGroup.add(osdAdvanced);

    this._addSliderRow(osdAdvanced, settings, 'osd-brightness', 'Brightness', 'Adjusts brightness', 0.5, 1.5, 0.01);
    this._addSliderRow(osdAdvanced, settings, 'osd-contrast', 'Contrast', 'Adjusts contrast', 0.5, 1.5, 0.01);
    this._addSliderRow(osdAdvanced, settings, 'osd-saturation', 'Saturation', 'Adjusts saturation', 0.0, 2.0, 0.01);


    // --- Applications タブ ---
    const appPage = new Adw.PreferencesPage({
      title: 'Applications',
      icon_name: 'applications-system-symbolic',
    });
    window.add(appPage);

    const appGroup = new Adw.PreferencesGroup({
      title: 'Application Window Settings',
      description: 'Apply the liquid glass effect to application windows',
    });
    appPage.add(appGroup);

    this._addSwitchRow(appGroup, settings, 'enable-application-glass', 'Enable Glass Effect', 'Apply to application
windows');
    // Switch to apply to all windows, bypassing the whitelist below.
    this._addSwitchRow(appGroup, settings, 'application-glass-all-windows', 'Apply to All Windows', 'Apply to all
application windows, bypassing the whitelist below');
    this._addWhitelistEditor(appPage, settings, 'application-window-whitelist');
    this._addColorRow(appGroup, settings, 'application-tint-color', 'Tint Color', 'Color of the glass tint');
    this._addSliderRow(appGroup, settings, 'application-tint-strength', 'Tint Strength', 'Intensity of the color
tint', 0.0, 1.0, 0.01);
    const appBlurRow = this._addSliderRow(appGroup, settings, 'application-blur-radius', 'Blur Radius', '', 0, 30, 1);
    blurRadiusRows.push(appBlurRow);
    this._addSliderRow(appGroup, settings, 'application-corner-radius', 'Corner Radius', 'Roundness of the corners',
0, 200, 1);
    this._addSliderRow(appGroup, settings, 'application-content-opacity', 'Window Content Opacity', 'Opacity of the
window content layer, so the glass shows through it', 0.0, 1.0, 0.01);

    // Advancedグループ（開閉可能）
    const appAdvanced = new Adw.ExpanderRow({
      title: 'Advanced',
      subtitle: 'Color adjustments (Brightness, Contrast, Saturation)'
    });
    appGroup.add(appAdvanced);

    this._addSliderRow(appAdvanced, settings, 'application-brightness', 'Brightness', 'Adjusts brightness', 0.5, 1.5,
0.01);
    this._addSliderRow(appAdvanced, settings, 'application-contrast', 'Contrast', 'Adjusts contrast', 0.5, 1.5, 0.01);
    this._addSliderRow(appAdvanced, settings, 'application-saturation', 'Saturation', 'Adjusts saturation', 0.0, 2.0,
0.01);


    // --- Glass Properties タブ ---
```

```
    const shaderPage = new Adw.PreferencesPage({
      title: 'Glass',
      icon_name: 'image-adjust-shadows-symbolic',
    });
    window.add(shaderPage);

    const physGroup = new Adw.PreferencesGroup({ title: 'Physical &amp;amp; Optical Properties' });
    shaderPage.add(physGroup);

    // Blur Method を選択する ComboRow を追加し、GSettingsにバインド
    const blurMethodRow = new Adw.ComboRow({
      title: 'Blur Method',
      model: Gtk.StringList.new([
        'Gaussian Blur (Recommended)',
        'Dual Kawase (Performance)'
      ])
    });
    this._addRowToContainer(physGroup, blurMethodRow);
    settings.bind('blur-method', blurMethodRow, 'selected', Gio.SettingsBindFlags.DEFAULT);

    this._addSliderRow(physGroup, settings, 'glass-max-z', 'Maximum Z Depth', 'Physical thickness of the glass', 0.0,
100.0, 1.0);
    this._addSliderRow(physGroup, settings, 'glass-displacement-scale', 'Displacement Scale', 'Strength of light
refraction', 0.0, 200.0, 1.0);
    this._addSliderRow(physGroup, settings, 'glass-edge-smoothing', 'Edge Smoothing', 'Anti-aliasing feathering
width', 0.0, 10.0, 0.1);
    this._addSliderRow(physGroup, settings, 'glass-profile-shape-n', 'Profile Shape N', 'Curvature shape of the
surface', 1.0, 20.0, 0.1);
    this._addSliderRow(physGroup, settings, 'glass-ior', 'Index of Refraction', 'Optical density (1.5 - 2.4)', 1.0,
4.0, 0.01);
    this._addSliderRow(physGroup, settings, 'glass-chroma-strength', 'Chroma Strength', 'RGB color separation', 0.0,
0.1, 0.001);

    const lightGroup = new Adw.PreferencesGroup({ title: 'Lighting &amp;amp; Reflections' });
    shaderPage.add(lightGroup);

    this._addSliderRow(lightGroup, settings, 'glass-specular-intensity', 'Specular Intensity', 'Brightness of
highlights', 0.0, 5.0, 0.1);
    this._addSliderRow(lightGroup, settings, 'glass-shininess', 'Shininess', 'Sharpness of reflections', 1.0, 200.0,
1.0);
    this._addSliderRow(lightGroup, settings, 'glass-rim-width', 'Rim Width', 'Width of the edge lighting', 0.0, 20.0,
0.1);
    this._addSliderRow(lightGroup, settings, 'glass-rim-intensity', 'Rim Intensity', 'Brightness of rim light', 0.0,
5.0, 0.1);
    this._addSliderRow(lightGroup, settings, 'glass-rim-directional-power', 'Rim Directional Power', 'Light direction
effect on rim', 0.0, 10.0, 0.1);
    this._addSliderRow(lightGroup, settings, 'glass-rim-power', 'Rim Fresnel Power', 'Fresnel falloff for rim light',
0.0, 20.0, 0.1);
    this._addSliderRow(lightGroup, settings, 'glass-rim-light-color-intensity', 'Rim Light Color Intensity',
'Multiplier for rim color', 0.0, 5.0, 0.1);
    this._addSliderRow(lightGroup, settings, 'glass-sheen-intensity', 'Sheen Intensity', 'Background sheen across
surface', 0.0, 2.0, 0.01);
    this._addSliderRow(lightGroup, settings, 'glass-light-angle-deg', 'Light Angle (Deg)', 'Directional angle of light
source', 0.0, 360.0, 1.0);

    const shadowGroup = new Adw.PreferencesGroup({
      title: 'Drop Shadow',
      description: 'Anchors the glass on light backgrounds (e.g. white wallpapers) so it does not visually disappear.'
    });
    shaderPage.add(shadowGroup);

    this._addSliderRow(shadowGroup, settings, 'shadow-radius', 'Shadow Radius (px)', 'How far the shadow extends past
the glass edge. Set to 0 to disable.', 0.0, 100.0, 1.0);
    this._addSliderRow(shadowGroup, settings, 'shadow-intensity', 'Shadow Intensity', 'How dark the shadow is. 0 =
invisible, 1 = pure black.', 0.0, 1.0, 0.01);

    const aoGroup = new Adw.PreferencesGroup({
      title: 'Inner Edge Darkening (AO)',
      description: 'A separate ambient-occlusion darkening just inside the glass edge, independent of the outer drop
shadow above.'
    });
    shaderPage.add(aoGroup);

    this._addSliderRow(aoGroup, settings, 'glass-ao-intensity', 'AO Intensity', 'How dark the inner edge band gets. 0
= invisible, 1 = pure black.', 0.0, 1.0, 0.01);
    this._addSliderRow(aoGroup, settings, 'glass-ao-radius', 'AO Radius (px)', 'How far inward from the edge the
darkening extends before fading out.', 0.0, 50.0, 0.5);

    const debugGroup = new Adw.PreferencesGroup({ title: 'Debug' });
    shaderPage.add(debugGroup);

    this._addSwitchRow(debugGroup, settings, 'output-logs', 'Output Logs', 'Output logs to the terminal');
```

```
    // Blur Methodの選択に応じて、各Blur Radiusの注釈（subtitle）を動的に切り替える処理
    const updateBlurSubtitles = () => {
      // get_int() が 1 (Dual Kawase) の時だけ注意書きを追加する
      const isDualKawase = settings.get_int('blur-method') === 1;
      const subtitle = isDualKawase
        ? 'Background blur intensity (Uses Dual Kawase blur; radius may not be pixel-accurate)'
        : 'Background blur intensity';

      // 登録しておいたすべての Blur Radius 行のサブタイトルを更新
      blurRadiusRows.forEach(row => {
        row.subtitle = subtitle;
      });
    };

    // 設定変更時のシグナル接続と、初期起動時の実行
    settings.connect('changed::blur-method', updateBlurSubtitles);
    updateBlurSubtitles();
  }

  // --- 便利メソッド群 ---

  _addRowToContainer(container, row) {
    if (container.add_row) {
      container.add_row(row);
    } else {
      container.add(row);
    }
  }

  // ON/OFFスイッチ
  _addSwitchRow(container, settings, key, title, subtitle = '') {
    const row = new Adw.SwitchRow({ title, subtitle });
    this._addRowToContainer(container, row);
    settings.bind(key, row, 'active', Gio.SettingsBindFlags.DEFAULT);
    return row;
  }

  // スライダーと標準のスピンボタン（+/-）を配置するメソッド
  _addSliderRow(container, settings, key, title, subtitle, min, max, step) {
    const row = new Adw.ActionRow({ title, subtitle });

    // 小数点のステップ数に応じて入力欄の表示桁数を自動判定
    let digits = 0;
    if (step < 1) digits = 2;
    if (step < 0.01) digits = 3;

    // スライダーとスピンボタンで共有する Adjustment
    const adjustment = new Gtk.Adjustment({
      lower: min,
      upper: max,
      step_increment: step
    });

    // 1. スライダー (Gtk.Scale) の生成
    const scale = Gtk.Scale.new(Gtk.Orientation.HORIZONTAL, adjustment);
    scale.set_hexpand(true);
    scale.set_valign(Gtk.Align.CENTER);
    scale.set_draw_value(false);
    scale.set_size_request(160, -1); // 最低限の横幅

    // 2. 標準の数値入力＆上下ボタン (Gtk.SpinButton) の生成
    const spinButton = new Gtk.SpinButton({
      adjustment: adjustment,
      climb_rate: step,
      digits: digits,
      numeric: true,
      valign: Gtk.Align.CENTER,
    });

    // スライダーとスピンボタンを並べるコンテナ
    const box = new Gtk.Box({
      orientation: Gtk.Orientation.HORIZONTAL,
      spacing: 12,
      valign: Gtk.Align.CENTER
    });
    box.append(scale);
    box.append(spinButton);

    row.add_suffix(box);
    this._addRowToContainer(container, row);
```

```
    // 設定とAdjustmentをバインド（これだけで両方が連動して保存・読み込みされます）
    settings.bind(key, adjustment, 'value', Gio.SettingsBindFlags.DEFAULT);

    return row;
  }

  // 数値入力（整数・小数両対応）※念のため維持
  _addSpinRow(container, settings, key, title, subtitle, min, max, step) {
    let digits = 0;
    if (step < 1) digits = 2;
    if (step < 0.01) digits = 3;
    const row = new Adw.SpinRow({
      title,
      subtitle,
      adjustment: new Gtk.Adjustment({ lower: min, upper: max, step_increment: step }),
      digits: digits,
    });
    this._addRowToContainer(container, row);
    settings.bind(key, row, 'value', Gio.SettingsBindFlags.DEFAULT);
    return row;
  }

  // アプリケーションウィンドウのホワイトリスト編集UI（WM_CLASS の追加/削除）
  _addWhitelistEditor(page, settings, key) {
    const listGroup = new Adw.PreferencesGroup({
      title: 'Whitelisted Windows',
      description: 'WM_CLASS values (run: xprop WM_CLASS on a window). Matching is case-insensitive. Ignored while
"Apply to All Windows" is on.',
    });
    page.add(listGroup);

    const listBox = new Gtk.ListBox({
      selection_mode: Gtk.SelectionMode.NONE,
    });
    listBox.add_css_class('boxed-list');

    const refreshList = () => {
      let child = listBox.get_first_child();
      while (child) {
        const next = child.get_next_sibling();
        listBox.remove(child);
        child = next;
      }

      const items = settings.get_strv(key);
      for (const item of items) {
        const row = new Adw.ActionRow({ title: item });
        const removeButton = new Gtk.Button({
          icon_name: 'user-trash-symbolic',
          valign: Gtk.Align.CENTER,
          css_classes: ['flat', 'error'],
        });
        removeButton.connect('clicked', () => {
          const current = settings.get_strv(key).filter((v) => v !== item);
          settings.set_strv(key, current);
          refreshList();
        });
        row.add_suffix(removeButton);
        listBox.append(row);
      }

      if (items.length === 0) {
        const emptyRow = new Adw.ActionRow({
          title: 'No windows whitelisted',
          subtitle: 'Add a WM_CLASS below to enable the effect on that application',
        });
        emptyRow.add_css_class('dim-label');
        listBox.append(emptyRow);
      }
    };

    settings.connect(`changed::${key}`, refreshList);
    refreshList();

    listGroup.add(listBox);

    const entryGroup = new Adw.PreferencesGroup({ title: 'Add Window Class' });
    page.add(entryGroup);

    const entryRow = new Adw.EntryRow({
      title: 'WM_CLASS',
      show_apply_button: true,
```

```
    });
    entryRow.connect('apply', () => {
      const value = entryRow.get_text().trim();
      if (!value) return;

      const normalized = value.toLowerCase();
      const items = settings.get_strv(key);
      if (!items.some((v) => v.toLowerCase() === normalized)) {
        settings.set_strv(key, [...items, value.trim()]);
      }
      entryRow.set_text('');
    });
    entryGroup.add(entryRow);
  }

  // 色選択
  _addColorRow(container, settings, key, title, subtitle) {
    const row = new Adw.ActionRow({ title, subtitle });
    const colorButton = new Gtk.ColorDialogButton({
      valign: Gtk.Align.CENTER,
      dialog: new Gtk.ColorDialog(),
    });

    // 保存されたHEX文字列をRGBAに変換してセット
    const rgba = new Gdk.RGBA();
    rgba.parse(settings.get_string(key));
    colorButton.rgba = rgba;

    // 色が変わったらHEXに変換して保存
    colorButton.connect('notify::rgba', () => {
      const color = colorButton.rgba;
      const r = Math.floor(color.red * 255).toString(16).padStart(2, '0');
      const g = Math.floor(color.green * 255).toString(16).padStart(2, '0');
      const b = Math.floor(color.blue * 255).toString(16).padStart(2, '0');
      const hex = `#${r}${g}${b}`;

      settings.set_string(key, hex);
    });

    row.add_suffix(colorButton);
    this._addRowToContainer(container, row);
  }
}
```

```
// shaders/glass.frag
uniform sampler2D cogl_sampler0; // Layer 0: 元のシャープな背景
uniform sampler2D cogl_sampler1; // Layer 1: ブラー済み背景

uniform float resolution_x;
uniform float resolution_y;
uniform float pointer_x;
uniform float pointer_y;
uniform float intensity;
uniform float corner_radius;
uniform float max_z;
uniform float displacement_scale;
uniform float edge_smoothing;
uniform float profile_shape_n;
uniform float ior;
uniform float chroma_strength;
uniform float blur_strength;
uniform float tint_strength;
uniform float tint_r;
uniform float tint_g;
uniform float tint_b;
uniform float specular_intensity;
uniform float rim_width;
uniform float rim_intensity;
uniform float rim_directional_power;
uniform float rim_power;
uniform float rim_light_color_intensity;
uniform float sheen_intensity;
uniform float shininess;
// Master on/off for the rim/specular/sheen "glass surface glint" group,
// set via LiquidEffect.setSurfaceLightEnabled(). Defaults to 1.0 (on) when
// unset, so surfaces that never call the setter (dock, menu, notification,
// quick-settings, OSD) are unaffected. applicationManager.ts sets this to
// 0.0 for application windows, which should read as plain "shadow + AO"
// (both computed independently of this uniform, see main()) rather than a
// dock-style glass glint hugging the window edge.
uniform float surface_light_enabled;
// [NEW] Inner edge ambient-occlusion darkening, independent of rim_width and
// of the outer drop shadow (shadow_radius / shadow_intensity above). Lets
// the user tune the "shadow under the glass" band on its own instead of it
// being derived from rim_width and gated by the drop shadow's radiusEnable.
uniform float ao_intensity; // 0 = no inner darkening, 1 = fully black at the edge
uniform float ao_radius;    // px inward from the edge over which the AO band fades out
uniform float light_angle_deg;
uniform float mouse_radius;
uniform float bg_glow_intensity;
uniform float shadow_radius;
uniform float shadow_intensity;
// [FIX] How much room (px) the drop shadow actually has to render outward,
// synced from dockManager's CLIP_PADDING. Previously the shadow reused
// `padding` (below) for this, which is a small, fixed value meant only for
// refraction/blur headroom (20px) — that silently capped shadow_radius's
// usable range at ~18px regardless of the 0-100 prefs.js slider.
uniform float shadow_max_radius;
uniform float padding;
uniform float isDock;

// [NEW] SCB (Saturation, Contrast, Brightness) 調整用の変数
uniform float brightness;
uniform float contrast;
uniform float saturation;

// [NEW] Dock geometry in monitor-local pixel coordinates.
// In full-screen FBO mode the actor covers the whole monitor, so the shader
// must know the dock's position and size within that space to correctly
// compute local_pos and box_size. Supplied each frame by LiquidEffect::setDockGeometry().
uniform float dock_x;  // dock background left edge  (monitor-relative px)
uniform float dock_y;  // dock background top  edge  (monitor-relative px)
uniform float dock_w;  // dock background width       (px, includes SHADER_PADDING)
uniform float dock_h;  // dock background height      (px, includes SHADER_PADDING)

// Signed Distance Field (SDF) function for a rounded rectangle.
// Returns negative values inside the shape, positive outside, and 0 on the exact edge.
float sdRoundRect(vec2 p, vec2 b, float r) {
    vec2 d = abs(p) - b + vec2(r);
    return min(max(d.x, d.y), 0.0) + length(max(d, 0.0)) - r;
}

// Normalizes the depth value based on the edge curvature.
float normalizedDepth(float d, vec2 b, float r) {
    // Limits the height build-up strictly to the pixel width defined by 'corner_radius'.
    // This prevents the glass from curving endlessly towards the center.
```

```
    float maxDepth = max(r, 1.0);

    float interiorDepth = max(-d, 0.0);
    return clamp(interiorDepth / maxDepth, 0.0, 1.0);
}

// Calculates the surface height profile using a superellipse formula.
float profileHeight(float t, float zScale) {
    // Superellipse profile: h = H * (1 - (1 - t)^n)^(1/n), t: edge=0 -> center=1.
    float n = max(profile_shape_n, 1.01);
    float invT = clamp(1.0 - t, 0.0, 1.0);
    float inner = max(1.0 - pow(invT, n), 0.0);
    float h = pow(inner, 1.0 / n);
    return h * zScale;
}

// Computes the absolute height at a specific 2D coordinate.
float getHeight(vec2 p, vec2 b, float r, float zScale) {
    float d = sdRoundRect(p, b, r);

    // [FIX 1] Soft boundary fade instead of a hard step at d=0.
    // A hard "if (d > 0) return 0" causes a discontinuous jump in the height
    // field. When heightGradient() straddles this boundary via finite
    // differences, it produces a large spurious gradient spike that manifests
    // as jaggy displacement especially against high-frequency backgrounds.
    // Allowing a smooth fade over ±edge_smoothing pixels eliminates the spike.
    float smoothZone = max(edge_smoothing, 1.0);
    if (d > smoothZone)
        return 0.0;

    float t = normalizedDepth(d, b, r);
    float h = profileHeight(t, zScale);

    // Taper height continuously to zero as d approaches the boundary from inside,
    // and continue fading through the thin outer fringe (0 < d < smoothZone).
    // [FIX] Same edge0>edge1 undefined-behavior pattern as insideMask/edgeBand
    // above (harmless here in practice since the `d > smoothZone` guard above
    // already bounds this call's domain, but corrected for consistency).
    float fade = 1.0 - smoothstep(-smoothZone, smoothZone, d);
    return h * fade;
}

// Dynamically adjusts the sampling step size for normal estimation based on resolution.
float gradientStep(vec2 resolution) {
    float minRes = max(min(resolution.x, resolution.y), 1.0);
    return clamp(minRes / 560.0, 0.45, 1.20);
}

// Estimates the height gradient (slope) by sampling neighboring pixels.
vec2 heightGradient(vec2 p, vec2 b, float r, float zScale, vec2 resolution) {
    float e = gradientStep(resolution);

    float hR = getHeight(p + vec2(e, 0.0), b, r, zScale);
    float hL = getHeight(p - vec2(e, 0.0), b, r, zScale);
    float hB = getHeight(p + vec2(0.0, e), b, r, zScale);
    float hT = getHeight(p - vec2(0.0, e), b, r, zScale);

    return vec2((hR - hL) / (2.0 * e), (hB - hT) / (2.0 * e));
}

// Converts the 2D gradient into a 3D normal vector.
vec3 getNormal(vec2 gradH) {
    return normalize(vec3(-gradH.x, -gradH.y, 1.0));
}

// Calculates the UV coordinate displacement caused by light refraction.
vec2 getDisplacement(float d, vec3 normal, vec2 resolution) {
    if (d > 0.0)
        return vec2(0.0);

    // Standard incident view vector (looking directly into the screen)
    vec3 viewDir = vec3(0.0, 0.0, -1.0);

    // Refract light using Snell's law (Air ~ 1.0 -> Glass ~ IOR)
    float eta = 1.0 / max(ior, 1.001);
    vec3 refractedRay = refract(viewDir, normal, eta);

    // If total internal reflection occurs, refractedRay is (0,0,0)
    if (length(refractedRay) < 0.0001)
        return vec2(0.0);

    float minRes = max(min(resolution.x, resolution.y), 1.0);
```

```
    float thicknessNorm = displacement_scale / minRes;

    // Safety clamp: Prevent infinite stretching artifacts near extreme curves
    // by ensuring the Z component never gets dangerously close to 0.
    float safe_z = max(-refractedRay.z, 0.15);
    vec2 displacement = (refractedRay.xy / safe_z) * thicknessNorm;
    float max_disp = 0.30;
    if (length(displacement) > max_disp) {
        displacement = normalize(displacement) * max_disp;
    }

    return displacement;
}

// Stabilizes UV coordinates near the edges to prevent black borders from bilinear filtering.
vec2 stabilizedUV(vec2 candidate, vec2 fallback) {
    vec2 clamped = clamp(candidate, vec2(0.001), vec2(0.999));
    float edgeDist = min(min(candidate.x, candidate.y), min(1.0 - candidate.x, 1.0 - candidate.y));
    float keep = smoothstep(-0.04, 0.03, edgeDist);
    return mix(fallback, clamped, keep);
}

// [NEW] Adjust color saturation, contrast, brightness
vec3 applySCB(vec3 color, float b, float c, float s) {
    // 1. 輝度 (Brightness): 単純な乗算
    color *= b;

    // 2. コントラスト (Contrast): 0.5のグレーを基準に距離を拡大・縮小
    color = mix(vec3(0.5), color, c);

    // 3. 彩度 (Saturation): グレースケール値とのブレンド
    // 人間の目の感度に基づいた重み (Rec. 601) を使用
    float luma = dot(color, vec3(0.299, 0.587, 0.114));
    color = mix(vec3(luma), color, s);

    // コントラスト調整等でマイナスになった値を0に丸める
    return max(color, 0.0);
}

void main() {
    vec2 resolution = vec2(resolution_x, resolution_y);
    vec2 uv = cogl_tex_coord_in[0].st;

    vec2 pixel_coord = uv * resolution;

    // [CHANGED] Full-screen FBO coordinate reconstruction.
    //
    // Previously the effect actor was sized to the dock area, so "center" was
    // simply resolution * 0.5 and local_pos was measured from that center.
    //
    // Now the effect actor spans the entire monitor (full-screen FBO).
    // We reconstruct the dock center from the dock_* uniforms supplied by
    // dockManager, giving us the same dock-centred local coordinate system
    // without any FBO size / BMS absolute-coordinate mismatch.
    //
    // dock_center is in the same pixel space as pixel_coord (monitor-local).
    vec2 dock_center = vec2(dock_x + dock_w * 0.5, dock_y + dock_h * 0.5);

    // local_pos: pixel offset from the dock center — identical semantics to
    // the old (pixel_coord - resolution*0.5) but now pinned to the dock, not
    // to the actor boundary.
    vec2 local_pos = pixel_coord - dock_center;

    // Pre-calculate geometry anti-aliasing feathering width.
    float edgeFeather = max(edge_smoothing, 0.75);

    // [CHANGED] box_size is derived from dock_w / dock_h instead of resolution.
    // Previously the whole actor was dock-sized so resolution == dock size.
    // Now resolution is the full monitor size; using it here would produce a
    // vastly oversized rounded rectangle.  dock_w/dock_h give the true extents
    // of the glass shape (including SHADER_PADDING on all sides).
    vec2 box_size;
    if (isDock > 0.5) {
        // For the dock, shrink the box so the feathered edges don't reach
        // the dock background boundary.
        vec2 actual_size = vec2(dock_w, dock_h) - vec2(padding * 2.0) - vec2(edgeFeather * 2.0);
        box_size = max(actual_size * 0.5, vec2(1.0));
    } else {
        vec2 actual_size = vec2(dock_w, dock_h) - vec2(padding * 2.0);
        box_size = max(actual_size * 0.5, vec2(1.0));
    }
```

```
    // Distance from the current pixel to the rounded rectangle boundary.
    float d = sdRoundRect(local_pos, box_size, corner_radius);

    // Geometry Anti-Aliasing: Smoothstep forces a sub-pixel soft transition.
    // Inside = 1.0, Outside = 0.0.
    //
    // [FIX] Root cause of the "whole screen darkens by a flat amount as
    // shadow_radius/shadow_intensity increase" bug.
    //
    // This used to be `smoothstep(edgeFeather, -edgeFeather, d)` — edge0
    // (+edgeFeather) is numerically LARGER than edge1 (-edgeFeather). Per the
    // GLSL spec, smoothstep()'s result is *undefined* whenever edge0 >= edge1;
    // it is only guaranteed to behave as documented when edge0 < edge1.
    //
    // With the standard textbook implementation
    // (t = clamp((x-edge0)/(edge1-edge0), 0, 1)) the swapped call still
    // happens to evaluate correctly, which is why the drop shadow itself
    // renders correctly right next to the dock (where |d| is small, so any
    // implementation difference is hard to notice). But several real
    // GLSL/GLSL-ES compilers (and fixed-function/optimized paths some
    // drivers substitute for smoothstep) implement it as sequential branches
    // that assume edge0 < edge1, e.g. `x <= edge0 -> 0`, `x >= edge1 -> 1`.
    // Evaluated with our swapped edges, ANY pixel with d > edgeFeather
    // (i.e. every pixel more than a couple of px outside the dock — the
    // entire rest of the monitor) satisfies `x >= edge1` (edge1 is negative)
    // and is misclassified as insideMask = 1.0 / outsideMask = 0.0 — or, in
    // the mirror-image cases, leaves outsideMask pinned to a non-decaying
    // constant far past where the umbra/penumbra falloff was supposed to
    // reach zero. Either way, the fixed, flat-looking tint over the whole
    // screen that scales with shadow_intensity (and is gated by
    // radiusEnable/shadow_radius) is exactly this outsideMask term failing
    // to fall back to 0 away from the dock.
    //
    // Fix: compute the same 0..1 transition using a call whose edges are
    // always in increasing order (-edgeFeather < edgeFeather), which is
    // well-defined on every driver, and derive insideMask as its complement.
    // Mathematically this produces IDENTICAL values to the original
    // (well-behaved) formula — only the undefined-behavior input ordering is
    // removed.
    float outsideTransition = smoothstep(-edgeFeather, edgeFeather, d);
    float insideMask = 1.0 - outsideTransition;
    float outsideMask = outsideTransition;

    // ------------------------------------------------------------------
    // Realistic drop shadow (anchors the glass on light backgrounds).
    //
    // Real shadows from a glass object have:
    //   * UMBRA   - a tight, dark core where the glass fully blocks the
    //               light source. Sharpest right at the edge.
    //   * PENUMBRA - a wider, softer halo where the glass partially blocks
    //                the light. Decays slowly with distance.
    //   * DIRECTIONAL EXTENSION - the shadow extends slightly further on
    //                the side opposite the light source (governed by
    //                light_angle_deg), not symmetrically in all directions.
    //   * COLOR TINT - real shadows are never pure black. They pick up
    //                ambient light (cool/blue cast for typical lighting).
    //
    // Composited via the Cogl premultiplied-alpha pipeline: a tinted-dark
    // color with alpha `s` darkens the destination by (1 - s) -> drop shadow.
    // ------------------------------------------------------------------

    // 1) Compute the 2D shadow direction in screen space (y points down).
    float lightAngleRad = radians(light_angle_deg);
    vec2 lightDir2D = vec2(cos(lightAngleRad), -sin(lightAngleRad));
    vec2 shadowDir   = -lightDir2D;   // shadow falls away from the light

    // 2) Outward direction from the glass center to this pixel. (Small
    //    epsilon avoids NaN at the exact center where local_pos == 0.)
    vec2 outwardDir = normalize(local_pos + vec2(1e-4));

    // 3) How much this pixel "faces" the shadow side. 0 = on the lit side,
    //    1 = directly opposite the light. max() clamps the lit side to 0.
    float lightAlignment = max(dot(outwardDir, shadowDir), 0.0);

    // 4) Subtle directional factor: 85% on the lit side, 100% on the shadow
    //    side. Gentle so the shadow still feels symmetric on bottom docks
    //    (where there's no room for it to extend "down" anyway).
    float dirRadius    = 0.85 + lightAlignment * 0.15;
    float dirIntensity = 0.85 + lightAlignment * 0.15;

    // 5) Clamp the effective radius to the bgActor's clipped render area so
    //    the shadow's penumbra cannot get hard-clipped at the actor boundary.
```

```
    //    [FIX] This used to derive from `padding` (a fixed 20px optical
    //    margin for refraction/blur, unrelated to the shadow feature), which
    //    capped every shadow_radius setting above ~18-21 to the same fixed,
    //    disproportionately dark ~18px band — no matter how far past that
    //    the 0-100 slider was pushed. shadow_max_radius instead reflects the
    //    real room available (synced from dockManager's CLIP_PADDING), so a
    //    large shadow_radius produces a correspondingly wide, gradual, soft
    //    shadow instead of the same intensity compressed into a tiny band.
    float maxRadius = max(shadow_max_radius, 5.0);
    float effectiveRadius    = min(shadow_radius * dirRadius, maxRadius);

    // 5b) [FIX] Master on/off switch driven purely by the user's radius
    //     setting. Previously effectiveIntensity depended only on
    //     shadow_intensity, so setting shadow_radius to 0 still left a
    //     visible shadow: the umbra term below stabilizes its divisor with
    //     a fixed max(..., 0.5) floor, which keeps producing a ~0.5px-wide,
    //     up-to-0.8-alpha dark band right at the glass edge no matter how
    //     small effectiveRadius gets. A short smoothstep ramp (rather than a
    //     hard step()) avoids a visible pop for very small nonzero radii.
    float radiusEnable = smoothstep(0.0, 0.75, shadow_radius);
    float effectiveIntensity = shadow_intensity * dirIntensity * radiusEnable;

    // [FIX] Guard against a 0/0 division when effectiveRadius collapses to
    // 0 (shadow_radius = 0): penumbra_t below used to divide by
    // effectiveRadius directly, which produces NaN exactly at d == 0 -
    // i.e. exactly on the glass boundary. NaN survives the later
    // "* effectiveIntensity" multiply (NaN * 0 is still NaN, not 0), so it
    // was reaching the final composite and rendering as a dark hairline
    // right on the glass edge even after the radiusEnable fix above.
    float safeRadius = max(effectiveRadius, 0.001);

    // 6) UMBRA: tight, dark core. Linear decay over 0.4 * effectiveRadius.
    //    This is the "contact shadow" band - very close to the glass.
    float umbra_t = clamp(d / max(safeRadius * 0.40, 0.5), 0.0, 1.0);
    float umbra  = (1.0 - umbra_t) * 0.80;

    // 7) PENUMBRA: wider, softer halo.
    //    [FIX] Previously a plain squared falloff (pow(1-t, 2)): its value
    //    reaches exactly 0 at t=1 with zero SLOPE, but its CURVATURE
    //    (2nd derivative) at t=1 is still nonzero (d^2/dt^2 of (1-t)^2 is a
    //    constant 2, not 0). Right where the curve meets the flat "outside
    //    shadow" region (which has zero value, slope, AND curvature), that
    //    curvature mismatch reads as a faint but visible bend/ring at the
    //    shadow's outer edge instead of a truly continuous fade.
    //
    //    Replaced with a quintic "smootherstep" ease (6t^5-15t^4+10t^3,
    //    Perlin's improved curve) applied to the fading term. It has zero
    //    value, zero 1st derivative, AND zero 2nd derivative at t=1, so the
    //    penumbra flattens smoothly INTO the surrounding zero rather than
    //    just touching it — matching curvature with the "no shadow" region
    //    outside, for a genuinely continuous transition.
    float penumbra_t = clamp(d / safeRadius, 0.0, 1.0);
    float penumbraFade = 1.0 - penumbra_t; // 1 at glass edge -> 0 at outer radius
    float penumbraEase = penumbraFade * penumbraFade * penumbraFade *
        (penumbraFade * (penumbraFade * 6.0 - 15.0) + 10.0);
    float penumbra = penumbraEase * 0.55;

    // 8) Combined shadow alpha, applied only OUTSIDE the glass shape.
    float shadowAlpha = clamp(
        (umbra + penumbra) * outsideMask * effectiveIntensity,
        0.0, 1.0
    );

    // [FIX] Use the same clamped `maxRadius` here that effectiveRadius/
    // safeRadius above are derived from, instead of the raw
    // `shadow_max_radius` uniform. They happen to be numerically identical
    // whenever shadow_max_radius >= 5 (the normal case, since dockManager
    // syncs it from CLIP_PADDING - 20 = 180), but keeping every cutoff in
    // this block anchored to the exact same value removes a latent
    // inconsistency: if shadow_max_radius were ever synced to something
    // below 5 (e.g. transiently, before the first geometry sync completes,
    // or on a future call site that forgets the CLIP_PADDING margin), this
    // bound and effectiveRadius's cap would silently disagree.
    // [FIX] Previously started fading at just 10% of maxRadius. Whenever
    // effectiveRadius (the penumbra's own natural falloff distance) was
    // smaller than maxRadius — the common case — this made boundsMask start
    // clipping the shadow *while the penumbra curve above was still
    // decaying*, layering a second, independent smoothstep on top of it.
    // Two smooth-but-different curves multiplied together bend the combined
    // curve at a point that has nothing to do with the shadow's actual
    // outer edge, which reads as an extra "shoulder" partway through the
    // fade. boundsMask only exists to guarantee the shadow can't get
```

```
    // hard-clipped at the bgActor's physical boundary (maxRadius) — it
    // should stay at 1.0 (no-op) through the entire range the penumbra
    // curve is already handling, and only engage in the last stretch right
    // before maxRadius. Starting at 85% keeps it out of the way for
    // virtually every shadow_radius setting, only smoothing the rare case
    // where effectiveRadius is pushed all the way up to maxRadius.
    float boundsFade = 1.0 - smoothstep(maxRadius * 0.85, maxRadius, d);
    // Same quintic ease as the penumbra above, so this safety fade also
    // reaches zero value/slope/curvature together at d = maxRadius,
    // continuous with the (curvature-free) fully-outside region beyond it.
    float boundsMask = boundsFade * boundsFade * boundsFade *
        (boundsFade * (boundsFade * 6.0 - 15.0) + 10.0);
    shadowAlpha *= boundsMask;

    // [FIX] Defense-in-depth hard cutoff, independent of any smoothstep().
    // `step(edge, x)` has only a single comparison point (no edge0/edge1
    // ordering to get wrong) and is well-defined on every driver: it
    // returns 0 once d passes maxRadius, guaranteeing the shadow can never
    // bleed past its own configured range no matter what a given GPU/driver
    // does with the smoothstep() calls above.
    shadowAlpha *= 1.0 - step(maxRadius, d);

    // 9) Shadow color: dark with a subtle cool/blue cast. Suggests ambient
    //    sky light bleeding into the shadow (a hallmark of realistic
    //    outdoor / window-lit shadow rendering). Avoids the "painted-on
    //    pure black" look.
    vec3 shadowColor = vec3(0.03, 0.04, 0.08);

    vec4 source = texture2D(cogl_sampler1, uv);

    vec2 gradH = heightGradient(local_pos, box_size, corner_radius, max_z, resolution);
    vec3 normal = getNormal(gradH);

    vec2 disp = getDisplacement(d, normal, resolution);

    // Dampen the refraction near the exact boundaries to eliminate jagged artifacts.
    float edgeDampen = smoothstep(0.0, edgeFeather * 3.0, -d);
    disp *= edgeDampen;

    vec2 refractedUv = stabilizedUV(uv + disp, uv);

    float minRes = max(min(resolution.x, resolution.y), 1.0);
    vec2 chromaDir = length(disp) > 0.00001 ? normalize(disp) : vec2(0.0);

    // Calculate Chromatic Aberration vectors (separating RGB channels slightly).
    vec2 chromaVec = chromaDir * (chroma_strength / minRes) * edgeDampen;
    vec2 uvR = stabilizedUV(refractedUv + chromaVec, refractedUv);
    vec2 uvG = refractedUv;
    vec2 uvB = stabilizedUV(refractedUv - chromaVec, refractedUv);

    // Step 1: RGSS (Rotated Grid Super-Sampling) Pattern Implementation
    // Instead of sampling in a simple square, sampling in a slanted diamond pattern
    // provides significantly better anti-aliasing for both horizontal and vertical edges.
    float edgeProximity = 1.0 - smoothstep(0.0, edgeFeather * 4.0, -d);
    float aa_spread = mix(0.75, 2.5, edgeProximity);
    vec2 texel = vec2(aa_spread) / resolution;

    vec2 off1 = vec2( 0.375, -0.125) * texel;
    vec2 off2 = vec2( 0.125,  0.375) * texel;
    vec2 off3 = vec2(-0.375,  0.125) * texel;
    vec2 off4 = vec2(-0.125, -0.375) * texel;

    // Hard limit sampling coordinates to 1.2px inside the texture bounds.
    // This prevents bilinear filtering from accidentally pulling in black/transparent
    // pixels from the void outside the texture space.
    vec2 margin = vec2(1.2) / resolution;
    #define SAFE(u) clamp(u, margin, 1.0 - margin)

    // Step 2: Multi-tap Sampling (Averaging 4 sub-pixels to smooth out the image)
    vec3 refractedRgb = vec3(
        (texture2D(cogl_sampler1, SAFE(uvR + off1)).r +
         texture2D(cogl_sampler1, SAFE(uvR + off2)).r +
         texture2D(cogl_sampler1, SAFE(uvR + off3)).r +
         texture2D(cogl_sampler1, SAFE(uvR + off4)).r) * 0.25,

        (texture2D(cogl_sampler1, SAFE(uvG + off1)).g +
         texture2D(cogl_sampler1, SAFE(uvG + off2)).g +
         texture2D(cogl_sampler1, SAFE(uvG + off3)).g +
         texture2D(cogl_sampler1, SAFE(uvG + off4)).g) * 0.25,

        (texture2D(cogl_sampler1, SAFE(uvB + off1)).b +
         texture2D(cogl_sampler1, SAFE(uvB + off2)).b +
```

```
         texture2D(cogl_sampler1, SAFE(uvB + off3)).b +
         texture2D(cogl_sampler1, SAFE(uvB + off4)).b) * 0.25
    );

    // Apply color saturation, contrast, brightness
    vec3 adjustedRefracted = applySCB(refractedRgb, brightness, contrast, saturation);
    vec3 refracted = adjustedRefracted;

    vec3 tintColor = vec3(tint_r, tint_g, tint_b);
    vec3 insideBaseColor = mix(refracted, tintColor, tint_strength);

    // [FIX] Do NOT multiply by insideMask here. insideMask is the same
    // value used as `alpha` below, and the final composite already
    // multiplies the fully-lit color by it exactly once
    // (finalRgb = litColor * alpha), matching Cogl/Clutter's
    // premultiplied-alpha compositing (out = src.rgb + dst.rgb*(1-src.a)).
    //
    // Baking insideMask into baseColor here AS WELL made rgb fall off as
    // insideMask^2 instead of insideMask^1 across the antialiased edge band
    // (the +-edge_smoothing px zone where insideMask is fractional, i.e.
    // exactly where the visible glass boundary sits). An under-premultiplied
    // color (rgb/alpha < true color) reads as a dark ring right at that
    // boundary once composited over the background — invisible at small
    // edge_smoothing (the transition band is only ~1-2px wide, too thin to
    // notice) but clearly visible once edge_smoothing is turned up, since
    // the band widens and the darkening becomes visible.
    //
    // This is independent of shadow_radius/shadow_intensity (radiusEnable
    // below already zeroes the drop shadow correctly at shadow_radius=0);
    // the ring persists even with the drop shadow fully disabled, which is
    // exactly the "Shadow Radius 0, Edge Smoothing >0" symptom reported.
    vec3 baseColor = insideBaseColor;

    // ------------------------------------------------------------------
    // Inner depth effects — make the glass look 3D on LIGHT backgrounds
    // where refraction and rim alone are not visible.
    //
    // A curved glass body has two visual cues that read as "3D" even
    // when the refracted background is invisible (e.g. on a white wall):
    //   (a) AMBIENT OCCLUSION near the inside edge — the glass body
    //       itself blocks light, so the inside edge is darker than
    //       the center. (This is the "shadow under the glass" the user
    //       asked about.)
    //   (b) A FOCAL HIGHLIGHT where light converges through the curved
    //       surface, biased slightly toward the light source.
    //
    // These are baked into the base color BEFORE the screen-blend
    // lighting pass, so they interact correctly with the rim / sheen
    // / specular that follow.
    // ------------------------------------------------------------------

    // (a) Inner shadow: dark band just inside the glass edge.
    //     aoMask = 1 at d=0 (right at the edge), fading to 0 at
    //     d = -ao_radius (ao_radius px inward from the edge).
    //     Strength is controlled independently by ao_intensity (0-1).
    //     [CHANGED] Previously this reused rim_width for its falloff
    //     distance and was multiplied by the drop shadow's radiusEnable
    //     (so it silently disappeared whenever shadow_radius was 0). Both
    //     couplings are removed here so the inner AO darkening has its own
    //     independent radius/intensity controls, matching the outer drop
    //     shadow's separate radius/intensity pair.
    float aoMask = 1.0 - smoothstep(0.0, max(ao_radius, 0.001), -d);
    baseColor *= (1.0 - aoMask * ao_intensity);

    // (b) Center focal highlight: bright spot offset slightly toward
    //     the light source, simulating where the curved glass focuses
    //     light. Uses lightDir2D (computed in the shadow block earlier
    //     in main()) so it tracks the user's light-angle setting.
    //     `-lightDir2D` because the focal point sits on the same side
    //     as the light, not the shadow side.
    /*
    vec2 focalOffset = -lightDir2D * box_size * 0.15;
    vec2 fromFocal  = (local_pos - focalOffset) / box_size;
    float radialDist = length(fromFocal);
    float focalHighlight = (1.0 - smoothstep(0.0, 0.85, radialDist)) * 0.20;
    baseColor += vec3(focalHighlight);
    */

    // lightAngleRad was declared earlier in main() (in the shadow block) and
    // is reused here for the 3D lighting direction.
    vec3 lightDir = normalize(vec3(cos(lightAngleRad), sin(lightAngleRad), 0.38));
    vec3 viewDir = vec3(0.0, 0.0, 1.0);
```

```
    vec3 reflectDir = reflect(-lightDir, normal);
    float response = 1.0;

    // Create a sharp band for the rim lighting near the edges.
    // [FIX] Same undefined-behavior ordering issue as insideMask above
    // (edge0 = rim_width > edge1 = 0.0). Rewritten with increasing edges and
    // complemented, so it's well-defined on every driver instead of only
    // "happening to work" with the textbook smoothstep formula.
    // [FIX 2] The above rewrite still hit smoothstep(0.0, rim_width, ...)
    // with rim_width == 0.0, i.e. edge0 == edge1 — GLSL spec: "results are
    // undefined if edge0 >= edge1", which includes equality. In practice
    // this meant setting "Rim Width" to 0 in prefs did NOT reliably disable
    // the rim highlight (observed: driver-dependent, could keep rendering
    // a full-strength or garbage edgeBand instead of 0). Clamp the width
    // used inside smoothstep away from 0, and separately force-zero the
    // whole band with an explicit step() gate on the *unclamped* rim_width,
    // so "0 truly means off" regardless of what the clamped smoothstep does.
    float safeRimWidth = max(rim_width, 0.001);
    float edgeBand = (1.0 - smoothstep(0.0, safeRimWidth, abs(d))) * step(0.0005, rim_width);

    float rimDot = 1.0 - max(dot(normal, viewDir), 0.0);
    float rimFresnel = pow(max(rimDot, 0.0), max(rim_power, 0.001));
    float lightMask = pow(abs(dot(normal, lightDir)), max(rim_directional_power, 1.0));

    // Mix the fresnel effect with the edge mask to keep light strictly on the bevels.
    float rimShape = mix(pow(edgeBand, 0.85), rimFresnel, 0.55) * edgeBand;
    float finalRimLight = rimShape * lightMask * rim_intensity * rim_light_color_intensity;
    finalRimLight *= response;

    // [FIX] Previously also multiplied by insideMask here to "mask out
    // light bleeding past the actual geometry boundary" — but the final
    // composite (finalRgb = litColor * alpha, alpha = insideMask) already
    // does that exactly once for the whole litColor (baseColor + all added
    // light, combined via screen blend below). Multiplying by insideMask
    // here too caused the same insideMask^2 under-premultiplication as
    // baseColor above, deepening the dark ring in the antialiased edge band
    // whenever edge_smoothing is large. No bleeding actually occurs from
    // removing this: outside the shape insideMask (and therefore the final
    // alpha) is still 0, so the pixel is still fully transparent regardless
    // of finalRimLight's magnitude.

    float specularDot = max(dot(reflectDir, viewDir), 0.0);
    float specularLight = pow(specularDot, max(shininess, 1.0));
    specularLight *= specular_intensity * response;
    float specMask = mix(0.25, 1.0, insideMask) * clamp(edgeBand + insideMask * 0.65, 0.0, 1.0);
    specularLight *= specMask;

    float idleRim = edgeBand * 0.008;
    // [FIX] Same redundant-insideMask issue as finalRimLight above — removed.
    // The final `litColor * alpha` composite already applies insideMask once.

    // Background sheen uses 3D surface normal directly (no 2D radial fallback).
    float sheenFacing = max(dot(normal, lightDir), 0.0);
    float surfaceSheen = pow(sheenFacing, 1.65);
    // [FIX] Dropped the extra insideMask factor here too — same
    // double-premultiplication issue as baseColor/finalRimLight/idleRim
    // above; the final `litColor * alpha` composite already covers it.
    surfaceSheen *= mix(1.0, 0.55, edgeBand);
    vec3 sheenColor = vec3(1.0) * surfaceSheen * sheen_intensity;

    float alpha = insideMask;

    // --- Light Blending Strategy ---

    // 1. Group all additive lighting components together
    // Gated by surface_light_enabled: application windows turn this whole
    // group off (see LiquidEffect.setSurfaceLightEnabled) so only the
    // (already-independent) outer drop shadow and inner AO darkening in
    // baseColor remain — no rim/specular/sheen glint hugging the edge.
    vec3 addedLight = (vec3(specularLight + finalRimLight + idleRim) + sheenColor) * surface_light_enabled;

    // 2. Screen Blend Mode (A + B - A*B)
    // Instead of simply adding lights (which causes intense overexposure and blows out
    // white backgrounds), this smoothly limits the maximum brightness to 1.0.
    vec3 litColor = baseColor + addedLight - (baseColor * addedLight);

    // [CHANGED] 3. Hue-preserving clamp (色相を維持する安全処理)
    // RGBのいずれかが1.0を超えた場合、白飛びによる色変（黄ばみ等）を防ぐため、
    // 最も強い色を基準にRGB全体の比率を保ったまま縮小する。
    float maxChannel = max(litColor.r, max(litColor.g, litColor.b));
    if (maxChannel > 1.0) {
        litColor /= maxChannel;
```

```
    }
    // マイナス値への侵入を防ぐ
    litColor = max(litColor, 0.0);

    // Composite the glass OVER the shadow, in PREMULTIPLIED-alpha space.
    //
    // Cogl/Clutter blends with `out = src.rgb + dst.rgb * (1 - src.a)`, which
    // adds src.rgb directly — independent of src.a. Therefore the emitted RGB
    // must already be multiplied by its coverage, otherwise any non-zero color
    // at zero coverage leaks as a constant tint across the whole actor.
    //
    // That leak is exactly what produced the dark rectangle behind the dock:
    // outside the shape `shadowColor` (a non-zero navy) was emitted even where
    // `shadowAlpha` had decayed to 0, painting the entire bgActor rectangle.
    //
    // Premultiplied "A over B":
    //   rgb = A.rgb*A.a + B.rgb*B.a*(1 - A.a)
    //   a   = A.a       + B.a*(1 - A.a)
    // with A = glass (litColor, alpha), B = shadow (shadowColor, shadowAlpha).
    // At zero total coverage this is exactly vec4(0) -> no rectangle.
    float shadowContribution = shadowAlpha * (1.0 - alpha);
    vec3 finalRgb   = litColor * alpha + shadowColor * shadowContribution;
    float finalAlpha = alpha + shadowContribution;

    // Output with premultiplied alpha format, required by Clutter/Cogl pipeline.
    cogl_color_out = vec4(finalRgb, finalAlpha) * cogl_color_in;
}
```